

FR-5

AI Coach น้องกล้า

โค้ชการเงิน AI ที่พูดได้
เข้าใจระบบทั้งหมดตั้งแต่ศูนย์ จนอธิบายโค้ดและตอบ
คำถามอาจารย์ได้ เหมือนเป็นคนสร้างระบบนี้เอง

ภาค ก · พื้นฐานทั้งระบบ (สำหรับคนไม่เคยเขียนโค้ด)

ภาค ข · FR-5 ละเอียดทุกส่วน

โค้ดจริงพร้อมคำอธิบายทีละบรรทัด



วิธีใช้เล่มนี้

เล่มนี้ทำให้คนที่รับผิดชอบ FR-5 AI Coach น้องกล้า เข้าใจระบบทั้งหมด ตั้งแต่ "แอปนี้สร้างด้วยอะไร" ไปจนถึง "บรรทัดนี้ของโค้ดทำอะไร" แม้ไม่เคยเขียนโปรแกรมมาก่อน

เล่มนี้แบ่งเป็น 2 ภาค

i ภาค ก • พื้นฐานทั้งระบบ

เหมือนกันทุกเล่ม (FR-1 ถึง FR-6) สอนตั้งแต่ศูนย์: เครื่องมือที่ใช้ (VS Code, Node.js, Git), ภาษา TypeScript, React, การทำแอปและเว็บจากโค้ดชุดเดียว, เซิร์ฟเวอร์ Supabase, การเชื่อม AI, การออกแบบหน้าจอ (UX/UI), การทดสอบ และ GitHub

★ ภาค ข • FR-5 AI Coach น้องกล้า

เฉพาะ FR ของเล่มนี้ เขียนในมุม "ฉันเป็นคนสร้างระบบนี้" มีหน้าจอจริง หลักการ ตัวอย่างคิดมือ โค้ดจริงพร้อมคำอธิบายทีละบรรทัด การทดสอบ เหตุผล การออกแบบ คำถามอาจารย์ แบบฝึกหัด และสรุปปต์นำเสนอ

เปิดโค้ดใน VS Code ได้ทุกจุดที่เห็นในเล่ม

โค้ดทุกก้อนในเล่มนี้ ดึงมาจากไฟล์จริงในโปรเจกต์ ไม่ได้พิมพ์ใหม่ เลขบรรทัดทางซ้ายจึงตรงกับในไฟล์ทุกตัว ทุกก้อนมีแถบล่างแบบนี้:

src/domain/runway.ts

ตัวอย่าง • บรรทัด 17-23

```
17 * so the status is "no_spending" and no date is shown.
18 */
19 import { addDays, bkkDayKey, daysBetween } from './dates';
20 import { confirmedOnly } from './summary';
21 import type { Transaction } from './types';
22
23 export const DEFAULT_FLOOR_SATANG = 50_000; // ฿500
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:17

- 1 **แตะชื่อไฟล์ หรือ "เปิดใน VS Code (เว็บ)"** ระบบจะเปิด VS Code ในเบราว์เซอร์ (vscode.dev) พร้อมโปรเจกต์ MindPay และกระโดดไปที่บรรทัดนั้นเลย ไม่ต้องติดตั้งอะไร ใช้ได้ทั้งคอมและแท็บเล็ต ครั้งแรกอาจถามให้เข้าสู่ระบบ GitHub ก่อน
- 2 **แตะ "GitHub"** เปิดไฟล์เดียวกันบนเว็บไซต์ GitHub และไฮไลต์บรรทัดเหล่านั้น (ใช้ได้บนมือถือ)
- 3 **ถ้าโคลนโปรเจกต์ลงเครื่องแล้ว** (วิธีอยู่ในบท A3) เปิด VS Code กด Ctrl+P (Mac: Cmd+P) แล้วพิมพ์ข้อความตัวหน้าที่มุมขวา เช่น **src/domain/runway.ts:17** แล้วกด Enter จะเปิดไฟล์และไปที่บรรทัด 17 ทันที

ชื่อไฟล์สีเขียวขีดเส้นใต้ในเนื้อหา เช่น [src/domain/runway.ts](#) ก็แตะเปิดได้เหมือนกัน

i ลิงก์ล็อกไว้ที่เวอร์ชันของเล่มนี้

ทุกลิงก์ชี้ไปที่โค้ดเวอร์ชันวันที่ 3 ต.ค. 2569 (commit 646b485) ซึ่งเป็นเวอร์ชันที่ใช้เขียนเล่มนี้ ถ้าทีมแก้ไขโค้ดหลังจากนี้ ลิงก์ในเล่มยังเปิดเวอร์ชันเดิมที่เลขบรรทัดตรงกับในเล่ม ถ้าอยากดูเวอร์ชันล่าสุด ให้เปลี่ยนไปที่ branch main ใน VS Code หรือ GitHub

อ่านคำอธิบายโค้ดยังไ้

หลังโค้ดแต่ละก้อนจะมีตาราง "อ่านทีละบรรทัด" คอลัมน์ซ้ายคือเลขบรรทัด คอลัมน์ขวาคือคำอธิบายเป็นภาษาคน ให้เปิดโค้ดกับตารางคู่กัน อ่านบรรทัดในโค้ดก่อน แล้วค่อยอ่านคำอธิบาย ถ้าเจอคำที่ไม่รู้จัก ให้กลับไปดูบท A4 ถึง A7 ซึ่งสอนพื้นฐานภาษาตั้งแต่ศูนย์

บรรทัด	ความหมาย
17	ตัวอย่าง: คำอธิบายของบรรทัด 17 จะอยู่แถวนี้
18–20	ถ้าหลายบรรทัดทำงานเรื่องเดียวกัน จะอธิบายรวมกันเป็นช่วง

กล่องสีต่าง ๆ ในเล่ม

≈ เปรียบเทียบ

เทียบกับของในชีวิตจริง ให้เห็นภาพก่อนลงรายละเอียด

★ จำให้ได้

เรื่องสำคัญที่อาจารย์มักถาม

!ระวัง

จุดที่คนมักเข้าใจผิด หรือนึกที่เคยเกิดขึ้นจริง

i เพิ่มเติม

รายละเอียดเสริม ข้ามได้ถ้ารีบ

ถ้ามีเวลาน้อย อ่านตามนี้

เวลาที่มี	อ่านอะไร
15 นาที	บท A1 (MindPay คืออะไร), บท A2 (แผนที่ระบบ), แล้วภาค ข บทที่ 1 และบทสุดท้าย (สรุปต้นน้ำเสนอ)
1–2 ชั่วโมง	ภาค ก บท A1, A2, A4, A9 แล้วอ่านภาค ข ทั้งหมดยกเว้นตารางโค้ดทีละบรรทัด
เตรียมตอบอาจารย์	ภาค ข บท "ทำไมฉันออกแบบแบบนี้", "ข้อจำกัด", "คำถามที่อาจารย์อาจถาม" และ "แบบฝึกหัด"
อยากอธิบายโค้ดได้จริง	อ่านทั้งเล่มตามลำดับ เปิด VS Code คู่กัน และลองทำแบบฝึกหัดโดยไม่ดูเฉลย

i ตัวเลขในเล่ม

ตัวอย่างคิดมือและเฉลยแบบฝึกหัดทุกข้อ ตรวจโดยรันฟังก์ชันจริงของแอปแล้ว ภาพหน้าจอต่่ายจากเว็บเวอร์ชันจริงใน
โหมดทดลอง (ข้อมูลตัวอย่าง) วันที่ 9 ต.ค. 2569 ข้อมูลตัวอย่างสร้างตามวันที่เปิดแอป ถ้าเปิดดูเองตัวเลขจะต่างไป
เล็กน้อย แต่หลักการเหมือนเดิมทุกอย่าง

สารบัญ

วิธีใช้เล่มนี้	1
เล่มนี้แบ่งเป็น 2 ภาค	1
เปิดโค้ดใน VS Code ได้ทุกจุดที่เห็นในเล่ม	1
อ่านคำอธิบายโค้ดยังไง	2
กล่องสีต่าง ๆ ในเล่ม	2
ถ้ามีเวลาน้อย อ่านตามนี้	2
พื้นฐานทั้งระบบ MINDPAY	10
MindPay คืออะไร และทีมเราสร้างอะไร	11
ปัญหาที่ทีมเราอยากแก้	11
6 ฟังก์ชันหลัก (Functional Requirements)	11
หน้าต่างแอปโดยรวม	12
ใช้แอปได้ 3 แบบ	13
ตัวเลขของโปรเจกต์	13
คำศัพท์ที่จะเจอบ่อยในเล่มนี้	14
แผนที่ระบบ: ชิ้นส่วนทั้งหมดต่อกันยังไง	15
ระบบมี 4 ส่วนใหญ่	15
ข้างในแอป: แบ่งโค้ดเป็น 5 ชั้น	15
ตามข้อมูล 1 รายการ: ตั้งแต่กดบันทึก จนถึงเลขบนหน้าหลักเปลี่ยน	16
ตามข้อมูลที่ใช้ AI: สแกนสลิป 1 ใบ	20
เครื่องมือ: เขียนโค้ดที่ไหน ใช้โปรแกรมอะไร และ VS Code ตั้งแต่ศูนย์	22
กล่องเครื่องมือของทีมเรา	22
ติดตั้งเครื่องมือ (ถ้าครั้งเดียว)	22
เอาโค้ด MindPay มาไว้ในเครื่อง	22
ไฟล์ package.json: บัตรประชาชนของโปรเจกต์	23
คำสั่งที่ใช้บ่อย	26
รู้จักหน้าจอส VS Code	26
คีย์ลัดที่ทำให้อ่านโค้ดเร็วขึ้น 10 เท่า	27
ส่วนเสริม (Extensions) ที่แนะนำ	28
ไม่อยากติดตั้งอะไร: ใช้ VS Code บนเว็บ	28
ฝึกใช้: ตามรอย Money Runway ใน 5 นาที	28
พื้นฐานการเขียนโปรแกรม สำหรับคนที่ไม่เคยเขียนเลย	29
โปรแกรมคืออะไร	29
1. ค่า (value) และชนิดของค่า	29
2. ตัวแปร: กล่องที่มีชื่อ	29
3. ตัวดำเนินการ: คำนวณและเปรียบเทียบ	30
4. เชื้อไข if: ถ้า... ให้ทำ...	30
5. ฟังก์ชัน: เครื่องจักรรับของเข้า ส่งของออก	31
6. Array: รายการของหลายค่า	31
7. Object: ของ 1 ชิ้นที่มีหลายช่อง	33
8. วงซ้ำ (loop)	34
9. ข้อความ (string) ที่ใช้บ่อย	34
10. คณิตศาสตร์ที่ใช้	34
แบบฝึกหัด A4	34

TypeScript: ภาษาที่ทีมเราใช้ และชนิดข้อมูลของ MindPay	36
ทำไมต้อง TypeScript	36
การบอชนิดข้อมูล (type annotation)	36
ไฟล์ชนิดข้อมูลกลาง: src/domain/types.ts	36
ชนิดข้อมูลแบบพิเศษที่เจอบ่อย	38
import และ export: แคร่โค้ดระหว่างไฟล์	39
งานที่ต้องรอ: async, await และ Promise	40
จัดการความผิดพลาด: try, catch, throw	40
ทำไมเก็บเงินเป็นสตางค์ (จำนวนเต็ม)	41
อ่านโค้ดจริง: แปลงข้อความที่ผู้ใช้พิมพ์เป็นสตางค์	41
Regular expression (regex) อ่านยังงี้	43
แปลงสตางค์กลับเป็นข้อความบาท	44
React: วาดหน้าจอจากข้อมูล	46
แนวคิดหลัก: หน้าจอเป็นผลลัพธ์ของข้อมูล	46
Component: ชิ้นส่วนหน้าจอที่เป็นฟังก์ชัน	46
State: ข้อมูลที่เปลี่ยนได้ และทำให้หน้าจอวาดใหม่	48
Hook ที่ใช้ในโปรเจกต์	49
Context: แคร่ข้อมูลให้ทุกหน้าโดยไม่ต้องส่งต่อเป็นทอด ๆ	51
ทุกอย่างประกอบกัน: หน้าจอใช้ข้อมูลกลางยังงี้	52
React Native และ Expo: โค้ดชุดเดียว ได้ทั้งแอปและเว็บ	54
React Native คืออะไร	54
Expo: ชุดเครื่องมือที่ทำให้ React Native ง่ายขึ้น	55
Expo Router: ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า	55
อ่านโค้ด: _layout.tsx โครงของทั้งแอป	57
แถบเมนูล่าง: (tabs)/_layout.tsx	61
โค้ดที่ต่างกันระหว่างมือถือกับเว็บ	62
จากโค้ด สู่เว็บและแอปที่ผู้ใช้ลง	64
วิธีอ่านโค้ด MindPay ด้วยตัวเอง	66
ขั้นตอนอ่านไฟล์ที่ไม่เคยเห็น	66
รูปแบบที่เจอซ้ำทั้งแอป (ตารางโกง)	66
ฟังก์ชันวันเวลาที่ต้องรู้ (ทุก FR ใช้)	67
แผนที่: อยากรู้อะไรไปเปิดไฟล์ไหน	68
เซิร์ฟเวอร์และฐานข้อมูล: Supabase	70
ฐานข้อมูลคืออะไร	70
Supabase ให้อะไรทีมเราบ้าง	70
Migration: ไฟล์สร้างฐานข้อมูล	71
Row Level Security (RLS): หัวใจความปลอดภัย	74
กฎแฉ 2 แบบ: อันไหนอยู่ในแอปได้ อันไหนห้าม	76
แอปต่อ Supabase ยังงี้: src/data/supabase.ts	76
การสมัครและเข้าสู่ระบบ	78
Repo: ชั้นที่ซ่อนว่าข้อมูลอยู่ที่ไหน	79
เปิดแอปแล้วเกิดอะไรขึ้น: AppProvider เลือกโหมด	84
การเชื่อม AI: ทำไมต้องผ่านเซิร์ฟเวอร์ และทำงานยังงี้	87
AI ในแอปคืออะไร	87
ทำไมแอปไม่เรียก AI เอง	87
เส้นทางของคำถาม 1 ข้อถึงน้องกล้า	87

ฝั่งแอป: src/services/coach.ts	88
ฝั่งเซิร์ฟเวอร์: Edge Function coach ที่ละบัสการ์ด	89
ตัวช่วยกลาง: supabase/functions/_shared/common.ts	91
เลือก AI ตัวไหน: Claude หรือ Gemini	94
ส่งคำขอไปที่ Claude: callClaude	96
ตั้งค่า AI (ผู้ดูแลทำครั้งเดียว)	99
หลักการใช้ AI อย่างรับผิดชอบที่ทีมเราใช้	99
UX/UI: การออกแบบหน้าจอและ Design System	100
หลักการออกแบบของ MindPay	100
Design System คืออะไร	100
สี: Green + Gold	100
ความอ่านง่ายของสี (WCAG) และเทสต์ที่บังคับ	102
ตัวอักษร	104
ชิ้นส่วนสำเร็จรูป: ปุ่ม (Button)	104
Accessibility: ทุกคนใช้ได้	107
แอนิเมชัน: สวยแต่ไม่กินแบต	107
เช็กลิสต์ก่อนเพิ่มหน้าจอใหม่ (จาก DESIGN_SYSTEM.md)	107
การทดสอบ: พิสูจน์ว่าระบบทำงานถูก	109
ทำไมต้องเขียนเทสต์	109
Unit test: ตรวจสอบฟังก์ชันทีละตัว	109
E2E: ทดสอบทั้งแอปเหมือนคนใช้จริง	111
Manual test: ทดสอบบนมือถือจริง	114
Traceability: ผูก Requirement → โค้ด → เทสต์	115
ตรวจสอบคุณภาพอื่น ๆ	115
Git และ GitHub: เก็บโค้ด ทำงานเป็นทีม และ build อัตโนมัติ	117
Git: ระบบบันทึกประวัติ	117
ใช้ Git ประจำวัน	117
ประวัติของ MindPay UU GitHub	118
GitHub Actions: หุ่นยนต์ที่ทำงานทุกครั้งที่เรา push	118
Secrets UU GitHub	123
ไฟล์เอกสารใน repo	123
6 FR ทำงานร่วมกันอย่างไร และคำถามระดับระบบ	125
สายข้อมูลของทั้งระบบ	125
ใครใช้ของใคร	125
ไฟล์ที่หลาย FR ใช้ร่วมกัน (แก้แล้วกระทบหลายคน)	125
คำถามระดับระบบ ที่อาจารย์ถามใครก็ได้	126
คำศัพท์รวม	127
FR-5 AI COACH น้องกล้า	129
FR-5 ในหน้าเดียว	130
อยู่ตรงไหนในแอป	130
4 ข้อที่ต้องจำ	130
ปัญหาที่ฉันทายากแท้	132
แต่การเอา AI มาเป็นโค้ด มีความเสี่ยง 5 ข้อ	132
เป้าหมายที่ฉันทั้งไว้ตอนออกแบบ	132
Requirement การเปลี่ยนแปลง และเงื่อนไขผ่าน	134

เดิม vs ตอนนี้	134
เงื่อนไขผ่าน (Acceptance criteria) → โค้ด → เทสต์	134
น้องกล้าคือใคร	136
อารมณ์ 6 แบบ ใช้เมื่อไร	136
ภูน้ำเสียง (ใช้ทั้งข้อความที่เขียนเองและคำตอบของ AI)	137
ตัวหน้าจอ: แท็บโค้ช	138
การ์ด "สิ่งที่ควรรู้นี้"	138
แตะการ์ด: น้องกล้าเล่าให้ฟัง	139
คุยกับน้องกล้า (และสิ่งที่เกิดในโหมดทดลอง)	140
เปลี่ยนชุดบนเวที และเมื่อเครื่องไม่มีเสียงไทย	141
ตัวหน้าจอ: คุยกับน้องกล้าด้วยบัญชีจริง	143
หน้าหลักและการ์ดสิ่งที่ควรรู้	143
ถามคำถามลัด: AI ตอบ น้องกล้าพูด	144
เช็กก่อนจ่าย → ถามโค้ชเรื่องนี้ (FR-6 ส่งต่อ FR-5)	145
เมื่อมีปัญหา: ข้อความที่ผู้ใช้เห็น	145
น้องกล้าที่อื่นในแอป	147
หน้าหลัก: น้องกล้าตัวเล็กกับประโยคเดียว	147
ตั้งค่า: เสียงน้องกล้า	147
ตู้สกินและการจัดการเงิน	148
หลักการขั้นที่ 1: สิ่งที่ควรรู้นี้ (กฎ ไม่ใช่ AI)	150
ลำดับการคิดของ buildInsights	150
กฎ ① การ์ด Runway	150
กฎ ② หมวดที่สูงกว่าปกติ (findCategoryIncrease)	151
ตัวอย่างคิดมือ: บัญชีมายด์ (ตรวจด้วยฟังก์ชันจริง)	151
กฎ ③ จังหวะการใช้งบ (budget_pace)	152
กฎ ④ ไม่มีข้อมูล และ ⑤ ทุกอย่างปกติ	153
ระดับของการ์ด → ไอคอน และหน้าต่างตอนเล่า	153
หลักการขั้นที่ 2: ถาม AI ยังไงให้ถูกต้องและปลอดภัย	154
เส้นทางของคำถาม 1 ข้อ	154
ข้อมูลที่แอปส่งจริง (ดัดมาจากบัญชีมายด์)	154
สิ่งที่ ไม่ ถูกส่ง	156
System prompt: กติกาที่ AI ต้องทำตาม	156
เลือก AI ตัวไหน และเมื่อ AI ไม่ว่าง	157
Error ทุกแบบ: จากเซิร์ฟเวอร์ถึงข้อความที่ผู้ใช้เห็น	158
ทำความเข้าใจคำตอบ (plainText)	158
เช็กก่อนจ่ายผ่านโค้ช: หาราคาในคำถาม	160
กติกา 4 ชั้น (ลองตามลำดับ เจอแล้วหยุด)	160
ตัวอย่าง 16 ข้อ (รับกับฟังก์ชันจริงแล้ว)	160
เจอราคาแล้ว: คัด Runway ก่อน/หลัง	161
หลักการขั้นที่ 3: ทำให้น้องกล้าพูดได้	162
ภาพรวม: จากข้อความ 1 ก้อน ถึงเสียงทีละท่อน	162
กติกาการตัดซับไต่เตส (talkChunks)	162
ตัวอย่างจริง: คำตอบ "สรุปสัปดาห์นี้" ของมายด์ (283 ตัวอักษร → 6 ท่อน)	163
ทำความเข้าใจเสียงอ่านถูก (speakable)	163
เวลาอ่านเมื่อไม่มีเสียง (talkMs)	163

เลือกเสียงภาษาไทย	164
เมื่อไรน้องกล้าเจียบ	164
เวทีของน้องกล้า: ภาพเคลื่อนไหวที่ลื่นบนมือถือราคาประหยัด	166
วาดด้วยโค้ด แยกเป็นชั้น	166
native driver คืออะไร ทำไมลื่น	166
ท่าทางทั้งหมด และจังหวะ	166
ประหยัดแบตเตอรี่และเคารพผู้ใช้	167
ตุ้สกินและการกิจการเงิน	168
ชุด 3 แบบ	168
การกิจการเงิน 8 การกิจ	168
สกินลิมิเต็ด 10 ชุด	168
ข้อมูลตุ้สกินเก็บที่ไหน	169
อ่านโค้ด: src/domain/insights.ts ทั้งไฟล์	170
ส่วนที่ 1: คอมเมนต์หัวไฟล์และ import	170
ส่วนที่ 2: ชนิดของการ์ด และตัวช่วยรวมรายจ่าย	171
ส่วนที่ 3: findCategoryIncrease หาหมวดที่สูงขึ้น	172
ส่วนที่ 4: buildInsights เริ่มต้น การ์ดไม่มีข้อมูล และการ์ด Runway	173
ส่วนที่ 5: การ์ดหมวดที่สูงขึ้น	174
ส่วนที่ 6: การ์ดงบ และการ์ดส่น้ำเสมอ	175
ส่วนที่ 7: buildCoachContext ข้อมูลเดียวกับที่ AI ได้รับ	176
อ่านโค้ด: src/domain/buddy.ts (น้องกล้าบนหน้าหลัก)	179
ส่วนที่ 1: หัวไฟล์ ชื่อ และชนิดข้อมูล	179
ส่วนที่ 2: ตัวช่วยนับเงินรายวัน และป้าย "ใหม่"	179
ส่วนที่ 3: buddyLine ประโยคประจำบนหน้าหลัก	180
ส่วนที่ 4: แถวน้องกล้า (เคล็ดลับ และจิกจี้)	182
ส่วนที่ 5: แถวต้นไม้เงิน	183
อ่านโค้ด: price.ts และ klaTalk.ts	185
Regular expression (regex) คืออะไร (ปูพื้น 2 นาที)	185
price.ts ทั้งไฟล์	185
klaTalk.ts ส่วนที่ 1: ประโยคประจำของโค้ช	186
klaTalk.ts ส่วนที่ 2: speakable ทำข้อความให้เสียงอ่าน	187
klaTalk.ts ส่วนที่ 3: talkChunks ตัดซับไตเติล	188
klaTalk.ts ส่วนที่ 4: talkMs เวลาอ่าน	190
อ่านโค้ด: ฟังก์ชันเวิร์กของโค้ช (ต่อจากบท A10)	191
ส่วนที่ 1: System prompt ที่ละบรรทัด (พร้อมคำแปล)	191
ส่วนที่ 2: ประกอบข้อความที่ส่งให้ AI	192
ส่วนที่ 3: รายชื่อรุ่น Gemini ที่จะลอง	193
ส่วนที่ 4: callGemini ถาม Gemini พร้อมแผนสำรอง	194
ส่วนที่ 5: อ่านคำตอบและทำความเข้าใจ	196
อ่านโค้ด: หน้าโค้ช src/app/(tabs)/coach.tsx	199
ส่วนที่ 1: คอมเมนต์ import และค่าคงที่	199
ส่วนที่ 2: ข้อมูลและสถานะของหน้า	200
ส่วนที่ 3: การ์ด การพูด และการทักทาย	201
ส่วนที่ 4: ส่งคำถาม แถวน้องกล้า และเปลี่ยนชุด	202
ส่วนที่ 5: อารมณ์ ขนาด และเวที	204

ส่วนที่ 6: แถบเลือกชุด และบรรทัดเมื่อไม่มีเสียงไทย	206
ส่วนที่ 7: การ์ดสิ่งที่ควรรู้ตอนนี้	208
ส่วนที่ 8: ห้องคุยและคำถามสด	209
ส่วนที่ 9: ช่องพิมพ์และปุ่มส่ง	210
ส่วนที่ 10: ฟองคำพูดบนเวที และปุ่มบนเวที	211
อ่านโค้ด: useKlaTalk.ts, tts.ts และ tts.web.ts	214
useKlaTalk ส่วนที่ 1: คำเปิด/ปิดเสียงที่ใช้ร่วมกันทั้งแอป	214
useKlaTalk ส่วนที่ 2: สถานะของการพูด	215
useKlaTalk ส่วนที่ 3: speak พูดทีละท่อน	217
tts.ts: เสียงบนมือถือ	219
tts.web.ts: เสียงบนเว็บ	221
อ่านโค้ด: เวทีและตัวน้องกล้า (KlaStage, KlaBackdrop, KlaPicture)	225
ส่วนที่ 1: หัวโพล์ จังหวะปาก และตัวช่วยองศา	225
ส่วนที่ 2: รับค่าและสร้างตัวแปรภาพเคลื่อนไหว	226
ส่วนที่ 3: ทำว่าง และการกะพริบ	227
ส่วนที่ 4: ตอนพูด และตอนกระโดด	228
ส่วนที่ 5: แปลงค่าเป็นการเลื่อนและหมุนของแต่ละชิ้น	229
ส่วนที่ 6: ซ่อนชั้นภาพ	230
KlaBackdrop: เวที	232
KlaPicture: น้องกล้าภาพนิ่ง	233
อ่านโค้ด: ตัวสกินและการกิจ (skins.ts, missions.ts, kla.ts)	235
skins.ts ส่วนที่ 1: ชนิดข้อมูลของชุด	235
skins.ts ส่วนที่ 2: สถานะของชุดและข้อความ	236
missions.ts ส่วนที่ 1: ชนิดข้อมูลและวันเปิดแอปติดกัน	238
missions.ts ส่วนที่ 2: computeMissions	240
kla.ts: เก็บตัวสกินในเครื่อง และแจกชุด	243
การทดสอบ FR-5	248
รายการเทสต์ของ FR-5	248
อ่านเทสต์ของการ์ดสิ่งที่ควรรู้	248
อ่านเทสต์การพูด (TC-76)	250
อ่านเทสต์การหาราคา (TC-89)	251
E2E: โค้ชในเบราว์เซอร์จริง (ข้อ 13)	251
ทดสอบด้วยมือ: MT-08 และเช็กลิสต์บนมือถือ	254
FR-5 เชื่อมกับ FR อื่นยังไ	255
ทำไมฉันออกแบบแบบนี้	256
ข้อจำกัด และสิ่งที่จะทำต่อ	257
ข้อจำกัดตอนนี้	257
บิกเล็ก 2 จุดที่ฉันเจอระหว่างทำเล่มนี้ (ยังไม่ได้แก้ไขในโปรเจกต์)	257
ไอเดียพัฒนาต่อ	258
คำถามที่อาจารย์อาจถาม พร้อมคำตอบ	259
แบบฝึกหัดเช็กความเข้าใจ	261
สคริปต์นำเสนอ 1 นาที และคำศัพท์	263
คำศัพท์ของ FR-5	263

ภาค ก

พื้นฐานทั้งระบบ MindPay

สอนตั้งแต่ศูนย์ สำหรับคนที่ไม่เคยเขียนโค้ด ภาคนี้เหมือนกับทุกเล่ม เพราะไม่ว่าจะรับผิดชอบ FR ไหน ทุกคนต้องเข้าใจว่าแอปทั้งตัวสร้างด้วยอะไร ทำงานที่ไหน และชิ้นส่วนต่างๆ คอยกันยังไง

- เครื่องมือ ภาษา TypeScript และ React ตั้งแต่พื้นฐาน
- โค้ดชุดเดียวกลายเป็นทั้งเว็บและแอป Android ได้ยังไง
- เซิร์ฟเวอร์ Supabase ฐานข้อมูล และความปลอดภัย
- การเชื่อม AI, การออกแบบ UX/UI, การทดสอบ และ GitHub

MindPay คืออะไร และทีมเราสร้างอะไร

ก่อนดูโค้ดสักบรรทัด ต้องรู้ก่อนว่าแอปนี้แก้ปัญหอะไร ใครใช้ และมีส่วนไหนบ้าง

ปัญหาที่ทีมเราอยากแก้

- นักศึกษาและคนเพิ่งเริ่มทำงาน (First Jobber) ส่วนใหญ่รับและจ่ายเงินผ่านแอปธนาคารบนมือถือ ทุกครั้งที่โอนเงินจะได้ "สลิป" เป็นรูปเก็บไว้ในแกลเลอรี แต่ปัญหาคือ
- **ขี้เกียจจด** แอปจดรายรับรายจ่ายทั่วไปต้องพิมพ์เองทุกรายการ ทั้งที่ข้อมูลอยู่ในสลิปแล้ว คนจึงเลื่อนการจดไปเรื่อย ๆ จนข้อมูลไม่ครบ
 - **เห็นแค่ยอดเงิน แต่ไม่รู้ว่าพอหรือเปล่า** แอปธนาคารบอกว่าเหลือ ฿3,500 แต่ไม่ได้บอกว่าเงินก้อนนี้จะอยู่ได้ถึงวันไหน
 - **รู้ตัวตอนเงินหมดแล้ว** ไม่มีใครช่วยเตือนหรือช่วยคิดก่อนจ่าย

ทีมเราจึงสร้าง **MindPay** แอปการเงินส่วนตัวที่ "ไม่ได้แค่จดว่าใช้ไปเท่าไร แต่บอกว่าเงินที่มีจะพอใช้อีกกี่วัน และช่วยคิดก่อนจ่าย" (ประโยคแรกของไฟล์ `README.md` ในโปรเจกต์)

6 ฟังก์ชันหลัก (Functional Requirements)

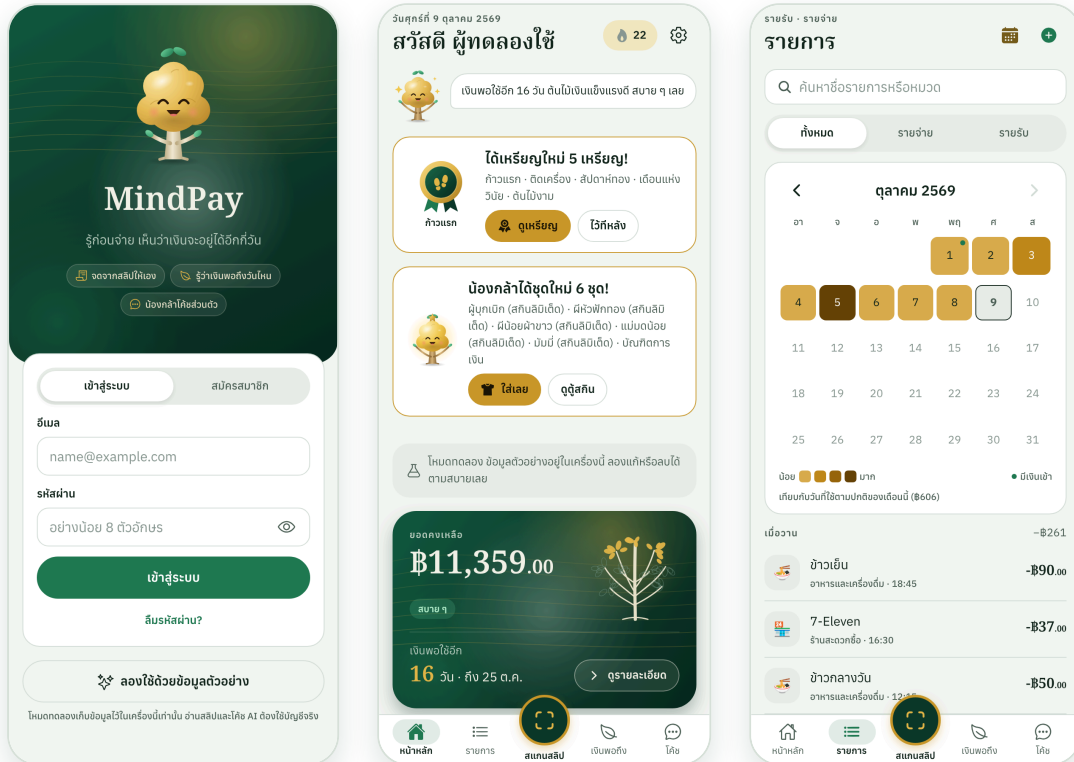
ในวิชา Software Engineering เราเรียกสิ่งที่ระบบ "ต้องทำได้" ว่า **Functional Requirement** หรือ **FR** แต่ละ FR มีเจ้าของ 1 คนในกลุ่ม MindPay มีทั้งหมด 6 ข้อ

FR	ชื่อ	ทำอะไร (ภาษาคน)	อยู่ตรงไหนในแอป
FR-1	Transaction Management	จดรายการเอง แก้ไข เลือกหมวด และลบรายการได้	แท็บ "รายการ" และหน้า "จดรายการ"
FR-2	Overview Dashboard	สรุปรายรับ รายจ่าย ยอดคงเหลือ และยอดแยกตามหมวด	แท็บ "หน้าหลัก"
FR-3	Voice Entry	พูดว่า "ข้าว 50 บาท" แล้วแอปจดให้	ปุ่มไมค์ทองบนหน้าหลัก
FR-4	Automatic Slip Detection	หาสลิปในแกลเลอรีเองแล้วให้ AI อ่านยอดเงิน วันที่ และชื่อผู้รับ	ปุ่มทวงกลางแถบล่าง "สแกนสลิป" และสแกนอัตโนมัติตอนเปิดแอป
FR-5	AI Coach (น้องกล้า)	โค้ช AI ที่อธิบายตัวเลขการเงินเป็นภาษาคน และพูดออกเสียงได้	แท็บ "โค้ช"
FR-6	Money Runway	คำนวณว่าเงินที่มีจะพอใช้อีกกี่วัน และหมดวันไหน	แท็บ "เงินพอถึง" และการ์ดเขี้ยวบนหน้าหลัก

★ จำให้ได้: ทุก FR ต่อกันเป็นสายเดี่ยว

FR-1, FR-3 และ FR-4 คือ **ทางเข้าของข้อมูล** (จุดเอง พุดจกด สแกนสลิป) ส่วน FR-2 และ FR-6 คือ **สมองที่คิดตัวเลข** (ยอดคงเหลือ และเงินพอถึงวันไหน) และ FR-5 คือ **ปากที่อธิบาย** ตัวเลขเหล่านั้นให้ผู้ฟัง ถ้า FR ไหนฟัง FR ที่อยู่ปลายน้ำจะได้ตัวเลขผิดตามไปด้วย

หน้าตาแอปโดยรวม



หน้าต้อนรับ: สมัคร เข้าสู่ระบบ หรือ "ลองใช้ด้วยข้อมูลตัวอย่าง"

หน้าหลัก (FR-2): ยอดคงเหลือ, การ์ดเงินพอถึง (FR-6), บอingkล้า พุดสรุป

แท็บรายการ (FR-1): รายการ ทั้งหมด ค้นหา กรองรายรับ/รายจ่าย



ด้านล่างของทุกหน้ามี **แถบเมนู 5 ช่อง**: หน้าหลัก, รายการ, ปุ่มทวง "สแกนสลิป" ตรงกลาง, เงินพอถึง และไอซ์ ปุ่มสแกนอยู่ตรงกลางและใหญ่ที่สุด เพราะเป็นสิ่งที่ผู้ใช้ทำบ่อยที่สุด (โค้ดอยู่ใน `src/app/(tabs)/_layout.tsx` ซึ่งจะอ่านทีละบรรทัดใน un A7)

ใช้แอปได้ 3 แบบ

แบบ	เปิดยังไ้	ต่างกันยังไ้
เว็บ	https://2550expo-tech.github.io/Socrates-and-Skeletons/ (เปิดในเบราว์เซอร์ได้ทั้งคอม iPhone และ Android)	ทำได้ทุกอย่าง ยกเว้นสแกนแคเลเลอร์ทั้งหมดเอง (บนเว็บต้องกดเลือกรูปสลิป)
แอป Android (.apk)	ดาวน์โหลดจากหน้า Releases ของ GitHub ชื่อ latest -apk	สแกนแคเลเลอร์อัตโนมัติ พุดจดด้วยไมค์น้องกล้าพุดออกเสียง
Expo Go	แอปตัวช่วยตอนพัฒนา สแกน QR จากคอมของนักพัฒนา	ใช้ทดสอบระหว่างเขียนโค้ดเท่านั้น

และไม่ว่าเปิดแบบไหน ผู้ใช้เลือกได้ 2 โหมด

- **บัญชีจริง** สมัครด้วยอีเมล ข้อมูลเก็บบนเซิร์ฟเวอร์ (Supabase) เปิดจากเครื่องไหนก็เห็นข้อมูลเดียวกัน ใช้ AI อ่านสลิปและไอซ์ได้
- **โหมดทดลอง** แตะ "ลองใช้ด้วยข้อมูลตัวอย่าง" แอปสร้างรายการตัวอย่างประมาณ 75 วันให้ เก็บไว้ในเครื่องอย่างเดียวไม่ต้องสมัคร ใช้ตอนฟรีเซนต์หรือลองเล่น (AI ยังใช้ไม่ได้ในโหมดนี้)

ตัวเลขของโปรเจกต์

~18,700

บรรทัดโค้ดของแอปและเซิร์ฟเวอร์ (ไม่นับ
เทสต์)

157

unit test ผ่านทั้งหมด

145

ข้อตรวจในเบราวเซอร์ (E2E) ผ่านทั้งหมด

โฟลเดอร์	มีอะไร	ไฟล์	บรรทัด
src/app	หน้าจอทุกหน้า	19	5,266
src/ui	ชิ้นส่วนหน้าตา: ปุ่ม การ์ด สี กราฟ ภาพน้องกล้า	31	6,042
src/domain	กฎและการคำนวณทั้งหมด (สมองของแอป)	24	3,457
src/data	การเก็บและโหลดข้อมูล	9	1,019
src/services	งานที่ต้องคุยกับโลกภายนอก: แกลเลอรี ไมค์ เสียง AI	15	1,776
supabase	ฐานข้อมูลและโค้ดฝั่งเซิร์ฟเวอร์	8	1,160
e2e, scripts	เทสต์ในเบราวเซอร์ และสคริปต์ช่วย (ไอคอน ภาพหน้าจอ)	10	1,867

นับจากโค้ดเวอร์ชัน 3 ต.ค. 2569 ไฟล์ .ts, .tsx, .sql และ .mjs ไม่รวมไฟล์เทสต์ (ไฟล์เทสต์อีกประมาณ 2,000 บรรทัด)

≈ เปรียบแอปเป็นร้านอาหาร

หน้าจอ (src/app, src/ui) คือหน้าร้านที่ลูกค้าเห็น **กฎการคำนวณ** (src/domain) คือสูตรอาหารของเชฟ **ข้อมูล** (src/data) คือตู้เย็นเก็บวัตถุดิบ **เซิร์ฟเวอร์** (supabase) คือโถงกลางที่ทุกสาขาใช้ร่วมกัน และ **AI** คือผู้เชี่ยวชาญภายนอกที่เราโทรถามเมื่อต้องอ่านสลิปหรืออธิบายตัวเลข บทต่อไปจะวาดแผนที่ว่าทั้งหมดนี้ต่อกันยังไง

คำศัพท์ที่จะเจอบ่อยในเล่มนี้

คำ	ความหมายแบบง่าย
รายการ (Transaction)	เงินเข้าหรือเงินออก 1 ครั้ง เช่น ข้าวมันไก่ ฿50
แบบร่าง (Draft)	รายการจากสลิปที่ AI อ่านแล้วแต่ยังไม่แน่ใจ รอผู้ใช้ตรวจ ยังไม่นับในยอดเงิน
ยืนยันแล้ว (Confirmed)	รายการที่นับในยอดเงินแล้ว
สตางค์ (satang)	หน่วยที่แอปใช้เก็บเงินในระบบ 100 สตางค์ = 1 บาท (เหตุผลอยู่บท A5)
เส้นเงินสำรอง (Floor)	เงินขั้นต่ำที่ผู้ใช้ไม่อยากให้ต่ำกว่า ค่าเริ่มต้น ฿500
น้องกล้า	มาสคอตของแอป เป็นต้นกล้าจากต้นไม้เงิน และเป็นโค้ช AI ใน FR-5
Supabase	บริการเซิร์ฟเวอร์ที่ให้ทั้งฐานข้อมูล ระบบบัญชีผู้ใช้ และที่รันโค้ดฝั่งเซิร์ฟเวอร์
Expo	เครื่องมือที่ทำให้โค้ดชุดเดียวเป็นได้ทั้งแอป Android, iPhone และเว็บ

แผนที่ระบบ: ชิ้นส่วนทั้งหมดต่อกันยังงี้

ภาพใหญ่ของ MindPay ตั้งแต่นิ้วแตะจอ จนข้อมูลไปถึงฐานข้อมูลบนคลาวด์และกลับมาเป็นตัวเลขบนหน้าจอ

ระบบมี 4 ส่วนใหญ่

① แอปบนมือถือหรือเว็บ (โค้ดในโฟลเดอร์ src/)

สร้างด้วย Expo + React Native ภาษา TypeScript โค้ดชุดเดียวได้ทั้งแอป Android, iPhone และเว็บ ทำหน้าที่แสดงหน้าจอ รับการแตะ คำนวณตัวเลขทั้งหมด (ยอดคงเหลือ, Runway) อ่าน QR บนสลิป และแปลงเสียงพูดเป็นรายการ

↕ อินเทอร์เน็ต (HTTPS) ด้วย "publishable key" ซึ่งเป็นกุญแจสาธารณะ

② Supabase: ฐานข้อมูล + ระบบบัญชีผู้ใช้ (โค้ดใน supabase/migrations/)

ฐานข้อมูล PostgreSQL เก็บตาราง profiles, transactions, savings_goals, ai_usage ทุกตารางเปิด Row Level Security (RLS) ทำให้แต่ละคนอ่านและแก้ไขได้เฉพาะแถวของตัวเอง ระบบ Auth จัดการสมัคร เข้าสู่ระบบ ยืนยันอีเมล

↕ เฉพาะงานที่ต้องใช้ AI

③ Supabase Edge Functions: โค้ดฝั่งเซิร์ฟเวอร์ (โค้ดใน supabase/functions/)

ฟังก์ชัน 2 ตัว: **parse-slip** (อ่านสลิป, FR-4) และ **coach** (โค้ชน้องกล้า, FR-5) ตรวจสอบว่าผู้ใช้เข้าสู่ระบบจริง นับโควตาต่อวัน แล้วคือ "กุญแจลับ" ของ AI ไว้เรียกแทนแอป

↕ API key ลับ (อยู่บนเซิร์ฟเวอร์เท่านั้น)

④ AI ภายนอก: Claude (Anthropic) หรือ Gemini (Google)

อ่านรูปสลิปแล้วตอบเป็น JSON และเขียนคำอธิบายของโค้ช ระบบเลือกใช้ตัวที่มีกุญแจ ถ้ามีทั้งสองตัวจะใช้ Claude ก่อน

★ **จำให้ได้: 4 กฎของสถาปัตยกรรม (จาก README)**

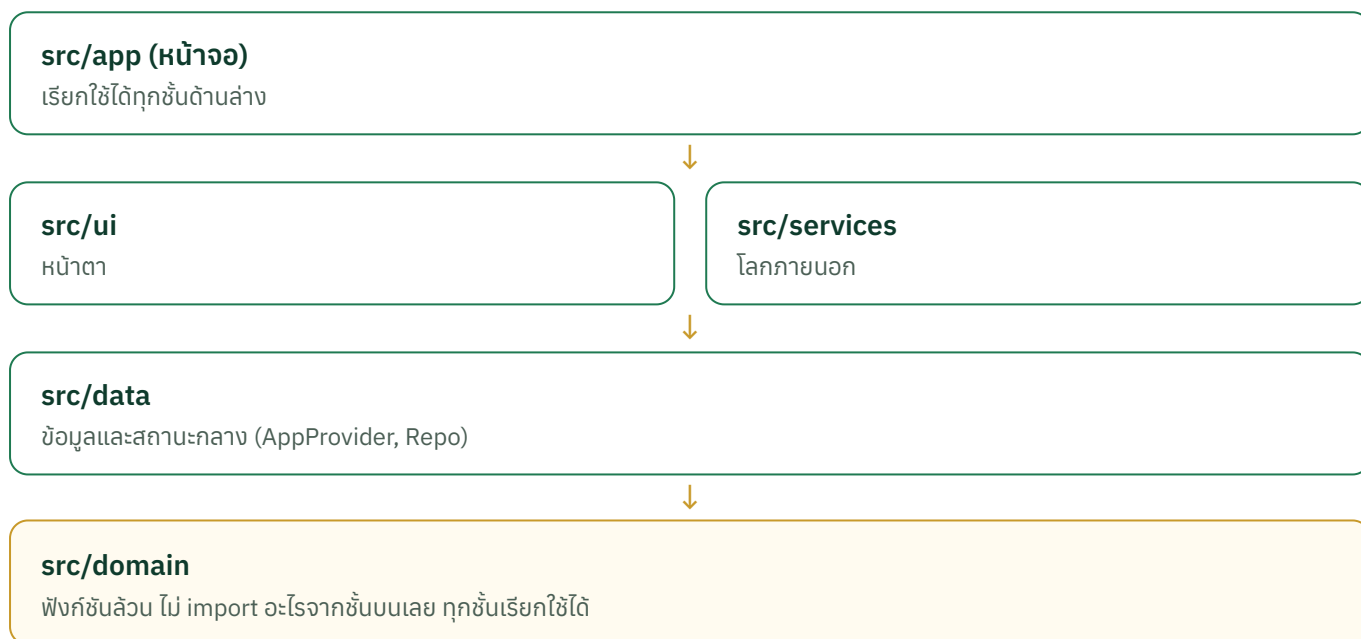
- เงินเก็บเป็นสตางค์ (จำนวนเต็ม) ไม่มีปัญหาทศนิยมเพี้ยน
- เวลาอิงเวลาไทย (UTC+7) ทุกจุด ทั้งการจัดกลุ่มรายวันและการอ่านวันที่บนสลิป
- สลิปที่ AI อ่านได้ไม่ชัดเป็น "แบบร่าง" จนกว่าผู้ใช้จะยืนยัน ไม่เข้ายอดเงินก่อน
- กุญแจ AI อยู่บนเซิร์ฟเวอร์เท่านั้น ในแอปไม่มีกุญแจลับเลยสักตัว

ข้างในแอป: แบ่งโค้ดเป็น 5 ชั้น

ถ้าเอาโค้ดทุกอย่างไปกองในไฟล์เดียว จะแก้ยากและทดสอบไม่ได้ ทีมเราจึงแยกตามหน้าที่ แต่ละโฟลเดอร์ใน src/ มีหน้าที่ชัดเจน และมีกฎว่าใครเรียกใครได้

ชั้น	โฟลเดอร์	หน้าที่	ตัวอย่างไฟล์
หน้าจอ	src/app/	แต่ละไฟล์คือ 1 หน้า (Expo Router) วางชิ้นส่วน รับการแตะ แล้วเรียกชั้นอื่นทำงาน	<code>src/app/transaction.tsx</code> หน้าจอดรายการ
ชิ้นส่วน หน้าตา	src/ui/	ปุ่ม การ์ด ช่องกรอก สี ตัวอักษร กราฟ ภาพวาด ใช้ซ้ำได้ทุกหน้า	<code>src/ui/components.tsx</code> , <code>src/ui/theme.ts</code>

ชั้น	โฟลเดอร์	หน้าที่	ตัวอย่างไฟล์
กฎและการคำนวณ	src/domain/	"สมอง" ของแอป เป็นฟังก์ชันล้วน รับตัวเลขเข้า ส่งตัวเลขออก ไม่ยุ่งกับหน้าจอหรืออินเทอร์เน็ท จึงทดสอบได้ง่าย มี unit test ทุกไฟล์	<code>src/domain/runway.ts</code> , <code>src/domain/money.ts</code>
ข้อมูล	src/data/	โหลดและบันทึกข้อมูล ทั้งบนคลาวด์ (Supabase) และในเครื่อง (โหมดทดลอง) และเก็บ "สถานะกลาง" ที่ทุกหน้าใช้ร่วมกัน	<code>src/data/repo.ts</code> , <code>src/data/AppProvider.tsx</code>
บริการภายนอก	src/services/	งานที่คุยกับโลกนอกแอป: แกลเลอรี่รูป ไมโครโฟน เสียงอ่าน และเรียก Edge Functions	<code>src/services/slips.ts</code> , <code>src/services/coach.ts</code>



≈ ทำไมแยก domain ออกมา

เหมือนแยก "สูตรคำนวณภาษี" ออกจาก "แบบฟอร์มกรอกภาษี" ถ้าสูตรเปลี่ยน แค่นี้เดียว แบบฟอร์มทุกฉบับได้ผลใหม่ทันที และเราตรวจสอบสูตรได้ด้วยการใส่ตัวเลขทดลองโดยไม่ต้องกรอกแบบฟอร์มจริง โค้ดใน `src/domain` ก็เช่นกัน มีเทสต์ 157 ข้อที่รันจบในไม่ถึง 2 วินาทีโดยไม่ต้องเปิดแอป

ตามข้อมูล 1 รายการ: ตั้งแต่กดบันทึก จนตัวเลขบนหน้าหลักเปลี่ยน

วิธีที่ดีที่สุดในการเข้าใจระบบ คือเลือกการกระทำหนึ่งอย่าง แล้วตามดูว่ามันวิ่งผ่านไฟล์ไหนบ้าง สมมติผู้ใช้กด "ข้ามมันไ้฿85" แล้วกด "บันทึกรายการ"

1. หน้าจอดรายการ [src/app/transaction.tsx](#)

ฟังก์ชัน `save()` แปลง "85" เป็น 8,500 สตางค์ ด้วย `parseBahtToSatang` แล้วเรียก `addTx(...)`



2. สถานะกลาง [src/data/AppProvider.tsx](#)

`addTx` ส่งต่อให้ Repo บันทึก รวบรวมกลับมา แล้วเพิ่มรายการใหม่เข้าไปในรายการทั้งหมด (`setTxs`)



3. Repo [src/data/repo.ts](#)

บัญชีจริง: แปลงชื่อช่องเป็นแบบฐานข้อมูล (`amountSatang` → `amount_satang`) แล้วส่งคำสั่ง `insert` ไปที่ Supabase · โหมดทดลอง: เก็บลงเครื่องแทน



4. ฐานข้อมูล Supabase [supabase/migrations/20260927000000_init.sql](#)

ตรวจกฎ: จำนวนเงินต้อง > 0, ผู้บันทึกต้องเป็นเจ้าของแถว (RLS) แล้วเพิ่มแถวใหม่ในตาราง `transactions` ส่งแถวที่บันทึกแล้วกลับไป



5. คำนวณใหม่อัตโนมัติ `useMoney()` ใน [src/data/AppProvider.tsx](#)

เพราะรายการเปลี่ยน React จึงคำนวณยอดคงเหลือ (FR-2) และ Money Runway (FR-6) ใหม่ด้วยฟังก์ชันใน `src/domain` แล้ววาดหน้าหลักใหม่ ตัวเลขลดลง ฿85 กันที

ดูโค้ดจริงของแต่ละชั้น (ยังไม่ต้องเข้าใจทุกคำ บท A4–A9 จะสอนทีละอย่าง ตอนนีดูแค่ว่ามันต่อกันยังไง)

ชั้น 1: หน้าจอดรายการเรียก `addTx`

`src/app/transaction.tsx`

บรรทัด 87–124

```
87  async function save(confirm: boolean) {
88    if (busy) return;
89    const satang = parseBahtToSatang(amount);
90    if (satang === null) {
91      setAmountError('ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50');
92      return;
93    }
94    setAmountError(null);
95    if (dayError || timeError) return;
96    const input = {
97      kind,
98      amountSatang: satang,
99      categoryKey: category,
100     title: title.trim(),
101     note: note.trim() || null,
102     occurredAt: bkkToIso(day, time),
103   };
104   setBusy(true);
105   try {
106     if (existing) {
107       await updateTx(existing.id, {
108         ...input,
109         ...(confirm ? { status: 'confirmed' as const, reviewFlags: [] } : {}),
110       });
111       toast({ message: confirm && isDraft ? 'ยืนยันรายการแล้ว รวมในยอดเงินแล้ว' : 'บันทึกการแก้ไขแล้ว' });
112     } else {
```

```

113     await addTx({
114         ...input,
115         source: 'manual',
116         status: 'confirmed',
117         slipRef: null,
118         slipImageHash: null,
119         ocrConfidence: null,
120         reviewFlags: [],
121     });
122     toast({ message: kind === 'income' ? 'เพิ่มเงินเข้าแล้ว' : 'บันทึกรายการแล้ว' });
123 }
124 leaveWhenDone();

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:87

บรรทัด	ความหมาย
87	ประกาศฟังก์ชัน save ซึ่งทำงานเมื่อกดปุ่มบันทึก คำว่า async แปลว่าข้างในมีงานที่ต้องรอ (เช่น รออินเทอร์เน็ต)
88	ถ้ากำลังบันทึกอยู่แล้ว (busy) ให้ออกเลย กันคนกดปุ่มรัว ๆ แล้วได้รายการซ้ำ
89–93	แปลงข้อความในช่องเงินเป็นสตริง ถ้าแปลงไม่ได้ (เช่น พิมพ์ "abc" หรือ 0) จะได้ null แล้วขึ้นข้อความเตือนใต้ช่อง และหยุดไม่บันทึก
95	ถ้าวันที่หรือเวลาที่พิมพ์ (บนเว็บ) ผิดรูปแบบ ก็ไม่บันทึก
96–103	รวมค่าจากทุกช่องในฟอร์มเป็นก้อนเดียวชื่อ input: ประเภท จำนวนเงิน หมวด ชื่อ บันทึก และเวลา (แปลงวันและเวลาไทยเป็นรูปแบบสากลด้วย bkkToIso)
104	ตั้ง busy เป็นจริง ปุ่มจะหมุนรอและกดซ้ำไม่ได้
106–111	ถ้าเป็นการแก้ไขรายการเดิม เรียก updateTx แทน
113–121	ถ้าเป็นรายการใหม่ เรียก addTx พร้อมบอกว่ามาจากการกดเอง (manual) และยืนยันแล้ว (confirmed) จึงนับในยอดเงินทันที
122	ขึ้นข้อความสั้น ๆ ด้านล่าง (toast) ว่า "บันทึกรายการแล้ว"
124	ปิดหน้าฟอร์มกลับไปหน้าก่อน

ขั้น 2: AppProvider ส่งต่อให้ Repo แล้วอัปเดตรายการ

src/data/AppProvider.tsx

บรรทัด 385–392

```

385     async addTx(input) {
386         const r = need();
387         const tx = await r.insert(input);
388         if (repoRef.current !== r) return tx;
389         noteLocal(tx.id, tx);
390         setTxs((list) => [tx, ...list]);
391         return tx;
392     },

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:385

บรรทัด	ความหมาย
386	หยาบ Repo ตัวที่ใช้อยู่ (บัญชีจริงหรือโหมดทดลอง) ถ้ายังไม่ได้เข้าสู่ระบบ need () จะโยน error
387	สั่ง Repo บันทึก แล้วรอผล (await) ได้รายการที่มี id จริงกลับมา
388	ถ้าระหว่างรอ ผู้ใช้ลบบัญชีไปแล้ว ไม่ต้องเอารายการนี้ไปใส่ในบัญชีใหม่
389	จดไว้ว่ามีการเปลี่ยนแปลงในเครื่อง (กันรายการหายถ้ากำลังโหลดข้อมูลชุดใหม่พร้อมกัน)
390	เอารายการใหม่ไปไว้หน้าสุดของรายการทั้งหมด การเรียก setTxS ทำให้ React วาดทุกหน้าที่ใช้รายการนี้ใหม่

ขั้น 3: Repo ส่งคำสั่งไป Supabase

src/data/repo.ts	บรรทัด 217–222
<pre> 217 async insert(input) { 218 const { data, error } = await db.from('transactions').insert(toRow(input)).select().single(); 219 if (isUniqueViolation(error)) throw new DuplicateSlipError(); 220 if (error) throw error; 221 return fromRow(data as TxRow); 222 }, </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:217	

บรรทัด	ความหมาย
218	อ่านเหมือนประโยคภาษาอังกฤษ: "จากตาราง transactions, เพิ่ม (insert) แถวนี้, แล้วเลือก (select) แถวที่เพิ่ง เพิ่มกลับมา 1 แถว (single)" toRow แปลงชื่อช่องให้ตรงกับฐานข้อมูล
219	ถ้าฐานข้อมูลตอบว่า "ซ้ำ" (รหัส 23505) แปลว่าสลิปนี้เคยบันทึกแล้ว โยน error แบบพิเศษให้หน้าจอบอกผู้ใช้ว่า "มีรายการจากสลิปนี้อยู่แล้ว"
220	error อื่น ๆ (เช่น อินเทอร์เน็ตหลุด) โยนต่อให้หน้าจอขึ้น "บันทึกไม่สำเร็จ ลองอีกครั้ง"
221	แปลงแถวจากฐานข้อมูลกลับเป็นรูปแบบที่แอปใช้ (amount_satang → amountSatang) แล้วส่งคืน

ขั้น 5: ตัวเลขคำนวณใหม่เอง

src/data/AppProvider.tsx	บรรทัด 442–459
<pre> 442 <i>/** Derived money numbers used across screens (FR-2 balance, FR-6 runway). */</i> 443 export function useMoney() { 444 const { profile, txs, carrySatang, goals, today } = useApp(); 445 return useMemo(() => { 446 const opening = profile?.openingBalanceSatang ?? 0; 447 const floor = profile?.runwayFloorSatang ?? 50_000; 448 <i>// Money from before the loaded window (over ~13 months ago) still counts.</i> 449 const balance = computeBalance(opening + carrySatang, txs); 450 const average = averageDailyExpense(txs); 451 <i>// Money in savings goals is set aside: it does not count as money to spend.</i> 452 const reserved = reservedSatang(goals); 453 const runway = computeRunway({ balanceSatang: balance, floorSatang: floor + reserved, averageSatang: average.averageSatang }); 454 const drafts = txs.filter((t) => t.status === 'draft'); 455 return { balance, average, runway, drafts, reserved, floor }; </pre>	

```

456 // `today` makes the runway and averages start over at midnight.
457 // eslint-disable-next-line react-hooks/exhaustive-deps
458 }, [profile, txs, carrySatang, goals, today]);
459 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:442`

บรรทัด	ความหมาย
443	useMoney คือ "hook" ที่ทุกหน้าเรียกใช้เมื่ออยากได้ตัวเลขเงิน (บท A6 อธิบายว่า hook คืออะไร)
444	ดึงข้อมูลดิบจากสถานะกลาง: โปรไฟล์ รายการ ยอดยกมา กระปุกลูก และวันนี้
445	useMemo แปลว่า "คำนวณแล้วจำไว้" คำนวณใหม่เฉพาะเมื่อข้อมูลในบรรทัด 458 เปลี่ยน
446–447	ยอดเงินตอนเริ่มใช้แอป และเส้นเงินสำรอง (ถ้ายังไม่ตั้ง ใช้ 50,000 สตางค์ = ฿500)
449	คำนวณยอดคงเหลือ (FR-2) จากรายการทั้งหมด
450–453	คำนวณค่าเฉลี่ยรายจ่าย เงินในกระปุก และ Money Runway (FR-6)
454	หารายการแบบร่างที่รอตรวจ (FR-4)
455	ส่งตัวเลขทั้งหมดออกไปให้หน้าจอใช้
458	รายการ "สิ่งที่ต้องจับตา" ถ้า txs เปลี่ยน (เพิ่มเพิ่มข้าวมันไก่) จะคำนวณใหม่ทั้งหมด และ today ทำให้คำนวณใหม่ตอนเที่ยงคืน

★ จำให้ได้: ตัวเลขบนหน้าจอไม่ได้เก็บไว้ แต่คิดใหม่ทุกครั้ง

ยอดคงเหลือ ค่าเฉลี่ย และ Runway **ไม่ได้เก็บในฐานข้อมูล** ฐานข้อมูลเก็บแค่ "รายการ" แล้วแอปคำนวณตัวเลขทั้งหมดสด ๆ จากรายการทุกครั้งที่มีอะไรเปลี่ยน ข้อดีคือตัวเลขไม่มีทางขัดกันเอง (ไม่มีกรณียอดในฐานข้อมูลบอกรายการหนึ่ง แต่รายการบอกอีกอย่าง) คอมเมนต์บนสุดของ `src/data/AppProvider.tsx` เขียนไว้ว่า "Numbers shown on screen are always derived from this state ... never stored twice"

ตามข้อมูลที่ใช้ AI: สแกนสลิป 1 ใบ

1. มือถือหารูปใหม่ในแคลเลอร์ [src/services/AutoScanProvider.tsx](#)

อ่าน QR บนรูปในเครื่องก่อน (ฟรี ไม่ใช้อินเทอร์เน็ต) รูปไหนไม่มี QR ของสลิปธนาคารจะไม่ถูกส่งไปไหนเลย

↓ เฉพาะรูปที่เป็นสลิป

2. ส่งรูปไป Edge Function [src/services/slips.ts](#) → [supabase/functions/parse-slip/index.ts](#)

เซิร์ฟเวอร์ตรวจว่าเข้าสู่ระบบจริง ตรวจโควตา (300 ใบต่อวัน) แล้วส่งรูปให้ AI อ่าน 2 ครั้งพร้อมกันด้วย 2 รุ่น

↓

3. AI ตอบเป็น JSON

ยอดเงิน วันที่ ชื่อผู้รับ ธนาคาร และ "ความมั่นใจ" ของแต่ละช่อง เซิร์ฟเวอร์รวมผลสองครั้ง ถ้าไม่ตรงกันจะให้ผู้ตรวจ

↓

4. แอปบันทึกรายการ

AI อ่านตรงกันทั้ง 2 ครั้งและมั่นใจทุกช่อง $\geq 80\%$ → เข้ายอดเงินเลย (ผู้ใช้ปิดตัวเลือกนี้ได้ในหน้าตั้งค่า) · ไม่มั่นใจหรืออ่านไม่ตรงกัน → เป็นแบบร่าง รอผู้ตรวจ แล้วตัวเลขทั้งแอปคำนวณใหม่เหมือนขึ้น 5 ด้านบน

รายละเอียดของ AI อยู่ในบท A10 และรายละเอียดเพิ่มเติมของการสแกนสลิปอยู่ในเล่ม FR-4

เครื่องมือ: เขียนโค้ดที่ไหน ใช้โปรแกรมอะไร และ VS Code ตั้งแต่ศูนย์

ทุกอย่างที่ต้องมีในเครื่อง วิธีเอาโค้ด MindPay มาเปิด รันแอป และใช้ VS Code อ่านโค้ดอย่างมืออาชีพ

กล่องเครื่องมือของทีมเรา

เครื่องมือ	คืออะไร	ใช้ทำอะไรใน MindPay
VS Code	โปรแกรมเขียนโค้ด (code editor) ฟรีจาก Microsoft	เปิด อ่าน แก้โค้ดทุกไฟล์ และรับคำสั่งผ่าน terminal ในตัว
Node.js (เวอร์ชัน 22 ขึ้นไป)	ตัวรันภาษา JavaScript นอกเบราว์เซอร์	รันเครื่องมือทั้งหมด: Expo, เทสต์, สคริปต์ build
npm	ตัวจัดการ "แพ็คเกจ" (โค้ดสำเร็จรูปที่คนอื่นเขียนไว้) มากับ Node.js	ติดตั้งไลบรารี 38 ตัวที่แอปใช้ (npm install) และรับคำสั่งใน package.json
Git	ระบบเก็บประวัติการแก้ไขโค้ด ย้อนเวอร์ชันได้	บันทึกการเปลี่ยนแปลงทุกครั้งเป็น "commit"
GitHub	เว็บที่เก็บโค้ดของ Git บนคลาวด์ ทำงานเป็นทีมได้	เก็บโค้ดที่ github.com/2550expo-tech/Socrates-and-Skeletons - และ build แอปให้อัตโนมัติ (un A13)
Expo	ชุดเครื่องมือทำแอปมือถือด้วย React Native	รันแอประหว่างพัฒนา (npx expo start), ทำเว็บ, build APK, ส่งอัปเดต
Expo Go	แอปบนมือถือสำหรับลองแอปที่กำลังเขียน	สแกน QR จากคอมแล้วเห็นแอปบนมือถือทันที
Supabase Dashboard	หน้าเว็บจัดการเซิร์ฟเวอร์ที่ supabase.com	ดูตาราง รัน SQL ใส่queryแฉ่ AI ดู log ของ Edge Functions
เบราว์เซอร์ Chrome	-	เปิดเว็บเวอร์ชัน และใช้ DevTools (F12) ดู error

ติดตั้งเครื่องมือ (ทำครั้งเดียว)

- VS Code:** ดาวน์โหลดจาก code.visualstudio.com ติดตั้งแบบค่าเริ่มต้น (Windows คลิก "Add to PATH" และ "Open with Code" ไปด้วย)
- Node.js:** ดาวน์โหลดรุ่น LTS (22 ขึ้นไป) จาก nodejs.org แล้วเปิด terminal พิมพ์ node -v ต้องขึ้น v22 หรือสูงกว่า
- Git:** ดาวน์โหลดจาก git-scm.com (Mac มีมาให้แล้ว) ตรวจสอบด้วย git --version
- บัญชี GitHub:** สมัครที่ github.com และขอให้เจ้าของ repo เพิ่มเป็นสมาชิกถ้าต้องแก้ไข (อ่านอย่างเดียวไม่ต้องขอ เพราะ repo เป็นสาธารณะ)
- Expo Go** บนมือถือ (ไม่บังคับ): ดาวน์โหลดจาก Play Store หรือ App Store

เอาโค้ด MindPay มาไว้ในเครื่อง

เปิด terminal (Windows: PowerShell, Mac: Terminal หรือใช้ terminal ใน VS Code) แล้วพิมพ์ทีละบรรทัด

Terminalคำสั่งที่พิมพ์จริง

```
1 git clone https://github.com/2550expo-tech/Socrates-and-Skeletons-.git
2 cd Socrates-and-Skeletons-
3 npm install
4 cp .env.example .env
5 code .
```

บรรทัด	ความหมาย
1	clone = คัดลอกโค้ดทั้งหมดพร้อมประวัติจาก GitHub มาไว้ในโฟลเดอร์ใหม่ชื่อ Socrates-and-Skeletons-
2	cd (change directory) = เข้าไปในโฟลเดอร์นั้น
3	อ่านรายชื่อแพ็คเกจใน package.json แล้วดาวน์โหลดทั้งหมดลงโฟลเดอร์ node_modules (ประมาณ 1-3 นาที ใช้พื้นที่หลายร้อย MB)
4	คัดลอกไฟล์ตั้งค่าตัวอย่างเป็น .env ซึ่งบอกแอปว่าเซิร์ฟเวอร์ Supabase อยู่ที่ไหน (Windows PowerShell ใช้ copy .env.example .env)
5	เปิด VS Code ที่โฟลเดอร์นี้ (จุด . แปลว่า "โฟลเดอร์ปัจจุบัน")

ไฟล์ .env มีอะไร

.env.exampleบรรทัด 1-6

```
1 # Copy this file to .env and fill in the values from
2 # Supabase Dashboard -> Project Settings -> API (or the green "Connect" button).
3 # Only the PUBLISHABLE (anon) key goes here. Never put the secret/service_role key
4 # or any AI key (Gemini / Anthropic) in the app: those live in Supabase Edge Function secrets.
5 EXPO_PUBLIC_SUPABASE_URL=https://dufvcdswcaqsfbxjwat.supabase.co
6 EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_LKQP3TU8Fr3t8JD6bzg7TA_5Uy6zcm
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → .env.example:1

บรรทัด	ความหมาย
1-4	คอมเมนต์ (บรรทัดที่ขึ้นต้นด้วย # คอมพิวเตอร์ไม่อ่าน) บอกว่าให้ใส่เฉพาะกุญแจ "publishable" ห้ามใส่กุญแจลับหรือกุญแจ AI ในแอปเด็ดขาด
5	ที่อยู่ของเซิร์ฟเวอร์ Supabase ของทีม
6	กุญแจสาธารณะ (publishable key) ใช้บอกว่า "แอปนี้เป็นของโปรเจกต์ไหน" ปลอดภัยที่จะอยู่ในแอป เพราะฐานข้อมูลใช้ RLS กันไว้ (บท A9) ชื่อที่ขึ้นต้นด้วย EXPO_PUBLIC_ คือค่าที่ Expo ยอมใส่เข้าไปในแอป

ไฟล์ package.json: บัตรประชาชนของโปรเจกต์

ไฟล์นี้บอกว่าโปรเจกต์ชื่ออะไร ใช้แพ็คเกจอะไร และมีคำสั่งลัดอะไรบ้าง

package.jsonบรรทัด 1-59

```

1 {
2   "name": "mindpay",
3   "version": "1.0.0",
4   "main": "expo-router/entry",
5   "dependencies": {
6     "@expo-google-fonts/anuphan": "^0.4.1",
7     "@expo-google-fonts/noto-serif-thai": "^0.4.2",
8     "@expo/vector-icons": "^15.0.2",
9     "@react-native-community/datetimepicker": "9.1.0",
10    "@supabase/supabase-js": "^2.117.2",
11    "expo": "~57.0.25",
12    "expo-camera": "~57.0.5",
13    "expo-constants": "~57.0.19",
14    "expo-crypto": "~57.0.3",
15    "expo-file-system": "~57.0.7",
16    "expo-font": "~57.0.4",
17    "expo-haptics": "~57.0.3",
18    "expo-image-manipulator": "~57.0.20",
19    "expo-image-picker": "~57.0.20",
20    "expo-linear-gradient": "~57.0.2",
21    "expo-linking": "~57.0.11",
22    "expo-media-library": "~57.0.5",
23    "expo-router": "~57.0.23",
24    "expo-speech": "~57.0.3",
25    "expo-speech-recognition": "^57.1.0",
26    "expo-splash-screen": "~57.0.9",
27    "expo-sqlite": "~57.0.3",
28    "expo-status-bar": "~57.0.1",
29    "expo-system-ui": "~57.0.4",
30    "expo-updates": "~57.0.23",
31    "react": "19.2.3",
32    "react-dom": "19.2.3",
33    "react-native": "0.86.3",
34    "react-native-safe-area-context": "~5.7.0",
35    "react-native-screens": "~4.26.0",
36    "react-native-svg": "15.15.4",
37    "react-native-web": "~0.21.0"
38  },
39  "devDependencies": {
40    "@types/react": "~19.2.2",
41    "eslint": "^9.0.0",
42    "eslint-config-expo": "~57.0.2",
43    "playwright": "^1.56.0",
44    "typescript": "~6.0.3",
45    "vitest": "^5.0.2"
46  },
47  "scripts": {
48    "start": "expo start",
49    "android": "expo start --android",
50    "ios": "expo start --ios",
51    "web": "expo start --web",
52    "test": "vitest run",
53    "typecheck": "tsc --noEmit",
54    "lint": "expo lint",
55    "e2e:web": "EXPO_WEB_BASE_URL=/Socrates-and-Skeletons- expo export --platform web --output-dir dist && EXPO_WEB_BASE_URL=/Socrates-and-Skeletons- node scripts/web-home-screen.mjs dist && cp dist/index.html dist/404.html && node e2e/web-auth.mjs dist e2e-shots",
56    "demo:video": "node e2e/demo-video.mjs dist demo-video light android && node e2e/demo-video.mjs dist demo-video dark android && node e2e/demo-video.mjs dist demo-video light web"
57  },

```

```
58 "private": true
59 }
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → package.json:1

บรรทัด	ความหมาย
2–3	ชื่อโปรเจกต์ mindpay และเวอร์ชัน
4	จุดเริ่มต้นของแอป: ให้ Expo Router เป็นคนเริ่ม แล้วมันจะไปหาหน้าจอในโฟลเดอร์ src/app เอง
5–38	dependencies แพ็กเกจที่แอปต้องใช้ตอนทำงานจริง ตัวเลขหลังชื่อคือเวอร์ชัน (^ = ยอมรุ่นย่อยที่ใหม่กว่า, ~ = ยอมเฉพาะรุ่นแก้บั๊ก)
6–7	ฟอนต์ไทย Anuphan (ข้อความทั่วไป) และ Noto Serif Thai (ตัวเลขเงินและหัวข้อ)
8	ไอคอนสำเร็จรูป (Icons) เช่น รูปบ้าน รูปใบไม้ในแถบล่าง
9	ตัวเลือกวันที่และเวลาแบบของมือถือ (FR-1)
10	ตัวเชื่อมต่อ Supabase (ฐานข้อมูล, เข้าสู่ระบบ, Edge Functions)
11	Expo ตัวหลัก เวอร์ชัน 57
12	expo-camera ใช้ อ่าน QR จากรูป (ไม่ได้ใช้ถ่ายรูป แอปบล็อกสิทธิ์กล้องไว้)
14	expo-crypto ทำ "ลายนิ้วมือ" (hash) ของรูปสลิป กันสแกนซ้ำ (FR-4)
15, 18	อ่านไฟล์ และย่อ/ตัดรูปก่อนส่งให้ AI
17	expo-haptics สั่นเบา ๆ เวลากดปุ่ม
19, 22	เลือกรูปเอง และเข้าถึงแคเลอรีทั้งหมด (FR-4)
20	สีไล่ระดับ (gradient) ของการ์ดเขียว
21	เปิดลิงก์ เช่น ลิงก์ยืนยันอีเมลที่พาเข้าแอป
23	Expo Router ระบบเปลี่ยนหน้าจอ (บท A7)
24–25	expo-speech ให้น้องกล่าวพูดออกเสียง (FR-5), expo-speech-recognition ฟังเสียงพูดจด (FR-3)
26	หน้าจอแรกตอนเปิดแอป (splash)
27	expo-sqlite ฐานข้อมูลเล็ก ๆ ในเครื่อง ใช้เก็บการตั้งค่าและข้อมูลโหมดทดลอง
30	expo-updates รับอัปเดตโค้ดใหม่โดยไม่ต้องลงแอปใหม่ (EAS Update)
31–33	React (ตัววาดหน้าจอ), react-dom (สำหรับเว็บ), React Native (สำหรับมือถือ)
34–35	จัดการขอมจ (รอยบาก แถบสถานะ) และหน้าจอแบบ native
36	วาดภาพเวกเตอร์ (SVG) เช่น ต้นไม้เงิน น้องกล่าวกราฟ
37	react-native-web แปลงชิ้นส่วนของมือถือเป็น HTML ให้แสดงบนเว็บได้
39–46	devDependencies ใช้ตอนพัฒนาเท่านั้น ไม่ได้อยู่ในแอปที่ผู้ใช้งาน: ตัวตรวจภาษา TypeScript, ESLint (ตรวจรูปแบบโค้ด), Playwright (ควบคุมเบราว์เซอร์อัตโนมัติสำหรับ E2E), Vitest (รัน unit test)

บรรทัด	ความหมาย
47–57	scripts คำสั่งลัด พิมพ์ <code>npm run</code> ชื่อ (หรือ <code>npm test</code> , <code>npm start</code>) แทนคำสั่งยาว ๆ ทางขวา
58	<code>private</code> = ไม่เผยแพร่เป็นแพ็คเกจให้คนอื่นติดตั้ง

คำสั่งที่ใช้บ่อย

คำสั่ง	ทำอะไร	ผลที่ควรเห็น
<code>npm run web</code>	เปิดแอปเวอร์ชันเว็บในเบราว์เซอร์ แก้โค้ดแล้วหน้าเว็บเปลี่ยนเอง	เบราว์เซอร์เปิด localhost:8081
<code>npx expo start</code>	เปิด "dev server" มี QR ให้สแกนด้วย Expo Go	QR ใน terminal กด w = เปิดเว็บ, r = โหลดใหม่
<code>npm test</code>	รัน unit test ทั้งหมด	"Tests 157 passed (157)"
<code>npm run typecheck</code>	ให้ TypeScript ตรวจสอบข้อมูลทั้งโปรเจกต์	ไม่มีข้อความ = ไม่มีปัญหา
<code>npx expo lint</code>	ตรวจสอบรูปแบบโค้ดตามกฎ ESLint	ไม่มี error
<code>npm run e2e:web</code>	build เว็บแล้วรันเทสต์ในเบราว์เซอร์ 145 ข้อ ถ่ายภาพหน้าจอเก็บใน e2e-shots	"145/145 checks passed"

! ถ้าเจอ error ตอนติดตั้ง

"node: command not found" = ยังไม่ได้ติดตั้ง Node.js หรือต้องปิดแล้วเปิด terminal ใหม่ · **npm install** ซ้ำ หรือค้าง = รอได้ หรือลบโฟลเดอร์ `node_modules` แล้วลองใหม่ · **แอปขึ้นแต่เข้าสู่ระบบไม่ได้** = ลืมสร้างไฟล์ `.env` (บรรทัด 4) · ห้ามใช้ `npm install` ชื่อแพ็คเกจ ตรง ๆ ในโปรเจกต์ Expo ให้ใช้ `npx expo install` ชื่อแพ็คเกจ เพื่อให้ได้รุ่นที่เข้ากับ Expo 57 (กฎใน AGENTS.md)

รู้จักหน้าจอ VS Code

ส่วน	อยู่ตรงไหน	ใช้ทำอะไร
Activity Bar	แถบไอคอนซ้ายสุด	สลับระหว่าง Explorer, Search, Source Control, Run, Extensions
Explorer	ไอคอนรูปกระดาษ (Ctrl+Shift+E)	ต้นไม้โฟลเดอร์และไฟล์ทั้งโปรเจกต์ คลิกเพื่อเปิดไฟล์
Editor	พื้นที่ใหญ่ตรงกลาง	อ่านและแก้ไขโค้ด เปิดหลายไฟล์เป็นแท็บได้ ลากแท็บไปข้าง ๆ เพื่อดู 2 ไฟล์ คู่กัน
Search	ไอคอนแว่นขยาย (Ctrl+Shift+F)	ค้นคำในทุกไฟล์ เช่นค้น <code>computeRunway</code> จะเห็นทุกที่ที่ใช้
Source Control	ไอคอนกิ่งไม้ (Ctrl+Shift+G)	ดูไฟล์ที่แก้ไข, commit, push (un A13)

ส่วน	อยู่ตรงไหน	ใช้ทำอะไร
Terminal	ด้านล่าง (Ctrl+` ปุ่มตัวหนอนใต้ Esc)	พิมพ์คำสั่ง npm, git ได้โดยไม่ต้องออกจาก VS Code
Problems	แถบข้าง Terminal (Ctrl+Shift+M)	รวม error จาก TypeScript และ ESLint คลิกแล้วกระโดดไปบรรทัดที่ผิด
Status Bar	แถบล่างสุด	บอก branch ของ Git, บรรทัด/คอลัมน์ที่เคอร์เซอร์อยู่, ภาษาของไฟล์

คีย์ลัดที่ทำให้อ่านโค้ดเร็วขึ้น 10 เท่า

Mac ใช้ Cmd แทน Ctrl และ Option แทน Alt

คีย์	ทำอะไร	ลองกับ MindPay
Ctrl+P	เปิดไฟล์จากชื่อ พิมพ์ : เลขบรรทัด ต่อกายเพื่อกระโดดไปบรรทัดนั้น	พิมพ์ runway.ts : 72 แล้ว Enter
Ctrl+G	ไปบรรทัดที่ต้องการในไฟล์ที่เปิดอยู่	Ctrl+G แล้วพิมพ์ 89
F12	Go to Definition ไปดูว่าฟังก์ชันหรือตัวแปรนี้ถูกสร้างไว้ที่ไหน (หรือ Ctrl+คลิก)	ใน AppProvider.tsx คลิกที่ computeRunway แล้วกด F12
Alt+F12	Peek Definition: ดูโค้ดปลายทางในกล่องเล็ก ๆ โดยไม่ต้องเปลี่ยนไฟล์	-
Shift+F12	Find All References หากทุกที่ที่เรียกใช้สิ่งนี้	ที่ computeRunway จะเห็นทั้งในแอปและในเทสต์
Alt+← / Alt+→	ย้อนกลับ/ไปข้างหน้า หลังกระโดดไปมาด้วย F12	-
เอาเมาส์ชี้ (hover)	เห็นชนิดข้อมูลและคอมเม้นต์อธิบายของสิ่งนั้น	ชี้ที่ Transaction จะเห็นทุกช่องของรายการ
Ctrl+Shift+0	รายการฟังก์ชันทั้งหมดในไฟล์นี้ (outline)	ใน runway.ts จะเห็น 6 ฟังก์ชัน
Ctrl+T	ค้นหาชื่อฟังก์ชันทั้งโปรเจกต์	พิมพ์ parseSpokenEntry
Ctrl+Shift+F	ค้นข้อความในทุกไฟล์	ค้น "เงินพอใช้อีก" เพื่อหาว่าข้อความบนจอมาจากไฟล์ไหน
Ctrl+F	ค้นในไฟล์นี้	-
F2	เปลี่ยนชื่อตัวแปรทุกที่พร้อมกันอย่างปลอดภัย	-
Ctrl+/,	เปลี่ยนบรรทัดเป็นคอมเมนต์ (ปิดชั่วคราว) หรือกลับคืน	-
Ctrl+Shift+P	Command Palette เรียกคำสั่งทุกอย่างของ VS Code	พิมพ์ "Format Document"
Ctrl+B	ซ่อน/แสดงแถบด้านข้าง ให้มีที่อ่านโค้ดมากขึ้น	-

★ เทคนิคหาว่าข้อความบนจอมาจากโค้ดตรงไหน

เห็นคำว่า "เงินพอใช้อีก 16 วัน" บนหน้าจอหลัก แล้วอยากรู้ว่าโค้ดอยู่ไหน: กด **Ctrl+Shift+F** พิมพ์ "เงินพอใช้อีก" จะเจอไฟล์ที่มีข้อความนี้ทั้งหมด คลิกผลลัพธ์แล้วจะไปบรรทัดนั้นทันที วิธีนี้ใช้ได้กับทุกข้อความภาษาไทยในแอป เพราะข้อความทั้งหมดเขียนอยู่ในโค้ดตรง ๆ

ส่วนเสริม (Extensions) ที่แนะนำ

กด **Ctrl+Shift+X** แล้วค้นชื่อ กด **Install**

ชื่อ	ช่วยอะไร
ESLint (Microsoft)	ขีดเส้นใต้โค้ดที่ผิดกฎทันทีที่พิมพ์ ใช้กฎเดียวกับ <code>npx expo lint</code>
Expo Tools	ช่วยเติมคำและตรวจไฟล์ <code>app.json</code> / <code>eas.json</code>
Vitest	มีปุ่ม ▶ ข้างเกสต์แต่ละข้อ กดรันทีละข้อได้
GitLens (ไม่บังคับ)	ขีดบรรทัดไหนก็เห็นว่าใครแก้ล่าสุด เมื่อไร ใน <code>commit</code> อะไร
Error Lens (ไม่บังคับ)	แสดงข้อความ error ท้ายบรรทัดเลย ไม่ต้องเอาเมาส์ชี้

ส่วน TypeScript ไม่ต้องลงเพิ่ม VS Code รองรับในตัวอยู่แล้ว และใช้ TypeScript รุ่นในโปรเจกต์ (`node_modules/typescript`) ตามไฟล์ `tsconfig.json`

ไม่อยากติดตั้งอะไร: ใช้ VS Code บนเว็บ

มี 2 ทางที่เปิดโค้ด MindPay ด้วย VS Code ได้ทันทีในเบราว์เซอร์

- **ลิงก์ในเล่มนี้** แต่ชื่อไฟล์บนหัวโค้ดค่อนข้างโหดก็ได้ จะเปิด **vscode.dev** ที่ไฟล์และบรรทัดนั้น
- **จากหน้า GitHub** เปิด repo บน github.com แล้วกดปุ่ม . (จุด) บนคีย์บอร์ด หน้าเว็บจะกลายเป็น VS Code (github.dev)

VS Code บนเว็บอ่านโค้ด ค้นหา (**Ctrl+Shift+F**) กด **F12** ไปดูฟังก์ชัน และแก้ไขได้ แต่ **รันคำสั่งไม่ได้** (ไม่มี terminal จึงรัน `npm test` หรือเปิดแอปไม่ได้) ถ้าจะรันแอปต้องโคลนลงเครื่องตามขั้นตอนด้านบน

ฝึกใช้: ตามรอย Money Runway ใน 5 นาที

1. **Ctrl+P** พิมพ์ `index.tsx` เลือกไฟล์ใน `src/app/` (tabs) (หน้าจอหลัก)
2. **Ctrl+F** ค้น `useMoney` แล้วกด **F12** ที่คำนั้น จะเดินไป `src/data/AppProvider.tsx` บรรทัด 443
3. ที่บรรทัด 453 คลิก `computeRunway` แล้วกด **F12** จะไปที่ `src/domain/runway.ts` บรรทัด 72 ซึ่งเป็นสูตรของ `FR-6`
4. กด **Shift+F12** ที่ชื่อฟังก์ชัน จะเห็นว่ามีเกสต์เรียกใช้ใน `src/domain/__tests__/domain.test.ts` ด้วย
5. กด **Alt+←** สองครั้ง ย้อนกลับไปหน้าจอหลัก

ทำแบบนี้ได้ แปลว่าพร้อมอ่านโค้ดส่วนไหนของแอปก็ได้แล้ว

พื้นฐานการเขียนโปรแกรม สำหรับคนที่ไม่เคยเขียนเลย

ทุกอย่างที่ต้องรู้เพื่ออ่านโค้ด MindPay ออก: ค่า ตัวแปร เงื่อนไข ฟังก์ชัน array และ object พร้อมตัวอย่างจากโค้ดจริง

โปรแกรมคืออะไร

โปรแกรมคือ รายการคำสั่งที่คอมพิวเตอร์ทำตามทีละบรรทัด จากบนลงล่าง เหมือนสูตรทำอาหาร ทีมเราเขียนด้วยภาษา **TypeScript** ซึ่งคือภาษา **JavaScript** (ภาษาของเว็บ) ที่เพิ่ม "ชนิดข้อมูล" เข้าไป (บท A5) ไฟล์ที่ลงท้ายด้วย `.ts` เป็น TypeScript ล้วน ส่วน `.tsx` คือ TypeScript ที่มีโค้ดหน้าจอบน React ปนอยู่ (บท A6)

1. ค่า (value) และชนิดของค่า

ชนิด	ตัวอย่าง	ใช้ใน MindPay
number ตัวเลข	8500, -3, 0.8	จำนวนเงินเป็นสตางค์ 8500 = ฿85, ความมั่นใจ 0.8 = 80%
string ข้อความ	'ข้าวมันไก่', "2026-10-09"	ชื่อรายการ วันที่ ข้อความบนจอ (ใส่ในเครื่องหมายคำพูด ' หรือ ")
boolean จริง/เท็จ	true, false	เข้าสู่ระบบแล้วหรือยัง, ตั้งค่าเสร็จหรือยัง (onboarded)
null	null	"ไม่มีค่า" โดยตั้งใจ เช่น รายการที่จองเองไม่มีเลขอ้างอิงสลิป (slipRef: null)
undefined	undefined	"ยังไม่ได้ใส่ค่า" เช่น ไม่ได้ส่งพารามิเตอร์นั้นมา

2. ตัวแปร: กล่องที่มีชื่อ

ตัวแปรคือ "กล่องที่ติดป้ายชื่อ" ไว้ใส่ค่า เพื่อเรียกใช้ทีหลัง

ตัวแปร

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const floor = 50_000;           // const = ค่าคงที่ ใส่แล้วเปลี่ยนไม่ได้
2 let total = 0;                  // let = เปลี่ยนค่าได้ภายหลัง
3 total = total + 8500;           // ตอนนี้ total = 8500
4 const title = 'ข้าวมันไก่';
```

- `const` ใช้บ่อยที่สุดในโค้ด MindPay เพราะค่าส่วนใหญ่ไม่ควรเปลี่ยน ใช้ `let` เฉพาะตอนต้องเปลี่ยนจริง ๆ
- `50_000` คือ 50000 เครื่องหมาย `_` ช่วยให้อ่านง่ายเหมือนคอมมา คอมพิวเตอร์ไม่สนใจมัน
- ข้อความหลัง `//` คือ **คอมเมนต์** คอมพิวเตอร์ข้าม มีไว้ให้คนอ่าน ส่วน `/* ... */` และ `/** ... */` เป็นคอมเมนต์หลายบรรทัด (แบบ `/**` VS Code จะเอาไปแสดงตอนเอาเมาส์ชี้)
- ตั้งชื่อแบบ **camelCase**: คำแรกตัวเล็ก คำต่อไปขึ้นต้นตัวใหญ่ เช่น `balanceSatang`, `averageDailyExpense` คำคงที่สำคัญใช้ตัวใหญ่ทั้งหมด เช่น `RUNWAY_CAP_DAYS`

★ กฎการตั้งชื่อของทีมเรา: ตัวแปรเงินลงท้ายด้วย Satang

ทุกตัวแปรที่เป็นจำนวนเงิน ลงท้ายด้วย Satang เสมอ เช่น amountSatang, floorSatang เห็นชื่อแล้วรู้ทันทีว่าต้องหาร 100 ก่อนแสดงเป็นบาท กันความผิดพลาดแบบเอาบาทไปบวกกับสตางค์

3. ตัวดำเนินการ: คำนวณและเปรียบเทียบ

เขียนว่า	ความหมาย	ตัวอย่าง → ผล
+ - * /	บวก ลบ คูณ หาร	300000 / 145 → 2068.96...
%	เศษจากการหาร	8550 % 100 → 50 (50 สตางค์)
=== !==	เท่ากับ / ไม่เท่ากับ (แบบเคร่ง ชนิดต้องตรงด้วย)	t.kind === 'income'
< <= > >=	น้อยกว่า น้อยกว่าหรือเท่ากับ ...	days < 7
&&	"และ" จริงเมื่อทั้งสองข้างจริง	d >= windowStart && d <= today
	"หรือ" จริงเมื่อข้างใดข้างหนึ่งจริง	event === 'SIGNED_IN' ...
!	"ไม่" กลับจริงเป็นเท็จ	!existing → ไม่มีรายการเดิม
ก ? ข : ค	ถ้า ก จริง ได้ ข ไม่จริงได้ ค (ternary)	kind === 'income' ? 'เพิ่มเงินเข้า' : 'ลดรายการ'
ก ?? ข	ใช้ ก ถ้า ก ไม่ใช่ null/undefined ไม่ขึ้นใช้ ข (ค่าสำรอง)	profile?.runwayFloorSatang ?? 50_000
ก?.ข	เข้าถึง ข ใน ก แต่ถ้า ก เป็น null ไม่ต้อง error ได้ undefined	profile?.openingBalanceSatang

ลองอ่านบรรทัดนี้จาก `src/data/AppProvider.tsx:447` ซึ่งใช้ทั้ง ?. และ ??

`src/data/AppProvider.tsx`

บรรทัด 447-447

```
447 const floor = profile?.runwayFloorSatang ?? 50_000;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:447`

อ่านว่า: "เส้นเงินสำรอง = ค่าที่ผู้ตั้งไว้ในโปรไฟล์ แต่ถ้ายังไม่มีโปรไฟล์ (?.) หรือไม่มีค่า ให้ใช้ 50,000 สตางค์ (??)"

4. เงื่อนไข if: ถ้า... ให้ทำ...

`if / else`

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 if (spendable <= 0) {
2   // ทำส่วนนี้เมื่อเงื่อนไขจริง
3 } else if (averageSatang <= 0) {
4   // ไม่จริง ถ้าเงื่อนไขนี้จริง ทำส่วนนี้
5 } else {
6   // ไม่ตรงอะไรเลย ทำส่วนนี้
```

ในโค้ด MindPay จะเห็นรูปแบบ "เช็กกรณีพิเศษก่อน แล้วออกเลย" (early return) บ่อยมาก เช่นในสูตร Money Runway

src/domain/runway.ts

บรรทัด 83-88

```
83  if (spendable <= 0) {
84    return { ...base, status: 'below_floor', days: 0, depletionDay: today, capped: false };
85  }
86  if (averageSatang <= 0) {
87    return { ...base, status: 'no_spending', days: null, depletionDay: null, capped: false };
88  }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:83

อ่านว่า: "ถ้าเงินที่ใช้ได้ ≤ 0 ให้ส่งผลกลับเลยว่าต่ำกว่าเงินสำรอง · ถ้าค่าเฉลี่ย ≤ 0 ส่งกลับว่ายังไม่มีรายจ่าย" พอผ่าน 2 ด่านนี้ไปได้ โค้ดด้านล่างมั่นใจได้ว่าหารได้แน่นอน ไม่หารด้วยศูนย์

5. ฟังก์ชัน: เครื่องจักรรับของเข้า ส่งของออก

ฟังก์ชันคือ "ชุดคำสั่งที่ตั้งชื่อไว้" เรียกใช้ซ้ำได้ รับ พารามิเตอร์ (ของที่ส่งเข้า) และ return ผลลัพธ์ออกมา

ฟังก์ชัน 2 แบบ

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1  // แบบที่ 1: function
2  function toBaht(satang: number): number {
3    return satang / 100;
4  }
5
6  // แบบที่ 2: arrow function (ลูกศร =>) ทำงานเหมือนกัน เขียนสั้นกว่า
7  const toBaht2 = (satang: number) => satang / 100;
8
9  toBaht(8500); // ได้ 85
```

- (satang: number) = รับ 1 พารามิเตอร์ชื่อ satang ต้องเป็นตัวเลข
- : number หลังวงเล็บ = ฟังก์ชันนี้จะคืนค่าเป็นตัวเลข
- arrow function ที่ไม่มี { } จะ return ค่าทางขวาของ => ให้อัตโนมัติ
- export หน้าฟังก์ชัน = ให้ไฟล์อื่น import ไปใช้ได้

★ Pure function: หัวใจของ src/domain

ฟังก์ชันใน src/domain ทุกตัวเป็น **pure function** คือ (1) ใส่ค่าเดิม ได้ผลเดิมทุกครั้ง และ (2) ไม่ไปทำอะไรข้างนอก ไม่เรียกอื่นเตอร์เน็ต ไม่แตะหน้าจอ เหมือนเครื่องคิดเลข จึงทดสอบง่ายมาก: ใส่ตัวเลขแล้วตรวจคำตอบ อาจารย์มักถามว่า "ทำไมแยก domain" ให้ตอบเรื่องนี้

6. Array: รายการของหลายค่า

Array คือรายการเรียงลำดับ เขียนใน [] ค้นด้วยคอมมา ตำแหน่งแรกคือ 0

Array พื้นฐาน

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 const amounts = [12000, 0, 8500, 30000];
2 amounts[0];           // 12000 (ตัวแรก)
3 amounts.length;       // 4 (มีกี่ตัว)
4 amounts.includes(0);   // true

```

MindPay เก็บรายการเงินทั้งหมดเป็น array ชื่อ txs (ย่อจาก transactions) และใช้ "เมธอด" ของ array ต่อไปนี้ตลอด ต้องจำให้ได้

เมธอด	ทำอะไร	ตัวอย่าง
filter	คัดเฉพาะตัวที่ผ่านเงื่อนไข ได้ array ใหม่ที่สั้นลง	txs.filter((t) => t.status === 'confirmed')
map	แปลงทุกตัว ได้ array ยาวเท่าเดิม	txs.map((t) => t.amountSatang)
reduce	รวมทุกตัวให้เหลือค่าเดียว เช่น ผลรวม	list.reduce((s, t) => s + t.amountSatang, 0)
find	หาตัวแรกที่ตรงเงื่อนไข (ไม่เจอได้ undefined)	txs.find((t) => t.id === id)
some	มีอย่างน้อย 1 ตัวที่ตรงไหม (true/false)	list.some((t) => t.slipRef === ref)
sort	เรียงลำดับ	days.sort()
slice	ตัดเอาบางส่วน	list.slice(0, 5) = 5 ตัวแรก

ตัว (t) => ... ที่ส่งเข้าไปในเมธอดเหล่านี้คือ arrow function ที่จะถูกเรียกกับสมาชิกแต่ละตัว t คือ "รายการที่กำลังดูอยู่"

ตัวอย่างจริง: ยอดคงเหลือคำนวณยังใจ (FR-2)

src/domain/summary.ts

บรรทัด 9-17

```

9 export const confirmedOnly = (txs: Transaction[]) => txs.filter((t) => t.status === 'confirmed');
10
11 /** Balance now = money at start + all confirmed income - all confirmed expense. */
12 export function computeBalance(openingBalanceSatang: number, txs: Transaction[]): number {
13   return confirmedOnly(txs).reduce(
14     (sum, t) => sum + (t.kind === 'income' ? t.amountSatang : -t.amountSatang),
15     openingBalanceSatang,
16   );
17 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/summary.ts:9

บรรทัด	ความหมาย
9	สร้างฟังก์ชัน confirmedOnly: รับรายการทั้งหมด คัดเฉพาะรายการที่ยืนยันแล้ว (กรองแบบร่างทิ้ง) ด้วย filter
11	คอมเมนต์อธิบายสูตร: ยอดตอนนี้ = เงินตอนเริ่ม + รายรับที่ยืนยัน - รายจ่ายที่ยืนยัน
12	ฟังก์ชัน computeBalance รับ 2 อย่าง: เงินตอนเริ่ม (สตางค์) และรายการทั้งหมด คำนวณเป็นตัวเลข
13	เอาเฉพาะรายการที่ยืนยันแล้ว แล้ว reduce รวมให้เหลือตัวเลขเดียว

บรรทัด	ความหมาย
14	สำหรับแต่ละรายการ t: ถ้าเป็นรายรับ บวกเข้า (+t.amountSatang) ถ้าเป็นรายจ่าย ลบออก (-t.amountSatang) แล้วเก็บผลไว้ใน sum
15	ค่าเริ่มต้นของ sum คือเงินตอนเริ่มใช้แอป

ลองคิดตาม: เริ่ม ฿3,000 (300,000 สตางค์) มีรายรับ ฿10,000 ที่ยืนยันแล้ว และรายจ่าย ฿85 กับแบบร่าง ฿200

```
sum เริ่ม          = 300,000
+ รายรับ 1,000,000 = 1,300,000
- รายจ่าย 8,500    = 1,291,500
(แบบร่าง 20,000 ถูกกรองทิ้งตั้งแต่บรรทัด 13)
ผล = 1,291,500 สตางค์ = ฿12,915.00
```

7. Object: ของ 1 ชิ้นที่มีหลายช่อง

Object คือกลุ่มของ "ชื่อช่อง: ค่า" อยู่ใน { } รายการเงิน 1 รายการใน MindPay คือ object แบบนี้

รายการ 1 รายการ (Transaction)

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const tx = {
2   id: 'demo-12',
3   kind: 'expense',
4   amountSatang: 8500,
5   categoryKey: 'food',
6   title: 'ข้าวมันไก่',
7   status: 'confirmed',
8 };
9 tx.title;           // 'ข้าวมันไก่' (เข้าถึงช่องด้วยจุด)
10 tx.amountSatang;   // 8500
```

เทคนิค object ที่เจอบ่อยในโค้ด

เขียนว่า	ชื่อเรียก	ความหมาย
<code>const { a, b } = obj;</code>	destructuring (แกะของ)	ดึงช่อง a และ b ออกมาเป็นตัวแปรในบรรทัดเดียว เช่น <code>const { balance, runway } = useMoney();</code>
<code>{ ...obj, status: 'confirmed' }</code>	spread (กระจาย)	คัดลอกทุกช่องของ obj มาเป็น object ใหม่ แล้วเปลี่ยนเฉพาะช่อง status ของเดิมไม่ถูกแก้
<code>[tx, ...list]</code>	spread กับ array	array ใหม่ที่มี tx อยู่หน้าสุด ตามด้วยของเดิมทั้งหมด (ใช้ตอนเพิ่มรายการใหม่ในบท A2)
<code>{ kind, title }</code>	shorthand	ย่อจาก <code>{ kind: kind, title: title }</code>

! ทำไมโค้ดชอบสร้าง object ใหม่แทนการแก้ไขเดิม

React (un A6) รู้ว่าข้อมูลเปลี่ยนด้วยการดูว่า "เป็น object ตัวใหม่หรือเปล่า" ถ้าเราแก้ไขใน object เดิมตรง ๆ React อาจไม่รู้ และหน้าจอก็ไม่อัปเดต โค้ดจึงใช้ { ...t, status: 'confirmed' } สร้างของใหม่เสมอ (ดูตัวอย่างใน [src/data/AppProvider.tsx:418](#))

8. วงซ้ำ (loop)

for ... of

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 let total = 0;
2 for (const t of txs) {           // ทำซ้ำกับรายการทีละตัว
3   if (t.kind !== 'expense') continue; // continue = ข้ามตัวนี้ ไปตัวถัดไป
4   total += t.amountSatang;       // += แปลว่า total = total + ...
5 }
```

ส่วน for (let i = 0; i < 20; i++) คือวนแบบนับรอบ: เริ่ม i = 0 ทำซ้ำตราบที่ i < 20 และเพิ่ม i ทีละ 1 (i++)
ส่วน break คือหยุดวนทันที

9. ข้อความ (string) ที่ใช้บ่อย

เขียนว่า	ผล
`เงินพอใช้อีก \${days} วัน`	template string (ใช้ backtick) แทรกค่าตัวแปรใน \${ } ได้ เช่น "เงินพอใช้อีก 16 วัน"
'2026-10-09'.slice(0, 7)	'2026-10' (ตัดตัวอักษรตำแหน่ง 0 ถึงก่อน 7)
'a,b'.split(',')	['a', 'b'] (แยกเป็น array)
' ข้าว '.trim()	'ข้าว' (ตัดช่องว่างหัวท้าย)
'KFC'.toLowerCase()	'kfc'
String(5).padStart(2, '0')	'05' (เติม 0 ข้างหน้าให้ครบ 2 ตัว ใช้กับวันที่)
s.replace(/บาง/g, '')	ลบคำว่า "บาง" ทุกที่ (/.../g คือ regular expression ดู un A5)

10. คณิตศาสตร์ที่ใช้

คำสั่ง	ผล	ใช้ที่ไหน
Math.floor(20.69)	20 (ปัดลงเสมอ)	จำนวนวัน Money Runway
Math.round(680.2857)	680 (ปัดตามปกติ)	ค่าเฉลี่ยต่อวันเป็นสตางค์
Math.max(0, -500)	0 (ตัวที่มากกว่า)	กันเงินที่ใช้ได้ติดลบ
Math.min(7, 3)	3 (ตัวที่น้อยกว่า)	จำนวนวันที่ใช้หารค่าเฉลี่ย ไม่เกิน 7
Math.abs(-85)	85 (ค่าสัมบูรณ์)	แสดงยอดเงินโดยไม่สนเครื่องหมาย

แบบฝึกหัด A4

1. ถ้า profile เป็น null ค่าของ profile?.runwayFloorSatang ?? 50_000 คืออะไร
2. [100, 200, 300].filter((x) => x > 150).map((x) => x / 100) ได้อะไร
3. เริ่ม ฿500 มีรายการยืนยันแล้ว: รายรับ ฿1,000 รายจ่าย ฿300 และแบบร่างรายจ่าย ฿200 computeBalance คืนที่
สตางค์
4. const t2 = { ...t, title: 'ข้าวผัด' } แล้ว t.title เปลี่ยนไหม

i เฉลย

- 1) 50000 · 2) [2, 3] (กรองเหลือ 200, 300 แล้วหาร 100) · 3) 50,000 + 100,000 – 30,000 = 120,000
สตางค์ (แบบร่างไม่นับ) · 4) ไม่เปลี่ยน t2 เป็น object ใหม่ t ยังเป็นค่าเดิม

TypeScript: ภาษาที่ทีมเราใช้ และชนิดข้อมูลของ MindPay

TypeScript คือ JavaScript ที่บังคับให้บอกว่าข้อมูลแต่ละตัวเป็นชนิดอะไร ทำให้จับผิดได้ตั้งแต่ตอนพิมพ์ ก่อนผู้ใช้จะเจอ

ทำไมต้อง TypeScript

ใน JavaScript ธรรมดา ถ้าพิมพ์ `tx.amountSatnag` (สะกดผิด) โปรแกรมจะรันได้ แต่ได้ค่า `undefined` แล้วยอดเงินเพี้ยนตอนผู้ใช้กำลังใช้ ส่วน TypeScript จะขีดเส้นแดงใน VS Code กันที่ว่า "ไม่มีช่องชื่อนี้" ทีมเราเปิดโหมดเข้มงวดที่สุด ("`strict`": `true` ในไฟล์ `tsconfig.json`) และรัน `npm run typecheck` ก่อน build APK ทุกครั้ง

≈ เปรียบเทียบ

TypeScript เหมือนแบบฟอร์มราชการที่มีช่องบังคับว่า "ช่องนี้ต้องเป็นตัวเลข 13 หลัก" กรอกผิดเจ้าหน้าที่ตักกลับทันทีที่โต๊ะ ตักว่าไปรู้ตอนเอกสารเข้าระบบแล้ว เมื่อ build แอป ชนิดข้อมูลทั้งหมดจะถูกลบออก เหลือ JavaScript ธรรมดา ชนิดข้อมูลจึงช่วยเราตอนเขียนโค้ดเท่านั้น ไม่ทำให้แอปช้าลง

การบอกชนิดข้อมูล (type annotation)

บอกชนิดด้วย :

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const days: number = 16;
2 const title: string = 'ข้าวมันไก่';
3 let slipRef: string | null = null; // เป็น string หรือ null ก็ได้
4 function toBaht(satang: number): number { return satang / 100; }
```

หลายครั้งไม่ต้องเขียนชนิดเอง TypeScript เดาเองได้ (type inference) เช่น `const days = 16` รู้เองว่าเป็น number เอาเมาส์ชี้ใน VS Code จะเห็นชนิดที่มันเดา

ไฟล์ชนิดข้อมูลกลาง: `src/domain/types.ts`

ไฟล์นี้สำคัญที่สุดไฟล์หนึ่ง เพราะกำหนดว่า "รายการเงิน" และ "โปรไฟล์ผู้ใช้" หน้าตาเป็นยังไง ทุก FR ใช้ร่วมกัน

`src/domain/types.ts`

บรรทัด 1-55

```
1 /**
2  * Core data types for MindPay.
3  *
4  * Money is always stored as an integer number of satang (1 baht = 100 satang)
5  * so that adding many amounts never produces floating-point errors
6  * (0.1 + 0.2 !== 0.3 in JavaScript).
7  */
8
9 export type TxKind = 'income' | 'expense';
10
```

```

11 /** draft = not counted in any total until the user confirms it (Charter: slip results stay drafts). */
12 export type TxStatus = 'draft' | 'confirmed';
13
14 export type TxSource = 'manual' | 'slip';
15
16 export interface Transaction {
17   id: string;
18   kind: TxKind;
19   amountSatang: number;
20   categoryKey: string;
21   title: string;
22   note: string | null;
23   /** ISO 8601 instant, e.g. "2026-09-27T01:15:00.000Z" */
24   occurredAt: string;
25   source: TxSource;
26   status: TxStatus;
27   /** Transaction reference read from the slip (QR or text). Used for duplicate detection. */
28   slipRef: string | null;
29   /** SHA-256 of the processed slip image. Second line of duplicate detection. */
30   slipImageHash: string | null;
31   /** Lowest field confidence (0..1) reported by the slip reader, null for manual entries. */
32   ocrConfidence: number | null;
33   /** Fields the slip reader was unsure about, shown to the user during review. */
34   reviewFlags: string[];
35   createdAt: string;
36 }
37
38 /** Everything the user can type or change in the form. */
39 export type TransactionInput = Omit<Transaction, 'id' | 'createdAt'>;
40
41 export type CoachTone = 'friend' | 'coach' | 'senior';
42
43 export interface Profile {
44   id: string;
45   displayName: string;
46   /** Money the user had when they started using MindPay. Balance = opening + income - expense. */
47   openingBalanceSatang: number;
48   /** FR-6: the "low" line. Runway counts the days until balance would reach this amount. */
49   runwayFloorSatang: number;
50   monthlyBudgetSatang: number | null;
51   coachTone: CoachTone;
52   onboarded: boolean;
53 }
54
55 export type RangeKey = 'today' | '7d' | '1m' | '6m' | '1y';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/types.ts:1

บรรทัด ความหมาย

- 1-7 คอมเมนต์อธิบายกฎสำคัญ: เงินเก็บเป็นจำนวนเต็มสตางค์เสมอ (1 บาท = 100 สตางค์) เพราะใน JavaScript 0.1 + 0.2 ไม่เท่ากับ 0.3
- 9 **union type:** ชนิด TxKind เป็นได้แค่ 2 ค่า คือข้อความ 'income' (รายรับ) หรือ 'expense' (รายจ่าย) เครื่องหมาย | แปลว่า "หรือ" ถ้าพิมพ์ 'incom' จะขึ้นเส้นแดงทันที

บรรทัด	ความหมาย
11–12	สถานะของรายการ: 'draft' (แบบร่าง ยังไม่นับ) หรือ 'confirmed' (ยืนยันแล้ว นับในยอด) คอมเมนต์อ้างถึง Charter ของทีมว่าผลจากสลิปเป็นแบบร่างก่อน
14	รายการมาจากไหน: จดเอง ('manual') หรือจากสลิป ('slip') รายการที่พุดจด (FR-3) นับเป็น manual
16	interface = พิมพ์เขียวของ object บอกว่า "รายการ 1 รายการ ต้องมีช่องอะไรบ้าง และแต่ละช่องชนิดอะไร"
17	id สหสัมพันธ์ของรายการ (ฐานข้อมูลสร้างให้)
18	kind ใช้ชนิด TxKind จากบรรทัด 9
19	amountSatang จำนวนเงินเป็นสตางค์ (บวกเสมอ ส่วนเข้าหรือออกดูจาก kind)
20	categoryKey สหสัมพันธ์ เช่น 'food', 'transport' (รายชื่อหมวดอยู่ใน src/domain/categories.ts)
21–22	ชื่อรายการ และบันทึกเพิ่มเติม (string null = ไม่มีบันทึกก็ได้)
23–24	เวลาที่เกิดรายการ เก็บแบบสากล ISO 8601 เช่น "2026-09-27T01:15:00.000Z" (Z = เวลา UTC ซึ่งช้ากว่าไทย 7 ชั่วโมง ตัวอย่างนี้คือ 08:15 น. เวลาไทย)
25–26	ที่มาและสถานะ ใช้ชนิดจากบรรทัด 14 และ 12
27–30	เลขอ้างอิงจากสลิป และ "ลายนิ้วมือ" ของรูป (SHA-256) ใช้กันสลิปซ้ำ 2 ชั้น (FR-4)
31–32	ความมั่นใจต่ำสุดที่ AI อ่านได้ (0 ถึง 1) รายการที่จดเองเป็น null
33–34	รายชื่อช่องที่ AI ไม่แน่ใจ เช่น ['amount'] ให้นำมาตรวจสอบสลิปไฮไลต์ช่องนั้น string[] = array ของข้อความ
35	เวลาที่สร้างแถวในระบบ
38–39	TransactionInput = Transaction ที่ ไม่มี id และ createdAt (Omit = ตัดช่องออก) ใช้ตอนสร้างรายการใหม่ เพราะสองช่องนี้ฐานข้อมูลเป็นคนสร้าง
41	โทนโคช 3 แบบเดิม (ไม่ได้ใช้แล้ว เพราะ 29 ก.ย. เปลี่ยนเป็นน้องกล้าเสียงเดียว แต่ยังเก็บชนิดไว้ให้ตรงกับคอลัมน์ในฐานข้อมูล)
43–53	Profile ข้อมูลผู้ใช้ 1 คน: ชื่อ, เงินตอนเริ่มใช้แอป (บรรทัด 47), เงินสำรองของ FR-6 (บรรทัด 49), จบต่อเดือน (ไม่ตั้งก็ได้ = null), โทนโคช และตั้งค่าครั้งแรกเสร็จหรือยัง
55	ช่วงเวลาที่เลือกดูได้ในหน้าหลัก: วันนี้, 7 วัน, 1 เดือน, 6 เดือน, 1 ปี (FR-2)

ชนิดข้อมูลแบบพิเศษที่เจอบ่อย

เขียนว่า	ความหมาย	ตัวอย่างใน MindPay
A B	เป็น A หรือ B (union)	'draft' 'confirmed'
ชื่อ?: ชนิด	ช่องนี้ไม่ใส่ก็ได้ (optional)	now?: Date
ชนิด[]	array ของชนิดนั้น	Transaction[]
Omit<T, 'a'>	T ที่ตัดช่อง a ออก	Omit<Transaction, 'id' 'createdAt'>

เขียนว่า	ความหมาย	ตัวอย่างใน MindPay
Partial<T>	T ที่ทุกช่องไม่บังคับ ใช้ตอน "แก้บางช่อง"	updateTx(id, patch: Partial<TransactionInput>)
Pick<T, 'a' 'b'>	T ที่เอาเฉพาะช่อง a และ b	Pick<TxRow, 'kind' 'amount_satang'>
Record<K, V>	object ที่ชื่อช่องเป็นชนิด K ค่าเป็นชนิด V	Record<string, unknown>
Promise<T>	"สัญญาว่าจะได้ T ในอนาคต" (งานที่ต้องรอ)	getProfile(): Promise<Profile>
type X = ...	ตั้งชื่อเล่นให้ชนิด	type TxKind = 'income' 'expense'
x as Y	บอก TypeScript ว่า "ชื่อฉัน x เป็นชนิด Y" (ใช้น้อย ๆ)	data as TxRow[]
'confirmed' as const	ให้ถือว่าเป็นค่าตายตัว ไม่ใช่ string ทั่วไป	status: 'confirmed' as const
unknown	ยังไม่รู้ชนิด ต้องตรวจสอบก่อนใช้ (ปลอดภัยกว่า any)	context: unknown

import และ export: แอร์โด้ระหว่างไฟล์

src/data/AppProvider.tsx

บรรทัด 6-17

```

6 import { Storage } from './storage';
7 import { createContext, useCallback, useContext, useEffect, useMemo, useRef, useState, type
  ReactNode } from 'react';
8 import { AppState } from 'react-native';
9 import { authErrorMessage, authLinkErrorMessage, parseAuthLink } from '../domain/auth';
10 import { bkkDayKey, msUntilNextBkkMidnight } from '../domain/dates';
11 import { applyDeposit, reservedSatang, type GoalInput, type SavingsGoal } from '../domain/goals';
12 import { averageDailyExpense, computeRunway } from '../domain/runway';
13 import { computeBalance } from '../domain/summary';
14 import type { Profile, Transaction, TransactionInput } from '../domain/types';
15 import { createCloudRepo, createDemoRepo, resetDemoData, type Repo } from './repo';
16 import { clearLinkFromAddressBar, onIncomingLink, openingLink } from './authLinks';
17 import { supabase } from './supabase';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:6

บรรทัด	ความหมาย
6	นำ Storage มาจากไฟล์ ./storage (จุด = โฟลเดอร์เดียวกัน ไม่ต้องใส่نامสกุลไฟล์)
7	นำเครื่องมือของ React มาใช้ ชื่อที่ไม่มีจุดนำหน้า เช่น 'react' คือแพ็คเกจใน node_modules
9-13	../domain/... = ถอยออกไป 1 โฟลเดอร์ (จาก data ไป src) แล้วเข้า domain นำฟังก์ชันคำนวณมาใช้
14	import type = นำมาเฉพาะชนิดข้อมูล (หายไปตอน build ไม่เพิ่มขนาดแอป)

11 `type GoalInput` ในวงเล็บปีกกา = import ทั้งฟังก์ชันและชนิดจากไฟล์เดียวกันในบรรทัดเดียว

ฝั่งไฟล์ต้นทางต้องเขียน `export` หน้าสิ่งที่ยอมให้คนอื่นใช้ ส่วน `export default` ใช้กับ "ของหลัก" ของไฟล์ เช่นหน้าจอแต่ละหน้า (Expo Router หาหน้าจอจาก default export)

งานที่ต้องรอ: `async`, `await` และ `Promise`

การคุยกับอินเทอร์เน็ตใช้เวลา (อาจครั้งวินาทีหรือหลายวินาที) ถ้าแอปหยุดรอเฉย ๆ หน้าจอจะค้าง JavaScript จึงใช้ **Promise** คือ "ใบรับของ" ที่บอกว่าเดี๋ยวผลจะมา

async / await

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 async function save() {           // async = ฟังก์ชันนี้มีงานที่ต้องรอ
2   const tx = await repo.insert(input); // await = รอจนจับได้ผล แต่หน้าจอไม่ค้าง
3   setTxs((list) => [tx, ...list]); // ทำต่อเมื่อได้ผลแล้ว
4 }
```

ถ้ามีหลายงานที่ไม่ต้องรอกัน ใช้ `Promise.all` ทำพร้อมกัน เช่น ตอนโหลดข้อมูลครั้งแรก แอปโหลดโปรไฟล์ รายการ และกระปุกออม พร้อมกันทั้ง 3 อย่าง

src/data/AppProvider.tsx

บรรทัด 131–133

```
131   try {
132     // Goals never block the app: if they cannot be loaded, the rest still works.
133     const [p, list, g] = await Promise.all([r.getProfile(), r.listTransactions(),
      r.listGoals().catch(() => [] as SavingsGoal[])]);
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:131`

บรรทัด 133: เรียก 3 งานพร้อมกัน แล้วรอจนเสร็จทั้งหมด ผลออกมาเป็น array 3 ช่อง แะใส่ตัวแปร `p`, `list`, `g` ส่วน `.catch(() => [])` ถ้างานที่ 3 แปลว่า "ถ้าโหลดกระปุกออมไม่ได้ ให้ถือว่าไม่มีกระปุก" แอปส่วนอื่นยังใช้ได้ (Supabase ฝั่งเซิร์ฟเวอร์ ใน Edge Function ก็ใช้ `Promise.allSettled` อ่านสลิป 2 ครั้งพร้อมกันแบบเดียวกัน un A10)

จัดการความผิดพลาด: `try`, `catch`, `throw`

try / catch / finally

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 try {
2   await addTx(input);           // ลองทำ
3   toast({ message: 'บันทึกรายการแล้ว' });
4 } catch (e) {                   // ถ้ามี error ระหว่างทาง กระโดดมาที่นี่
5   toast({ message: 'บันทึกไม่สำเร็จ ลองอีกครั้ง', tone: 'error' });
6 } finally {                     // ทำเสมอ ไม่ว่าสำเร็จหรือพลาด
7   setBusy(false);
8 }
```

`throw` คือ "โยน error" ออกไปให้ผู้เรียกจัดการ ทีมเราสร้าง error ชนิดของตัวเองเพื่อแยกเหตุผล เช่น `DuplicateSlipError` (สลิปซ้ำ) ใน [src/data/repo.ts:14](#)

```

14 export class DuplicateSlipError extends Error {
15   constructor() {
16     super('รายการนี้มีอยู่แล้ว');
17     this.name = 'DuplicateSlipError';
18   }
19 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:14

`class ... extends Error` = สร้างชนิด error ใหม่โดยต่อยอดจาก Error ธรรมดา `super(...)` ส่งข้อความภาษาไทยให้ Error ตัวแม่ และตั้ง name ไว้ให้หน้าจอลูกได้ว่าเป็น error แบบไหน (ดู [src/app/transaction.tsx:126](#))

ทำไมเก็บเงินเป็นสตางค์ (จำนวนเต็ม)

คอมพิวเตอร์เก็บทศนิยมเป็นเลขฐานสอง ซึ่งเก็บบางค่าได้ไม่เป๊ะ ทดลองรันใน Node.js จริง:

```

0.1 + 0.2      → 0.30000000000000004
19.99 * 100    → 1998.9999999999998

```

ถ้าบวกเงินเป็นบาททศนิยมหลายพันรายการ เศษเล็ก ๆ จะสะสมจนยอดเพี้ยน ทางแก้คือเก็บเป็น **สตางค์ (จำนวนเต็ม)** ทั้งหมด เช่น ฿85.50 เก็บเป็น 8550 จำนวนเต็มบวกหลบกันได้แม่นยำ 100% แล้วค่อยแปลงเป็นบาทตอนแสดงผลบนจอ ฐานข้อมูลก็เก็บเป็นจำนวนเต็ม (คอลัมน์ `amount_satang bigint`)

อ่านโค้ดจริง: แปลงข้อความที่ผู้ใช้พิมพ์เป็นสตางค์

ฟังก์ชัน `parseBahtToSatang` ใน [src/domain/money.ts](#) ใช้ทุกที่ที่ผู้ใช้กรอกจำนวนเงิน (FR-1 จดรายการ, ตั้งค่า, เช็กร่อนจ่ายของ FR-6, กระทบยอด) และใช้กับยอดเงินที่ AI อ่านจากสลิปด้วย

```

1 /**
2  * Money helpers. All amounts are integers in satang.
3  */
4
5 /** The largest amount the app takes: ฿1,000,000,000 (the database caps goals at the same value).
6  */
7 export const MAX_SATANG = 100_000_000_000;
8
9 /** "๑๒๓" -> "123": Thai digits typed on a Thai keyboard. */
10 export function toAsciiDigits(s: string): string {
11   return s.replace(/[\u0E50-\u0E59]/g, (d) => String(d.charCodeAt(0) - 0x0e50));
12 }
13
14 /**
15  * Parse what a user (or a slip) wrote into satang.
16  * Accepts "1,234.50", "1234.5", "฿85", "85 บาท", " 1 234.00 ", Thai digits "๘๕", and a decimal
17  * comma as typed on phones set to such regions ("85,50" = 85.50, "12,34" = 12.34; never 1,234).
18  * Other commas must group thousands ("1,234.50"); anything else ("1,23,4") is refused.
19  * Returns null for anything that is not a positive amount with at most 2 decimals, or above
20  * MAX_SATANG. With `allowZero`, "0" and "0.00" give 0 (an opening balance may be zero).

```

```

21 export function parseBahtToSatang(input: string | number | null | undefined, opts: { allowZero?:
    boolean } = {}): number | null {
22   if (input === null || input === undefined) return null;
23   const ok = (satang: number) => (satang > MAX_SATANG ? null : satang > 0 || (opts.allowZero &&
    satang === 0) ? satang : null);
24   if (typeof input === 'number') {
25     if (!Number.isFinite(input) || input < 0) return null;
26     return ok(Math.round(input * 100));
27   }
28   let cleaned = toAsciiDigits(input)
29     .replace(/฿|บาท|THB|baht/gi, '')
30     .replace(/,/g, '');
31   if (/^d+,d{1,2}$/.test(cleaned)) cleaned = cleaned.replace(',', '.');
32   else if (cleaned.includes(',')) {
33     if (!/^d{1,3}(,d{3})+(\.d{1,2})?$/i.test(cleaned)) return null;
34     cleaned = cleaned.replace(/,/g, '');
35   }
36   if (!/^d+(\.d{0,2})?$/i.test(cleaned)) return null;
37   const [whole, frac = ''] = cleaned.split('.');
38   if (whole.length > 12) return null;
39   return ok(Number(whole) * 100 + Number((frac + '00').slice(0, 2)));
40 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/money.ts:1`

บรรทัด	ความหมาย
5-6	ค่าคงที่ MAX_SATANG = 100,000,000,000 สตางค์ = ฿1,000,000,000 เป็นจำนวนเงินสูงสุดที่แอปรับ (ฐานข้อมูลตั้งเพดานเดียวกัน กับค่าที่อ่านผิดใหญ่เกินจริง)
8-11	toAsciiDigits แปลงเลขไทย "๑๒๓" เป็น "123" คนที่ใช้แป้นพิมพ์ไทยอาจพิมพ์เลขไทยมา ทำงานโดยหาตัวอักษรในช่วงเลขไทย (รหัส U+0E50 ถึง U+0E59) แล้วลบด้วยรหัสของ "๐" (0x0E50) จะได้ 0-9
13-20	คอมเมนต์บอกว่ารับรูปแบบไหนได้บ้าง: "1,234.50", "1234.5", "฿85", "85 บาท", เลขไทย และทศนิยมแบบคอมมาที่มีมือถือบางประเทศใช้ ("85,50" = 85.50) แต่ "1,23,4" ไม่รับ
21	รับค่าเป็นข้อความหรือตัวเลขก็ได้ และมีตัวเลือก allowZero (ปกติ 0 ไม่ผ่าน เพราะรายการเงิน 0 บาทไม่มีความหมาย แต่ยอดเงินตั้งต้นเป็น 0 ได้) คืนค่าเป็นสตางค์ หรือ null ถ้าไม่ใช่จำนวนเงินที่ถูกต้อง
22	ไม่มีค่าเลย คืน null
23	ฟังก์ชันย่อย ok ตรวจสอบสุดท้าย: เกินเพดานไม่ผ่าน, มากกว่า 0 ผ่าน, เท่ากับ 0 ผ่านเฉพาะเมื่อ allowZero
24-27	ถ้าส่งเข้ามาเป็นตัวเลขอยู่แล้ว ต้องเป็นตัวเลขจริงและไม่ติดลบ แล้วคูณ 100 ปัดเป็นจำนวนเต็ม (ปัดเพื่อแก้ปัญหา $19.99 \times 100 = 1998.999\dots$)
28-30	ถ้าเป็นข้อความ: แปลงเลขไทย ลบสัญลักษณ์ ฿, คำว่า "บาท", THB, baht (ไม่สนตัวพิมพ์เล็กใหญ่) และลบช่องว่างทั้งหมด
31	ถ้ารูปแบบคือ "ตัวเลข,ตัวเลข 1-2 หลัก" เช่น "85,50" ถือว่าคอมมาคือจุดทศนิยม
32-35	ถ้ามีคอมมาแบบอื่น ต้องเป็นคอมมาคั่นหลักพันที่ถูกต้อง (เช่น 1,234 หรือ 1,234.50) ถึงจะลบคอมมาออก ไม่จิ้นคืน null
36	สุดท้ายต้องเหลือแค่ตัวเลข กับทศนิยมไม่เกิน 2 ตำแหน่ง ไม่จิ้นคืน null (เช่น "85.555" หรือ "abc")
37	แยกส่วนจำนวนเต็มกับทศนิยมที่จุด ถ้าไม่มีทศนิยมให้ frac เป็นข้อความว่าง

บรรทัด	ความหมาย
38	เกิน 12 หลักไม่รับ (กันตัวเลขยาวผิดปกติ)
39	คำนวณสตางค์ด้วยจำนวนเต็มล้วน: บาท $\times 100$ + สตางค์ (เติม "00" แล้วตัด 2 ตัว ทำให้ ".5" กลายเป็น 50 สตางค์) ไม่มีการคูณทศนิยมเลย จึงไม่เพี้ยน

ผลจากการรับฟังค์ชันจริง

ผู้ใช้พิมพ์	ได้สตางค์	เหตุผล
85	8500	-
85.5	8550	".5" $\rightarrow$ 50 สตางค์
1,234.50	123450	คอมมาคั่นหลักพันถูกต้อง
฿85 / 85 บาท / ๘๕	8500	ลบสัญลักษณ์และแปลงเลขไทย
85,50	8550	คอมมาเป็นทศนิยม
1,23,4	null	คอมมาผิดรูปแบบ
0	null	ต้องมากกว่า 0 (เว้นแต่ allowZero)
85.555	null	ทศนิยมเกิน 2 ตำแหน่ง
-5	null	มีเครื่องหมายลบ ไม่ใช่ตัวเลขล้วน
1000000000.01	null	เกินเพดาน ฿1,000,000,000

Regular expression (regex) อ่านยังงี้

บรรทัด 10 และ 29–36 มีของแปลก ๆ ใน / / เรียกว่า **regular expression** คือ "แพตเทิร์นสำหรับค้นหาข้อความ" ไม่ต้องจำทั้งหมด แค่อ่านออก

สัญลักษณ์	แปลว่า
\d	ตัวเลข 0–9 หนึ่งตัว
\s	ช่องว่าง (space, tab, ขึ้นบรรทัด)
+	ตัวข้างหน้ามี 1 ตัวขึ้นไป (\d+ = ตัวเลขที่หลักก็ได้)
{1,3}	ตัวข้างหน้ามี 1 ถึง 3 ตัว
?	ตัวข้างหน้ามีหรือไม่มีก็ได้
^ และ \$	ต้นข้อความ และท้ายข้อความ (ต้องตรงทั้งข้อความ ไม่ใช่แค่บางส่วน)
()	จับกลุ่ม
a b	a หรือ b
[๐-๙]	ตัวอักษร 1 ตัวในช่วงเลขไทย ๐–๙

สัญลักษณ์	แปลว่า
g, i (ท้าย /)	g = หากจุด, i = ไม่สนตัวพิมพ์เล็กใหญ่

ตัวอย่าง `/^\d{1,3}(\,\d{3})+(\.\d{1,2})?$/` อ่านว่า: "ขึ้นต้นด้วยตัวเลข 1-3 หลัก ตามด้วย (คอมมา + ตัวเลข 3 หลัก) อย่างน้อย 1 ชุด แล้วอาจมี (จุด + ตัวเลข 1-2 หลัก) และจบข้อความ" ซึ่งก็คือรูปแบบ 1,234 หรือ 12,345,678.50

แปลงสตริงกลับเป็นข้อความบาท

src/domain/money.ts

บรรทัด 42-63

```

42 /** 123450 -> "1,234.50" */
43 export function formatSatang(satang: number, opts: { decimals?: boolean } = {}): string {
44   const decimals = opts.decimals ?? true;
45   const negative = satang < 0;
46   const abs = Math.abs(Math.round(satang));
47   // Whole baht only: round to the nearest baht (#99.99 reads #100, not #99).
48   const whole = decimals ? Math.floor(abs / 100) : Math.round(abs / 100);
49   const frac = abs % 100;
50   const grouped = String(whole).replace(/\B(?=(\d{3})+(?!\d))/g, ',');
51   const body = decimals ? `${grouped}.${String(frac).padStart(2, '0')}` : grouped;
52   return negative ? `-${body}` : body;
53 }
54
55 /** 123450 -> "฿1,234.50"; with sign: "+฿1,234.50" / "-฿1,234.50" */
56 export function formatBaht(
57   satang: number,
58   opts: { decimals?: boolean; sign?: boolean } = {},
59 ): string {
60   const body = formatSatang(Math.abs(satang), opts);
61   if (opts.sign) return `${satang < 0 ? '-' : '+'}฿${body}`;
62   return `${satang < 0 ? '-' : ''}฿${body}`;
63 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/money.ts:42

บรรทัด	ความหมาย
42-43	formatSatang แปลง 123450 เป็น "1,234.50"
44	ตัวเลือก decimals ถ้าไม่ส่งมาใช้ค่าเริ่มต้น true (แสดงสตางค์)
45-46	จำไว้ว่าติดลบไหม แล้วทำงานกับค่าบวกที่ปิดเป็นจำนวนเต็มแล้ว
47-48	ส่วนบาท: ถ้าแสดงสตางค์ด้วย ใช้ปัดลง (สตางค์จะแสดงแยก) ถ้าไม่แสดงสตางค์ ปิดตามปกติ #99.99 จึงแสดงเป็น #100 ไม่ใช่ #99
49	สตางค์คือเศษจากการหาร 100
50	ใส่คอมมาคั่นหลักพันด้วย regex (หาตำแหน่งที่ด้านหลัมีตัวเลขเป็นชุดละ 3 ตัวพอดี)
51-52	ต่อสตางค์ 2 หลัก (5 สตางค์ → "05") และเติมเครื่องหมายลบถ้าติดลบ
55-63	formatBaht เพิ่ม "฿" ข้างหน้า และถ้าขอเครื่องหมาย (sign) จะใส่ + หรือ - เช่น formatBaht(-8500, {sign: true}) → "-฿85.00"

i ผลจริง

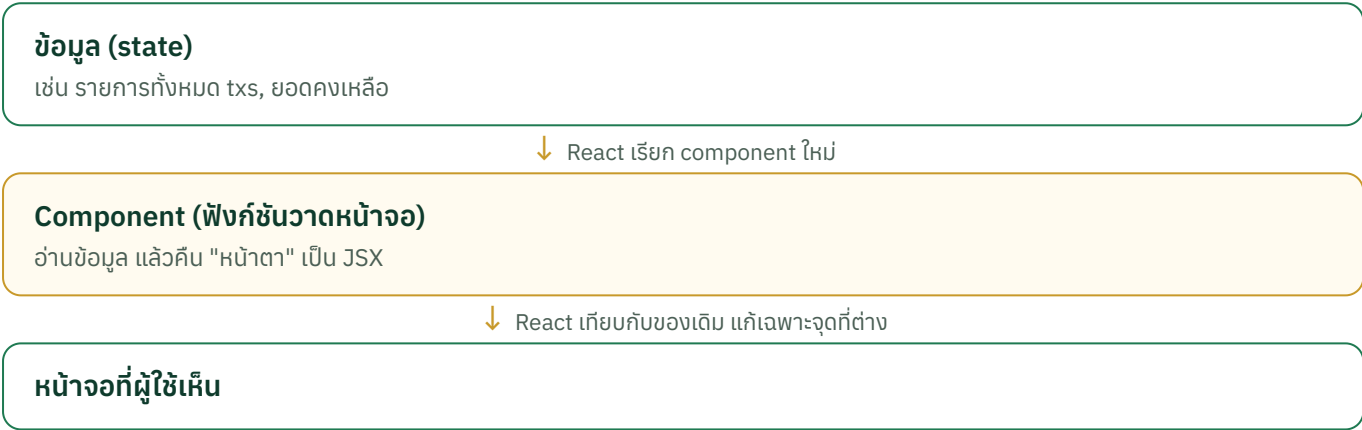
```
formatBaht(123450) → "฿1,234.50" · formatBaht(-8500, { sign: true }) → "-฿85.00" ·  
formatBaht(9999, { decimals: false }) → "฿100"
```

React: วาดหน้าจจากข้อมูล

React คือวิธีคิดที่ว่า "หน้าจ = ฟังก์ชันของข้อมูล" ข้อมูลเปลี่ยนเมื่อไร หน้าจวาดใหม่เอง ไม่ต้องสั่งแก้ทีละจุด

แนวคิดหลัก: หน้าจเป็นผลลัพธ์ของข้อมูล

สมัยก่อน ถ้ายอดเงินเปลี่ยน โปรแกรมเมอร์ต้องเขียนเองว่า "ไปหาข้อความยอดเงิน แก่ตัวเลข ไปหาต้นไม้ เปลี่ยนจำนวนใบ ไปหาป้ายสถานะ เปลี่ยนสี..." พลาดจุดเดียวหน้าจก็ขัดกันเอง React เปลี่ยนวิธีคิด: เราเขียนแค่ว่า "ถ้าข้อมูลเป็นแบบนี้ หน้าจหน้าตาแบบนี้" แล้ว React จะวาดใหม่ให้เองทุกครั้งที่ข้อมูลเปลี่ยน



Component: ชิ้นส่วนหน้าจที่เป็นฟังก์ชัน

Component คือฟังก์ชันที่ชื่อขึ้นต้นด้วยตัวใหญ่ และคืนค่าเป็น **JSX** (หน้าตาที่เขียนคล้าย HTML อยู่ในโค้ด) หน้าจทั้งหน้าก็เป็น component และปุ่มเล็ก ๆ ก็เป็น component ซ้อนกันเป็นชั้น ๆ

Component เล็กที่สุด

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 function Hello({ name }: { name: string }) {
2   return <Text>สวัสดี {name}</Text>;
3 }
4
5 // เรียกใช้ที่อื่น:
6 <Hello name="มินท์" />    // แสดง "สวัสดี มินท์"
```

ส่วน	ความหมาย
<Text>...</Text>	JSX แท็กเปิดและปิด เหมือน HTML (Text คือชิ้นส่วนแสดงข้อความของ React Native)
{name}	วงเล็บปีกกาใน JSX = "ใส่ค่าจากโค้ด JavaScript ตรงนี้"
{ name }: { name: string }	props คือค่าที่ผู้เรียกส่งเข้ามา (เหมือนพารามิเตอร์) ในที่นี้แกะเอาช่อง name ออกมา
<Hello name="มินท์" />	ใช้ component เหมือนแท็ก ส่ง props เป็น attribute แท็กที่ไม่มีของข้างในปิดด้วย />

ตัวอย่างจริง: T ชิ้นส่วนตัวหนังสือที่ทุกหน้าใช้

ทุกข้อความในแอปใช้ <T> แทนการตั้งขนาดและสีเอง เพื่อให้ทั้งแอปหน้าตาเหมือนกัน (กฎใน Design System un A11)

```

36 type Variant = 'display' | 'h1' | 'h2' | 'h3' | 'body' | 'small' | 'micro' | 'label';
37
38 const VARIANT_FONT: Record<Variant, string> = {
39   display: fonts.serif,
40   h1: fonts.serif,
41   h2: fonts.sansBold,
42   h3: fonts.sansSemi,
43   body: fonts.sans,
44   small: fonts.sans,
45   micro: fonts.sansMedium,
46   label: fonts.sansSemi,
47 };
48
49 export function T({
50   v = 'body',
51   color,
52   style,
53   children,
54   numberOfLines,
55   center,
56   selectable,
57   accessibilityRole,
58 }: {
59   v?: Variant;
60   color?: string;
61   style?: StyleProp<TextStyle>;
62   children: ReactNode;
63   numberOfLines?: number;
64   center?: boolean;
65   selectable?: boolean;
66   accessibilityRole?: 'header' | 'text' | 'alert';
67 }): {
68   const theme = useTheme();
69   const size = v === 'label' ? type.micro : type[v];
70   return (
71     <Text
72       accessibilityRole={accessibilityRole}
73       numberOfLines={numberOfLines}
74       selectable={selectable}
75       style={[
76         size,
77         {
78           fontFamily: VARIANT_FONT[v],
79           color: color ?? (v === 'small' || v === 'micro' ? theme.inkSoft : theme.ink),
80           textAlign: center ? 'center' : undefined,
81         },
82         // Labels are text: inkSoft (inkFaint is for placeholders, rules and quiet icons, below
4.5:1).
83         v === 'label' && { letterSpacing: 0.8, textTransform: 'uppercase', color: color ??
theme.inkSoft },
84         style,
85       ]}
86     >
87       {children}
88     </Text>
89   );

```

บรรทัด	ความหมาย
36	ระดับตัวหนังสือมี 8 แบบ ตั้งแต่ใหญ่สุด display (ยอดเงินบนการ์ด) ไปจนเล็กสุด micro
38–47	ตารางจับคู่ว่าแต่ละระดับใช้ฟอนต์อะไร: หัวข้อใหญ่ใช้ Noto Serif Thai (มีหัว) ที่เหลือใช้ Anuphan น้ำหนักต่างกัน Record<Variant, string> บังคับให้ต้องใส่ครบทั้ง 8 ระดับ
49–57	ประกาศ component ชื่อ T พร้อมแกะ props ออกมา <code>v = 'body'</code> แปลว่าถ้าไม่ระบุระดับ ใช้ body
58–67	ชนิดของ props แต่ละตัว (เครื่องหมาย ? = ไม่บังคับ) children คือสิ่งที่อยู่ระหว่าง <T> กับ </T> ส่วน accessibilityRole บอกโปรแกรมอ่านหน้าจอสำหรับคนตาบอดว่าเป็นหัวข้อหรือข้อความแจ้งเตือน
68	เรียก <code>useTheme()</code> เอาชุดสีของธีมที่ผู้ใช้เลือก (สว่าง/มืด, ป่าทอง/ม่วง ฯลฯ)
69	เลือกขนาดและความสูงบรรทัดจากตาราง type (label ใช้ขนาดเท่า micro)
70–86	คืน JSX: ชิ้นส่วน Text ของ React Native พร้อม style ที่ซ้อนกันเป็นชั้น (array ของ style ตัวหลังทับตัวหน้า)
78	ฟอนต์ตามระดับ
79	สี: ถ้าผู้เรียกส่งสีมาใช้สีนั้น ไม่จันข้อความเล็กใช้สีรอง (inkSoft) ข้อความอื่นใช้สีหลัก (ink)
80	จัดกึ่งกลางถ้าส่ง center
82–83	ระดับ label (หัวข้อเล็กตัวพิมพ์ใหญ่ เช่น "MONEY RUNWAY") ห่างตัวอักษรกว้างขึ้น และใช้สีที่ผ่านเกณฑ์อ่านง่าย
84	style ที่ผู้เรียกส่งมาเองใส่ท้ายสุด จึงทับค่าทั้งหมดได้
87	แสดงสิ่งที่อยู่ข้างใน (ข้อความ)

ใช้งานแบบนี้: `<T v="h3">จัดรายการ</T>` หรือ `<T v="small" color={theme.watch}>อ่านวันที่ไม่ชัด</T>`

State: ข้อมูลที่เปลี่ยนได้ และทำให้หน้าจอลoadใหม่

ตัวแปรธรรมดาในฟังก์ชันจะหายไปทุกครั้งที React วาดใหม่ ถ้าอยากให้ component "จำ" ค่าได้ ต้องใช้ **state** ผ่าน hook ชื่อ `useState`

useState

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const [amount, setAmount] = useState(''); // ค่าเริ่มต้น = ข้อความว่าง
2 // amount    = ค่าปัจจุบัน
3 // setAmount = ฟังก์ชันเปลี่ยนค่า เรียกแล้ว React วาดหน้าจอใหม่
4
5 <AmountField value={amount} onChangeText={setAmount} />
6 // ผู้ใช้พิมพ์ "8" → setAmount('8') → วาดใหม่ ช่องแสดง 8
```

หน้าจอดรายการ (FR-1) ใช้ state จำค่าทุกช่องในฟอร์ม ดูจากโค้ดจริง:

src/app/transaction.tsx

บรรทัด 35–46

```
35 const startKind: TxKind = existing?.kind ?? (kindParam === 'income' ? 'income' : 'expense');
```

```

36 const [kind, setKind] = useState<TxKind>(startKind);
37 const [amount, setAmount] = useState(existing ? satangToInput(existing.amountSatang) : '');
38 const [title, setTitle] = useState(existing?.title ?? '');
39 const [category, setCategory] = useState(existing?.categoryKey ?? defaultCategory(startKind));
40 const [categoryTouched, setCategoryTouched] = useState(!existing);
41 const [day, setDay] = useState(existing ? bkkDayKey(existing.occurredAt) : bkkDayKey(new
Date()));
42 const [time, setTime] = useState(existing ? bkkTime(existing.occurredAt) : bkkTime(new
Date()));
43 const [note, setNote] = useState(existing?.note ?? '');
44 const [busy, setBusy] = useState(false);
45 const [askDelete, setAskDelete] = useState(false);
46 const [amountError, setAmountError] = useState<string | null>(null);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:35

บรรทัด	ความหมาย
35	ประเภทเริ่มต้น: ถ้าแก้ไขรายการเดิม ใช้ประเภทเดิม ไม่จําเป็นดูว่าเปิดมาจากปุ่ม "เพิ่มเงินเข้า" ใหม่ (เป็นรายรับ) ไม่จําเป็นรายจ่าย
36	state ประเภท (รายรับ/รายจ่าย) useState<TxKind> บอกว่าเก็บได้แค่ 'income' หรือ 'expense'
37	state ช่องจํานวนเงิน เก็บเป็น ข้อความ ตามที่ผู้ใช้พิมพ์ (แปลงเป็นสตริงตอนกดบันทึก) ถ้าแก้ไขรายการเดิม เติมค่าเดิมให้
38–39	ชื่อรายการ และหมวด
40	จํานวนที่ผู้ใช้เลือกหมวดเองแล้วหรือยัง (ถ้ายัง แอปจะเดาหมวดจากชื่อที่พิมพ์ให้)
41–42	วันและเวลา (ค่าเริ่มต้นคือตอนนี้ ตามเวลาไทย)
43	บันทึกเพิ่มเติม
44–46	สถานะของหน้าจอเอง: กำลังบันทึกอยู่ไหม, กำลังถามยืนยันการลบไหม, ข้อความ error ใต้ช่องเงิน

! กฎของ Hook

ฟังก์ชันที่ขึ้นต้นด้วย use เรียกว่า **hook** มีกฎ 2 ข้อ: (1) เรียกได้เฉพาะใน component หรือใน hook อื่น (2) ต้องเรียกที่ระดับบนสุดของ component ห้ามอยู่ใน if หรือ loop เพราะ React จํา state ตามลำดับการเรียก ESLint ของโปรเจกต์ตรวจกฎนี้ให้อัตโนมัติ

Hook ที่ใช้ในโปรเจกต์

Hook	ทำอะไร	ตัวอย่างใน MindPay
useState	จําค่าที่เปลี่ยนได้ เปลี่ยนแล้ววาดใหม่	ทุกช่องในฟอร์ม, รายการทั้งหมดใน AppProvider
useEffect	ทำงาน "ข้างเคียง" หลังวาดเสร็จ เช่น ตั้งเวลา ฟัง event โหลดข้อมูล และเก็บกวาดเมื่อเลิกใช้	ตั้งนาฬิกาให้วันเปลี่ยนตอนเที่ยงคืน
useMemo	คำนวณแล้วจําผลไว้ คำนวณใหม่เฉพาะเมื่อข้อมูลที่ระบุเปลี่ยน	useMoney () คำนวณยอดเงินและ Runway

Hook	ทำอะไร	ตัวอย่างใน MindPay
useCallback	จำ "ฟังก์ชัน" ไว้ไม่ให้สร้างใหม่ทุกครั้ง	loadAll, activateRepo
useRef	กล่องจำค่าที่เปลี่ยนแล้ว ไม่ ต้องวาดใหม่	จำว่ากำลังใช้บัญชีไหนอยู่ (activeUserRef)
useContext	หยิบข้อมูลที่ "แชร์ทั้งแอป" มาใช้	useApp()
hook ของเรา	รวม hook หลายตัวเป็นชิ้นเดียว ตั้งชื่อขึ้นต้นด้วย use	useMoney, useTheme, useToast

ตัวอย่าง useEffect: วันใหม่เริ่มตอนเที่ยงคืนตามเวลาไทย

src/data/AppProvider.tsx

บรรทัด 182–212

```

182 // A new day starts at 00:00 Bangkok time: while the app is open (timer) or
183 // when it comes back to the screen the next morning (AppState / tab focus).
184 useEffect(() => {
185   let timer: ReturnType<typeof setTimeout>;
186   const check = () => {
187     const now = bkkDayKey(Date.now());
188     if (now !== todayRef.current) {
189       todayRef.current = now;
190       setToday(now);
191       setNewDay(true);
192       Storage.setItem(LAST_DAY_KEY, now).catch(() => {});
193     }
194     clearTimeout(timer);
195     timer = setTimeout(check, msUntilNextBkkMidnight() + 1000);
196   };
197   check();
198   // First open on a different day from the last one.
199   Storage.getItem(LAST_DAY_KEY)
200     .then((last) => {
201       if (last && last !== todayRef.current) setNewDay(true);
202       return Storage.setItem(LAST_DAY_KEY, todayRef.current);
203     })
204     .catch(() => {});
205   const sub = AppState.addEventListener('change', (s) => {
206     if (s === 'active') check();
207   });
208   return () => {
209     clearTimeout(timer);
210     sub.remove();
211   };
212 }, []);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:182

บรรทัด	ความหมาย
182–183	คอมเมนต์: วันใหม่เริ่ม 00:00 เวลากรุงเทพ ทั้งตอนเปิดแอปค้างไว้ (ใช้นาฬิกาจับเวลา) และตอนกลับมาเปิดแอปตอนเช้า
184	useEffect(() => {...}, []) array วางท้ายสุดแปลว่า "ทำครั้งเดียวตอน component เริ่มทำงาน"
185	ตัวแปรเก็บนาฬิกาจับเวลา

บรรทัด	ความหมาย
186–196	ฟังก์ชัน check: ดูวันที่ตอนนี้ตามเวลาไทย ถ้าไม่ตรงกับ "วันนี้" ที่จำไว้ แปลว่าข้ามวันแล้ว ให้อัปเดตวันนี้ (setToday) ซึ่งทำให้ตัวเลขทุกอย่างที่ขึ้นกับวันนี้คำนวณใหม่ (เช่น ค่าเฉลี่ย 7 วันของ FR-6)
191	ตั้งธง "วันใหม่" ให้น้องกลั๊กทายและสรุปเมื่อวาน
192	จดวันที่ล่าสุดลงเครื่อง
194–195	ยกเลิกลูกาพิคาเดิม แล้วตั้งใหม่ให้ปลุกตอนเที่ยงคืนครั้งหน้า (+1 วินาทีเพื่อ) msUntilNextBkkMidnight คำนวณว่าเหลืออีกกี่มิลิวินาที
197	เช็กกันทีหนึ่งครั้งตอนเริ่ม
198–204	อ่านวันที่เปิดแอปครั้งล่าสุดจากเครื่อง ถ้าไม่ใช่วันนี้ แปลว่าเป็นการเปิดครั้งแรกของวัน
205–207	ฟังเหตุการณ์ "แอปกลับมาอยู่หน้าจอ" (เช่น เปิดแอปค้างไว้ข้ามคืน แล้วเข้ามาสลับกลับมา) แล้วเช็กวันใหม่อีกครั้ง
208–211	ฟังก์ชันเก็บกวาด (cleanup): ถ้า component นี้ถูกปิด ให้ยกเลิกนาฬิกาและเลิกฟังเหตุการณ์ ไม่ให้ทำงานค้างกินแบตเตอรี่

Context: แชรข้อมูลให้ทุกหน้าโดยไม่ต้องส่งต่อเป็นทอด ๆ

ถ้าหน้าหลัก หน้ารายการ และหน้าเงินพอถึง ต้องใช้รายการเงินชุดเดียวกัน จะส่ง props ต่อกันเป็นทอด ๆ ก็ยุ่ง React มี **Context** เหมือน "ประกาศเสียงตามสาย" ตัวให้ข้อมูล (Provider) อยู่บนสุด ใครอยู่ข้างในจะหยิบไปใช้ได้เลย MindPay มี Provider หลายชั้นครอบคลุมทั้งแอปไว้ใน `src/app/_layout.tsx` (un A7) ตัวสำคัญที่สุดคือ **AppProvider**

`src/data/AppProvider.tsx`

บรรทัด 87–121

```

87 const AppContext = createContext<AppContextValue | null>(null);
88
89 export function useApp() {
90   const ctx = useContext(AppContext);
91   if (!ctx) throw new Error('useApp must be used inside AppProvider');
92   return ctx;
93 }
94
95 /** Same as useApp, but null outside the provider (for small shared pieces such as the
96     companion). */
97 export function useAppMaybe() {
98   return useContext(AppContext);
99 }
100 export function AppProvider({ children }: { children: ReactNode }) {
101   const [status, setStatus] = useState<AuthStatus>('loading');
102   const [repo, setRepo] = useState<Repo | null>(null);
103   const [userId, setUserId] = useState<string | null>(null);
104   const [profile, setProfile] = useState<Profile | null>(null);
105   const [txs, setTxs] = useState<Transaction[]>([]);
106   const [carrySatang, setCarry] = useState(0);
107   const [goals, setGoals] = useState<SavingsGoal[]>([]);
108   const [loadError, setLoadError] = useState<string | null>(null);
109   const [refreshing, setRefreshing] = useState(false);
110   const [authNotice, setAuthNotice] = useState<AuthNotice | null>(null);
111   const [today, setToday] = useState(() => bkkDayKey(Date.now()));
112   const [newDay, setNewDay] = useState(false);

```

```

113 const todayRef = useRef(today);
114 const repoRef = useRef<Repo | null>(null);
115 /** The account whose data is loaded (or being loaded): prevents loading the same account
    twice. */
116 const activeUserRef = useRef<string | null>(null);
117 const handledLinks = useRef(new Set<string>());
118 /** Each load gets a number; an older load that finishes late is ignored (it may be another
    account's). */
119 const loadGen = useRef(0);
120 /** Changes made on this phone while a load is running (null = deleted), put back over what it
    returns. */
121 const localChanges = useRef(new Map<string, Transaction | null>());

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:87

บรรทัด	ความหมาย
87	สร้าง Context เปล่า ๆ ไว้ (เริ่มต้นเป็น null)
89–93	useApp(): hook ที่ทุกหน้าใช้หับข้อมูลกลาง ถ้าเรียกนอก Provider จะ error กันที (กันลืมหวน)
95–98	แบบที่ไม่ error แต่คืน null สำหรับชิ้นส่วนเล็กที่อาจอยู่นอก Provider
100	ประกาศ component AppProvider ซึ่งครอบหน้าจอทั้งหมด (children)
101	สถานะการเข้าสู่ระบบ: กำลังโหลด / ยังไม่เข้า / พร้อม
102–103	Repo ที่ใช้อยู่ (คลาวด์หรือเครื่อง) และ id ของผู้ใช้
104–107	ข้อมูลหลักของแอป: โปรไฟล์, รายการทั้งหมด, ยอดยกมาจากรายการเก่ามาก, กระปุกลูก
108–110	ข้อความ error ตอนโหลด, คำสั่งรีเฟรชใหม่, ข้อความแจ้งเรื่องบัญชี (เช่น "สมัครสำเร็จ")
111	วันนี้ตามเวลาไทย (ฟังก์ชันใน useState ทำงานครั้งเดียวตอนเริ่ม)
112	รวันวันใหม่
113–121	คำที่ใช้ useRef เพราะไม่ต้องวาดหน้าจอใหม่เมื่อเปลี่ยน: วันนี้ (ไว้ใช้ในนาฬิกา), Repo ปัจจุบัน, บัญชีที่กำลังโหลด, ลิงก์อีเมลที่จัดการแล้ว, เลขรอบการโหลด และการเปลี่ยนแปลงในเครื่องระหว่างโหลด

ส่วนท้ายของ AppProvider (บรรทัด 325–439) รวมข้อมูลและฟังก์ชันทั้งหมด (เช่น addTx, removeTx, saveProfile) เป็น object เดียว แล้วส่งให้ทุกหน้าผ่าน <AppContext.Provider value={value}> ที่บรรทัด 439

ทุกอย่างประกอบกัน: หน้าจอใช้ข้อมูลกลางยังไง

รูปแบบที่ทุกหน้าใช้

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 export default function SomeScreen() {
2   const theme = useTheme();           // สีของธีม
3   const { txs, addTx } = useApp();     // ข้อมูลและคำสั่งกลาง
4   const { balance, runway } = useMoney(); // ตัวเลขที่คำนวณแล้ว
5   const [open, setOpen] = useState(false); // state ของหน้านี้เอง
6
7   return (
8     <Screen>

```

```

9      <T v="h1">ยอด {formatBaht(balance)}</T>
10      {runway.days !== null ? <T>เงินพอใช้ซัก {runway.days} วัน</T> : null}
11    </Screen>
12  );
13 }

```

บรรทัด `{runway.days !== null ? ... : null}` คือ **การแสดงผลแบบมีเงื่อนไข** ใน JSX: ถ้ามีจำนวนวัน แสดงข้อความ ถ้าไม่มี (null) ไม่แสดงอะไร จะเจอรูปแบบนี้ทุกหน้า อีกแบบที่เจอบ่อยคือ `{list.map((x) => <Row key={x.id} ... />)}` แสดงรายการหลายแถวจาก array (ต้องใส่ key ที่ไม่ซ้ำ ให้ React รู้ว่าแถวไหนเป็นแถวไหน)

★ สรุปบท A6 ใน 4 ประโยค

(1) Component คือฟังก์ชันที่คืนหน้าตา (JSX) (2) props คือของที่ส่งเข้า component, state คือความจำของ component (3) เปลี่ยน state ด้วย `setXxx` แล้ว React วาดใหม่เอง (4) ข้อมูลที่ทั้งแอปใช้ร่วมกันอยู่ใน `AppProvider` หยิบใช้ด้วย `useApp()` และตัวเลขเงินหยิบด้วย `useMoney()`

React Native และ Expo: โค้ดชุดเดียว ได้ทั้งแอปและเว็บ

ทำไมทีมเราเขียนโค้ดครั้งเดียว แล้วได้ทั้งแอป Android, iPhone และเว็บ และการเปลี่ยนหน้าจอด้วย Expo Router ทำงานยังไ้

React Native คืออะไร

React (un A6) เดิมใช้ทำเว็บ ซึ่งวาดหน้าจอด้วย HTML แต่แอปมือถือไม่มี HTML **React Native** จึงเปลี่ยน "ชิ้นส่วนพื้นฐาน" ให้เป็นของจริงบนมือถือ เราเขียน `<View>` แล้วบน Android กลายเป็นกล่องของ Android บน iPhone กลายเป็นกล่องของ iOS แอปจึงสั้นเหมือนแอปที่เขียนด้วยภาษาของมือถือโดยตรง

สำหรับเว็บ มีแพ็คเกจ **react-native-web** (bussกั 37 ใน package.json) แปลชิ้นส่วนเดียวกันเป็น HTML เช่น `<View>` → `<div>` `<Text>` → `<span>` โค้ดชุดเดียวจึงได้ 3 แพลตฟอร์ม

ชิ้นส่วน React Native	คล้ายกับ (เว็บ)	ใช้ทำอะไร
View	div	กล่องสำหรับจัดวาง (การ์ด แถว คอลัมน์)
Text	span / p	ข้อความ (ข้อความทุกตัวต้องอยู่ใน Text)
Pressable	button	พื้นที่ที่แตะได้ มี onPress, onPressIn, onPressOut
ScrollView / FlatList / SectionList	หน้าที่เลื่อนได้	เลื่อนเนื้อหายาว ๆ (FlatList/SectionList เหมาะกับรายการยาวมาก เพราะวาดเฉพาะแถวที่เห็น)
TextInput	input	ช่องพิมพ์ (ห่ออยู่ใน Field / AmountField ของเรา)
Image	img	รูปภาพ
SafeAreaView	-	ไม่ให้เนื้อหาไปซ้อนกับรอยบาก แถบสถานะ หรือขอบล่างของมือถือ
Animated.View	-	กล่องที่ขยับได้ (แอนิเมชัน)

การจัดวางด้วย style

React Native ไม่มีไฟล์ CSS แต่ใส่ `style={{ ... }}` เป็น object ตรง ๆ ชื่อเหมือน CSS แต่เป็น camelCase เช่น `backgroundColor` หน่วยเป็นตัวเลขเฉย ๆ (หมายถึง "จุด" ที่ปรับตามความละเอียดจอเอง) การจัดวางใช้ **Flexbox**:

style	ความหมาย
<code>flexDirection: 'row'</code>	เรียงลูกในแนวนอน (ค่าเริ่มต้นคือแนวตั้ง 'column')
<code>flex: 1</code>	ขยายเต็มพื้นที่ที่เหลือ
<code>alignItems: 'center'</code>	จัดลูกให้อยู่กลาง (แกนขวาง)
<code>justifyContent: 'space-between'</code>	กระจายลูกให้ชิดหัวท้าย (แกนหลัก)

style	ความหมาย
gap: 8	ระยะห่างระหว่างลูกแต่ละตัว
padding / margin	ระยะห่างด้านใน / ด้านนอก
borderRadius	มุมโค้ง

ตัวอย่างจากชิ้นส่วน Row ที่ใช้ทั้งแอป: บรรทัดเดียวจบ

src/ui/components.tsx

บรรทัด 189–203

```

189 export function Row({
190   children,
191   gap = space.sm,
192   style,
193   align = 'center',
194   justify,
195 }: {
196   children: ReactNode;
197   gap?: number;
198   style?: StyleProp<ViewStyle>;
199   align?: ViewStyle['alignItems'];
200   justify?: ViewStyle['justifyContent'];
201 }) {
202   return <View style={[{ flexDirection: 'row', alignItems: align, justifyContent: justify, gap },
203     style] >{children}</View>;
204 }

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/ui/components.tsx:189

บรรทัด 202: สร้าง View ที่เรียงลูกแนวนอน (row) จัดกลางแนวตั้ง และเว้นระยะ gap ค่าเริ่มต้น 8 แล้วต่อด้วย style ที่ผู้เรียกส่งมา

Expo: ชุดเครื่องมือที่ทำให้ React Native ง่ายขึ้น

React Native เปล่า ๆ ต้องติดตั้ง Android Studio, Xcode และตั้งค่ามากมาย **Expo** ห่อทุกอย่างไว้ และมีโมดูลสำเร็จรูป (expo-camera, expo-speech ฯลฯ) ที่ใช้ได้เลย สิ่งที่ Expo ให้กับเรา:

- **Expo Go + dev server:** แก้โค้ดแล้วเห็นผลบนมือถือภายในไม่กี่วินาที
- **Expo Router:** ระบบเปลี่ยนหน้าจอจากโครงสร้างไฟล์ (หวัข้อถัดไป)
- **expo export:** ทำเว็บเวอร์ชันเป็นไฟล์นิ่ง ๆ เอาไปวางบนเว็บไซต์ได้
- **prebuild + EAS Build:** สร้างโปรเจกต์ Android/iOS และไฟล์ APK
- **EAS Update:** ส่งโค้ดใหม่ไปถึงแอปที่ลงแล้วโดยไม่ต้องลง APK ใหม่

! Expo ที่เราใช้เวอร์ชัน 57

Expo เปลี่ยนเร็วมาก ทุกเวอร์ชันมีคำสั่งที่เปลี่ยนชื่อหรือย้ายที่ ไฟล์ AGENTS.md ในโปรเจกต์จึงเตือนว่าก่อนใช้ API ของ Expo ให้เปิดเอกสารของเวอร์ชัน 57 (docs.expo.dev/versions/v57.0.0/) ไม่ใช่จากที่เคยเห็น

Expo Router: ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า

ใน src/app/ ทุกไฟล์คือหน้าจอ ชื่อไฟล์คือที่อยู่ (URL) ของหน้านั้น บนเว็บพิมพ์ที่อยู่ได้ตรง ๆ เลย

ไฟล์	ที่อยู่	หน้าจอ	FR
_layout.tsx	-	โครงทั้งแอป: ฟอนต์, Provider, กฎว่าเข้าหน้าไหนได้ (ไม่ใช่หน้าจอ)	ทุก FR
welcome.tsx	/welcome	สมัคร เข้าสู่ระบบ ลืมรหัสผ่าน ลองใช้ข้อมูลตัวอย่าง	บัญชี
onboarding.tsx	/onboarding	ตั้งค่าเงินครั้งแรก (ยอดเงินตอนนี้, เงินสำรอง, บง)	FR-2, FR-6
(tabs)/_layout.tsx	-	แถบเมนูล่าง 4 แท็บ + ปุ่มสแกน	-
(tabs)/index.tsx	/	หน้าหลัก	FR-2 (+FR-6)
(tabs)/transactions.tsx	/transactions	แท็บรายการ ปฏิทินการใช้จ่าย	FR-1, FR-2
(tabs)/runway.tsx	/runway	แท็บเงินพอถึง	FR-6
(tabs)/coach.tsx	/coach	แท็บโค้ชน้องกล้า	FR-5
transaction.tsx	/transaction	จด/แก้/ลบ/ตรวจสอบสลิป 1 รายการ	FR-1, FR-4
voice.tsx	/voice	พูดจด	FR-3
scan.tsx	/scan	สแกนสลิป	FR-4
drafts.tsx	/drafts	สลิปรอยืนยัน	FR-4
settings.tsx	/settings	ตั้งค่า (เงินสำรอง, ธีม, เสียงน้องกล้า ฯลฯ)	หลาย FR
goals.tsx, goal.tsx	/goals, /goal	กระปุกออม	FR-6
recap.tsx	/recap	สรุปเดือนแบบเรื่องเล่า	FR-2
achievements.tsx, skins.tsx	/achievements, /skins	เหรียญ การกิจ ตูสกินน้องกล้า	FR-5 (เสริม)
whatsnew.tsx	/whatsnew	มีอะไรใหม่	-

- โฟลเดอร์ในวงเล็บ เช่น (tabs) เป็น "กลุ่ม" โมไฟล์ในที่อยู่ ใช้จัดหน้าที่อยู่ใต้แถบเมนูเดียวกัน
- index.tsx คือหน้าแรกของโฟลเดอร์นั้น (ที่อยู่ /)
- ไฟล์ _layout.tsx ไม่ใช่หน้าจอ แต่เป็น "กรอบ" ที่ครอบหน้าจอในโฟลเดอร์เดียวกัน

เปลี่ยนหน้าในโค้ด

โค้ด	ผล
<code>router.push('/scan')</code>	เปิดหน้าจอสแกนซ้อนขึ้นมา (กดกลับได้)
<code>router.push({ pathname: '/transaction', params: { kind: 'income' } })</code>	เปิดหน้าจอรายการ พร้อมส่งค่า kind=income (ปุ่ม "เพิ่มเงินเข้า")
<code>useLocalSearchParams()</code>	หน้าปลายทางอ่านค่าที่ส่งมา (src/app/transaction.tsx:28)
<code>router.navigate('/runway')</code>	ไปที่เก็บเงินพอถึง
<code>goBack()</code>	กลับหน้าก่อน ถ้าไม่มี (เช่น เปิดลิงก์ตรงบนเว็บ) กลับหน้าหลัก (src/ui/nav.ts)

อ่านโค้ด: `_layout.tsx` โครงของทั้งแอป

ไฟล์นี้ทำงานเป็นไฟล์แรกเมื่อเปิดแอป ทุกหน้าจออยู่ข้างในมัน

`src/app/_layout.tsx`

บรรทัด 1-51

```
1 import { useFonts } from 'expo-font';
2 import { Stack } from 'expo-router';
3 import * as SplashScreen from 'expo-splash-screen';
4 import { StatusBar } from 'expo-status-bar';
5 import { useCallback, useState, type ReactNode } from 'react';
6 import { Platform, View } from 'react-native';
7 import { SafeAreaProvider } from 'react-native-safe-area-context';
8 import {
9   Anuphan_400Regular,
10  Anuphan_500Medium,
11  Anuphan_600SemiBold,
12  Anuphan_700Bold,
13 } from '@expo-google-fonts/anuphan';
14 import { NotoSerifThai_600SemiBold, NotoSerifThai_700Bold } from '@expo-google-fonts/noto-serif-thai';
15 import { AppProvider, useApp } from '../data/AppProvider';
16 import { AutoScanProvider } from '../services/AutoScanProvider';
17 import { AuthNoticeHost } from '../ui/AuthNotice';
18 import { CelebrationProvider } from '../ui/effects';
19 import { ToastProvider } from '../ui/feedback';
20 import { LaunchIntro } from '../ui/LaunchIntro';
21 import { UpdateBanner } from '../ui/UpdateBanner';
22 import { mix, useTheme } from '../ui/theme';
23
24 // The native splash stays until the opening animation (LaunchIntro) takes over.
25 SplashScreen.preventAutoHideAsync().catch(() => {});
26
27 export default function RootLayout() {
28   const [fontsLoaded, fontError] = useFonts({
29     Anuphan_400Regular,
30     Anuphan_500Medium,
31     Anuphan_600SemiBold,
32     Anuphan_700Bold,
33     NotoSerifThai_600SemiBold,
34     NotoSerifThai_700Bold,
```

```

35 });
36 return (
37   <SafeAreaProvider>
38     <WebFrame>
39       <CelebrationProvider>
40         <AppProvider>
41           <ToastProvider>
42             <AutoScanProvider>
43               <Navigator ready={fontsLoaded || !!fontError} />
44             </AutoScanProvider>
45           </ToastProvider>
46         </AppProvider>
47       </CelebrationProvider>
48     </WebFrame>
49   </SafeAreaProvider>
50 );
51 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/_layout.tsx:1

บรรทัด	ความหมาย
1–22	นำเครื่องมือมาใช้: โหลดฟอนต์, Stack (การซ้อนหน้า), หน้าจอเปิดแอป, แถบสถานะ, ฟอนต์ Anuphan 4 น้ำหนัก และ Noto Serif Thai 2 น้ำหนัก, Provider ต่าง ๆ ของเรา
24–25	สั่งให้หน้าจอเปิดแอป (splash สีเขียวรูปน้องกล้า) ค้างไว้ก่อน จนกว่าแอนิเมชันเปิดแอปจะมารับช่วง .catch(() => {}) = ถ้าทำไม่ได้ก็ไม่เป็นไร
27	export default = นี่คือ component หลักของไฟล์ Expo Router จะเรียกตัวนี้
28–35	โหลดฟอนต์ทั้ง 6 ตัว ได้ผลว่าโหลดเสร็จหรือยัง (fontsLoaded) หรือมี error ไหม
36–50	ซ้อน Provider เป็นชั้น ๆ เหมือนตุ๊กตาแม่ลูกดก ชั้นนอกสุดถึงชั้นในสุด: ขอบจอปลอดภัย → กรอบเว็บ → ระบบพลูลอง → ข้อมูลกลาง (AppProvider) → ข้อความแจ้งเตือน (toast) → สแกนสลิปอัตโนมัติ (FR-4) → ตัวเปลี่ยนหน้า ชั้นในใช้ของชั้นนอกได้ เช่น AutoScanProvider ใช้ข้อมูลจาก AppProvider ได้เพราะอยู่ข้างใน
43	ส่ง ready = ฟอนต์โหลดเสร็จ หรือโหลดพังกี่ให้ไปต่อ (ไม่ค้างหน้าขาวตลอดไป)

src/app/_layout.tsx

บรรทัด 53–73

```

53 /** On a computer browser, show the app at phone width in the middle of the screen. */
54 function WebFrame({ children }: { children: ReactNode }) {
55   const theme = useTheme();
56   if (Platform.OS !== 'web') return <>{children}</>;
57   return (
58     <View style={{ flex: 1, alignItems: 'center', backgroundColor: theme.dark ? mix(theme.bg,
59       '#000000', 0.55) : theme.surfaceAlt }}>
60       <View
61         style={{
62           flex: 1,
63           width: '100%',
64           maxWidth: 440,
65           backgroundColor: theme.bg,
66           overflow: 'hidden',
67           boxShadow: '0 0 40px rgba(0, 0, 0, 0.18)',
68         }}
69       >
70         {children}
71       </View>
72     </View>
73   );

```

```

70     </View>
71   </View>
72 );
73 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/_layout.tsx:53

บรรทัด ความหมาย

53–54 WebFrame: บนคอม แสดงแอปกว้างเท่ามือถือไว้ตรงกลางจอ

56 ถ้าไม่ใช่เว็บ (Platform.OS !== 'web') ไม่ต้องทำอะไร ค้นของข้างในตรง ๆ (<>...</> คือกล่องที่มองไม่เห็น)

58 พื้นหลังเต็มจอ จัดกลาง สีเข้มขึ้นเล็กน้อยในโหมดมืด

59–68 กล่องแอป กว้างสูงสุด 440 จุด มีเงารอบ ๆ ให้ดูเหมือนมือถือวางบนโต๊ะ

src/app/_layout.tsx

บรรทัด 75–126

```

75 function Navigator({ ready }: { ready: boolean }) {
76   const theme = useTheme();
77   const { status, repo, profile } = useApp();
78   const loading = !ready || status === 'loading';
79   const [introDone, setIntroDone] = useState(false);
80   const finishIntro = useCallback(() => setIntroDone(true), []);
81
82   const signedIn = status === 'ready' && !!repo;
83   const needsOnboarding = signedIn && !!profile && !profile.onboarded;
84   const inApp = signedIn && !needsOnboarding;
85
86   return (
87     <>
88       <StatusBar style={!introDone || theme.dark ? 'light' : 'dark'} />
89       {loading ? null : <Screens signedIn={signedIn} needsOnboarding={needsOnboarding} inApp=
{inApp} />}
90       {loading || !introDone ? null : <AuthNoticeHost />}
91       {loading || !introDone ? null : <UpdateBanner />}
92       <LaunchIntro ready={!loading} fontsReady={ready} onDone={finishIntro} />
93     </>
94   );
95 }
96
97 function Screens({ signedIn, needsOnboarding, inApp }: { signedIn: boolean; needsOnboarding:
boolean; inApp: boolean }) {
98   const theme = useTheme();
99   return (
100     <>
101       /* Screens slide in from the side; scanning and adding a transaction rise from the bottom.
*/
102       <Stack screenOptions={{ headerShown: false, contentStyle: { backgroundColor: theme.bg },
animation: 'slide_from_right' }}>
103         <Stack.Protected guard={!signedIn}>
104           <Stack.Screen name="welcome" options={{ animation: 'fade' }} />
105         </Stack.Protected>
106         <Stack.Protected guard={needsOnboarding}>
107           <Stack.Screen name="onboarding" options={{ animation: 'fade' }} />
108         </Stack.Protected>
109         <Stack.Protected guard={inApp}>
110           <Stack.Screen name="(tabs)" options={{ animation: 'fade' }} />

```

```

111     <Stack.Screen name="transaction" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
112     <Stack.Screen name="scan" options={{ presentation: 'fullScreenModal', animation:
'slide_from_bottom' }} />
113     <Stack.Screen name="voice" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
114     <Stack.Screen name="achievements" />
115     <Stack.Screen name="skins" />
116     <Stack.Screen name="recap" options={{ presentation: 'fullScreenModal', animation:
'fade' }} />
117     <Stack.Screen name="goals" />
118     <Stack.Screen name="whatsnew" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
119     <Stack.Screen name="goal" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
120     <Stack.Screen name="drafts" />
121     <Stack.Screen name="settings" />
122   </Stack.Protected>
123 </Stack>
124 </>
125 );
126 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/_layout.tsx:75`

บรรทัด	ความหมาย
75–78	Navigator ดูสถานะจากข้อมูลกลาง: เข้าสู่ระบบหรือยัง มี Repo ใหม่ โปรไฟล์ตั้งค่าเสร็จไหม และยังโหลดอยู่ไหม
79–80	จำว่าแอนิเมชันเปิดแอปจบหรือยัง
82–84	คำนวณ 3 สถานะ: signedIn (เข้าสู่ระบบแล้ว), needsOnboarding (เข้าแล้วแต่ยังไม่ตั้งค่าเงิน), inApp (เข้าแล้วและตั้งค่าแล้ว ใช้แอปได้เต็มที่)
88	สไลด์คอนแทกสถานะด้านบน (สว้างบนพื้นเข้ม)
89	ถ้ายังโหลดอยู่ ยังไม่แสดงหน้าจอ
90–91	หลังแอนิเมชันจบ แสดงหน้าต่างแจ้งเรื่องบัญชี และแถบ "มีอัปเดตใหม่"
92	แอนิเมชันเปิดแอป (น้องกล้วยไฟล์) ช้อนอยู่บนสุดจนจบ
97–102	Stack = หน้าจอซ้อนกันเป็นกอง หน้าใหม่กับหน้าเก่า กดกลับก็หิบบหน้าบนออก ตั้งค่าทุกหน้า: ไม่มีหัวหน้าสำเร็จรูป, พื้นตามืด, เลื่อนเข้ามาจากขวา
103–105	Stack.Protected = ประตูกุญแจ: หน้า welcome เข้าได้เฉพาะตอน ยังไม่ เข้าสู่ระบบ
106–108	หน้า onboarding เข้าได้เฉพาะตอนเข้าแล้วแต่ยังไม่ตั้งค่าเงิน
109–122	หน้าในแอปทั้งหมดเข้าได้เฉพาะตอนพร้อมใช้งาน บางหน้าแสดงแบบ modal (เลื่อนขึ้นจากล่าง) เช่น จดรายการ พุดจด, บางหน้าเต็มจอ เช่น สแกนสลิป สรุปเดือน

★ แลของ Stack.Protected

ผู้ใช้ไม่ต้องทำอะไร พอสมัครเสร็จ สถานะเปลี่ยนเป็น needsOnboarding ยามจะปิดประตู welcome และเปิดประตู onboarding แอปจึงพาไปหน้าตั้งค่าเงินเอง พอตั้งค่าเสร็จก็ไปหน้าหลักเอง และกดย้อนกลับก็ไม่เจอหน้าสมัครอีก (ตรงกับ acceptance criteria ในเอกสาร Traceability)

แถบเมนูล่าง: (tabs)/_layout.tsx

src/app/(tabs)/_layout.tsx

บรรทัด 23–28

```
23 const TABS: Record<string, { label: string; icon: IconName; iconActive: IconName }> = {
24   index: { label: 'หน้าหลัก', icon: 'home-outline', iconActive: 'home' },
25   transactions: { label: 'รายการ', icon: 'list-outline', iconActive: 'list' },
26   runway: { label: 'เงินพอถึง', icon: 'leaf-outline', iconActive: 'leaf' },
27   coach: { label: 'โค้ช', icon: 'chatbubble-ellipses-outline', iconActive: 'chatbubble-ellipses' },
28 };
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:23

บรรทัด 23–28: ตารางของ 4 แท็บ แต่ละแท็บมีชื่อไทย ไอคอนตอนไม่ได้เลือก และไอคอนตอนเลือก (แบบทึบ) ชื่อช่อง (index, transactions, runway, coach) ตรงกับชื่อไฟล์ในโฟลเดอร์ (tabs)

src/app/(tabs)/_layout.tsx

บรรทัด 134–186

```
134   {routes.slice(0, half).map((r, i) => tab(r, i))}
135   <View style={{ width: 76, alignItems: 'center' }}>
136     <View style={{ width: 60, height: 60, marginTop: -26, alignItems: 'center',
137       justifyContent: 'center' }}>
138       <PulseRing size={60} active times={auto.state.phase === 'scanning' ? 'always' : 3}
139         color={theme.accent} />
140       <Animated.View style={{ transform: [{ scale: press.scale }] }}>
141         <Pressable
142           accessibilityRole="button"
143           accessibilityLabel="สแกนสลิป"
144           onPressIn={press.onPressIn}
145           onPressOut={press.onPressOut}
146           onPress={() => {
147             Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Medium).catch(() => {});
148             router.push('/scan');
149           }}
150           style={{
151             width: 60,
152             height: 60,
153             borderRadius: 30,
154             backgroundColor: theme.hero,
155             borderWidth: 3,
156             borderColor: theme.accent,
157             alignItems: 'center',
158             justifyContent: 'center',
159             shadowColor: theme.accent,
160             shadowOpacity: 0.35,
161             shadowRadius: 10,
162             shadowOffset: { width: 0, height: 4 },
163             elevation: 8,
164           }}
165         />
166       </Animated.View>
167     </View>
168   </View>
```

```

164         <Ionicons name="scan" size={26} color={theme.heroAccent} />
165     </Pressable>
166 </Animated.View>
167 </View>
168     <Text style={[type.micro, { fontFamily: fonts.sansSemi, color: theme.ink, marginTop: 4
    ]}]>สแกนสลิป</Text>
169 </View>
170     {routes.slice(half).map((r, i) => tab(r, i + half))}
171 </View>
172 );
173 }
174
175 export default function TabsLayout() {
176 // Records today's visit and hands out น้องกล้า's skins for finished missions and events.
177 useKlaSync();
178 return (
179     <Tabs tabBar={props => <TabBar {...props} />} screenOptions={{ headerShown: false }}>
180         <Tabs.Screen name="index" />
181         <Tabs.Screen name="transactions" />
182         <Tabs.Screen name="runway" />
183         <Tabs.Screen name="coach" />
184     </Tabs>
185 );
186 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:134

บรรทัด	ความหมาย
134	วาดแท็บครั้งแรก (หน้าหลัก, รายการ) ทางซ้าย
135–169	ปุ่มทองตรงกลาง: วงกลม 60 จุด ลอยขึ้นมา 26 จุด (marginTop: -26) มีวงแหวนกระเพื่อม (PulseRing) 3 รอบ ตอนเปิดแอป และกระเพื่อมตลอดระหว่างสแกนอัตโนมัติ
144–147	แตะแล้วมือถือสั่นแรงปานกลาง และเปิดหน้าสแกน
170	วาดแท็บครั้งหลัง (เงินพอถึง, โค้ช) ทางขวา
175–186	component หลัก: ใช้แถบเมนูที่ทีมออกแบบเอง (tabBar=) แทนของสำเร็จรูป และประกาศ 4 แท็บ useKlaSync() บันทึกว่าวันนี้เปิดแอป (ใช้บันทึกการกิจ "เปิดแอป 30 วันติด")

โค้ดที่ต่างกันระหว่างมือถือกับเว็บ

บางอย่างทำได้ต่างกัน เช่น มือถือเก็บข้อมูลด้วย SQLite แต่เว็บใช้ localStorage ของเบราว์เซอร์ ทีมเราแก้ด้วย **ไฟล์คู่**: ตั้งชื่อ storage.ts สำหรับมือถือ และ storage.web.ts สำหรับเว็บ ตอน build ระบบ (Metro bundler) เลือกไฟล์ให้เองตามแพลตฟอร์ม ที่เหลือของแอป import แค่ './storage'

src/data/storage.ts

บรรทัด 1–12

```

1 /**
2  * Small key-value store used for app
  settings and demo data.
3  * Phone: SQLite-backed store (survives
  app restarts).
4  * Web: see storage.web.ts (browser
  localStorage).

```

src/data/storage.web.ts

บรรทัด 1–24

```

1 /** Web version of storage.ts: the
  browser's localStorage, wrapped so it
  never throws. */
2 export const Storage = {
3   async getItem(key: string):
  Promise<string | null> {
4     try {

```

```

5  */
6  import KV from 'expo-sqlite/kv-store';
7
8  export const Storage = {
9    getItem: (key: string) =>
10     KV.getItem(key),
11    setItem: (key: string, value: string) =>
12     KV.setItem(key, value),
13    removeItem: (key: string) =>
14     KV.removeItem(key),
15  };

```

► เปิดใน VS
Code (เว็บ)

GitHub

VS Code ในเครื่อง: Ctrl+P →
src/data/storage.ts:1

```

5      return
6      window.localStorage.getItem(key);
7    } catch {
8      return null;
9    },
10   async setItem(key: string, value:
11     string): Promise<void> {
12     try {
13       window.localStorage.setItem(key,
14         value);
15     } catch {
16       // Private mode or storage full:
17       // keep working in memory.
18     }
19   },
20   async removeItem(key: string):
21     Promise<void> {
22     try {
23       window.localStorage.removeItem(key);
24     } catch {
25       // ignore
26     }
27   },
28 };

```

► เปิดใน VS
Code (เว็บ)

GitHub

VS Code ในเครื่อง: Ctrl+P →
src/data/storage.web.ts:1

ทั้งสองไฟล์มี getItem, setItem, removeItem หน้าตาเดียวกัน ฉบับเว็บห่อด้วย try/catch เพราะเบราว์เซอร์โหมดส่วนตัวอาจห้ามเก็บข้อมูล คู่มือฉบับนี้มีทั้งหมด 5 คู่มือ:

ไฟล์	มือถือ	เว็บ
data/storage	SQLite (expo-sqlite)	localStorage
data/sessionStorageSetup	ที่เก็บการเข้าสู่ระบบของ Supabase บนมือถือ	ของเบราว์เซอร์
services/gallery	อ่านแกลเลอรีทั้งหมด (expo-media-library)	ให้ผู้ใช้เลือกรูปเอง (เบราว์เซอร์ไม่อนุญาตให้เว็บเปิดแกลเลอรี)
services/speech	ตัวฟังเสียงของมือถือ (Google/Apple)	Web Speech API ของเบราว์เซอร์
services/tts	เสียงอ่านของมือถือ (expo-speech)	speechSynthesis ของเบราว์เซอร์

ส่วนที่ต่างเล็กน้อยใช้ Platform.OS ในไฟล์เดียวกัน เช่น หน้าจอดรายการบนเว็บให้พิมพ์วันที่ แต่บน Android เปิดปฏิทินของเครื่อง ([src/app/transaction.tsx:288](#))

Native module แบบไม่บังคับ

src/services/speech.ts

บรรทัด 1–24

```

1  /**
2   * Speech to text on the phone ("พูดจด"), in Thai, with the phone's own speech

```

```

3  * recognizer (Google on Android, Apple on iOS) through expo-speech-recognition.
4  *
5  * The native module is looked up optionally: app builds made before it was
6  * added do not have it, and an over-the-air update must not crash them. There
7  * the voice screen simply offers typing instead (the keyboard's own mic works).
8  * The web version is speech.web.ts.
9  */
10 import { requireOptionalNativeModule } from 'expo';
11
12 interface Subscription {
13   remove(): void;
14 }
15
16 interface NativeSpeech {
17   start(options: Record<string, unknown>): void;
18   stop(): void;
19   abort(): void;
20   requestPermissionsAsync(): Promise<{ granted: boolean; canAskAgain?: boolean }>;
21   isRecognitionAvailable(): boolean;
22   addListener(event: string, listener: (e: never) => void): Subscription;
23 }
24
25 const Speech = requireOptionalNativeModule<NativeSpeech>('ExpoSpeechRecognition');

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/speech.ts:1

บรรทัด 24 ใช้ requireOptionalNativeModule = "ขอโมดูลฟังเสียง ถ้าไม่มีก็ให้ null ไม่ต้อง crash" เพราะแอปที่ลงไว้ก่อนเพิ่มฟีเจอร์พูดจดไม่มีโมดูลนี้ แต่ยังสามารถอัปเดตใหม่ผ่าน EAS Update (หัวข้อถัดไป) ถ้าเรียกแบบบังคับ แอปเก่าจะแจ้งเตือนแบบนี้แอปเก่าแค่แสดงช่องพิมพ์แทนไมค์

จากโค้ด สู่เว็บและแอปที่ผู้ใช้ลง

โค้ดใน src/ (TypeScript + React Native)



เว็บ

expo export --platform web รวมโค้ดเป็นไฟล์ JavaScript + HTML ในโฟลเดอร์ dist แล้ววางบน GitHub Pages

APK (Android)

expo prebuild สร้างโปรเจกต์ Android จาก app.json แล้ว Gradle build เป็นไฟล์ .apk

EAS Update

ส่งเฉพาะโค้ด JavaScript ใหม่ แอปที่ลงแล้วโหลดเองตอนเปิดแอป ใช้ตอนเปิดครั้งถัดไป

ทั้ง 3 ทางทำอัตโนมัติทุกครั้งที่เรา push โค้ดขึ้น branch main บน GitHub (บน A13) ไฟล์ตั้งค่าของแอปคือ app.json ส่วนสำคัญ:

ค่าใน app.json	ความหมาย
"name": "MindPay", "slug": "mindpay"	ชื่อแอปที่เห็นใต้ไอคอน
"version": "1.0.0" + "runtimeVersion": {"policy": "appVersion"}	EAS Update ส่งโค้ดใหม่ให้เฉพาะแอปที่เวอร์ชันตรงกัน ถ้าเพิ่ม native module ใหม่ ต้องเพิ่มเวอร์ชัน แล้วลง APK ใหม่
"android.package"	รหัสแอป com.socratesandskeletons.mindpay

ค่าใน app.json	ความหมาย
"blockedPermissions"	บล็อกสิทธิ์กล้อง วัตถุโอ เสียงในเครื่อง ที่ไม่ได้ใช้ (ความเป็นส่วนตัว)
"plugins"	ตั้งค่า native ของโมดูล เช่น ข้อความขอสิทธิ์แคลเลอร์/ไมโครโฟนเป็นภาษาไทย
"scheme": "mindpay"	ลิงก์ที่ขึ้นต้นด้วย mindpay:// เปิดแอปได้ (ใช้กับลิงก์ยืนยันอีเมล)
"updates"	ที่อยู่ของเซิร์ฟเวอร์ EAS Update และให้เช็กอัปเดตทุกครั้งที่เปิดแอป
"web": {"output": "single"}	เว็บเป็นหน้าเดียว (single page app) ให้ JavaScript เปลี่ยนหน้าเอง

ไฟล์ `app.config.js` เพิ่มค่าเฉพาะตอน build เว็บ: เว็บอยู่ที่ `/Socrates-and-Skeletons - /` บน GitHub Pages จึงต้องตั้ง `baseUrl` แต่ห้ามใส่ค่านี้ในแอปมือถือ (แอปจะหาฟอนต์และรูปผิดที่)

วิธีอ่านโค้ด MindPay ด้วยตัวเอง

โค้ดทั้งแอปมีเกือบ 19,000 บรรทัด ไม่มีใครจำได้หมด แต่ทุกไฟล์ใช้รูปแบบซ้ำ ๆ กัน รู้รูปแบบแล้วจะอ่านไฟล์ไหนก็ออก

ขั้นตอนอ่านไฟล์ที่ไม่เคยเห็น

- อ่านคอมเมนต์บนสุดของไฟล์ก่อน** ทุกไฟล์ในโปรเจกต์เริ่มด้วยคอมเมนต์ `/** ... */` บอกว่าไฟล์นี้ทำอะไร เป็นของ FR ไหน และมีกฎอะไรสำคัญ (ภาษาอังกฤษง่าย ๆ ใช้ Google Translate ช่วยได้)
- ดูบรรทัด import** บอกว่าไฟล์นี้พึ่งใครบ้าง ถ้า import จาก `../domain` แปลว่ามีการคำนวณ ถ้าจาก `../data/AppProvider` แปลว่าใช้ข้อมูลกลาง
- กด Ctrl+Shift+O** ดูรายชื่อฟังก์ชันทั้งหมดในไฟล์ เหมือนดูสารบัญ
- หาฟังก์ชันที่มี export** นั่นคือ "ประตูหน้าบ้าน" ที่ไฟล์อื่นเรียก ฟังก์ชันที่ไม่มี export คือผู้ช่วยภายใน
- อ่านชนิดข้อมูลก่อนอ่านตัวโค้ด** ดูว่ารับอะไรเข้า คืนอะไรออก ก็เข้าใจไปครึ่งหนึ่งแล้ว
- หาเทสต์ของมัน** กด Shift+F12 ที่ชื่อฟังก์ชัน ถ้ามีเทสต์ใน `__tests__` จะเห็นตัวอย่างการใช้งานจริงพร้อมคำตอบที่ถูกต้อง อ่านง่ายกว่าตัวโค้ดเสียอีก

รูปแบบที่เจอซ้ำทั้งแอป (ตารางโยง)

เห็นอะไร	แปลว่าอะไร
ชื่อลงท้าย Satang	จำนวนเงินเป็นสตางค์ (จำนวนเต็ม) หาร 100 ได้บาท
ชื่อขึ้นต้น bkk เช่น bkkDayKey, bkkToIso	ฟังก์ชันจัดการวันเวลาแบบเวลาไทย (กรุงเทพ UTC+7) อยู่ใน <code>src/domain/dates.ts</code>
ข้อความวันที่ '2026-10-09'	"day key" วันที่ไทยรูปแบบ ปี-เดือน-วัน (ค.ศ.) เอามาเทียบมากกว่า/น้อยกว่า กันได้ตรง ๆ เพราะเรียงตามตัวอักษรก็ตรงกับเรียงตามเวลา
useApp()	หยิบข้อมูลกลาง (ไปรไฟล์ รายการ คำสั่ง addTx ฯลฯ)
useMoney()	หยิบตัวเลขที่คำนวณแล้ว: ยอดคงเหลือ ค่าเฉลี่ย Runway แบบร่าง
useTheme() แล้วตามด้วย theme.ink, theme.primary	สีจากธีมที่ผู้ใช้เลือก (ห้ามใส่รหัสสีเองในหน้าจอ)
<T v="h3">, <Money satang={...}>, <Card>, <Button>	ชิ้นส่วนหน้าตาจาก <code>src/ui/components.tsx</code>
space.lg, radius.md	ระยะห่างและมุมโค้งมาตรฐาน (16 จุด, 14 จุด) จาก <code>src/ui/theme.ts</code>
toast({ message })	ข้อความแจ้งเตือนสั้น ๆ ด้านล่างจอ
router.push('/xxx')	เปิดหน้า <code>src/app/xxx.tsx</code>
accessibilityLabel="..."	ข้อความที่โปรแกรมอ่านหน้าจออ่านให้คนตาบอดฟัง และเทสต์ E2E ใช้หาปุ่ม

เห็นอะไร	แปลว่าอะไร
status === 'confirmed' หรือ confirmedOnly(tx)	นับเฉพาะรายการที่ยืนยันแล้ว
Reveal, usePressSpring, useReduceMotion	แอนิเมชัน (ค่อย ๆ โฟกัส, ปุ่มยุบตอนกด, เคารพ "ลดการเคลื่อนไหว")
คอมเมนต์มีรหัส TC-12	รหัสเทสต์ ตรงกับในเอกสาร docs/TRACEABILITY.md
ไฟล์ชื่อ xxx.web.ts	ฉบับเว็บของไฟล์ xxx.ts
ชื่อแบบ snake_case เช่น amount_satang	ชื่อคอลัมน์ในฐานข้อมูล (ฝั่งแอปใช้ camelCase amountSatang) แปลงกันใน src/data/repo.ts

ฟังก์ชันวันเวลาที่ต้องรู้ (ทุก FR ใช้)

เวลาในฐานข้อมูลเก็บเป็นเวลาสากล (UTC) แต่ผู้ใช้ชาวไทย (เร็วกว่า 7 ชั่วโมง) ถ้าไม่แปลง รายการตอน 06:00 น. ของวันที่ 8 จะถูกนับเป็นวันที่ 7 (เพราะ UTC ยังเป็น 23:00 ของวันที่ 7) ทุก FR จึงใช้ฟังก์ชันจาก [src/domain/dates.ts](#)

src/domain/dates.ts

บรรทัด 1-51

```

1  /**
2   * Date helpers fixed to Thailand time (Asia/Bangkok, UTC+7, no daylight saving).
3   *
4   * Everything the user sees ("today", "this week", the date printed on a slip)
5   * is a Bangkok calendar day, so all grouping and range logic goes through
6   * `bkkDayKey`. Because Thailand has no DST, a fixed +7h offset is exact.
7   */
8  import type { RangeKey } from './types';
9
10 export const BKK_OFFSET_MS = 7 * 60 * 60 * 1000;
11 export const DAY_MS = 24 * 60 * 60 * 1000;
12
13 /** Instant -> "YYYY-MM-DD" in Bangkok. */
14 export function bkkDayKey(at: Date | string | number): string {
15   const ms = new Date(at).getTime() + BKK_OFFSET_MS;
16   return new Date(ms).toISOString().slice(0, 10);
17 }
18
19 /** "YYYY-MM-DD" (Bangkok day) -> the UTC instant of 00:00 that day in Bangkok. */
20 /** Milliseconds until the next 00:00 in Bangkok (when "วันนี้" starts over). */
21 export function msUntilNextBkkMidnight(nowMs: number = Date.now()): number {
22   const next = bkkDayStart(addDays(bkkDayKey(nowMs), 1)).getTime();
23   return Math.max(0, next - nowMs);
24 }
25
26 export function bkkDayStart(dayKey: string): Date {
27   return new Date(Date.parse(`${dayKey}T00:00:00.000Z`) - BKK_OFFSET_MS);
28 }
29
30 /** Add whole days to a "YYYY-MM-DD" key. */
31 export function addDays(dayKey: string, days: number): string {
32   return new Date(Date.parse(`${dayKey}T00:00:00.000Z`) + days * DAY_MS)
33     .toISOString()
34     .slice(0, 10);
35 }

```

```

36
37 /** Whole days from a to b (both "YYYY-MM-DD"). */
38 export function daysBetween(a: string, b: string): number {
39   return Math.round((Date.parse(`${b}T00:00:00Z`) - Date.parse(`${a}T00:00:00Z`)) / DAY_MS);
40 }
41
42 /** Combine a Bangkok day and "HH:MM" into an ISO instant. */
43 export function bkkToIso(dayKey: string, time: string | null = '12:00'): string {
44   const t = time && /\d{1,2}:\d{2}$/.test(time) ? time.padStart(5, '0') : '12:00';
45   return new Date(Date.parse(`${dayKey}T${t}:00.000Z`) - BKK_OFFSET_MS).toISOString();
46 }
47
48 /** "HH:MM" of an instant in Bangkok. */
49 export function bkkTime(at: Date | string): string {
50   return new Date(new Date(at).getTime() + BKK_OFFSET_MS).toISOString().slice(11, 16);
51 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/dates.ts:1

บรรทัด	ความหมาย
1–7	คอมเมนต์: ทุกอย่างที่ใช้ให้เห็น (วันนี้, สัปดาห์นี้, วันที่บนสลิป) เป็นวันตามปฏิทินกรุงเทพ และเพราะไทยไม่มีการปรับเวลา ฤดูร้อน (daylight saving) การบวก 7 ชั่วโมงตายตัวจึงถูกต้องเสมอ
10	7 ชั่วโมงในหน่วยมิลลิวินาที ($7 \times 60 \times 60 \times 1000 = 25,200,000$)
11	1 วันในหน่วยมิลลิวินาที ($86,400,000$)
13–17	bkkDayKey : รับเวลาแบบไหนก็ได้ (Date, ข้อความ, ตัวเลข) บวก 7 ชั่วโมง แล้วตัดเอา 10 ตัวอักษรแรกของรูปแบบ สากล ได้วันที่ไทย เช่น <code>bkkDayKey('2026-10-07T23:00:00Z')</code> → <code>'2026-10-08'</code> (23:00 UTC = 06:00 น. ของวันที่ 8 เวลาไทย)
20–24	msUntilNextBkkMidnight : เหลืออีกกี่มิลลิวินาทีถึงเที่ยงคืนไทยครั้งหน้า (เอาวันพรุ่งนี้ 00:00 ลบเวลาตอนนี้) ใช้ตั้ง นาฬิกาใน AppProvider (un A6)
26–28	bkkDayStart : วันที่ไทย → เวลาสากลของ 00:00 น. วันนั้น (ลบ 7 ชั่วโมงกลับ)
30–35	addDays : บวกหรือลบจำนวนวันจากวันที่ เช่น <code>addDays('2026-10-08', 20)</code> → <code>'2026-10-28'</code> ข้ามเดือน ข้ามปีให้เอง
37–40	daysBetween : นับจำนวนวันจาก a ถึง b เช่น <code>daysBetween('2026-10-08', '2026-10-31')</code> → 23
42–46	bkkToIso : เอาวันไทย + เวลา "HH:MM" มารวมเป็นเวลาสากลสำหรับเก็บในฐานข้อมูล ถ้าเวลาผิดรูปแบบใช้ 12:00 แทน เช่น <code>bkkToIso('2026-10-08', '06:00')</code> → <code>'2026-10-07T23:00:00.000Z'</code>
48–51	bkkTime : เวลาสากล → "HH:MM" ตามเวลาไทย ใช้แสดงเวลาของรายการ

แผนที่: อยากรู้อะไรไปเปิดไฟล์ไหน

อยากรู้	เปิดไฟล์
หน้าจอนี้หน้าตาแบบนี้เพราะอะไร	src/app/ชื่อหน้า.tsx (ตารางใน un A7)

อยากรู้	เปิดไฟล์
ตัวเลขนี้คิดยังไง	src/domain/: src/domain/summary.ts (ยอด), src/domain/runway.ts (Runway), src/domain/insights.ts (การ์ดโค้ช)
ข้อมูลโหลด/บันทึกยังไง	src/data/repo.ts , src/data/AppProvider.tsx
ตารางในฐานข้อมูลมีคอลัมน์อะไร	supabase/migrations/20260927000000_init.sql
AI ได้รับคำสั่งอะไร	supabase/functions/coach/index.ts , supabase/functions/parse-slip/index.ts
สี ขนาด ฟอนต์	src/ui/theme.ts , docs/DESIGN_SYSTEM.md
ฟีเจอร์นี้ตอบ requirement ข้อไหน มีทดสอบอะไร	docs/TRACEABILITY.md
ทีมใช้ AI ช่วยเขียนตรงไหน ตรวจอะไรไปแล้ว	docs/AI_USAGE_LOG.md
build / deploy อัตโนมัติ	.github/workflows/*.yaml

★ ถ้าอ่านแล้วงง

ใส่ `console.log('ค่า', ตัวแปร)` ไว้ในโค้ดแล้วเปิดแอปด้วย `npm run web` กด F12 ในเบราว์เซอร์ แถบ Console จะเห็นค่าที่พิมพ์ออกมา (อย่าลืมลบก่อน commit) หรือเขียนเทสต์เล็ก ๆ ลองส่งค่าเข้าฟังก์ชันใน `src/domain` แล้วดูผล

เซิร์ฟเวอร์และฐานข้อมูล: Supabase

ข้อมูลของผู้ใช้เก็บที่ไหน เก็บยังไง แอปต่อเซิร์ฟเวอร์ยังไง และทำไมคนอื่นแอบดูข้อมูลเราไม่ได้ ทั้งที่กุญแจอยู่ในแอป

ฐานข้อมูลคืออะไร

ฐานข้อมูล (database) คือที่เก็บข้อมูลแบบมีระเบียบ คิดว่าเป็น **สมุด Excel บนคลาวด์** ที่มีหลายชีต แต่ละชีตเรียกว่า **ตาราง (table)** แต่ละแถวคือข้อมูล 1 ชัน (row) แต่ละคอลัมน์คือช่องข้อมูล (column) เช่น ตาราง transactions มี 1 แถวต่อรายการเงิน 1 รายการ

id	user_id	kind	amount_satang	title	status	occurred_at
a1b2...	มินท์	expense	8500	ข้าวมันไก่	confirmed	2026-10-08T05:10Z
c3d4...	มินท์	income	1000000	ค่าขนมจากที่บ้าน	confirmed	2026-10-01T02:00Z
e5f6...	ต้น	expense	125000	โอนให้ นาย ส***	draft	2026-10-07T11:42Z

ตัวอย่างสมมติ คอลัมน์ user_id จริงเป็นรหัสยาว (UUID) ไม่ใช่ชื่อ

ภาษาที่ใช้สั่งฐานข้อมูลคือ **SQL** คำสั่งพื้นฐานมี 4 คำ

SQL	ความหมาย	ในแอปเขียนว่า (supabase-js)
SELECT * FROM transactions	อ่านข้อมูล	db.from('transactions').select('*')
INSERT INTO transactions ...	เพิ่มแถว	db.from('transactions').insert(row)
UPDATE transactions SET ... WHERE id = ...	แก้แถว	db.from('transactions').update(row).eq('id', id)
DELETE FROM transactions WHERE id = ...	ลบแถว	db.from('transactions').delete().eq('id', id)

แอปไม่ได้เขียน SQL ตรง ๆ แต่ใช้ไลบรารี supabase-js ซึ่งแปลงคำสั่งเป็นคำขอผ่านอินเทอร์เน็ต แล้วเซิร์ฟเวอร์แปลงเป็น SQL ให้อีกที

Supabase ให้อะไรกับเราบ้าง

ส่วน	คืออะไร	ใช้ทำอะไรใน MindPay
PostgreSQL	ฐานข้อมูลแบบตาราง ที่นิยมที่สุดตัวหนึ่งของโลก	เก็บ 4 ตาราง: profiles, transactions, savings_goals, ai_usage
Auth	ระบบบัญชีผู้ใช้สำเร็จรูป	สมัคร เข้าสู่ระบบ ส่งรหัส 6 หลักทางอีเมล ลืมรหัสผ่าน

ส่วน	คืออะไร	ใช้ทำอะไรใน MindPay
REST API อัตโนมัติ	สร้าง "ประตู" ให้เข้าถึงตารางผ่านอินเทอร์เน็ทให้เอง	แอปอ่าน/เขียนตารางได้โดยไม่ต้องเขียนเซิร์ฟเวอร์เอง
Row Level Security	กฎความปลอดภัยรายแถว	แต่ละคนเห็นเฉพาะแถวของตัวเอง
Edge Functions	ที่รันโค้ด TypeScript ผังเซิร์ฟเวอร์ (Deno)	parse-slip และ coach ซึ่งถือกุญแจ AI (un A10)
Dashboard	หน้าเว็บจัดการทั้งหมด	ดูตาราง รัน SQL ตั้ง Secrets ดู log

โปรเจกต์ของทีมอยู่ภูมิภาค Singapore (ใกล้ไทย ตอบเร็ว) และใช้แพ็คเกจฟรี

Migration: ไฟล์สร้างฐานข้อมูล

แทนที่จะคลิกสร้างตารางในหน้าเว็บ ทีมเราเขียนคำสั่ง SQL ไว้ในไฟล์ เรียกว่า **migration** เรียงตามวันที่ในชื่อไฟล์ ใครจะสร้างฐานข้อมูลใหม่ก็แค่รันไฟล์ตามลำดับ ได้ฐานข้อมูลเหมือนกันทุกตัว และเก็บประวัติใน Git ได้

ไฟล์	ทำอะไร
<code>supabase/migrations/20260927000000_init.sql</code>	สร้างตารางหลัก 3 ตาราง กฎ RLS และ trigger (27 ก.ย. 2569)
<code>supabase/migrations/20260927000100_harden.sql</code>	แก้ตามคำแนะนำของตัวตรวจความปลอดภัย Supabase Advisor
<code>supabase/migrations/20260929000000_savings_goals.sql</code>	เพิ่มตารางกระปุกออม (FR-6)
<code>supabase/migrations/20261001000000_atomic_ai_quota.sql</code>	ฟังก์ชันนับโควตา AI แบบกันแย่งกัน และเพดานจำนวนเงิน

ตาราง profiles: ผู้ใช้ 1 คน = 1 แถว

supabase/migrations/20260927000000_init.sqlบรรทัด 1-22

```
1 -- MindPay database schema (Supabase / PostgreSQL)
2 -- Run once: Supabase Dashboard -> SQL Editor -> paste -> Run
3 -- or with the CLI: supabase db push
4 --
5 -- Security model: every row belongs to one user and Row Level Security (RLS)
6 -- makes sure a user can only ever read or change their own rows, even though
7 -- the app talks to the database directly with the public (publishable) key.
8
9 -- -----
10 -- profiles: one row per user (created automatically at sign-up)
11 -- -----
12 create table if not exists public.profiles (
13   id uuid primary key references auth.users (id) on delete cascade,
14   display_name text not null default '',
15   opening_balance_satang bigint not null default 0,
16   runway_floor_satang bigint not null default 50000 check (runway_floor_satang >= 0), -- FR-6 low
17   line, default #500
```

```

17 monthly_budget_satang bigint check (monthly_budget_satang is null or monthly_budget_satang >
0),
18 coach_tone text not null default 'friend' check (coach_tone in ('friend', 'coach', 'senior')),
-- FR-5
19 onboarded boolean not null default false,
20 created_at timestamptz not null default now(),
21 updated_at timestamptz not null default now()
22 );

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:1

บรรทัด	ความหมาย
1-7	คอมเมนต์ SQL (ขึ้นต้นด้วย --) อธิบายวิธีรัน และหลักความปลอดภัย: ทุกแถวเป็นของผู้ใช้คนเดียว RLS ทำให้อ่านแก้ไขได้เฉพาะของตัวเอง แม้แอปจะคุยกับฐานข้อมูลตรง ๆ ด้วยคุณดูแลสาธารณะ
12	สร้างตาราง profiles ในส่วน public (ถ้ายังไม่มี)
13	id ชนิด uuid (รหัสยาวไม่ซ้ำ) เป็น primary key (ตัวระบุแถว) และ อ้างอิง ผู้ใช้ในระบบ Auth ถ้าลบผู้ใช้ โพรไฟล์ถูกลบตาม (on delete cascade)
14	ชื่อเล่น ห้ามว่าง (not null) ค่าเริ่มต้นเป็นข้อความว่าง
15	เงินตอนเริ่มใช้แอป เป็นสตางค์ ชนิด bigint (จำนวนเต็มขนาดใหญ่)
16	เส้นเงินสำรองของ FR-6 ค่าเริ่มต้น 50,000 สตางค์ (฿500) และมี check บังคับว่าต้อง ≥ 0 ถ้าแอปส่งค่าติดลบมา ฐานข้อมูลปฏิเสธเอง
17	งบท่อเดือน ไม่ตั้งก็ได้ (null) ถ้าตั้งต้องมากกว่า 0
18	โทนโค้ช (ของเดิม FR-5) ต้องเป็น 1 ใน 3 ค่าเท่านั้น
19	ตั้งค่าเงินครั้งแรกเสร็จหรือยัง (ใช้กับ Stack.Protected ใน un A7)
20-21	เวลาสร้างและแก้ไขล่าสุด ใส่เวลาปัจจุบันให้เอง (now())

ตาราง transactions: รายการเงิน (FR-1)

supabase/migrations/20260927000000_init.sql

บรรทัด 24-53

```

24 -- -----
25 -- transactions (FR-1). Money is stored in satang (integer) to avoid rounding errors.
26 -- -----
27 create table if not exists public.transactions (
28   id uuid primary key default gen_random_uuid(),
29   user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
30   kind text not null check (kind in ('income', 'expense')),
31   amount_satang bigint not null check (amount_satang > 0),
32   category_key text not null,
33   title text not null default '' check (char_length(title) <= 120),
34   note text check (note is null or char_length(note) <= 500),
35   occurred_at timestamptz not null,
36   source text not null default 'manual' check (source in ('manual', 'slip')),
37   status text not null default 'confirmed' check (status in ('draft', 'confirmed')),
38   slip_ref text, -- FR-4 duplicate key #1 (QR / printed reference)
39   slip_image_hash text, -- FR-4 duplicate key #2 (SHA-256 of the image)
40   ocr_confidence numeric(3, 2) check (ocr_confidence is null or (ocr_confidence >= 0 and
ocr_confidence <= 1)),

```

```

41 review_flags text[] not null default '{}',
42 created_at timestamptz not null default now(),
43 updated_at timestamptz not null default now()
44 );
45
46 create index if not exists transactions_user_time_idx on public.transactions (user_id,
    occurred_at desc);
47
48 -- The database itself refuses a second copy of the same slip, even if two
49 -- phones scan at the same moment (FR-4 "ตรวจสอบรายการซ้ำ").
50 create unique index if not exists transactions_user_slip_ref_uidx
51 on public.transactions (user_id, slip_ref) where slip_ref is not null;
52 create unique index if not exists transactions_user_slip_hash_uidx
53 on public.transactions (user_id, slip_image_hash) where slip_image_hash is not null;

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:24

บรรทัด	ความหมาย
28	id สร้างให้อัตโนมัติด้วยฟังก์ชันสุ่ม (gen_random_uuid())
29	user_id เจ้าของรายการ ค่าเริ่มต้นคือ auth.uid() = id ของคนที่ส่งคำขอมา แอปจึงไม่ต้องส่ง user_id เอง (และปลอมเป็นคนอื่นไม่ได้)
30	ประเภทต้องเป็น income หรือ expense เท่านั้น (กฎเดียวกับ TxKind ในแอป)
31	จำนวนเงินเป็นสตริง ต้องมากกว่า 0
32	รหัสหมวด
33–34	ชื่อยาวไม่เกิน 120 ตัวอักษร บันทึกไม่เกิน 500 (ตรงกับ maxLength ในหน้าจอลายการ)
35	เวลาที่เกิดรายการ ชนิด timestamptz (เวลาพร้อมเขตเวลา)
36–37	ที่มา (manual/slip) และสถานะ (draft/confirmed) ค่าเริ่มต้นคือยืนยันแล้ว
38–39	เลขอ้างอิงสลิป และ hash ของรูป สำหรับกันซ้ำ (FR-4)
40	ความมั่นใจของ AI ตัวเลข 0.00 ถึง 1.00
41	ช่องที่ต้องตรวจ เป็น array ของข้อความ (text[])
46	index = สารบัญของตาราง ช่วยให้ค้น "รายการของผู้ใช้คนนี้ เรียงตามเวลาล่าสุด" ได้เร็ว ไม่ต้องไล่ทุกแถว
48–53	unique index = ห้ามซ้ำ: ผู้ใช้คนเดียวกันจะมีเลขอ้างอิงสลิปซ้ำกันไม่ได้ และ hash รูปซ้ำไม่ได้ ถ้ามือถือ 2 เครื่องสแกนสลิปใบเดียวกันพร้อมกัน ฐานข้อมูลปฏิเสธใบที่ 2 เอง (ตอบรหัส 23505 ที่ repo.ts แปลงเป็น DuplicateSlipError)

ตาราง ai_usage: นับการใช้ AI

supabase/migrations/20260927000000_init.sql

บรรทัด 55–65

```

55 -- -----
56 -- ai_usage: counts calls to the AI so one account cannot run up the bill.
57 -- Written only by the Edge Functions (service role), readable by the owner.
58 -- -----
59 create table if not exists public.ai_usage (
60   id bigserial primary key,
61   user_id uuid not null references auth.users (id) on delete cascade,

```

```

62 kind text not null check (kind in ('slip', 'coach')),
63 created_at timestamptz not null default now()
64 );
65 create index if not exists ai_usage_user_day_idx on public.ai_usage (user_id, kind, created_at
desc);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:55

ทุกครั้งที่ AI ถูกเรียก (อ่านสลิป 1 ใบ หรือถามโค้ช 1 ครั้ง) Edge Function จะเพิ่ม 1 แถว เพื่อบันทึกว่าวันนี้ใช้ไปกี่ครั้งแล้ว กันบัญชีเดียวใช้ AI จนค่าใช้จ่ายบานปลาย (ผู้ใช้อ่านของตัวเองได้ แต่เขียนได้เฉพาะเซิร์ฟเวอร์)

Row Level Security (RLS): หัวใจความปลอดภัย

≈ เปรียบเทียบ: ตู้ล็อกเกอร์ในห้องสมุด

กุญแจสาธารณะ (publishable key) ในแอปเหมือน "บัตรเข้าห้องสมุด" ใครมีก็เดินเข้าห้องได้ แต่ละคนมีล็อกเกอร์ของตัวเอง RLS คือ "ยามประจำล็อกเกอร์" ที่ดูบัตรประชาชน (การเข้าสู่ระบบ) ทุกครั้ง และเปิดให้เฉพาะล็อกเกอร์ที่มีชื่อเรา แม้แขกเกอร์จะโยมบัตรเข้าห้องสมุด (กุญแจในแอป) ไปได้ ก็เปิดล็อกเกอร์คนอื่นไม่ได้

supabase/migrations/20260927000000_init.sql

บรรทัด 67-87

```

67 -- -----
68 -- Row Level Security
69 -- -----
70 alter table public.profiles enable row level security;
71 alter table public.transactions enable row level security;
72 alter table public.ai_usage enable row level security;
73
74 drop policy if exists "own profile read" on public.profiles;
75 create policy "own profile read" on public.profiles
76   for select using (auth.uid() = id);
77 drop policy if exists "own profile update" on public.profiles;
78 create policy "own profile update" on public.profiles
79   for update using (auth.uid() = id) with check (auth.uid() = id);
80
81 drop policy if exists "own transactions" on public.transactions;
82 create policy "own transactions" on public.transactions
83   for all using (auth.uid() = user_id) with check (auth.uid() = user_id);
84
85 drop policy if exists "own ai usage read" on public.ai_usage;
86 create policy "own ai usage read" on public.ai_usage
87   for select using (auth.uid() = user_id);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:67

บรรทัด	ความหมาย
70-72	เปิด RLS ทั้ง 3 ตาราง พอเปิดแล้ว ค่าเริ่มต้นคือ ห้ามทุกอย่าง ต้องมี policy อนุญาตทีละเรื่อง
74-76	policy "own profile read": อ่านโปรไฟล์ได้ เฉพาะแถวที่ id ตรงกับคนที่เข้าสู่ระบบ (auth.uid() = id) drop policy if exists ด้านบนคือลบของเก่าก่อน ทำให้รันไฟล์ซ้ำได้
77-79	แก้โปรไฟล์ได้เฉพาะของตัวเอง using = ตรวจสอบแถวเดิม, with check = ตรวจสอบแถวหลังแก้ (กันแค่ id ให้เป็นของคนอื่น) ไม่มี policy insert/delete เพราะโปรไฟล์สร้างโดย trigger เท่านั้น

บรรทัด	ความหมาย
81–83	รายการเงิน: ทำได้ทุกอย่าง (for all = อ่าน เพิ่ม แก้ ลบ) เฉพาะแถวที่ user_id เป็นของตัวเอง
85–87	ai_usage อ่านได้อย่างเดียวและเฉพาะของตัวเอง

ทีมทดสอบเรื่องนี้ด้วยแผน MT-10 "บัญชี A ไม่เห็นรายการของบัญชี B" และรับตัวตรวจ Security Advisor ของ Supabase แล้วแก้ไขไม่เหลือคำเตือนเรื่องตาราง ไฟล์ harden.sql เปลี่ยน auth.uid() เป็น (select auth.uid()) เพื่อให้ฐานข้อมูลคำนวณครั้งเดียวต่อคำสั่ง แทนที่จะคำนวณทุกแถว (เร็วขึ้นเมื่อมีรายการเยอะ)

supabase/migrations/20260927000100_harden.sql
บรรทัด 1–25

```

1 -- Hardening after running the Supabase security/performance advisors on the first migration.
2
3 -- 1. Fixed search_path on trigger functions (advisor 0011).
4 create or replace function public.touch_updated_at() returns trigger
5 language plpgsql set search_path = '' as $$
6 begin
7     new.updated_at := now();
8     return new;
9 end $$;
10
11 -- 2. The sign-up trigger must not be callable through the REST API (advisors 0028/0029).
12 revoke execute on function public.handle_new_user() from public, anon, authenticated;
13
14 -- 3. Evaluate auth.uid() once per query instead of once per row (advisor 0003).
15 drop policy if exists "own profile read" on public.profiles;
16 create policy "own profile read" on public.profiles
17     for select using ((select auth.uid()) = id);
18
19 drop policy if exists "own profile update" on public.profiles;
20 create policy "own profile update" on public.profiles
21     for update using ((select auth.uid()) = id) with check ((select auth.uid()) = id);
22
23 drop policy if exists "own transactions" on public.transactions;
24 create policy "own transactions" on public.transactions
25     for all using ((select auth.uid()) = user_id) with check ((select auth.uid()) = user_id);

```

▶ เปิดใน VS Code (เร็ว)
GitHub
VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000100_harden.sql:1

Trigger: งานที่ฐานข้อมูลทำเองอัตโนมัติ

supabase/migrations/20260927000000_init.sql
บรรทัด 89–118

```

89 -- -----
90 -- Triggers
91 -- -----
92 create or replace function public.touch_updated_at() returns trigger
93 language plpgsql as $$
94 begin
95     new.updated_at := now();
96     return new;
97 end $$;
98
99 drop trigger if exists profiles_touch on public.profiles;
100 create trigger profiles_touch before update on public.profiles
101     for each row execute function public.touch_updated_at();

```

```
102 drop trigger if exists transactions_touch on public.transactions;
103 create trigger transactions_touch before update on public.transactions
104   for each row execute function public.touch_updated_at();
105
106 -- Create the profile row when a user signs up.
107 create or replace function public.handle_new_user() returns trigger
108 language plpgsql security definer set search_path = public as $$
109 begin
110   insert into public.profiles (id, display_name)
111   values (new.id, coalesce(new.raw_user_meta_data ->> 'display_name', ''))
112   on conflict (id) do nothing;
113   return new;
114 end $$;
115
116 drop trigger if exists on_auth_user_created on auth.users;
117 create trigger on_auth_user_created after insert on auth.users
118   for each row execute function public.handle_new_user();
```

▶ เปิดใน VS Code (เว็บ) GitHub VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:89

บรรทัด	ความหมาย
92–97	ฟังก์ชัน touch_updated_at (เขียนด้วยภาษา plpgsql ของ PostgreSQL) ตั้งเวลาแก้ไขล่าสุดเป็นตอนนี้
99–104	trigger = "ทุกครั้งก่อนแก้ไขในตาราง profiles หรือ transactions ให้เรียกฟังก์ชันนั้น" updated_at จึงถูกต้องเสมอ ไม่ต้องพึ่งแอป
107–114	ฟังก์ชัน handle_new_user: สร้างแถวโปรไฟล์ใหม่ โดยเอาชื่อเล่นที่ผู้ใช้กรอกตอนสมัคร (เก็บใน raw_user_meta_data) ถ้าไม่มีใช้ข้อความว่าง และถ้ามีแถวนี้อยู่แล้วไม่ต้องทำ (on conflict do nothing) security definer = รันด้วยสิทธิ์ของเจ้าของฟังก์ชัน (เพราะผู้ใช้ไม่มีสิทธิ์ insert โปรไฟล์เอง)
116–118	trigger บนตารางผู้ใช้ของระบบ Auth: หลังจากมีคนสมัครใหม่ ให้สร้างโปรไฟล์ทันที แอปจึงไม่ต้องสร้างเอง

กฎแฉ 2 แบบ: อันไหนอยู่ในแอปได้ อันไหนห้าม

	Publishable key (anon)	Service role key (secret)
อยู่ที่ไหน	ในแอป (.env, eas.json) เผยแพร่ได้	บนเซิร์ฟเวอร์ใน Edge Functions เท่านั้น (Supabase ใส่ให้เอง)
ทำอะไรได้	ทุกคำขอต้องผ่าน RLS เห็นเฉพาะข้อมูลของคน que เข้าสู่ระบบ	ข้าม RLS ได้ทั้งหมด อ่านเขียนทุกแถว
ใช้ทำอะไร	แอปอ่าน/เขียนรายการของผู้ใช้	ตรวจตัวตนผู้ใช้ และนับโควตา AI ใน Edge Function
ถ้ารั่ว	ไม่เป็นไร (ออกแบบมาให้เปิดเผยได้)	อันตรายมาก ต้องเปลี่ยนกฎแฉทันที

กฎแฉ AI (Claude หรือ Gemini) ก็เป็นความลับระดับเดียวกับ service role เก็บใน Supabase → Edge Functions → Secrets เท่านั้น

แอปต่อ Supabase ยังไง: src/data/supabase.ts

src/data/supabase.ts

บรรทัด 1-38

```
1 /**
2  * Supabase client. Keys come from environment variables (see .env.example).
3  * The publishable key is safe to ship in the app: Row Level Security in the
4  * database decides what each signed-in user can read and write.
5  * AI keys are NOT here; they live only in the Edge Functions.
6  *
7  * detectSessionInUrl is off: email links are handled by AppProvider (via
8  * authLinks.ts) the same way on the web and in the phone app.
9  */
10 import './sessionStorageSetup';
11 import './authLinks'; // records the address the web page was opened with
12 import { createClient, type SupabaseClient } from '@supabase/supabase-js';
13 import { AppState } from 'react-native';
14
15 const url = process.env.EXPO_PUBLIC_SUPABASE_URL;
16 const key = process.env.EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY;
17
18 /** null when the app was built without Supabase settings: only demo mode is available. */
19 export const supabase: SupabaseClient | null =
20   url && key
21     ? createClient(url, key, {
22       auth: {
23         storage: localStorage,
24         autoRefreshToken: true,
25         persistSession: true,
26         detectSessionInUrl: false,
27       },
28     })
29     : null;
30
31 if (supabase) {
32   AppState.addEventListener('change', (state) => {
33     if (state === 'active') supabase.auth.startAutoRefresh();
34     else supabase.auth.stopAutoRefresh();
35   });
36 }
37
38 export const cloudAvailable = supabase !== null;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/supabase.ts:1

บรรทัด	ความหมาย
1-9	คอมเมนต์: คุญแจมาจากตัวแปรสภาพแวดล้อม (.env) คุญแจ publishable ปลอดภัยเพราะ RLS และคุญแจ AI ไม่อยู่ที่นี่
10	ตั้งที่เก็บการเข้าสู่ระบบ (บนมือถือใช้ SQLite เก็บไว้ ปิดแอปแล้วยังเข้าสู่ระบบอยู่)
11	จุดที่อยู่ของหน้าเว็บตอนเปิด (ไว้จัดการลิงก์ยืนยันอีเมล)
15-16	อ่านที่อยู่เซิร์ฟเวอร์และคุญแจจาก .env (process.env.EXPO_PUBLIC_...)
18-29	ถ้ามีทั้งสองค่า สร้าง "client" สำหรับคุยกับ Supabase ถ้าไม่มี (build แบบไม่มีการตั้งค่า) ได้ null แล้วแอปใช้ได้เฉพาะโหมดทดลอง

บรรทัด	ความหมาย
23–26	ตั้งค่าการเข้าสู่ระบบ: เก็บ session ไว้ (persistSession), ต่ออายุ token อัตโนมัติ (token ของ Supabase มีอายุสั้น ค่าปกติคือ 1 ชั่วโมง), และไม่ให้อัปเดตไลบรารีอ่านลิงก์ในแอปที่อยู่อ้างอิง (AuthProvider จัดการเองเพื่อให้เว็บกับแอปทำงานเหมือนกัน)
31–36	ต่ออายุ token เฉพาะตอนแอปเปิดอยู่บนหน้าจอ พอแอปถูกพับไว้หยุดต่ออายุ (ประหยัดแบตเตอรี่)
38	บอกส่วนอื่นของแอปว่าใช้บัญชีจริงได้ไหม

การสมัครและเข้าสู่ระบบ

1. กรอกชื่อเล่น อีเมล รหัสผ่าน ≥ 8 ตัว [src/app/welcome.tsx](#)

↓ `supabase.auth.signUp`

2. Supabase สร้างผู้ใช้

trigger `handle_new_user` สร้างแถวโปรไฟล์ให้ทันที แล้วส่งอีเมลที่มีรหัส 6 หลักและลิงก์ยืนยัน

↓ `supabase.auth.verifyOtp` (กรอกรหัส) หรือแตะลิงก์ในอีเมล

3. ได้ session

แอปเก็บ token ไว้ในเครื่อง AppProvider เห็นเหตุการณ์ `SIGNED_IN` แล้วสร้าง `CloudRepo` โหลดข้อมูล

↓

4. โปรไฟล์ยังไม่ onboarded

`Stack.Protected` พาไปหน้าตั้งค่าเงิน กรอกรายละเอียดตอนนี้และเงินสำรอง แล้วเข้าหน้าหลัก

`src/app/welcome.tsx`

บรรทัด 148–172

```

148     const displayName = name.trim();
149     const { data, error } = await supabase!.auth.signUp({
150       email: address,
151       password,
152       options: { data: { display_name: displayName }, emailRedirectTo: authRedirectUrl() },
153     });
154     if (error) return setNotice({ text: authErrorMessage(error), tone: 'error' });
155     if (isExistingAccountSignUp(data.user)) {
156       setMode('signin');
157       setPassword('');
158       setNotice({
159         text: 'อีเมลนี้มีบัญชีอยู่แล้ว เข้าสู่ระบบด้วยรหัสผ่านเดิม หรือกด "ลืมรหัสผ่าน" เพื่อตั้งใหม่',
160         tone: 'info',
161         action: 'forgot',
162       });
163       return;
164     }
165     if (data.session) {
166       showAuthNotice({ kind: 'signed_up', name: displayName });
167       resetForm();
168       return;
169     }
170     setSent({ kind: 'confirm', email: address, name: displayName });
171     setResendIn(RESEND_WAIT_S);

```

บรรทัด	ความหมาย
148	ตัดช่องว่างหัวท้ายของชื่อเล่น
149–153	ส่งสมัคร: ส่งอีเมล รหัสผ่าน ชื่อเล่น (เก็บใน metadata ของผู้ใช้ ซึ่ง trigger จะคัดลอกไปใส่โปรไฟล์) และที่อยู่ลิงก์ในอีเมลจะพากลับมา
154	ถ้า error (เช่น รหัสผ่านสั้นเกินไป) แสดงข้อความภาษาไทยที่บอกวิธีแก้
155–164	กรณีพิเศษ: อีเมลนี้เคยสมัครแล้ว Supabase จะตอบเหมือนสำเร็จ (เพื่อไม่ให้คนนอกรู้ว่าอีเมลไหนมีบัญชี) แอปตรวจจบได้แล้วบอกให้ไปเข้าสู่ระบบหรือกดลิ้งรหัสผ่าน แทนที่จะขึ้น "สมัครสำเร็จ" หลอก ๆ
165–169	ถ้าโปรเจกต์ปิดการยืนยันอีเมลไว้ จะได้ session ทันที ขึ้นหน้าต่าง "สมัครบัญชีสำเร็จ"
170–171	ไม่ขึ้นไปหน้ากรอกรหัส 6 หลัก และเริ่มนับถอยหลังก่อนให้กดส่งรหัสใหม่ได้

Repo: ชั้นที่ซ่อนว่าข้อมูลอยู่ที่ไหน

หน้าจอไม่เคยคุยกับ Supabase หรือที่เก็บในเครื่องโดยตรง แต่คุยผ่าน **Repo** (ย่อจาก repository แปลว่าคลัง) ซึ่งมี 2 แบบที่หน้าตาเหมือนกันทุกประการ

src/data/repo.ts

บรรทัด 1–38

```

1  /**
2   * Data access. The screens never talk to Supabase or storage directly; they use
3   * a Repo. Two implementations:
4   *   - CloudRepo: Supabase (real accounts, data in PostgreSQL with RLS)
5   *   - DemoRepo: on-device only, starts with sample data (for trying the app
6   *                 without an account and as a fallback during a presentation)
7   */
8  import { Storage } from '../storage';
9  import { buildSampleTransactions, DEMO_OPENING_BALANCE_SATANG } from '../domain/sample';
10 import type { Profile, Transaction, TransactionInput } from '../domain/types';
11 import type { GoalInput, SavingsGoal } from '../domain/goals';
12 import { supabase } from '../supabase';
13
14 export class DuplicateSlipError extends Error {
15   constructor() {
16     super('รายการนี้มีอยู่แล้ว');
17     this.name = 'DuplicateSlipError';
18   }
19 }
20
21 export interface Repo {
22   mode: 'cloud' | 'demo';
23   getProfile(): Promise<Profile>;
24   updateProfile(patch: Partial<Omit<Profile, 'id'>>): Promise<Profile>;
25   /**
26    * Transactions of about the last 13 months, plus the net of confirmed money before them
27    * (it still counts in the balance, but is not needed as a list).
28    */
29   listTransactions(): Promise<{ txs: Transaction[]; carrySatang: number }>;
30   insert(input: TransactionInput): Promise<Transaction>;

```

```

31 update(id: string, patch: Partial<TransactionInput>): Promise<Transaction>;
32 remove(id: string): Promise<void>;
33 confirmMany(ids: string[]): Promise<void>;
34 listGoals(): Promise<SavingsGoal[]>;
35 insertGoal(input: GoalInput): Promise<SavingsGoal>;
36 updateGoal(id: string, patch: Partial<GoalInput>): Promise<SavingsGoal>;
37 removeGoal(id: string): Promise<void>;
38 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:1

บรรทัด	ความหมาย
1-7	คอมเมนต์: หน้าจอใช้ Repo เสมอ มี 2 ตัว คือ CloudRepo (Supabase, บัญชีจริง) และ DemoRepo (เก็บในเครื่อง มีข้อมูลตัวอย่าง ใช้ลองแอปหรือสำรองตอนพรีเซนต์)
14-19	error เฉพาะสำหรับสลิปซ้ำ (อธิบายในบท A5)
21	interface Repo = สัญญาว่า Repo ทุกตัวต้องทำอะไรเหล่านี้ได้
22	บอกว่าเป็นแบบไหน
23-24	อ่านและแก้ไขไฟล์
25-29	โหลดรายการประมาณ 13 เดือนล่าสุด และ "ยอดยกมา" ของรายการที่เก่ากว่านั้น (ยังต้องนับในยอดคงเหลือ แต่ไม่ต้องโหลดมาเป็นรายการ)
30-33	เพิ่ม แก้ ลบ รายการ และยืนยันหลายรายการพร้อมกัน (สลิปรอยืนยัน)
34-37	คำสั่งของกระปุกออม (FR-6)

≈ ทำไมต้องมี 2 แบบ

เหมือนปลั๊กไฟ: หน้าจอคือเครื่องใช้ไฟฟ้า Repo คือเต้าเสียบ จะต่อกับไฟบ้าน (Supabase) หรือแบตเตอรี่สำรอง (เครื่อง) หน้าจอไม่ต้องรู้ เสียบแล้วใช้ได้เหมือนกัน ทีมเราจึงพรีเซนต์ได้แม้อินเทอร์เน็ตในห้องเรียนล่ม และ E2E ทดสอบได้โดยไม่แตะบัญชีจริง

แปลงชื่อระหว่างแอปกับฐานข้อมูล

src/data/repo.ts

บรรทัด 44-93

```

44 type TxRow = {
45   id: string;
46   kind: 'income' | 'expense';
47   amount_satang: number;
48   category_key: string;
49   title: string;
50   note: string | null;
51   occurred_at: string;
52   source: 'manual' | 'slip';
53   status: 'draft' | 'confirmed';
54   slip_ref: string | null;
55   slip_image_hash: string | null;
56   ocr_confidence: number | null;
57   review_flags: string[] | null;

```

```

58   created_at: string;
59 };
60
61 const fromRow = (r: TxRow): Transaction => ({
62   id: r.id,
63   kind: r.kind,
64   amountSatang: Number(r.amount_satang),
65   categoryKey: r.category_key,
66   title: r.title,
67   note: r.note,
68   occurredAt: r.occurred_at,
69   source: r.source,
70   status: r.status,
71   slipRef: r.slip_ref,
72   slipImageHash: r.slip_image_hash,
73   ocrConfidence: r.ocr_confidence === null ? null : Number(r.ocr_confidence),
74   reviewFlags: r.review_flags ?? [],
75   createdAt: r.created_at,
76 });
77
78 function toRow(p: Partial<TransactionInput>) {
79   const row: Record<string, unknown> = {};
80   if (p.kind !== undefined) row.kind = p.kind;
81   if (p.amountSatang !== undefined) row.amount_satang = p.amountSatang;
82   if (p.categoryKey !== undefined) row.category_key = p.categoryKey;
83   if (p.title !== undefined) row.title = p.title;
84   if (p.note !== undefined) row.note = p.note;
85   if (p.occurredAt !== undefined) row.occurred_at = p.occurredAt;
86   if (p.source !== undefined) row.source = p.source;
87   if (p.status !== undefined) row.status = p.status;
88   if (p.slipRef !== undefined) row.slip_ref = p.slipRef;
89   if (p.slipImageHash !== undefined) row.slip_image_hash = p.slipImageHash;
90   if (p.ocrConfidence !== undefined) row.ocr_confidence = p.ocrConfidence;
91   if (p.reviewFlags !== undefined) row.review_flags = p.reviewFlags;
92   return row;
93 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:44

บรรทัด	ความหมาย
44–59	TxRow = หน้าตาของแถวในฐานข้อมูล ใช้ชื่อแบบ snake_case (คั่นด้วยขีดล่าง) ตามธรรมเนียมของ SQL
61–76	fromRow: แถวฐานข้อมูล → Transaction ของแอป ใช้ Number(. . .) ครอบตัวเลข เพราะ bigint บางครั้งส่งมาเป็นข้อความ และ review_flags ที่เป็น null ให้เป็น array ว่าง
78–93	toRow: กลับทาง ใส่เฉพาะช่องที่มีค่า (ไม่ใช่ undefined) เพราะตอนแก้รายการ เราส่งแค่บางช่อง ถ้าใส่ทุกช่อง ช่องที่ไม่ได้ส่งจะถูกเขียนทับเป็นค่าว่าง

โหลดรายการจากคลาวด์

src/data/repo.ts

บรรทัด 166–216

```

166 export function createCloudRepo(userId: string): Repo {
167   const db = supabase!;
168   return {
169     mode: 'cloud',
170     async getProfile() {
171       const { data, error } = await db.from('profiles').select('*').eq('id', userId).single();

```

```

172     if (error) throw error;
173     return profileFromRow(data as ProfileRow);
174 },
175 async updateProfile(patch) {
176     const { data, error } = await db.from('profiles').update(profileToRow(patch)).eq('id',
userId).select().single();
177     if (error) throw error;
178     return profileFromRow(data as ProfileRow);
179 },
180 async listTransactions() {
181     // Up to ~13 months: enough for the 1-year range. Paged to stay under the API row limit.
182     const since = new Date(Date.now() - 400 * 24 * 3600 * 1000).toISOString();
183     const all: Transaction[] = [];
184     for (let from = 0; ; from += 1000) {
185         const { data, error } = await db
186             .from('transactions')
187             .select('*')
188             .gte('occurred_at', since)
189             .order('occurred_at', { ascending: false })
190             // A unique tiebreaker, so rows with the same time are never skipped or read twice
across pages.
191             .order('id', { ascending: true })
192             .range(from, from + 999);
193         if (error) throw error;
194         all.push...(data as TxRow[]).map(fromRow);
195         if (!data || data.length < 1000) break;
196     }
197     // Older confirmed money still counts in the balance: add it up (two columns only).
198     let carrySatang = 0;
199     for (let from = 0; ; from += 1000) {
200         const { data, error } = await db
201             .from('transactions')
202             .select('kind,amount_satang')
203             .eq('status', 'confirmed')
204             .lt('occurred_at', since)
205             .order('occurred_at', { ascending: false })
206             // A unique tiebreaker, so rows with the same time are never skipped or read twice
across pages.
207             .order('id', { ascending: true })
208             .range(from, from + 999);
209         if (error) throw error;
210         for (const r of (data ?? []) as Pick<TxRow, 'kind' | 'amount_satang'>[]) {
211             carrySatang += r.kind === 'income' ? Number(r.amount_satang) : -
Number(r.amount_satang);
212         }
213         if (!data || data.length < 1000) break;
214     }
215     return { txs: all, carrySatang };
216 },

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:166

บรรทัด	ความหมาย
166–169	สร้าง CloudRepo ของผู้ใช้คนนี้ supabase! (เครื่องหมายตกใจ) บอก TypeScript ว่า "มั่นใจว่าไม่ใช่ null"
170–174	อ่านโปรไฟล์: จากตาราง profiles เลือกทุกคอลัมน์ ที่ id เท่ากับผู้ใช้ เอาแถวเดียว (RLS ก็กันไว้อีกชั้นอยู่แล้ว)
175–179	แก้ไขโปรไฟล์แล้วขอแถวใหม่กลับมา

บรรทัด	ความหมาย
182	คำนวณวันที่ย้อนหลัง 400 วัน (ประมาณ 13 เดือน พอสำหรับช่วง "1 ปี" ในหน้าหลัก)
184–196	โหลดทีละหน้า หน้าละ 1,000 แถว (API จำกัดจำนวนแถวต่อครั้ง) เรียงจากใหม่ไปเก่า ถ้าได้น้อยกว่า 1,000 แปลว่าหมดแล้ว หยุด
190–191	เรียงด้วย id เป็นตัวตัดสินรอง ถ้ามีหลายรายการเวลาเดียวกัน จะไม่หลุดหรือซ้ำข้ามหน้า (บ๊ีกที่ทีมเจอและแก้ตอนตรวจทั้ง repo)
197–214	ยอดยกมา: รายการที่ ยืนยันแล้ว และเก่ากว่า 400 วัน โหลดมาแค่ 2 คอลัมน์ (ประเภทและจำนวน) แล้วรวมเป็นตัวเลขเดียว รายรับบวก รายจ่ายลบ ยอดคงเหลือจึงยังถูกต้องแม้ผู้ใช้ใช้แอปมาหลายปี

โมดกทดลอง: DemoRepo

src/data/repo.ts

บรรทัด 264–305

```

264 // v2: rebalanced sample data (v1 could show a negative balance). Old v1 data is simply ignored.
265 const DEMO_KEY = 'mindpay.demo.v2';
266
267 interface DemoState {
268   profile: Profile;
269   txs: Transaction[];
270   goals?: SavingsGoal[];
271 }
272
273 const demoProfile: Profile = {
274   id: 'demo',
275   displayName: 'ผู้ทดลองใช้',
276   openingBalanceSatang: DEMO_OPENING_BALANCE_SATANG,
277   runwayFloorSatang: 50_000,
278   monthlyBudgetSatang: 1_000_000,
279   coachTone: 'friend',
280   onboarded: true,
281 };
282
283 const newId = () => `local-${Date.now().toString(36)}-${Math.random().toString(36).slice(2, 8)}`;
284
285 export async function resetDemoData() {
286   await Storage.removeItem(DEMO_KEY);
287 }
288
289 export function createDemoRepo(): Repo {
290   let cache: DemoState | null = null;
291
292   async function load(): Promise<DemoState> {
293     if (cache) return cache;
294     try {
295       const raw = await Storage.getItem(DEMO_KEY);
296       if (raw) cache = JSON.parse(raw) as DemoState;
297     } catch {
298       cache = null;
299     }
300     if (!cache) {
301       cache = { profile: demoProfile, txs: buildSampleTransactions() };
302       await save();
303     }
304     return cache;

```

บรรทัด 265: ข้อมูลโหมดทดลองเก็บในเครื่องด้วยชื่อ `mindpay.demo.v2` (v2 เพราะข้อมูลชุดแรกเคยทำให้ยอดติดลบ จึงเปลี่ยนชุดใหม่) บรรทัด 273–281 คือโปรไฟล์ของผู้ทดลองใช้: เงินเริ่มต้น ฿3,000 เลื่อนสำรอง ฿500 จบ ฿10,000 บรรทัด 292–305: ครั้งแรกที่เปิดยังไม่มีข้อมูล จะสร้างรายการตัวอย่าง (`buildSampleTransactions` ใน [src/domain/sample.ts](#)) ประมาณ 75 วันย้อนหลังจากวันนี้ แล้วเก็บไว้ ข้อมูลตัวอย่างจึงสดใหม่เสมอไม่ว่าเปิดวันไหน

เปิดแอปแล้วเกิดอะไรขึ้น: AppProvider เลือกโหมด

src/data/AppProvider.tsx

บรรทัด 271–318

```

271 // Decide the starting mode once, then follow Supabase auth changes.
272 useEffect(() => {
273   let cancelled = false;
274   (async () => {
275     // An email link that opened the app wins over the saved mode.
276     if (await handleAuthLink(await openingLink())) return;
277     const mode = await Storage.getItem(MODE_KEY).catch(() => null);
278     if (mode === 'demo') {
279       if (!cancelled) await activateRepo(createDemoRepo(), 'demo');
280       return;
281     }
282     if (!supabase) {
283       if (!cancelled) setStatus('signedOut');
284       return;
285     }
286     const { data } = await supabase.auth.getSession();
287     if (cancelled) return;
288     const id = data.session?.user.id;
289     if (id) {
290       if (activeUserRef.current !== id) await activateRepo(createCloudRepo(id), id);
291     } else if (!activeUserRef.current) setStatus('signedOut');
292   })();
293
294   const sub = supabase?.auth.onAuthStateChange((event, session) => {
295     // Supabase warns against calling other supabase methods inside this callback
296     // (it can deadlock the auth lock), so the actual work is deferred.
297     setTimeout(() => {
298       // A password-reset code signs in with the PASSWORD_RECOVERY event instead of SIGNED_IN.
299       // Opened offline with an expired token, the session only comes back later as
300       // TOKEN_REFRESHED:
301       // that signs in too, but only while nothing is open (never over the demo or another
302       // account).
303       const signedIn =
304         event === 'SIGNED_IN' || event === 'PASSWORD_RECOVERY' || (event === 'TOKEN_REFRESHED'
305         && !activeUserRef.current);
306       if (signedIn && session && activeUserRef.current !== session.user.id) {
307         Storage.setItem(MODE_KEY, 'cloud').catch(() => {});
308         activateRepo(createCloudRepo(session.user.id), session.user.id);
309       }
310       if (event === 'SIGNED_OUT' && repoRef.current?.mode === 'cloud') clear();
311     }, 0);
312   });
313
314   const stopLinks = onIncomingLink((url) => {
315     handleAuthLink(url);
316   });

```

```

313     return () => {
314         cancelled = true;
315         sub?.data.subscription.unsubscribe();
316         stopLinks();
317     };
318 }, [activateRepo, clear, handleAuthLink]);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:271`

บรรทัด	ความหมาย
272	ทำครั้งเดียวตอนแอปเริ่ม
275–276	ถ้าแอปถูกเปิดจากลิงก์ในอีเมล (ยืนยันสมัคร/ตั้งรหัสใหม่) จัดการลิงก์ก่อน ถ้าเข้าสู่ระบบสำเร็จก็จบ
277–281	อ่านโหมดที่จำไว้ในเครื่อง ถ้าเป็นโหมดทดลอง ใช้ DemoRepo
282–285	build ที่ไม่มี Supabase: ไปหน้าต้อนรับ
286–291	ถามว่ามี session ค้างอยู่ไหม (เคยเข้าสู่ระบบไว้) ถ้ามี ใช้ CloudRepo ของคนนั้น ไม่มีก็ไปหน้าต้อนรับ
294–309	ฟังเหตุการณ์ของระบบ Auth: เข้าสู่ระบบ (SIGNED_IN), กู้รหัสผ่าน (PASSWORD_RECOVERY) หรือ token กลับมา หลังออฟไลน์ → เปิดบัญชีนั้น · ออกจากระบบ (SIGNED_OUT) → ล้างข้อมูล ทั้งหมดห่อด้วย <code>setTimeout(..., 0)</code> เพราะ Supabase เตือนว่าเรียกคำสั่งอื่นในนี้ตรง ๆ อาจค้าง
310–312	ฟังลิงก์ที่เข้ามาระหว่างแอปเปิดอยู่
313–317	เก็บกวาดเมื่อปิด: เลิกฟังทุกอย่าง

`src/data/AppProvider.tsx`

บรรทัด 127–167

```

127     const loadAll = useCallback(async (r: Repo) => {
128         const gen = ++loadGen.current;
129         localChanges.current = new Map();
130         setLoadError(null);
131         try {
132             // Goals never block the app: if they cannot be loaded, the rest still works.
133             const [p, list, g] = await Promise.all([r.getProfile(), r.listTransactions(),
134             r.listGoals().catch(() => [] as SavingsGoal[])]);
135             if (gen !== loadGen.current || repoRef.current !== r) return;
136             // A slip saved by the automatic scan during the load is not lost when the list arrives.
137             const changes = localChanges.current;
138             const fresh = list.txs.filter((t) => !changes.has(t.id));
139             const kept = [...changes.values()].filter((t): t is Transaction => t !== null);
140             setProfile(p);
141             setTxs([...kept, ...fresh]);
142             setCarry(list.carrySatang);
143             setGoals(g);
144             setStatus('ready');
145         } catch (e) {
146             if (gen !== loadGen.current || repoRef.current !== r) return;
147             setLoadError(e instanceof Error ? e.message : 'โหลดข้อมูลไม่สำเร็จ');
148             setStatus('ready');
149         }
150     }, []);
151
152     const activateRepo = useCallback(
153         async (r: Repo, id: string) => {

```

```

153     if (activeUserRef.current !== id) {
154         // Another account (or leaving the demo): never show or use the previous one's data
           meanwhile.
155         setProfile(null);
156         setTxs([]);
157         setCarry(0);
158         setGoals([]);
159     }
160     activeUserRef.current = id;
161     repoRef.current = r;
162     setRepo(r);
163     setUserId(id);
164     await loadAll(r);
165 },
166 [loadAll],
167 );

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:127

บรรทัด	ความหมาย
127–128	loadAll: โหลดข้อมูลทั้งหมด แต่ละครั้งได้เลขรอบใหม่ (gen)
129–130	เริ่มจัดการเปลี่ยนแปลงในเครื่องใหม่ และล้าง error เก่า
133	โหลด 3 อย่างพร้อมกัน (un A5)
134	ถ้ามีการโหลดรอบใหม่กว่าเริ่มไปแล้ว หรือสลับบัญชีไปแล้ว ทั้งผลรอบนี้ (กันข้อมูลบัญชีเก่าโผล่ในบัญชีใหม่)
135–140	ถ้าระหว่างโหลด สแกนสลิปอัตโนมัติบันทึกรายการใหม่ไป เอารายการนั้นใส่กลับไว้ด้านบน ไม่ให้หายเพราะผลโหลดมากับ
139–143	ใส่ข้อมูลลง state ทั้งหมด แล้วบอกว่าพร้อม (หน้าจอวาดใหม่)
144–147	ถ้าโหลดพลาด เก็บข้อความ error ให้หน้าหลักแสดงปุ่ม "ลองอีกครั้ง"
151–167	activateRepo: เปลี่ยนไปใช้ Repo ของบัญชีใหม่ ถ้าเป็นคนละบัญชีกับเดิม ล้างข้อมูลเก่าออกก่อนทันที แล้วจํา Repo และโหลดข้อมูล

★ สรุป un A9

ฐานข้อมูลมี 4 ตาราง ทุกตารางเปิด RLS ให้แต่ละคนเห็นเฉพาะของตัวเอง เงินเก็บเป็นสตางค์และมี check กันค่าผิด สลิปซ้ำถูกกันด้วย unique index ระดับฐานข้อมูล แอปคุยกับฐานข้อมูลผ่าน Repo ซึ่งมีแบบคลาวด์และแบบในเครื่อง และ AppProvider เป็นคนเลือกว่าจะใช้แบบไหน

การเชื่อม AI: ทำไมต้องผ่านเซิร์ฟเวอร์ และทำงานยังไง

MindPay ใช้ AI 2 งาน คืออ่านสลิป (FR-4) และโค้ชน้องกล้า (FR-5) บทนี้อธิบายเส้นทางตั้งแต่แอปถามจนได้คำตอบ ทีละบรรทัด

AI ในแอปคืออะไร

AI ที่ใช้คือ โมเดลภาษาขนาดใหญ่ (Large Language Model, LLM) เช่น Claude ของบริษัท Anthropic และ Gemini ของ Google เราส่ง "ข้อความ" (และรูป) ไปทางอินเทอร์เน็ต มันตอบกลับเป็นข้อความ บริษัทเหล่านี้คิดเงินตามจำนวนที่ใช้ จึงต้องมี **API key** (กุญแจ) ไว้ระบุว่าใครเป็นคนจ่าย

คำ	ความหมาย
API	"ประตู" ให้โปรแกรมคุยกับโปรแกรม ส่งคำขอเป็น JSON ทาง HTTPS แล้วได้คำตอบเป็น JSON
API key	รหัสลับยาว ๆ ที่แนบไปกับคำขอ ใครได้กุญแจไป ก็ใช้ AI โดยให้เราจ่ายเงินได้
System prompt	"คำสั่งประจำตัว" ที่บอก AI ว่าต้องเป็นใคร ตอบยังไง ห้ามอะไร (ผู้ใช้ไม่เห็น)
Token	หน่วยนับข้อความของ AI (ประมาณคำหรือพยางค์) ใช้คิดราคาและจำกัดความยาวคำตอบ
JSON Schema	แบบฟอร์มที่บังคับให้ AI ตอบเป็นโครงสร้างตายตัว เช่น ต้องมีช่อง amount, dateIso
JSON	รูปแบบข้อมูลที่เขียนเหมือน object ของ JavaScript เช่น {"text": "สวัสดี"}

ทำไมแอปไม่เรียก AI เอง

! ห้ามใส่กุญแจ AI ไว้ในแอปเด็ดขาด

ไฟล์ APK หรือเว็บที่ผู้ใช้โหลดไป ใครก็แกะดูข้างในได้ ถ้ากุญแจ AI อยู่ในแอป คนแกะจะเอาไปใช้ฟรีโดยไม่ให้เราจ่าย (หรือทำให้กุญแจโดนแบน) ทีมเราจึงเก็บกุญแจไว้บนเซิร์ฟเวอร์ใน **Edge Functions** เท่านั้น แอปเรียก Edge Function แล้ว Edge Function ไปเรียก AI แทน

การมีเซิร์ฟเวอร์คั่นกลางให้ประโยชน์อีกหลายอย่าง

1. **ตรวจตัวตน:** เฉพาะผู้ใช้ที่เข้าสู่ระบบแล้วเท่านั้นที่ใช้ AI ได้
2. **จำกัดโควตา:** อ่านสลิปได้ 300 ใบ/วัน/คน ถามโค้ชได้ 60 ครั้ง/วัน/คน กันค่าใช้จ่ายบานปลาย
3. **เปลี่ยน AI ได้โดยไม่ต้องอัปเดตแอป:** สลับ Claude กับ Gemini แค่เปลี่ยน Secret บนเซิร์ฟเวอร์
4. **ทำความเข้าใจคำตอบ:** ตรวจสอบและแก้คำตอบของ AI ก่อนส่งถึงแอป (เช่น ปี พ.ศ. ที่ AI สัมแปลง)
5. **ความเป็นส่วนตัว:** ควบคุมได้ว่าส่งอะไรให้ AI บ้าง

เส้นทางของคำถาม 1 ข้อยิ่งน้องกล้า

1. แอป `src/app/(tabs)/coach.tsx`

ผู้ใช้พิมพ์ "ชื่อรองเท้า 1,290 ได้ไหม" แอปคำนวณ "ข้อเท็จจริง" เอง (ยอดเงิน ค่าเฉลี่ย Runway ก่อนและหลังชื่อ) ด้วยโค้ดใน `src/domain`

↓ `askCoach()` ใน `src/services/coach.ts`

2. `supabase.functions.invoke('coach')`

ส่ง `{context, question}` พร้อม token การเข้าสู่ระบบของผู้ใช้ ทาง HTTPS



3. Edge Function `coach` `supabase/functions/coach/index.ts`

ตรวจสอบ token → ตรวจสอบขนาดคำขอ → ตรวจสอบโควตา → สร้างข้อความให้ AI (system prompt + DATA + QUESTION)

↓ คุกกี้แวลู (Secret)

4. Claude หรือ Gemini

เขียนคำตอบภาษาไทย ไม่เกิน 90 คำ ใช้เฉพาะตัวเลขที่ได้รับ

↑ คำตอบย้อนกลับทางเดิม

5. แอปแสดงและอ่านออกเสียง

น้องกล้าขยับปากพร้อมซับไต้เตล (FR-5)

ฝั่งแอป: `src/services/coach.ts`

`src/services/coach.ts`

บรรทัด 1-25

```
1 /** FR-5: ask น้องกล้า, the AI coach (Edge Function "coach"). */
2 import { FunctionsHttpError } from '@supabase/supabase-js';
3 import { supabase } from '../data/supabase';
4
5 export class CoachError extends Error {}
6
7 export async function askCoach(params: { context: unknown; question: string }): Promise<string> {
8   if (!supabase) throw new CoachError('น้องกล้าตอบคำถามได้เมื่อเข้าสู่ระบบด้วยบัญชีจริง ระหว่างนี้แตะการ์ด "สิ่งที่ควรรู้ตอนนี้" ให้กล้าเล่าให้ฟังได้');
9   const { data, error } = await supabase.functions.invoke('coach', { body: params });
10  if (error) {
11    if (error instanceof FunctionsHttpError) {
12      const body = await (error.context as Response).json().catch(() => null);
13      if (body?.error?.code === 'quota') throw new CoachError('วันนี้คุยกับน้องกล้าครบโควตาแล้ว พรุ่งนี้มาคุยกันใหม่');
14      if (body?.error?.code === 'not_configured') {
15        throw new CoachError('โค้ช AI ยังไม่พร้อม: ผู้ดูแลต้องใส่ GEMINI_API_KEY ใน Supabase → Edge Functions → Secrets (หรือ key ที่ใส่ไว้ใช้ไม่ได้) ระหว่างนี้ดูคำแนะนำด้านบนได้เลย');
16      }
17      if (body?.error?.code === 'busy') throw new CoachError('ตอนนี้มีคนคุยกับน้องกล้าเยอะ รอสักครู่แล้วถามใหม่');
18      if (body?.error?.code === 'unauthorized') throw new CoachError('เซสชันหมดอายุ กรุณาเข้าสู่ระบบอีกครั้ง');
19    }
20    throw new CoachError('ติดต่อน้องกล้าไม่ได้ตอนนี้ ลองใหม่อีกครั้ง');
21  }
22  const text = (data as { text?: string })?.text;
```

```

23 if (!text) throw new CoachError('น้องกล้าไม่ได้ตอบกลับ ลองถามใหม่อีกครั้งนะ');
24 return text;
25 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/coach.ts:1

บรรทัด	ความหมาย
2	นำชนิด error ของ Supabase ที่บอกว่า "เซิร์ฟเวอร์ตอบกลับมาว่าผิดพลาด" มาใช้
5	สร้าง error ชนิดของโค้ชเอง ข้อความของมันคือสิ่งที่จะแสดงให้ผู้ใช้เห็น
7	ฟังก์ชัน askCoach รับข้อมูลสรุป (context) และคำถาม คืนข้อความคำตอบ
8	ไม่มี Supabase (เช่น build ที่ไม่ได้ตั้งค่า) → บอกผู้ใช้ว่าต้องใช้บัญชีจริง
9	เรียก Edge Function ชื่อ coach ส่ง context และ question ไป ไสวราวีแบบ token การเข้าสู่ระบบให้เอง
10–21	ถ้าผิดพลาด อ่านรหัส error ที่เซิร์ฟเวอร์ส่งมา แล้วแปลงเป็นข้อความภาษาไทยที่เป็นมิตร: ครบโควตา (quota), ยังไม่ใส่ กุญแจ AI (not_configured), AI ไม่ว่าง (busy), หมดอายุการเข้าสู่ระบบ (unauthorized) อย่างอื่นบอกให้ลองใหม่
22–24	ดึงข้อความคำตอบ ถ้าไม่มีก็แจ้ง ถ้ามีส่งกลับไปให้หน้าโค้ช

ฝั่งเซิร์ฟเวอร์: Edge Function coach ที่ละบรรทัด

Edge Function เขียนด้วย TypeScript เหมือนแอป แต่รันบนเซิร์ฟเวอร์ของ Supabase ด้วยโปรแกรมชื่อ **Deno** (คล้าย Node.js) ฟังก์ชันหนึ่งตัวคือไฟล์เดอร์หนึ่งใน supabase/functions/ มีไฟล์ index.ts และไฟล์เดอร์ _shared เก็บโค้ดที่ใช้ร่วมกัน

supabase/functions/coach/index.ts

บรรทัด 1–26

```

1 // FR-5: the AI coach is น้องกล้า, the MindPay mascot. It explains the user's
2 // CONFIRMED numbers in one calm, friendly voice; the app reads the answer
3 // aloud, so it is written to be spoken. The app sends only totals by category
4 // (no slip images, no names of people, no account numbers).
5 import { aiProvider, BusyError, callAI, corsHeaders, envLimit, fail, json, NotConfiguredError,
6   requireUser, takeQuota } from '../_shared/common.ts';
7 import { plainText } from '../_shared/helpers.ts';
8
9 const DAILY_LIMIT = envLimit('COACH_DAILY_LIMIT', 60);
10 /** A question and a summary of totals: far below this. */
11 const MAX_BODY_BYTES = 64_000;
12
13 const SYSTEM = `You are น้องกล้า (Nong Kla), the mascot of MindPay, a Thai personal finance app for
14   students and first-jobbers.
15   You are a small golden money-tree sapling and the user's personal money coach.
16   Voice: calm, composed and warm (สุขุม เป็นมิตร), like a thoughtful friend who is good with money.
17   - Speak natural, everyday Thai. Call yourself "กล้า". Do not use ครับ orค่ะ; soft endings such as
18     "นะ" are fine.
19   - Your answer is read aloud by a text-to-speech voice and shown as subtitles, so write short
20     spoken sentences separated by spaces,
21     no emoji, no bullet points, no markdown, no lists, no headings.
22   Rules:
23   - Reply in Thai, at most 90 words.

```

- 22 - Use ONLY the numbers in the DATA block. Never invent amounts, dates, shops or trends. If the data cannot answer, say what is missing.
- 23 - Structure: what the numbers show -> why it matters -> one or two small suggestions -> leave the decision to the user.
- 24 - Never shame or scold ("ใช้เยอะเกินไป!" is not allowed). Be kind about mistakes.
- 25 - When the user asks "can I buy X", use the "purchaseCheck" numbers if present and state the runway before and after.
- 26 - You are not a licensed financial advisor: no investment, loan or crypto recommendations.`;

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/coach/index.ts:1`

บรรทัด	ความหมาย
1-4	คอมเมนต์: โค้ชคือน้องกล้า อธิบายตัวเลขที่ยืนยันแล้ว คำตอบจะถูกอ่านออกเสียงจึงเขียนแบบภาษาพูด แอปส่งเฉพาะยอดรวมตามหมวด (ไม่มีรูปสลิป ชื่อคน หรือเลขบัญชี)
5-6	นำตัวช่วยจาก <code>_shared</code> มาใช้ (นามสกุล <code>.ts</code> ต้องเขียนเต็มใน Deno)
8	โควตาต่อวัน อ่านจาก <code>Secret COACH_DAILY_LIMIT</code> ถ้าไม่ได้ตั้งใช้ 60
10	ขนาดคำขอสองสูงสุด 64,000 ไบต์ (คำถามกับตัวเลขสรุปเล็กกว่านี้มาก กันคนส่งข้อมูลขยะก้อนใหญ่)
12-26	System prompt (เขียนเป็นภาษาอังกฤษเพราะ AI ทำตามคำสั่งภาษาอังกฤษได้แม่นยำกว่า แต่สั่งให้ตอบเป็นไทย) บอกว่า: เป็นน้องกล้า มาสคอตต้นกล้าสีทอง · น้ำเสียงสุ่ม เป็นมิตร เรียกตัวเองว่า "กล้า" ไม่ใช่ครับ/ค่ะ · คำตอบถูกอ่านออกเสียง จึงห้ามอีโมจิ ห้ามรายการ ห้าม markdown · ตอบไม่เกิน 90 คำ · ใช้เฉพาะตัวเลขในส่วน DATA ห้ามแต่งตัวเลขขึ้นมาเอง · โครงสร้างคำตอบ: ตัวเลขบอกอะไร → สำคัญยัง → คำแนะนำเล็ก ๆ 1-2 ข้อ → ให้ผู้ใช้ตัดสินใจเอง · ห้ามตำหนิ · ถ้าถาม "ซื้อได้ไหม" ให้ใช้ตัวเลข <code>purchaseCheck</code> บอก Runway ก่อนและหลัง · ไม่ใช้ที่ปรึกษาการเงินที่มีใบอนุญาต ห้ามแนะนำลงทุน กู้ยืม คริปโต

`supabase/functions/coach/index.ts`

บรรทัด 28-76

```

28 Deno.serve(async (req) => {
29   if (req.method === 'OPTIONS') return new Response('ok', { headers: corsHeaders });
30   if (req.method !== 'POST') return fail('method_not_allowed', 'Use POST', 405);
31
32   const userId = await requireUser(req);
33   if (!userId) return fail('unauthorized', 'Sign in again', 401);
34
35   if (Number(req.headers.get('content-length') ?? 0) > MAX_BODY_BYTES) return fail('too_large',
'Request is too large', 413);
36   let body: { context?: unknown; question?: string };
37   try {
38     body = await req.json();
39   } catch {
40     return fail('bad_request', 'Body must be JSON', 400);
41   }
42   if (!body || typeof body !== 'object') return fail('bad_request', 'Body must be a JSON object',
400);
43   const question = (body.question ?? '').toString().slice(0, 400).trim();
44   const data = JSON.stringify(body.context ?? {}).slice(0, 6000);
45
46   if (!aiProvider()) return fail('not_configured', 'AI key is not set on the server', 503);
47
48   try {
49     if (!(await takeQuota(userId, 'coach', DAILY_LIMIT))) {
50       return fail('quota', 'Daily coach limit reached, try again tomorrow', 429);
51     }
52     const text = await callAI({

```

```

53     task: 'coach',
54     system: SYSTEM,
55     maxTokens: 500,
56     content: [
57       {
58         type: 'text',
59         text: `DATA:\n${data}\n\nQUESTION:\n${question} || 'สรุปสถานะการเงินของเงินช่วงนี้ และบอกสิ่งที่คว
60       },
61     ],
62   });
63   return json({ text: plainText(text) });
64 } catch (e) {
65   if (e instanceof NotConfiguredError) {
66     console.error('coach: AI key missing or refused', e.message);
67     return fail('not_configured', 'AI key is not set on the server or was refused', 503);
68   }
69   if (e instanceof BusyError) {
70     console.warn('coach: AI busy', e.message);
71     return fail('busy', 'The AI service is busy, try again shortly', 503);
72   }
73   console.error('coach failed', e);
74   return fail('coach_error', 'The coach is unavailable right now', 502);
75 }
76 });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/coach/index.ts:28

บรรทัด	ความหมาย
28	Deno.serve = "เปิดร้าน" รอรับคำขอ ทุกคำขอที่เข้ามาจะเรียกฟังก์ชันนี้ 1 ครั้ง
29	คำขอแบบ OPTIONS คือเบราว์เซอร์ "ถามทาง" ก่อนส่งจริง (กฎ CORS ของเว็บ) ตอบ ok พร้อม header ที่อนุญาต
30	รับเฉพาะ POST อย่างอื่นตอบ 405
32–33	ตรวจตัวตน: อ่าน token จากคำขอแล้วถาม Supabase ว่าเป็นใคร ไม่ผ่าน → 401 (แอปจะบอก "เซสชันหมดอายุ")
35	คำขอใหญ่เกิน → 413 (ตรวจจากหัวคำขอ ก่อนอ่านเนื้อหา จึงไม่เปลืองหน่วยความจำ)
36–42	อ่านเนื้อหาเป็น JSON ถ้าอ่านไม่ได้หรือไม่ใช่ object → 400
43	คำถามตัดเหลือไม่เกิน 400 ตัวอักษร และตัดช่องว่างหัวท้าย
44	ข้อมูลสรุปแปลงเป็นข้อความ JSON ตัดเหลือไม่เกิน 6,000 ตัวอักษร (คุมค่าใช้จ่าย)
46	ไม่มีกุญแจ AI ใน Secrets → 503 not_configured
49–51	ตรวจโควตา และนับเพิ่ม 1 ในคำสั่งเดียว เกิน 60 ครั้งใน 24 ชั่วโมง → 429
52–62	เรียก AI ส่ง system prompt, จำกัดคำตอบ 500 token, และข้อความที่มี 2 ส่วน: DATA: ตามด้วยตัวเลข และ QUESTION: ตามด้วยคำถาม (ถ้าผู้ใช้ไม่ได้ถาม ใช้คำถามตั้งต้น "สรุปสถานะการเงินของเงินช่วงนี้...")
63	ล้าง markdown ที่ AI อาจใส่มา (plainText) แล้วตอบแอปเป็น {"text": "..."}
64–75	จัดการ error ตามชนิด: กุญแจผิด/ไม่มี → 503 not_configured, AI ไม่ว่าง → 503 busy, อื่น ๆ → 502 และบันทึก log ไว้ดูใน Dashboard (บันทึกเฉพาะชนิด error ไม่บันทึกคำถามหรือคำตอบของผู้ใช้)

ตัวอย่างกลาง: supabase/functions/_shared/common.ts

บรรทัด 17-83

```
17 /** Read one environment variable; a name the runtime refuses counts as "not set". */
18 const env = (name: string) => {
19   try {
20     return Deno.env.get(name);
21   } catch {
22     return undefined;
23   }
24 };
25 const geminiKey = () => readKey(env, GEMINI_KEY_NAMES);
26 const claudeKey = () => readKey(env, CLAUDE_KEY_NAMES);
27
28 export const corsHeaders = {
29   'Access-Control-Allow-Origin': '*',
30   'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
31   'Access-Control-Allow-Methods': 'POST, OPTIONS',
32 };
33
34 export function json(body: unknown, status = 200) {
35   return new Response(JSON.stringify(body), {
36     status,
37     headers: { ...corsHeaders, 'Content-Type': 'application/json' },
38   });
39 }
40
41 /** Error body the app understands: { error: { code, message } } */
42 export function fail(code: string, message: string, status: number) {
43   return json({ error: { code, message } }, status);
44 }
45
46 /** Returns the signed-in user's id, or null when the request has no valid session. */
47 export async function requireUser(req: Request): Promise<string | null> {
48   const token = req.headers.get('Authorization')?.replace(/^Bearer\s+/i, '');
49   if (!token) return null;
50   const { data, error } = await admin().auth.getUser(token);
51   if (error || !data.user) return null;
52   return data.user.id;
53 }
54
55 function admin() {
56   return createClient(Deno.env.get('SUPABASE_URL')!, Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!,
57     {
58       auth: { persistSession: false },
59     });
60 }
61
62 /** Thrown when no AI key is set in the function secrets, or the key was refused. */
63 export class NotConfiguredError extends Error {}
64
65 /** The AI service is rate-limiting us (e.g. Gemini free tier) or overloaded: try again shortly. */
66 export class BusyError extends Error {}
67
68 /** A daily limit from the function settings; a missing or non-number setting uses the default. */
69 export function envLimit(name: string, fallback: number): number {
70   const n = Number(Deno.env.get(name));
```

```

70 return Number.isFinite(n) && n > 0 ? Math.floor(n) : fallback;
71 }
72
73 /**
74  * Daily cap per user so a leaked account or a bug cannot run up the AI bill.
75  * Returns false when the user is over the limit; otherwise records one use.
76  * Counting and recording happen in one database call under a per-user lock
77  * (take_ai_quota), so many requests at once cannot all slip under the limit.
78  */
79 export async function takeQuota(userId: string, kind: 'slip' | 'coach', dailyLimit: number) {
80   const { data, error } = await admin().rpc('take_ai_quota', { p_user: userId, p_kind: kind,
81     p_limit: dailyLimit });
82   if (error) throw error;
83   return data === true;
84 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/_shared/common.ts:17`

บรรทัด	ความหมาย
17–24	อ่านค่า Secret จากเชิร์ฟเวอร์ (<code>Deno.env.get()</code>) ถ้าชื่อแปลจนอ่านไม่ได้ ถือว่าไม่ได้ตั้ง
25–26	หากุญแจ Gemini และ Claude จากรายชื่อที่ยอมรับ (ดู <code>readKey</code> ด้านล่าง)
28–32	header CORS: ยอมให้เว็บจากทุกที่เรียกได้ (ความปลอดภัยมาจาก token ไม่ใช่จากที่อยู่เว็บ)
34–39	<code>json()</code> สร้างคำตอบ HTTP ที่เป็น JSON พร้อม header
41–44	<code>fail()</code> สร้างคำตอบ error รูปแบบเดียวกันทุกครั้ง <code>{ "error": { "code", "message" } }</code> ที่แอปอ่าน code ได้ (บรรทัด 13–18 ของ <code>coach.ts</code> ผังแอป)
46–53	<code>requireUser</code> : ตัดคำว่า "Bearer " ออกจาก header Authorization ได้ token แล้วถาม Auth ว่า token นี้เป็นของใคร คืน id ผู้ใช้ หรือ null
55–59	<code>admin()</code> : client ที่ใช้ service role key (ข้าม RLS ได้) มีเฉพาะบนเชิร์ฟเวอร์
61–65	error 2 ชนิด: ไม่ได้ตั้งกุญแจ/กุญแจใช้ไม่ได้ และ AI ไม่ว่าง (ติดลิมิตหรือโหลดเกิน)
67–71	อ่านไควตาจาก Secret ถ้าไม่ใช่ตัวเลขบวกใช้ค่าตั้งต้น
73–83	<code>takeQuota</code> : เรียกฟังก์ชัน SQL <code>take_ai_quota</code> ในฐานข้อมูล (rpc = เรียกฟังก์ชันระยะไกล) ซึ่งนับและบันทึกในชั้นตอนเดียว

ทำไมนับไควตาต้อง "ทำในชั้นตอนเดียว"

`supabase/migrations/20261001000000_atomic_ai_quota.sql`

บรรทัด 1–44

```

1 -- Atomic daily AI quota (audit 1 Oct 2569).
2 -- Before: the Edge Functions counted today's uses and inserted a new one in two
3 -- separate calls, so many requests sent at once could all pass the limit.
4 -- Now counting and recording happen in one call, under a lock per user and kind.
5 -- Called only by the Edge Functions with the service role.
6
7 create or replace function public.take_ai_quota(p_user uuid, p_kind text, p_limit integer)
8 returns boolean
9 language plpgsql
10 security definer
11 set search_path = ''

```

```

12 as $$
13 declare
14     used integer;
15 begin
16     if p_kind not in ('slip', 'coach') or p_limit is null or p_limit < 1 then
17         return false;
18     end if;
19     perform pg_advisory_xact_lock(hashtextextended(p_user::text || ':' || p_kind, 0));
20     select count(*) into used
21         from public.ai_usage
22         where user_id = p_user and kind = p_kind and created_at >= now() - interval '24 hours';
23     if used >= p_limit then
24         return false;
25     end if;
26     insert into public.ai_usage (user_id, kind) values (p_user, p_kind);
27     return true;
28 end;
29 $$;
30
31 revoke execute on function public.take_ai_quota(uuid, text, integer) from public, anon,
    authenticated;
32 grant execute on function public.take_ai_quota(uuid, text, integer) to service_role;
33
34 -- Amounts have the same upper bound as savings goals ($1,000,000,000), so a misread
35 -- amount can never overflow what the app handles.
36 alter table public.transactions
37     drop constraint if exists transactions_amount_max,
38     add constraint transactions_amount_max check (amount_satang <= 1000000000000);
39 alter table public.profiles
40     drop constraint if exists profiles_amounts_max,
41     add constraint profiles_amounts_max check (
42         abs(opening_balance_satang) <= 1000000000000
43         and (monthly_budget_satang is null or monthly_budget_satang <= 1000000000000)
44     );

```

► เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20261001000000_atomic_ai_quota.sql:1

บรรทัด	ความหมาย
1-5	คอมเมนต์เล่าบ๊วกเดิม: เดิมนับจำนวนครั้งวันนี้นับกับบันทึกครั้งใหม่เป็น 2 คำสั่งแยกกัน ถ้าส่งคำขอพร้อมกันหลายอัน ทุกอันนับได้ "ยังไม่เกิน" แล้วผ่านหมด (race condition) แก้ให้นับและบันทึกในคำสั่งเดียวภายใต้กุญแจล็อก
7-14	สร้างฟังก์ชัน SQL รับ id ผู้ใช้ ชนิตงาน (slip/coach) และเพดาน คำนวณจริง/เท็จ รับด้วยสิทธิ์เจ้าของ (security definer)
16-18	ค่าที่ส่งมาผิด → ไม่อนุญาต
19	ล็อก ต่อผู้ใช้ต่อชนิตงาน: คำขอที่ 2 ของคนเดียวกันต้องรอคำขอแรกเสร็จก่อน (ล็อกปลดเองเมื่อจบคำสั่ง)
20-22	นับจำนวนการใช้ใน 24 ชั่วโมงล่าสุด
23-25	ถึงเพดานแล้ว → ไม่อนุญาต
26-27	ยังไม่ถึง → บันทึกการใช้ 1 ครั้ง แล้วอนุญาต
31-32	ห้ามผู้ใช้ทั่วไปเรียกฟังก์ชันนี้ตรง ๆ ให้เฉพาะเซิร์ฟเวอร์ (service_role)
34-44	(ไฟล์เดียวกัน) เพิ่มเพดานจำนวนเงิน \$1,000,000,000 ให้ตารางรายการและโปรไฟล์ ตรงกับ MAX_SATANG ในแอป

เลือก AI ตัวไหน: Claude หรือ Gemini

supabase/functions/_shared/helpers.ts

บรรทัด 1-45

```
1 // Pure helpers for the Edge Functions. No Deno APIs here, so the same file is
2 // unit-tested with vitest (see __tests__/helpers.test.ts).
3
4 export type AiProvider = 'claude' | 'gemini';
5
6 /**
7  * Secret names accepted for each service. The first is the documented one;
8  * the rest are Google's other standard name and common typing slips seen in
9  * the Supabase dashboard (a secret saved as "GEMINI API KEY", 28 ก.ย. 2569).
10 */
11 export const GEMINI_KEY_NAMES = [
12   'GEMINI_API_KEY',
13   'GOOGLE_API_KEY',
14   'GOOGLE_GENERATIVE_AI_API_KEY',
15   'GEMINI API KEY',
16   'GEMINI-API-KEY',
17   'GEMINI_KEY',
18 ];
19 export const CLAUDE_KEY_NAMES = ['ANTHROPIC_API_KEY', 'CLAUDE_API_KEY', 'ANTHROPIC API KEY'];
20
21 /**
22  * The first secret that is set, with spaces, line breaks or quotes from
23  * copy-paste removed. Returns null when none is set.
24 */
25 export function readKey(get: (name: string) => string | undefined | null, names: string[]):
  string | null {
26   for (const name of names) {
27     const raw = get(name);
28     const clean = raw?.trim().replace(/^[\"']|['\"]$/g, '').trim();
29     if (clean) return clean;
30   }
31   return null;
32 }
33
34 /**
35  * Which AI service answers: Claude when its key is set, otherwise Gemini.
36  * AI_PROVIDER=claude|gemini forces one. Returns null when no usable key is set.
37 */
38 export function pickProvider(env: { forced?: string | null; hasClaude: boolean; hasGemini:
  boolean }): AiProvider | null {
39   const forced = env.forced?.trim().toLowerCase();
40   if (forced === 'claude') return env.hasClaude ? 'claude' : null;
41   if (forced === 'gemini') return env.hasGemini ? 'gemini' : null;
42   if (env.hasClaude) return 'claude';
43   if (env.hasGemini) return 'gemini';
44   return null;
45 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/helpers.ts:1

บรรทัด ความหมาย

1-2 โฟลน์นี้เป็นฟังก์ชันล้วน ไม่ใช้คำสั่งของ Deno จึงเอาไปเทสต์ด้วย vitest ได้เหมือนโค้ดในแอป

บรรทัด	ความหมาย
6–19	รายชื่อ Secret ที่ยอมรับ ตัวแรกคือชื่อที่ถูกต้อง ตัวอื่นเป็นชื่อสำรองและ ชื่อที่พิมพ์ผิดบ่อย คอมเมนต์เล่าว่าเคยมีคนตั้งชื่อ Secret เป็น "GEMINI API KEY" (มีเว้นวรรค) เมื่อ 28 ก.ย. 2569 ทำให้ AI ไม่ทำงาน ทีมจึงให้เซิร์ฟเวอร์รับชื่อเหล่านี้ด้วย
25–32	readKey: ใส่ชื่อที่ละตัว เจอตัวแรกที่มีค่า ตัดช่องว่าง การขึ้นบรรทัด และเครื่องหมายคำพูดที่ติดมาตอนถือปวงออก
38–45	pickProvider: ถ้าตั้ง AI_PROVIDER บังคับไว้ ใช้ตัวนั้น (ต้องมีคีย์แจด้วย) ไม่จันมีคีย์แจ Claude ใช้ Claude ก่อน ไม่มีก็ใช้ Gemini ไม่มีเลยคืน null

	Claude (Anthropic)	Gemini (Google)
Secret	ANTHROPIC_API_KEY	GEMINI_API_KEY
รุ่นที่ใช้ (ค่าเริ่มต้น)	claude-haiku-4-5 (เปลี่ยนได้ด้วย SLIP_MODEL, COACH_MODEL)	gemini-3.5-flash-lite แล้วลองรุ่นถัดไปเองถ้าใช้ไม่ได้
ค่าใช้จ่าย	ต้องเติมเครดิต อ่านสลิปใบละประมาณ 10 สตางค์	มีแบบฟรี (มีเพดานต่อนาที/ต่อวัน)
ข้อมูลของผู้ใช้	ไม่นำไปฝึกโมเดล	แบบฟรี Google อาจนำข้อมูลไปปรับปรุงโมเดล (README บอกไว้ตรง ๆ)

ส่งคำขอไปที่ Claude: callClaude

supabase/functions/_shared/common.ts

บรรทัด 114–200

```
114 /** Ask the configured AI and return its text reply. */
115 export async function callAI(req: AiRequest): Promise<string> {
116   return (await callAIWithModel(req)).text;
117 }
118
119 /**
120  * Longest wait for one call to the AI service, and for all tries of one request together
121  * (the platform stops a function after 150 s; both slip reads run at the same time).
122  */
123 const FETCH_TIMEOUT_MS = 45_000;
124 const REQUEST_DEADLINE_MS = 110_000;
125
126 /** Ask the configured AI; also says which model answered (the two slip reads compare it). */
127 export async function callAIWithModel(req: AiRequest): Promise<{ text: string; model: string }> {
128   const provider = aiProvider();
129   if (!provider) throw new NotConfiguredError('No AI key is set');
130   const started = Date.now();
131   const deadline = started + REQUEST_DEADLINE_MS;
132   const result = provider === 'claude' ? await callClaude(req, deadline) : await callGemini(req,
133     deadline);
134   // Metadata only: never log images, questions or replies.
135   console.log(JSON.stringify({ ai: provider, model: result.model, task: req.task, variant:
136     req.variant ?? 'primary', ms: Date.now() - started }));
137   return result;
138 }
139
140 /** fetch with a time limit; running out of time counts as the AI being busy (the app waits and
141   retries). */
142 async function timedFetch(url: string, init: RequestInit, deadline: number): Promise<Response> {
```

```

140   const left = deadline - Date.now();
141   if (left <= 1000) throw new BusyError('AI request deadline reached');
142   try {
143     return await fetch(url, { ...init, signal: AbortSignal.timeout(Math.min(FETCH_TIMEOUT_MS,
144   left)) });
145   } catch (e) {
146     const name = (e as Error)?.name;
147     if (name === 'TimeoutError' || name === 'AbortError') throw new BusyError(`AI request timed
148   out (${url.split('/')[2]})`);
149     throw e;
150   }
151 }
152
153 const sleep = (ms: number) => new Promise((resolve) => setTimeout(resolve, ms));
154
155 async function callClaude(req: AiRequest, deadline: number): Promise<{ text: string; model:
156   string }> {
157   const key = claudeKey();
158   const main = Deno.env.get(req.task === 'slip' ? 'SLIP_MODEL' : 'COACH_MODEL') ??
159   CLAUDE_DEFAULT_MODEL;
160   const model = req.variant === 'verify' ? (Deno.env.get('VERIFY_MODEL') ?? main) : main;
161   const body: Record<string, unknown> = {
162     model,
163     max_tokens: req.maxTokens,
164     system: req.system,
165     messages: [
166       {
167         role: 'user',
168         content: req.content.map((c) =>
169           c.type === 'text' ? c : { type: 'image', source: { type: 'base64', media_type:
170   c.mediaType, data: c.data } },
171       ),
172     ],
173   };
174   if (req.schema) {
175     body.output_config = { format: { type: 'json_schema', schema: req.schema } };
176   }
177   const res = await timedFetch(
178     'https://api.anthropic.com/v1/messages',
179     {
180       method: 'POST',
181       headers: {
182         'x-api-key': key,
183         'anthropic-version': '2023-06-01',
184         'content-type': 'application/json',
185       },
186       body: JSON.stringify(body),
187     },
188     deadline,
189   );
190   if (!res.ok) {
191     const detail = await res.text();
192     const msg = `Claude API ${res.status}: ${detail.slice(0, 300)}`;
193     if (isInvalidKey(res.status, detail)) throw new NotConfiguredError(msg);
194     if (res.status === 429 || res.status === 529) throw new BusyError(msg);
195     throw new Error(msg);
196   }
197   const data = await res.json();
198   const text = (data.content ?? []).
199     .filter((b: { type: string }) => b.type === 'text')

```

```

196     .map((b: { text: string }) => b.text)
197     .join('');
198     if (!text) throw new Error('Claude API returned no text');
199     return { text, model };
200 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/common.ts:114

บรรทัด	ความหมาย
114–117	callAI เรียกตัวที่บอกชื่อรุ่นด้วย แล้วคืนแค่ข้อความ
119–124	เวลาสูงสุด: รอ AI ครั้งละไม่เกิน 45 วินาที และทั้งคำขอไม่เกิน 110 วินาที (Supabase จะตัดฟังก์ชันที่ 150 วินาที)
127–136	เลือก AI แล้วเรียก จับเวลา และเขียน log เฉพาะข้อมูลคำกับ (ตัวไหน รุ่นไหน งานอะไร ใช้กี่มิลลิวินาที) ไม่เคยบันทึกรูปคำถาม หรือคำตอบ
139–149	timedFetch: ส่งคำขอทางอินเทอร์เน็ตแบบมีเวลาจำกัด ถ้าหมดเวลาถือว่า AI ไม่ว่าง (แอปจะรอแล้วลองใหม่)
153–156	เลือกรุ่นของ Claude ตามงาน ส่วนการอ่านสลิปครั้งที่ 2 (verify) ใช้รุ่น VERIFY_MODEL ถ้าตั้งไว้
157–169	สร้างเนื้อหาคำขอตามรูปแบบของ Anthropic Messages API: รุ่น, จำนวน token สูงสุด, system prompt และข้อความของผู้ใช้ (รูปแปลงเป็นรูปแบบ base64 ที่ Claude รับ)
170–172	ถ้างานนี้ต้องการ JSON ตามแบบฟอร์ม (อ่านสลิป) ใส่ schema ไปด้วย Claude จะตอบตามแบบฟอร์มเป็น
173–185	ส่งไปที่ https://api.anthropic.com/v1/messages แบบถูกใจใน header x-api-key
186–192	ตอบกลับไม่สำเร็จ: คุญแฉผิด → NotConfigured, 429/529 (ส่งถี่เกิน/ระบบเต็ม) → Busy, อื่น ๆ → error ทั่วไป
193–199	สำเร็จ: เอาเฉพาะส่วนที่เป็นข้อความมาต่อกัน คืนพร้อมชื่อรุ่น

ฝั่ง Gemini (callGemini บรรทัด 202–268) หลักการเดียวกันแต่ซับซ้อนกว่า เพราะแบบฟรีติดลิมิตบ่อย: ไล่ลองหลายรุ่นตามลำดับ ถ้ารุ่นไหนใช้ไม่ได้กับคุญแฉนี้ข้ามไปรุ่นถัดไป ถ้าทุกรุ่น "ไม่ว่าง" รอตามเวลาที่ Google บอก (ไม่เกิน 12 วินาที) แล้วลองทั้งหมดอีก 1 รอบ ก่อนจะบอกแอปว่า busy การตัดสินใจหลังแต่ละ error อยู่ในฟังก์ชัน geminiStep

supabase/functions/_shared/helpers.ts

บรรทัด 75–93

```

75 /**
76  * What to do after a failed Gemini call:
77  * - invalid_key: stop, the key is wrong (tell the app "not configured")
78  * - plain_schema: retry the same model with the schema described in the prompt
79  * - next_model: this model is not available to this key; try the next one
80  * - busy: rate limit or overload on this model. Each model has its own free
81  *   quota, so try the next model first and wait only when all are busy.
82  * - fail: anything else
83  */
84 export type GeminiStep = 'invalid_key' | 'plain_schema' | 'next_model' | 'busy' | 'fail';
85
86 export function geminiStep(status: number, body: string, usingJsonSchema: boolean): GeminiStep {
87   if (isInvalidKey(status, body)) return 'invalid_key';
88   if (status === 404) return 'next_model';
89   if (status === 429 && isZeroQuota(body)) return 'next_model';
90   if (status === 400 && usingJsonSchema) return 'plain_schema';
91   if (status === 429 || status === 500 || status === 503) return 'busy';
92   return 'fail';

```

บรรทัด	ความหมาย
87	กุญแจผิด (401/403 หรือข้อความ API key not valid) → หยุด บอกว่ายังไม่ได้ตั้งค่า
88	404 = ไม่มีรุ่นนี้ → ลองรุ่นถัดไป
89	429 ที่บอกว่าโควตาเป็น 0 (กุญแจนี้ไม่มีสิทธิ์ใช้รุ่นนี้เลย) → รุ่นถัดไป
90	400 ตอนใช้ JSON schema (รุ่นเก่าบางรุ่นไม่รองรับ) → ลองใหม่โดยเขียนแบบฟอร์มลงใน prompt แทน
91	429/500/503 → ไม่ว่าง ลองรุ่นอื่นก่อน แล้วค่อยรอ
92	อย่างอื่น → ล้มเหลว

ตั้งค่า AI (ผู้ดูแลทำครั้งเดียว)

1. สร้างกุญแจ: Gemini ที่ aistudio.google.com/apikey (ฟรี) หรือ Claude ที่ console.anthropic.com
2. เปิด Supabase Dashboard → **Edge Functions** → **Secrets** (ไม่ใช่ Vault และไม่ใช่หน้า API keys)
3. เพิ่ม Secret ชื่อ GEMINI_API_KEY หรือ ANTHROPIC_API_KEY ใส่ค่ากุญแจ
4. ไม่ต้อง deploy ใหม่ ฟังก์ชันอ่าน Secret ทุกครั้งที่มีคำขอ
5. ถ้าแก้ไขโค้ดใน `supabase/functions` ต้อง deploy ด้วย `npx supabase functions deploy coach` (หรือ `parse-slip`)

หลักการใช้ AI อย่างรับผิดชอบที่ทีมเราใช้

หลักการ	ในโค้ด
AI ไม่คิดเลข โค้ดที่ทดสอบแล้วคิด AI แค่อธิบาย	แอปคำนวณ Runway และ purchaseCheck เองก่อนส่ง (FR-5, FR-6) system prompt ห้ามแต่งตัวเลข
AI ผิดได้ ต้องมีคนตรวจ	สลิปที่ไม่มั่นใจเป็นแบบร่าง อ่าน 2 ครั้งด้วย 2 รุ่น ไม่ตรงกันให้ผู้ใช้เช็ค (FR-4)
ส่งข้อมูลให้ AI น้อยที่สุด	โค้ชได้เฉพาะยอดรวม รูปที่ส่งอ่านคือรูปที่มี QR สลิปเท่านั้น และไม่เก็บรูปบนเซิร์ฟเวอร์
คุมค่าใช้จ่าย	โควตาต่อวันแบบ atomic, ตัดความยาวคำถาม, จำกัด token คำตอบ
บันทึกการใช้ AI ในการพัฒนา	docs/AI_USAGE_LOG.md บันทึกตาม SRS ว่าใช้ AI ช่วยอะไร แก้อะไร และสมาชิกตรวจอะไร

UX/UI: การออกแบบหน้าจอและ Design System

UI คือหน้าตา (สี ตัวอักษร ปุ่ม) UX คือประสบการณ์ (ใช้ง่ายไหม รู้สึกยังไง) บทนี้อธิบายหลักการออกแบบของ MindPay และโค้ดที่บังคับให้ทุกหน้าทำตาม

หลักการออกแบบของ MindPay

หลักการ	ทำอย่างไรในแอป
ตัวเลขเดียว เข้าใจทันที	หน้าหลักมีการ์ดเขียวที่บอกยอดเงินและ "เงินพอใช้อีก 16 วัน" ตัวใหญ่ที่สุด ผู้ใช้ไม่ต้องคิดเลขเอง
ทำสิ่งที่ทำบ่อยให้ง่ายที่สุด	ปุ่มสแกนสลิปใหญ่สุดกลางแถบล่าง, สแกนอัตโนมัติตอนเปิดแอป, พูดจได้, ปุ่มลัด + ฿100/500/1,000/5,000 ตอนเพิ่มเงินเข้า
ไม่ดู ไม่ทำให้รู้สึกผิด	บอกล่าพุดว่า "ลองพักของที่ไม่จำเป็นสักหน่อยนะ ค่อย ๆ ไปด้วยกัน" แทน "ใช้เงินเยอะเกินไป!" (system prompt ของโค้ชห้ามตำหนิ)
บอกตรง ๆ ว่ามันใจแค่นี้	สลิปที่ไม่ชัดเป็น "แบบร่าง" พร้อมบอกว่าช่องไหนต้องตรวจ, Runway ใช้คำว่า "ถึงประมาณ"
อธิบายตัวเองได้	การ์ด "คำนวณยังง" ในหน้าเงินพอถึงแสดงสูตรที่ละบรรทัดเหมือนใบเสร็จ
ย้อนกลับได้	ลบรายการแล้วกด "เลิกทำ" ได้ (FR-1), สลิปที่เข้ายอดเองกด "ยังไม่รวมยอด" ได้ (FR-4)
ข้อความผิดพลาดต้องบอกวิธีแก้	"ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50" แทน "Invalid input"
ทุกคนใช้ได้ (accessibility)	สีผ่านเกณฑ์อ่านง่าย, ปุ่มใหญ่พอแตะ, มีข้อความสำหรับโปรแกรมอ่านหน้าจอ, ปิดแอนิเมชันได้

Design System คืออะไร

Design System คือ "กฎกลาง + ชิ้นส่วนสำเร็จรูป" ที่ทุกหน้าใช้ร่วมกัน เหมือนชุดตัวต่อเลโก้ที่มีขนาดและสีมาตรฐาน หน้าจอทุกหน้าจึงดูเป็นแอปเดียวกัน และเปลี่ยนธีมสีได้ทั้งแอปในจุดเดียว ของ MindPay อยู่ใน

- `src/ui/theme.ts` ค่ากลาง (token): สี ฟอนต์ ขนาดตัวอักษร ระยะห่าง มุมโค้ง
- `src/ui/components.tsx`, `src/ui/inputs.tsx`, `src/ui/feedback.tsx` ชิ้นส่วน: ตัวหนังสือ ปุ่ม การ์ด ช่องกรอก ข้อความแจ้งเตือน
- `docs/DESIGN_SYSTEM.md` เอกสารกฎ กฎข้อแรก: ห้ามหน้าจอไหนคิดสีหรือขนาดเอง

สี: Green + Gold

ทิศทางแบรนด์คือ สีเขียว (เงิน การเติบโต ความมั่นคง) ใช้กับโครงสร้าง และ สีทอง ใช้ "น้อย ๆ" กับสิ่งที่สำคัญที่สุดบนหน้า เช่น ยอดเงิน ปุ่มหลัก ใบของต้นไม้เงิน คอมเมนต์บนสุดของ theme.ts เขียนว่า "Never paint a whole screen green"

`src/ui/theme.ts`

บรรทัด 1-21

```
1 /**
```

```

2  * MindPay design system tokens.
3  *
4  * Brand: Green (money, growth, stability) + Gold (accent for what matters most).
5  * Rule from the brand direction: green is for structure, gold is used sparingly
6  * for the single most important thing on a screen (the balance, the main action).
7  * Never paint a whole screen green.
8  */
9  import { useColorThemeKey, useSchemeChoice } from './themeMode';
10
11 export const palette = {
12   forest: '#0E3B2C', // hero surfaces, brand
13   forestDeep: '#082A1F',
14   leaf: '#1F7A52', // primary actions, positive
15   mint: '#DDEEE3',
16   gold: '#C8992A', // accent: balance, main CTA ring, tree leaves
17   goldBright: '#E2B64A',
18   goldSoft: '#F4E7C2',
19   clay: '#B4442F', // critical / expense emphasis
20   amber: '#B7791F', // watch
21 };

```

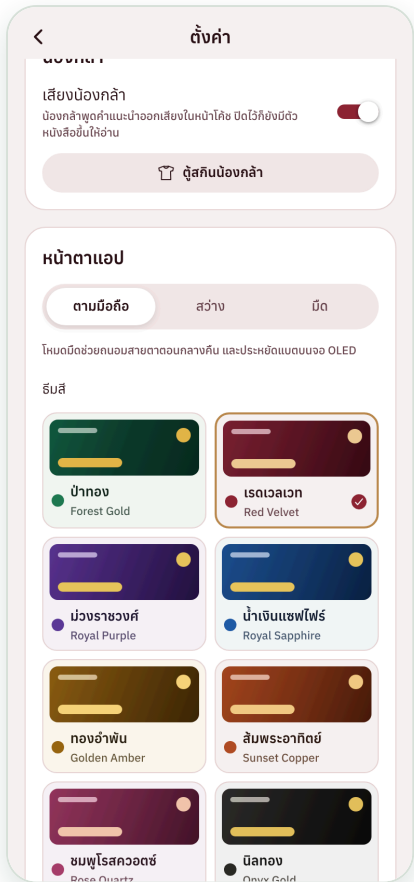
▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:1

สีทั้งหมดถูกตั้งชื่อตาม **หน้าที่** (token) ไม่ใช่ตามสี เช่น ink = สีตัวหนังสือหลัก, primary = สีปุ่มหลัก หน้าจอเรียก theme.ink โดยไม่สนว่าตอนนี้เป็นธีมไหน

Token	ใช้กับ	ป่าทอง สว่าง	ป่าทอง มืด
bg	พื้นหลังหน้าจอ	#F2F5F1	#0A1410
surface	การ์ด ช่องกรอก	#FFFFFF	#111E18
ink	ตัวหนังสือหลัก	#11231B	#E7EFE9
inkSoft	ตัวหนังสือรอง คำอธิบาย	#4B5F55	#A2B6AA
inkFaint	placeholder ไอคอนเจียม ๆ (ห้ามใช้กับข้อความที่ต้องอ่าน)	#8A9A91	#6B8076
primary	ปุ่มหลัก สิ่ง que เลือกอยู่	#1F7A52	#3FA774
accent	ปุ่มทอง ขอบการ์ดที่ต้องสนใจ	#C8992A	#E2B64A
income / good / watch / critical	สีที่มีความหมาย: เงินเข้า, ดี, ระวัง, วิกฤต เหมือนกันทุกธีม ผู้ใช้จึงไม่สับสน	เขียว / เขียว / ส้มอำพัน / แดงอิฐ	

9 ธีม × สว่าง/มืด



ตั้งค่า → ธีมสี: เลือก 1 ใน 9 ธีม และโหมด ตามมือถือ / สว่าง / มืด

ผู้ใช้เลือกธีมได้ 9 แบบ: ป่าทอง (ค่าเริ่มต้น), เรดเวลเวท, ม่วงราชวงศ์, น้ำเงินแซฟไฟร์, ทองอำพัน, ส้มพระอาทิตย์, ชมพูโรสควอตซ์, นิลทอง และธีมเทศกาลฮาโลวีน แต่ละธีมมีชุดสีสว่างและมืด (COLOR_THEMES ใน theme.ts บรรทัด 93–204) เลือกแล้วจำไว้ในเครื่อง

ฟังก์ชัน buildTheme สร้างชุดสีเต็ม (ประมาณ 30 token) จากค่าพื้นฐานของแต่ละธีม และคำนวณสีบางตัวให้อ่านง่ายอัตโนมัติ เช่น สีตัวหนังสือบนปุ่มทอง (onAccent) เลือกระหว่างสีเข้มกับสีขาว ตัวที่อ่านออกบนพื้นทงนั้น

หน้าจอเรียกแค่ const theme = useTheme() ได้ชุดสีของธีมและโหมดที่ผู้ใช้เลือกตอนนี้ เปลี่ยนธีมแล้วทุกหน้าเปลี่ยนตามทันที

ความอ่านง่ายของสี (WCAG) และเทสต์ที่บังคับ

มาตรฐานสากล WCAG AA กำหนดว่าตัวหนังสือกับพื้นหลังต้องต่างกันอย่างน้อย 4.5 : 1 (1 = สีเดียวกัน, 21 = ดำบนขาว) คนสายตาไม่ดีหรือใช้มือถือกลางแดดจึงอ่านออก

คู่สี (ป่าทอง สว่าง)	อัตราส่วน	ผ่านไหม
ink uu bg	14.93	ผ่าน
inkSoft uu surface	6.85	ผ่าน
ตัวหนังสือขาว uu primary	5.30	ผ่าน
inkFaint uu surface	2.95	ไม่ผ่าน → จึงห้ามใช้กับข้อความ ใช้กับ placeholder และไอคอนตกแต่งเท่านั้น
ทอง accent เป็นตัวหนังสือบนพื้นขาว	2.61	ไม่ผ่าน → จึงมี token accentInk ที่เข้มกว่าไว้ใช้แทน

คำนวณด้วยสูตรเดียวกับฟังก์ชัน contrast() ในแอป

src/ui/theme.tsบรรทัด 261–279

```
261 function luminance(hex: string): number {
262   const n = parseInt(hex.replace('#', '').slice(0, 6), 16);
263   const ch = (v: number) => {
```

```

264     const c = v / 255;
265     return c <= 0.03928 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4;
266 };
267 return 0.2126 * ch((n >> 16) & 255) + 0.7152 * ch((n >> 8) & 255) + 0.0722 * ch(n & 255);
268 }
269
270 /** WCAG contrast ratio between two hex colours (1 to 21). */
271 export function contrast(a: string, b: string): number {
272     const [hi, lo] = [luminance(a), luminance(b)].sort((x, y) => y - x);
273     return (hi + 0.05) / (lo + 0.05);
274 }
275
276 /** The first colour that reads on `bg` at 4.5:1 or better (the last one otherwise). */
277 export function readableOn(bg: string, ...colors: string[]): string {
278     return colors.find((c) => contrast(c, bg) >= 4.5) ?? colors[colors.length - 1];
279 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:261

บรรทัด ความหมาย

- 261–268** luminance: ความสว่างที่ตาคนรับรู้ของสีหนึ่ง (0 = ดำ ถึง 1 = ขาว) แยกสีแดง เขียว น้ำเงิน แปลงค่า แล้วถ่วงน้ำหนัก เขียวหนักสุด (0.7152) เพราะตาคนไวต่อสีเขียวที่สุด
- 270–274** contrast: เอาสีที่สว่างกว่าเป็นตัวตั้ง (บวก 0.05) หาสีที่มืดกว่า (บวก 0.05) ตามสูตรของ WCAG
- 276–279** readableOn: ให้รายการสีมา ค้นสีแรกที่อ่านออกบนพื้นนั้น (≥ 4.5)

src/ui/__tests__/theme.test.ts

บรรทัด 34–58

```

34 for (const spec of COLOR_THEMES) {
35     for (const dark of [false, true]) {
36         it(`${spec.key} ${dark ? 'dark' : 'light'}: every text colour reads on the background it is
        used on`, () => {
37             const t = buildTheme(spec.key, dark);
38             const pairs: [string, string, string][] = [
39                 ['inkSoft on bg', t.inkSoft, t.bg],
40                 ['inkSoft on surface', t.inkSoft, t.surface],
41                 ['onPrimary on primary', t.onPrimary, t.primary],
42                 ['onAccent on accent', t.onAccent, t.accent],
43                 ['accentInk on surface', t.accentInk, t.surface],
44                 ['accentInk on accentSoft', t.accentInk, t.accentSoft],
45                 ['good on goodSoft', t.good, t.goodSoft],
46                 ['watch on watchSoft', t.watch, t.watchSoft],
47                 ['critical on criticalSoft', t.critical, t.criticalSoft],
48                 ['good on bg', t.good, t.bg],
49                 ['watch on bg', t.watch, t.bg],
50                 ['critical on bg', t.critical, t.bg],
51                 ['heroInkSoft on hero', t.heroInkSoft, t.hero],
52                 ['heroInk on heroTop', t.heroInk, t.heroTop],
53             ];
54             const low = pairs.filter(([a, b]) => contrast(a, b) < 4.5).map(([a, b]) => `${n}
            ${contrast(a, b).toFixed(2)}`);
55             expect(low).toEqual([]);
56         });
57     }
58 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/__tests__/theme.test.ts:34

เทสต์ TC-78 ตรวจสอบทุกธีม (9) × ทุกโหมด (สว่าง/มืด) = 18 ชุด แต่ละชุดตรวจ 14 คู่มือ (บรรทัด 38–53) ถ้าคู่มือต่ำกว่า 4.5 เทสต์ล้ม และบอกชื่อคู่มือพร้อมค่าที่ได้ (บรรทัด 54) ใครเพิ่มธีมใหม่แล้วเลือกสีอ่านยาก npm test จะไม่ผ่าน และ GitHub จะไม่ build แอปให้

ตัวอักษร

src/ui/theme.ts

บรรทัด 301–328

```
301 /**
302  * Two typefaces:
303  * - Noto Serif Thai for money and headings: the "international banking" feel
304  *   from the logo direction, and clear numerals.
305  * - Anuphan for everything else: a modern Thai sans that reads well small.
306  */
307 export const fonts = {
308   serif: 'NotoSerifThai_600SemiBold',
309   serifBold: 'NotoSerifThai_700Bold',
310   sans: 'Anuphan_400Regular',
311   sansMedium: 'Anuphan_500Medium',
312   sansSemi: 'Anuphan_600SemiBold',
313   sansBold: 'Anuphan_700Bold',
314 };
315
316 export const space = { xs: 4, sm: 8, md: 12, lg: 16, xl: 20, xxl: 28, xxxl: 40 };
317 export const radius = { sm: 8, md: 14, lg: 20, xl: 28, pill: 999 };
318
319 /** Type scale (size / line height). Thai needs generous line height for vowels above and below.
320  */
321 export const type = {
322   display: { fontSize: 38, lineHeight: 52 },
323   h1: { fontSize: 26, lineHeight: 38 },
324   h2: { fontSize: 20, lineHeight: 30 },
325   h3: { fontSize: 17, lineHeight: 26 },
326   body: { fontSize: 15, lineHeight: 24 },
327   small: { fontSize: 13, lineHeight: 20 },
328   micro: { fontSize: 11, lineHeight: 16 },
329 };
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:301

- **Noto Serif Thai** (มีหัว) ใช้กับตัวเลขเงินและหัวข้อใหญ่ ให้ความรู้สึก "ธนาคารสากล" และตัวเลขอ่านชัด
- **Anuphan** (ไม่มีหัว กันสมัย) ใช้กับข้อความอื่นทั้งหมด อ่านง่ายแม้ตัวเล็ก
- บรรทัด 316–317: ระยะห่างมาตรฐาน 4/8/12/16/20/28/40 และมุมโค้ง 8/14/20/28/999 (999 = ปลายมนเป็นเม็ดยา)
- บรรทัด 319–328: ขนาดตัวอักษรคู่กับความสูงบรรทัดเสมอ ภาษาไทยมีสระบนล่างและวรรณยุกต์ ต้องมีความสูงบรรทัดประมาณ 1.4–1.6 เท่าของขนาดตัวอักษร ไม่ขึ้นสระจะชนบรรทัดข้างบน

ชิ้นส่วนสำเร็จรูป: ปุ่ม (Button)

src/ui/components.tsx

บรรทัด 261–338

```
261 export function Button({
262   label,
```

```

263   onPress,
264   kind = 'primary',
265   icon,
266   loading,
267   disabled,
268   style,
269   small,
270   shine,
271 }: {
272   label: string;
273   onPress: () => void;
274   /** onDark: outline button on the dark green hero surfaces */
275   kind?: 'primary' | 'gold' | 'ghost' | 'danger' | 'soft' | 'onDark';
276   icon?: IconName;
277   loading?: boolean;
278   disabled?: boolean;
279   style?: StyleProp<ViewStyle>;
280   small?: boolean;
281   /** A band of light sweeps across (for the one main action on a screen). */
282   shine?: boolean;
283 }) {
284   const theme = useTheme();
285   const press = usePressSpring(0.95);
286   const colors = {
287     primary: { bg: theme.primary, fg: theme.onPrimary, border: theme.primary },
288     gold: { bg: theme.accent, fg: theme.onAccent, border: theme.accent },
289     ghost: { bg: 'transparent', fg: theme.ink, border: theme.line },
290     danger: { bg: 'transparent', fg: theme.critical, border: theme.critical },
291     soft: { bg: theme.surfaceAlt, fg: theme.ink, border: theme.surfaceAlt },
292     onDark: { bg: 'rgba(244,241,230,0.10)', fg: '#F4F1E6', border: 'rgba(244,241,230,0.45)' },
293   }[kind];
294   const off = disabled || loading;
295   return (
296     <AnimatedPressable
297       onPress={() => {
298         Haptics.selectionAsync().catch(() => {});
299         onPress();
300       }}
301       onPressIn={off ? undefined : press.onPressIn}
302       onPressOut={press.onPressOut}
303       disabled={off}
304       // Small buttons are about 40pt tall: the touch area reaches 44pt and more (up and down
305       only,
306       // so buttons side by side never share a touch area).
307       hitSlop={small ? { top: 4, bottom: 4 } : undefined}
308       accessibilityRole="button"
309       aria-disabled={!off}
310       aria-busy={!loading}
311       style={[
312         {
313           backgroundColor: colors.bg,
314           borderColor: colors.border,
315           borderWidth: 1.5,
316           borderRadius: radius.pill,
317           paddingVertical: small ? 8 : 12,
318           paddingHorizontal: small ? 14 : 20,
319           flexDirection: 'row',
320           alignItems: 'center',
321           justifyContent: 'center',
322           gap: 8,
323           opacity: off ? 0.5 : 1,

```

```

323         overflow: shine ? 'hidden' : undefined,
324     },
325     style,
326     { transform: [{ scale: press.scale }] },
327   ]}
328   >
329   {shine && !off ? <Shine times={1} delay={900} color={kind === 'gold' ?
'rgba(255,255,255,0.45)' : 'rgba(255,255,255,0.28)'} /> : null}
330   {loading ? (
331     <ActivityIndicator color={colors.fg} />
332   ) : icon ? (
333     <Icons name={icon} size={small ? 16 : 19} color={colors.fg} />
334   ) : null}
335     <Text style={[small ? type.small : type.body, { fontFamily: fonts.sansSemi, color:
colors.fg }]}>{label}</Text>
336   </AnimatedPressable>
337 );
338 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/components.tsx:261`

บรรทัด	ความหมาย
261–283	props ของปุ่ม: ข้อความ, สิ่งที่ทำเมื่อกด, ชนิด 6 แบบ (ค่าเริ่มต้น primary), ไอคอน, คำสั่งโหลด, ปิดใช้งาน, ขนาดเล็ก, แสงวิ่งผ่าน
284–285	สีของธีม และแอนิเมชันยุบตัวตอนกด (เหลือ 95%)
286–293	ตารางสีของปุ่มแต่ละชนิด: primary = พื้นสีขาว, gold = พื้นทอง, ghost = โปร่งมีขอบ, danger = ขอบแดง (ปุ่มลบ), soft = พื้นเทาอ่อน, onDark = โปร่งบนการ์ดเขียว ทุกสีมาจาก theme
294	ปุ่ม "ปิด" ถ้าถูกปิดใช้งานหรือคำสั่งโหลด (กันกดซ้ำ)
297–300	กดแล้วมือถือสั่นเบา ๆ (haptic) แล้วทำสิ่งที่ผู้เรียกกำหนด
304–306	ปุ่มเล็กสูงประมาณ 40 จุด ขยายพื้นที่แตะขึ้นลงอีก 4 จุด ให้ถึง 44 จุดตามแนวทางของ Apple/Google (นิ้วคนกว้างกว่าที่คิด) ขยายเฉพาะแนวตั้ง ปุ่มที่วางติดกันจึงไม่แย่งพื้นที่แตะ
307–309	บอกโปรแกรมอ่านหน้าจอว่าเป็นปุ่ม และสถานะปิด/คำสั่งโหลด
310–327	รูปร่าง: ขอบ 1.5, มุมโค้งแบบเบียด, ระยะในตามขนาด, ไอคอนกับข้อความเรียงแนวนอนตรงกลาง, จางลง 50% เมื่อปิดใช้งาน
329	แสงวิ่งผ่าน 1 ครั้ง (ใช้กับปุ่มหลักปุ่มเดียวของหน้า)
330–334	คำสั่งโหลดแสดงวงหมุนแทนไอคอน
335	ข้อความบนปุ่ม ขนาดตามปุ่ม ฟอนต์หนาถึงหนา

ชิ้นส่วน	ใช้เมื่อ
Screen	ทุกหน้า: พื้นหลัง ขอบ เลื่อนช่องที่คำสั่งพิมพ์ให้พื้นที่บอร์ด
Card	กลุ่มเนื้อหา มี 3 โท: ปกติ, พิเศษ (alt), ท่องอ่อน (accent)
Button / IconButton	การกระทำ 1 หน้ามีปุ่ม primary แค่ 1 ปุ่ม / ปุ่มไอคอนต้องมี label เสมอ

ชิ้นส่วน	ใช้เมื่อ
Segmented	สลับมุมมอง 2–4 แบบ เช่น วันนี้ / 7 วัน / 1 เดือน, รายจ่าย / รายรับ
Chip	ตัวเลือกหลายอัน เช่น หมวดหมู่, จำนวนเงินลด
Field / AmountField	ช่องกรอก ขอบเปลี่ยนสีตอนพิมพ์และตอนผิด บนเว็บตัวหนังสือ 16 เพื่อไม่ให้ iPhone ชุมหน้า
Badge	ป้ายสถานะสั้น ๆ เช่น "สบาย ๆ"
useToast	ยืนยันผล เช่น "บันทึกรายการแล้ว" มีปุ่ม "เลิกทำ" ได้
ConfirmSheet	ถามยืนยันก่อนทำสิ่งที่ย้อนไม่ได้ เช่น ลบ
EmptyState	ยังไม่มีข้อมูล: น้องกล้า + คำอธิบาย + ปุ่มให้เริ่ม

Accessibility: ทุกคนใช้ได้

เรื่อง	ทำอย่างไร
คนตาบอด/สายตาเลือนราง	ปุ่มทุกปุ่มมี accessibilityLabel ให้ TalkBack (Android) / VoiceOver (iPhone) อ่าน เช่นการ์ดยอดเงินอ่านว่า "ยอดคงเหลือ ฿11,359.00 เงินพอใช้อีก 16 วัน"
แยกสีไม่ออก (ตาบอดสี)	สถานะไม่ใช่สีอย่างเดียว มีข้อความกำกับเสมอ เช่น "ใกล้เส้นสำรอง"
เมารถจากอาการเคลื่อนไหว	เปิด "ลดการเคลื่อนไหว" (Reduce motion) ในมือถือ ทุกแอนิเมชันหยุดและแสดงผลทันที (useReduceMotion())
นิ้วใหญ่ มือสั่น	พื้นที่แตะอย่างน้อย 44 จุด
อ่านยาก	สีผ่าน WCAG AA ทุกธีม (TC-78)

แอนิเมชัน: สวยแต่ไม่กินแบตเตอรี่

แอปมีเอฟเฟกต์เยอะ (ส่วนต่าง ๆ ค่อย ๆ โผล่, ปุ่มดัง, ตันไม้เงินโยก, พลุตอนสำเร็จ) แต่มีกฎ 4 ข้อจาก README:

- ขยับเฉพาะ transform (ย้าย หมุน ย่อขยาย) กับ opacity (ความจาง) และใช้ **native driver** ให้การขยับทำงานส่วน native ของมือถือ ไม่ผ่าน JavaScript จึงสิ้นแม่มือถือราคาประหยัด
- ทุกเอฟเฟกต์หยุดเอง ไม่มีอะไรยับยั้งค้างตอนผู้ใช้ไม่ได้ทำอะไร (มี E2E ตรวจสอบว่าหน้าหลักนิ่งภายใน 9 วินาที)
- เคส "ลดการเคลื่อนไหว"
- ไม่ใช้ native module ใหม่ แอปที่ลงแล้วจึงได้เอฟเฟกต์ใหม่ทาง OTA โดยไม่ต้องลง APK ใหม่

เช็กลิสต์ก่อนเพิ่มหน้าจอใหม่ (จาก DESIGN_SYSTEM.md)

- ไม่มีรหัสสี #... หรือ rgba (...) ในหน้าจอ ใช้ token หรือ alpha()
- ตัวหนังสือใช้ <T> และเงินใช้ <Money>
- ปุ่ม ชิป ช่องกรอก การ์ด มาจาก component ไม่สร้างใหม่เอง
- ลองเปิดหน้านั้น 2–3 ธีม ทั้งสว่างและมืด

5. รัน `npm test, npx tsc --noEmit, npx expo lint, npm run e2e:web`

i ทบทวนศึกษา UX จากที่ไหน

บันทึกใน [docs/AI_USAGE_LOG.md](#) แถว 10 และ 19: ศึกษาแอปเหมียวจวด (ผู้ช่วยที่พูดกับผู้ใช้, "แค่เปิดแอปก็จดให้") และแนวการออกแบบของ Hugging Face (มาสคอตตัวเดียวหลายอารมณ์, สีอุ้งบนพื้นเรียบ, การ์ดโค้ง เส้นบาง) และหลัก Laws of UX เช่น aesthetic-usability effect (ของสวยทำให้รู้สึกว่ายาง่าย) และ peak-end rule (คนจำช่วงพีคและตอนจบ จึงมีพลุดอนทำสำเร็จ) แล้วออกแบบตัวละครน้องกล้าของ MindPay เอง ไม่ลอกของใคร

การทดสอบ: พิสูจน์ว่าระบบทำงานถูก

ทีมเรามีการทดสอบ 3 ระดับ: unit test 157 ข้อ, E2E ในเบราว์เซอร์ 145 ข้อ และแผนทดสอบบนมือถือจริง 12 ข้อ บทนี้สอนอ่านและเขียนเทสต์

ทำไมต้องเขียนเทสต์

เวลาแก้ไขโค้ดจุดหนึ่ง อาจทำให้อีกจุดพังโดยไม่รู้ตัว (เรียกว่า **regression**) ถ้าต้องกดทดสอบทุกหน้าเองทุกครั้งก็แย่ จะใช้เวลาหลายชั่วโมงและล้มบางจุดแน่นอน เทสต์อัตโนมัติคือ "โปรแกรมที่ตรวจโปรแกรม" รันเสร็จในไม่กี่วินาที และทีมตั้งไว้ว่าถ้าเทสต์ไม่ผ่าน GitHub จะไม่ปล่อยเว็บและ APK ให้ผู้ใช้

157

Unit test (Vitest)

ตรวจฟังก์ชันทีละตัว · รันไม่ถึง 2 วินาที

145

E2E (Playwright)

เปิดเว็บจริงแล้วกดเหมือนคน · ประมาณ 3 นาที

12

Manual test (MT-01-12)

คนทดสอบบนมือถือจริง

Unit test: ตรวจฟังก์ชันทีละตัว

ไฟล์เทสต์อยู่ในโฟลเดอร์ `__tests__` ชื่อลงท้าย `.test.ts` รันด้วยคำสั่ง `npm test` ใช้เครื่องมือ **Vitest** ตัวอย่างจากเทสต์ของ FR-6:

src/domain/__tests__/domain.test.ts

บรรทัด 35-58

```
35 // Sunday 27 Sep 2026, 14:00 in Bangkok
36 const NOW = new Date('2026-09-27T07:00:00.000Z');
37
38 let seq = 0;
39 function tx(p: Partial<Transaction> & { day: string; amount: number }): Transaction {
40   const { day, amount, ...rest } = p;
41   return {
42     id: `t${++seq}`,
43     kind: 'expense',
44     amountSatang: amount * 100,
45     categoryKey: 'food',
46     title: 'test',
47     note: null,
48     occurredAt: bkkToIso(day, '12:00'),
49     source: 'manual',
50     status: 'confirmed',
51     slipRef: null,
52     slipImageHash: null,
53     ocrConfidence: null,
54     reviewFlags: [],
55     createdAt: NOW.toISOString(),
56     ...rest,
57   };
58 }
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:35

บรรทัด	ความหมาย
35–36	ล็อก "วันนี้" ให้ตายตัว เป็นวันอาทิตย์ที่ 27 ก.ย. 2569 เวลา 14:00 น. ไม่ว่ารันเทสต์วันไหน ผลก็เหมือนเดิม (ถ้าใช้วันจริง เทสต์จะผ่านบ้างไม่ผ่านบ้างตามวันที่)
38–58	ฟังก์ชันช่วยสร้างรายการปลอม tx({ day, amount }) ใส่แค่วันกับจำนวนบาท ที่เหลือใช้ค่าตั้งต้น (รายจ่าย หมวดอาหาร ยืนยันแล้ว) ...rest ท้ายสุดให้เปลี่ยนช่องอื่นได้ เช่น tx({ day, amount, status: 'draft' })

src/domain/__tests__/domain.test.ts

บรรทัด 184–210

```

184 it('TC-14 computes days and depletion date', () => {
185     const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 20_000,
now: NOW });
186     expect(r.days).toBe(15);
187     expect(r.depletionDay).toBe('2026-10-12');
188     expect(r.status).toBe('healthy');
189 });
190
191 it('TC-15 handles zero average without dividing by zero', () => {
192     const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 0, now:
NOW });
193     expect(r.status).toBe('no_spending');
194     expect(r.days).toBeNull();
195     expect(r.depletionDay).toBeNull();
196 });
197
198 it('TC-16 flags balance at or below the low line', () => {
199     const r = computeRunway({ balanceSatang: 50_000, floorSatang: 50_000, averageSatang: 10_000,
now: NOW });
200     expect(r.status).toBe('below_floor');
201     expect(r.days).toBe(0);
202 });
203
204 it('TC-17 status thresholds', () => {
205     const at = (days: number) =>
206         computeRunway({ balanceSatang: 50_000 + days * 10_000, floorSatang: 50_000, averageSatang:
10_000, now: NOW }).status;
207     expect(at(6)).toBe('critical');
208     expect(at(7)).toBe('watch');
209     expect(at(14)).toBe('healthy');
210 });

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:184

บรรทัด	ความหมาย
184	it('ชื่อเทสต์', () => {...}) = เทสต์ 1 ข้อ ชื่อขึ้นต้นด้วยรหัส TC ที่ตรงกับเอกสาร Traceability
185	Arrange + Act: เตรียมข้อมูล (ยอด ฿3,500, เงินสำรอง ฿500, ใช้วันละ ฿200) แล้วเรียกฟังก์ชันจริง
186–188	Assert: expect(ค่าจริง).toBe(ค่าที่ต้องได้) ตรวจสอบว่าได้ 15 วัน, วันที่ 12 ต.ค. 2569 และสถานะ healthy ถ้าไม่ตรงข้อใดข้อหนึ่ง เทสต์ล้มพร้อมบอกว่าได้อะไรแทน
191–196	TC-15 ทดสอบ กรณีพิเศษ : ค่าเฉลี่ยเป็น 0 ต้องไม่หารด้วยศูนย์ ต้องได้ no_spending และวันเป็น null
198–202	TC-16 ทดสอบ ขอบ : ยอดเท่ากับเงินสำรองพอดี ต้องนับว่า "ต่ำกว่าเงินสำรอง"

บรรทัด	ความหมาย
204–210	TC-17 ทดสอบ ขอบของเกณฑ์ : 6 วัน critical, 7 วัน watch, 14 วัน healthy ถ้ามีคนเผลอแก้ < เป็น <= เทสต์นี้จะจับได้ทันที

★ เขียนเทสต์ที่ดี: ทดสอบขอบ ไม่ใช่แค่ค่ากลาง ๆ

บ๊ิกชอบซ่อนอยู่ตรงขอบ เช่น 0, ค่าเท่ากันพอดี, ค่าว่าง, วันแรกของเดือน, เกียงคืน, ข้อมูลภาษาไทยแปลก ๆ เทสต์ของทิมจึงมีทั้งกรณีปกติ กรณีพิเศษ และกรณีขอบ อาจารย์มักถามว่า "ทดสอบกรณีไหนบ้าง" ให้ตอบเป็น 3 กลุ่มนี้

ไฟล์ unit test หลัก (มีทั้งหมด 18 ไฟล์)

ไฟล์	ทดสอบ
domain.test.ts	เงิน, วันที่ไทย, ยอดคงเหลือ (FR-2), Runway (FR-6), สลิป (FR-4), คำแนะนำโค้ช (FR-5), หมวด
voice.test.ts	แยกประโยคพูดเป็นรายการ ตัวเลขคำไทย (FR-3)
slipAccuracy.test.ts, autoScan.test.ts	ความแม่นยำการอ่านสลิป และสแกนอัตโนมัติ (FR-4)
recurring.test.ts, recap.test.ts	รายการประจำเดือน (FR-1) และสรุปเดือนปฏิทิน (FR-2)
goals.test.ts	กระปุกออม (FR-6)
buddy.test.ts, klaTalk.test.ts	คำพูดน้องกล้า และการตัดซับไตเติลอ่านออกเสียง (FR-5)
auth.test.ts	ลิงก์อีเมลและข้อความ error ภาษาไทย
audit.test.ts	บ๊ิกที่เจอตอนตรวจทั้ง repo (1–3 ต.ค.) กันไม่ให้กลับมาอีก
ui/__tests__/theme.test.ts	สีทุกธีมอ่านง่าย (TC-78)
supabase/functions/_shared/__tests__/helpers.test.ts	ตัวช่วยของเซิร์ฟเวอร์: เลือก AI, ทำความสะอาดผลอ่านสลิป, รวมผลอ่าน 2 ครั้ง

E2E: ทดสอบทั้งแอปเหมือนคนใช้จริง

End-to-End (E2E) test เปิดเว็บเวอร์ชันจริงในเบราว์เซอร์อัตโนมัติ (เครื่องมือ **Playwright**) แล้วกด พิมพ์ เลื่อน ตามขั้นตอนเหมือนคน และตรวจว่าเห็นสิ่งที่ควรเห็น ไฟล์หลักคือ [e2e/web-auth.mjs](#) (ยาวกว่า 1,000 บรรทัด)

≈ ไม่แตะบัญชีจริง: เซิร์ฟเวอร์ปลอม

ถ้า E2E ใช้ Supabase จริง จะสร้างบัญชีขยะ ส่งอีเมลจริง และเปลืองโควตา AI ทีมเราจึงเขียน

`e2e/fake-backend.mjs` เป็น "Supabase ปลอม" ที่ตอบเหมือนของจริงแต่อยู่ในหน่วยความจำ ทุกคำขอที่ไม่ได้อยู่ของ Supabase จะถูกดักมาตอบด้วยตัวปลอมนี้ (`page.route`) เหมือนซ้อมละครกับนักแสดงแทน

`e2e/web-auth.mjs`

บรรทัด 29–60

```
29 // ----- helpers
30 const results = [];
31 function check(name, ok, detail = '') {
32   results.push({ name, ok, detail });
33   console.log(`${ok ? 'PASS' : 'FAIL'} ${name}${detail ? ` - ${detail}` : ''}`);
34   // In GitHub Actions a failed check also becomes an annotation on the run.
35   if (!ok && IN_CI) console.log(`::error title=${ENGINE} browser test::${name}${detail ? `
36   (${detail.replace(/\n/g, ' ').slice(0, 300)}):` : ''}`);
37 }
38 const browser = await (ENGINE === 'webkit' ? webkit : chromium).launch();
39 const phone = {
40   ...(ENGINE === 'webkit' ? { isMobile: true, hasTouch: true } : {}),
41   // E2E_SCHEME=dark runs everything with the phone in dark mode.
42   colorScheme: process.env.E2E_SCHEME === 'dark' ? 'dark' : 'light',
43 };
44 async function freshPage(tag) {
45   const ctx = await browser.newContext({ viewport: { width: 390, height: 844 },
46     deviceScaleFactor: 2, locale: 'th-TH', ...phone });
47   const page = await ctx.newPage();
48   page.on('pageerror', (e) => console.log(`[${tag}] pageerror`, e.message));
49   page.on('console', (m) => { if (m.type() === 'error') console.log(`[${tag}] console.error`,
50     m.text().slice(0, 200)); });
51   page.on('requestfailed', (r) => { if (!r.url().startsWith(SUPA)) console.log(`[${tag}] request
52     failed`, r.url().slice(0, 160)); });
53   await page.route(`${SUPA}/**`, supabase);
54   // The web QR reader downloads its decoder from a CDN: serve the same file from node_modules.
55   await routeQrDecoder(page);
56   return { ctx, page };
57 }
58 const shot = (page, name) => page.screenshot({ path: join(SHOTS, `${name}.png`) });
59 const visible = (page, text, timeout = 6000) => page.getByText(text, { exact: false
60   }).first().waitFor({ state: 'visible', timeout }).then(() => true, () => false);
61 const gone = (page, text, timeout = 6000) => page.getByText(text, { exact: false
62   }).first().waitFor({ state: 'hidden', timeout }).then(() => true, () => false);
63 const introGone = (page) => page.getByLabel('กำลังเปิด MindPay').waitFor({ state: 'detached',
64   timeout: 8000 }).catch(() => {});
65 const button = (page, name) => page.getByRole('button', { name, exact: false }).first();
66 /** Waits (up to `timeout`) until the element is visible. */
67 const shown = (locator, timeout = 6000) => locator.waitFor({ state: 'visible', timeout }).then(()
68   => true, () => false);
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `e2e/web-auth.mjs:29`

บรรทัด ความหมาย

30–36 `check` (ชื่อ, ผ่านไหม, รายละเอียด) บันทึกผลแต่ละข้อ พิมพ์ PASS หรือ FAIL ถ้ารันบน GitHub และไม่ผ่าน จะขึ้นเป็นป้าย error บนหน้า Actions

บรรทัด	ความหมาย
37–42	เปิดเบราว์เซอร์ Chromium (หรือ WebKit แบบ Safari) โหมดสว่างหรือมืด
43–53	freshPage: เปิดแท็บใหม่ขนาดมือถือ (390×844) ภาษาไทย ดักคำขอไป Supabase ให้ตัวปลอมตอบ และพิมพ์ error ของหน้าเว็บถ้ามี
54	shot: ถ่ายภาพหน้าจอเก็บไว้ (ภาพหน้าจอหลายภาพในเล่มนี้มาจากตรงนี้)
55–56	visible / gone: รอจนเห็นข้อความ (หรือข้อความหายไป) ภายใน 6 วินาที
58	button: หาปุ่มจากชื่อที่โปรแกรมอ่านหน้าจออ่าน (accessibilityLabel) แอปที่ทำ accessibility ดีจึงเขียนทดสอบง่าย

ตัวอย่าง: ทดสอบแถบ "ลองปรับ" ของ FR-6

e2e/web-auth.mjs

บรรทัด 572–627

```

572 // 14. Runway: "can I buy this?" shows the days before/after and hands the question to the
    coach
573 {
574   const { ctx, page } = await freshPage('runway');
575   await page.goto(APP);
576   await visible(page, 'ลองใช้ด้วยข้อมูลตัวอย่าง', 8000);
577   await introGone(page);
578   await page.getByText('ลองใช้ด้วยข้อมูลตัวอย่าง').click();
579   await visible(page, 'ยอดคงเหลือ', 8000);
580   await page.goto(`${APP}runway`);
581   await introGone(page);
582   // What-if slider: drag the gold thumb to spend 30% less a day.
583   const slider = page.getByRole('slider', { name: 'ใช้น้อยลงวันละกี่เปอร์เซ็นต์' }).first();
584   await slider.scrollIntoViewIfNeeded();
585   // A raw mouse drag does not wait like click() does: make sure the opening animation has
    fully gone.
586   const introWait = Date.now();
587   await page.getByLabel('กำลังเปิด MindPay').waitFor({ state: 'detached', timeout: 20000
    }).catch(() => {});
588   const introMs = Date.now() - introWait;
589   // Wait until the page stops moving (cards slide in), so the drag starts on the thumb.
590   let box = await slider.boundingBox();
591   for (let i = 0; i < 12; i++) {
592     await page.waitForTimeout(250);
593     const again = await slider.boundingBox();
594     const still = box && again && Math.abs(again.y - box.y) < 0.5 && Math.abs(again.x - box.x)
    < 0.5;
595     box = again;
596     if (still) break;
597   }
598   const under = box
    ? await page.evaluate(({ x, y }) => {
599       const el = document.elementFromPoint(x, y);
600       return el ? `${el.tagName} role=${el.getAttribute('role')}`
601         label=${el.getAttribute('aria-label')}` : 'nothing';
602     }, { x: box.x + 15, y: box.y + box.height / 2 })
    : 'no box';
603   if (box) {
604     // hover() waits until the slider itself receives the pointer (nothing on top of it).
605     await slider.hover({ position: { x: 15, y: box.height / 2 } }).catch(() => {});
606     await page.mouse.down();

```

```

608     await page.mouse.move(box.x + 15 + (box.width - 30) * 0.3, box.y + box.height / 2, { steps:
8 });
609     await page.mouse.move(box.x + 15 + (box.width - 30) * 0.6, box.y + box.height / 2, { steps:
8 });
610     await page.mouse.up();
611 }
612 const slid = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
613 await page.waitForTimeout(900);
614 await shot(page, '28-runway-slider');
615 check('Runway: dragging the slider to 30% less a day shows the extra days', slid === '30' &&
(await visible(page, 'เพิ่มขึ้น'), `value=${slid} box=${JSON.stringify(box)} under=${under}
introWaitMs=${introMs}`);
616 // The arrow keys move it too (keyboard users on the web).
617 await slider.focus();
618 await page.keyboard.press('ArrowRight');
619 const keyed = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
620 check('Runway: the slider also moves with the arrow keys', keyed === String(Number(slid || 0)
+ 5), `value=${keyed}`);
621 await page.keyboard.press('Home');
622 await page.fill('#runway-price', '2000');
623 check('Runway: buying ฿2,000 shows the days before and after', (await visible(page, 'ถ้าซื้อ
฿2,000')) && (await visible(page, 'ตอนนี้')));
624 await button(page, 'ถามโค้ชเรื่องนี้').click();
625 const asked = await page.locator('#coach-input').inputValue().catch(() => '');
626 check('Runway: "ask the coach" opens the coach with the question ready',
asked.includes('฿2,000'), asked);
627 await ctx.close();

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:572

บรรทัด	ความหมาย
574–579	เปิดแท็บใหม่ ไปหน้าต้อนรับ รอแอนิเมชันเปิดแอปหาย และ "ลองใช้ด้วยข้อมูลตัวอย่าง" แล้วรอเห็นคำว่า "ยอดคงเหลือ"
580	ไปที่หน้าเงินพอถึง
583	หาแถบเลื่อนจากชื่อ "ใช้น้อยลงวันละกี่เปอร์เซ็นต์"
589–597	รอนหน้าจอหยุดขยับ (การ์ดค่อย ๆ เลื่อนเข้ามา) ไม่เงินจะลากพลาด
604–611	จำลองเมาส์: กดที่ปุ่มกลมของแถบ ลากไป 30% แล้ว 60% ของความกว้าง แล้วปล่อย
612–615	อ่านค่าปัจจุบันของแถบ ถ่ายภาพ และตรวจสอบว่าได้ 30 และมีคำว่า "เพิ่มขึ้น"
617–620	ทดสอบคีย์บอร์ด: กดลูกศรขวา ค่าต้องเพิ่ม 5
622–623	พิมพ์ราคา 2000 ในเช็ก่อนจ่าย ต้องเห็น "ถ้าซื้อ ฿2,000" และ "ตอนนี้"
624–626	กด "ถามโค้ชเรื่องนี้" ช่องพิมพ์ในหน้าโค้ชต้องมีคำถามที่มี "฿2,000" รออยู่ (เชื่อม FR-6 กับ FR-5)

ผลล่าสุด: **145/145 ข้อผ่าน** ทั้งโหมดสว่างและมืด (รันซ้ำตอนทำเล่มนี้ วันที่ 9 ต.ค. 2569 ก็ได้ 145/145 และ unit test 157/157) และมี workflow รันชุดเดียวกันบน WebKit (เอนจินของ Safari) แบบ iPhone โหมดมืดด้วย

Manual test: ทดสอบบนมือถือจริง

บางอย่างทดสอบอัตโนมัติไม่ได้ เช่น ไมโครโฟนจริง แกลเลอรีจริง สลิปจริงจากธนาคาร ทีมจึงมีแผน MT-01 ถึง MT-12 ใน [docs/TRACEABILITY.md](#) ให้สมาชิกทดสอบบนมือถือแล้วกรอกผล

ID	ขั้นตอน	ผลที่คาดหวัง
MT-02	จดรายจ่าย ฿85 หมวดอาหาร	ยอดคงเหลือลด ฿85, หมวดอาหารเพิ่ม, Runway เปลี่ยน
MT-03	ลบรายการ แล้วกด "เลิกทำ"	รายการกลับมา ยอดเท่าเดิม
MT-08	ถามโค้ช "ชื่อของ 500 บาทวันนี้ได้ไหม"	ตอบไทย อ้าง Runway ก่อน/หลัง ไม่ตำหนิ
MT-10	บัญชี A ไม่เห็นรายการของบัญชี B	RLS กันไว้
MT-11	บันทึกสลิปจริงลงแกลเลอรี แล้วเปิดแอป	ภายในไม่กี่วินาทีขึ้น "เพิ่ม 1 รายการจากสลิปใหม่"
MT-12	พูด "ข้าว 50 บาท น้ำ 15"	เห็น 2 รายการ บันทึกแล้วยอดลด ฿65

! ตรวจสอบนำเสนอ

ในเอกสาร Traceability ช่อง "ผล" ของ MT ทุกข้อยังเป็น ☐ (ยังไม่ได้กรอก) และในบันทึกการใช้ AI ช่อง "Human verification" ยังเป็น ☐ หลายข้อ ทีมควรทดสอบจริงและกรอกผลก่อนส่งงาน เพราะอาจารย์อาจถามว่า "ทดสอบบนมือถือจริงหรือยัง"

Traceability: ผูก Requirement → โค้ด → ทดสอบ

วิชา Software Engineering เน้น "Golden Thread" คือทุก requirement ต้องโยงไปหาโค้ดที่ทำ และเทสต์ที่พิสูจน์ได้ ทีมเราเก็บไว้ใน [docs/TRACEABILITY.md](#) เป็นตาราง 3 คอลัมน์: Acceptance criteria | Code | Test เช่น

Acceptance criteria (FR-6)	Code	Test
ค่าเฉลี่ยนับวันที่ไม่ได้ใช้เงินด้วย	averageDailyExpense	TC-12
ยอด ≤ เส้นสำรอง → สถานะ below_floor	computeRunway	TC-16

แต่ละเล่มในภาค ข มีตารางแบบนี้ครบทุกข้อของ FR นั้น

ตรวจคุณภาพอื่น ๆ

คำสั่ง	ตรวจอะไร
npm run typecheck	TypeScript ตรวจชนิดข้อมูลทั้งโปรเจกต์ (0 error)
npx expo lint	ESLint ตรวจรูปแบบโค้ดและกฎของ React Hooks
Supabase Advisors	ตัวตรวจความปลอดภัยและประสิทธิภาพของฐานข้อมูล (แก้จนเหลือ 0 ปัญหาเรื่องตาราง)

คำสั่ง	ตรวจสอบอะไร
ผู้ตรวจอิสระ	ตอนตรวจทั้ง repo (1–3 ต.ค.) ให้ผู้ตรวจที่ไม่ได้เห็นงานเทียบพฤติกรรมก่อน/หลังแก้ (เช่น ประโยคพูด 260 ประโยค ชื่อผู้รับ 120 ชื่อ) เจอ regression 10 ข้อ แก่ครบพร้อมทดสอบ

Git และ GitHub: เก็บโค้ด ทำงานเป็นทีม และ build อัตโนมัติ

ทุกการแก้ไขโค้ดถูกบันทึกเป็นประวัติ และทุกครั้งที่เราส่งโค้ดขึ้น GitHub จะมีหุ่นยนต์ 4 ตัวช่วยทดสอบ สร้างเว็บ สร้าง APK และส่งอัปเดตให้ผู้ใช้

Git: ระบบบันทึกประวัติ

≈ เปรียบเทียบ

Git เหมือนปุ่ม "Save game" ของเกม ทุกครั้งที่ทำสำเร็จหนึ่งด่าน (แก้ฟิเจอร์เสร็จ) เราเซฟไว้พร้อมโน้ตว่าทำอะไร ถ้าพลาดภายหลังก็โหลดเซฟเก่ากลับมาได้ และดูย้อนได้ว่าใครเปลี่ยนอะไรเมื่อไร

คำ	ความหมาย
repository (repo)	โฟลเดอร์โปรเจกต์ที่ Git ดูแล รวมประวัติทั้งหมด (ซ่อนอยู่ในโฟลเดอร์ .git)
commit	"จุดเซฟ" 1 ครั้ง มีรหัส (เช่น 646b485) ข้อความอธิบาย ผู้ทำ และเวลา
branch	"เส้นทางคู่ขนาน" ของโค้ด แยกไปทำฟิเจอร์โดยไม่กระทบเส้นหลัก เส้นหลักของทีมชื่อ main
remote / origin	สำเนาของ repo บนเซิร์ฟเวอร์ (GitHub)
push	ส่ง commit ในเครื่องขึ้น GitHub
pull	ดึง commit ใหม่จาก GitHub ลงเครื่อง
clone	คัดลอก repo ทั้งหมดจาก GitHub ลงเครื่องครั้งแรก
pull request (PR)	คำขอรวม branch หนึ่งเข้า main ให้เพื่อนรีวิวก่อน
merge conflict	สองคนแก้บรรทัดเดียวกัน Git ไม่รู้จะเลือกของใคร ต้องตัดสินใจเอง
.gitignore	รายชื่อไฟล์ที่ไม่ต้องเก็บใน Git เช่น node_modules (ใหญ่และติดตั้งใหม่ได้), .env (อาจมีความลับ), dist (สร้างใหม่ได้)

ใช้ Git ประจำวัน

Terminal

วงจรทำงาน 1 รอบ

```

1 git pull                                # 1. ดึงของล่าสุดก่อนเริ่ม
2 git checkout -b fix-runway-text         # 2. แยก branch ใหม่ (ไม่บังคับ)
3 # ... แก้โค้ดใน VS Code ...
4 npm test                                # 3. รันเทสต์ให้ผ่านก่อน
5 git status                              # 4. ดูว่าแค่ไฟล์ไหนไปบ้าง
6 git add src/domain/runway.ts            # 5. เลือกไฟล์ที่จะเซฟ
  
```

```
7 git commit -m "Runway: say 'ประมาณ' in the date" # 6. เซฟพร้อมข้อความ
8 git push -u origin fix-runway-text # 7. ส่งขึ้น GitHub แล้วเปิด Pull Request
```

ใน VS Code ทำแบบเดียวกันได้ด้วยเมาส์ที่เก็บ **Source Control** (Ctrl+Shift+G): ไฟล์ที่แก้ไขขึ้นในรายการ กด + เพื่อเลือก
พิมพ์ข้อความ กด Commit แล้วกด Sync Changes

★ ข้อความ commit ที่ดี

บอกว่า "ทำอะไร และทำไม" สั้น ๆ เช่นของทีมเรา: *"Runway: arrow keys on the web; steadier drag test"*, *"Audit follow-up: fix regressions found by independent review"* อ่านประวัติแล้วรู้เรื่องโดยไม่ต้องเปิดโค้ด ดูประวัติทั้งหมดด้วย `git log --oneline`

ประวัติของ MindPay บน GitHub

repo อยู่ที่ github.com/2550expo-tech/Socrates-and-Skeletons - (สาธารณะ ใครก็อ่านได้) branch main มี **66 commit** ตั้งแต่ 27 ก.ย. ถึง 3 ต.ค. 2569

วันที่	commit	งานหลักของวันนั้น (จากข้อความ commit)
27 ก.ย.	20	MVP แรก FR-1, FR-2, FR-4, FR-5, FR-6 · build APK อัตโนมัติ · เว็บบน GitHub Pages · EAS Update · AI ใช้ Gemini หรือ Claude · อ่านสลิปใหม่อัตโนมัติตอนเปิดแอป
28 ก.ย.	22	ระบบสมัครแบบรหัสอีเมล, นื่องกล้า, สแกนเองแยกตามช่วงเวลา, แก้ปัญหาคุกกี้ AI, อ่าน QR มุมขวาล่าง, เทสต์ในเบราว์เซอร์ (รวม Safari)
29 ก.ย.	19	พุดจด (FR-3), นื่องกล้าพุดได้, ธีมสี 8 แบบ, เพิ่มเงินเข้า, รายการประจำ, กระปุกออม
30 ก.ย.	2	ธีมฮาโลวีน, เทสต์ความอ่านง่ายของสี
1-3 ต.ค.	3	ตรวจทั้ง repository แก่บ๊ิก จัด Design System ให้ตรงกัน

GitHub Actions: หุ่นยนต์ที่ทำงานทุกครั้ง push

CI/CD (Continuous Integration / Continuous Delivery) คือให้คอมพิวเตอร์ทดสอบและส่งของให้อัตโนมัติทุกครั้งที่ได้เปลี่ยน ของทีมเราเขียนเป็นไฟล์ YAML ใน `.github/workflows/` มี 4 ตัว

ไฟล์	ทำอะไร	ผลลัพธ์
<code>web.yml</code>	build เว็บ → unit test → E2E 145 ข้อ → ถ้าผ่านทั้งหมดจึงเผยแพร่	เว็บที่ 2550expo-tech.github.io/Socrates-and-Skeletons/
<code>android-apk.yml</code>	unit test → typecheck → สร้างโปรเจกต์ Android → build APK (15-25 นาที)	ไฟล์ <code>mind-pay.apk</code> ในหน้า Releases "latest-apk"
<code>eas-update.yml</code>	unit test → ส่งโค้ด JavaScript ใหม่ขึ้น EAS Update	แอปที่ลงแล้วได้โค้ดใหม่ตอนเปิดครั้งถัดไป

ไฟล์	ทำอะไร	ผลลัพธ์
e2e-webkit.yml	E2E ชัดเดียวกันบนบนแอนดรอยด์ Safari แบบ iPhone โหมดมืด	รายงานผล (ไม่บล็อกการเผยแพร่)

git push ขึ้น main

↓ GitHub เปิดเครื่องเสมือน Ubuntu ให้ฟรีตัวละ 1 เครื่อง ทำพร้อมกัน

web.yml

เทสไม่ผ่าน = เว็บไม่เปลี่ยน ผู้ใช้ยังได้เวอร์ชันเดิมที่ต่ออยู่

android-apk.yml

เทสหรือ typecheck ไม่ผ่าน = ไม่มี APK ใหม่

eas-update.yml

ต้องมี secret EXPO_TOKEN และเทสผ่าน

e2e-webkit.yml

ตรวจบน Safari เพิ่ม

อ่าน web.yml ที่ละบรรทัด

.github/workflows/web.yml

บรรทัด 1-76

```

1 # Publishes the web version of MindPay to GitHub Pages on every push to main:
2 # https://2550expo-tech.github.io/Socrates-and-Skeletons-/
3 # The web version shows the app at phone width, works with real accounts and
4 # demo data, and reads slips from photos the user uploads. Scanning the whole
5 # gallery is only in the phone app. On iPhone: open in Safari -> Share ->
6 # Add to Home Screen, and it opens full screen with the MindPay icon.
7 name: Deploy web app
8
9 on:
10   push:
11     branches: [main]
12   workflow_dispatch:
13
14 permissions:
15   contents: write
16
17 concurrency:
18   group: web-pages
19   cancel-in-progress: true
20
21 jobs:
22   web:
23     runs-on: ubuntu-latest
24     timeout-minutes: 20
25     steps:
26       - uses: actions/checkout@v4
27
28       - uses: actions/setup-node@v4
29         with:
30           node-version: 22
31           cache: npm
32
33       - run: npm ci
34
35       - name: App settings (public Supabase URL and publishable key)
36         run: cp .env.example .env

```

```

37
38   - name: Export web build
39     env:
40       EXPO_WEB_BASE_URL: /Socrates-and-Skeletons-
41     run: npx expo export --platform web --output-dir dist
42
43   - name: Home-screen icon and full-screen mode (iPhone "Add to Home Screen")
44     env:
45       EXPO_WEB_BASE_URL: /Socrates-and-Skeletons-
46     run: node scripts/web-home-screen.mjs dist
47
48   - name: Single-page app fallback for deep links
49     run: cp dist/index.html dist/404.html
50
51   - name: Unit tests
52     run: npm test
53
54   # Browser test of every sign-in path (sign up, email code, links, reset
55   # password, sign out) against a local stand-in for Supabase. If anything
56   # fails, the website is NOT published and the screenshots are attached.
57   - name: Install the test browser
58     run: npx playwright install --with-deps chromium
59
60   - name: Browser tests (sign-in flows)
61     run: node e2e/web-auth.mjs dist e2e-shots
62
63   - name: Keep screenshots when the browser tests fail
64     if: failure()
65     uses: actions/upload-artifact@v4
66     with:
67       name: e2e-screenshots
68       path: e2e-shots
69
70   - name: Publish to the gh-pages branch
71     uses: peaceiris/actions-gh-pages@v4
72     with:
73       github_token: ${ secrets.GITHUB_TOKEN }
74       publish_dir: ./dist
75       force_orphan: true
76       commit_message: 'Deploy web app'

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `.github/workflows/web.yml:1`

บรรทัด	ความหมาย
1-6	คอมเมนต์ (YAML ใช้ #) บอกว่าเผยแพร่เว็บไปที่ไหน และวิธีเพิ่มลงหน้าจอโฮมบน iPhone
7	ชื่อที่แสดงในแท็บ Actions ของ GitHub
9-12	เมื่อไรจะทำงาน: ทุกครั้งที่ push ขึ้น branch main และกดสั่งเองได้ (workflow_dispatch)
14-15	สิทธิ์เขียนลง repo (ต้องใช้ตอนเผยแพร่ไป branch gh-pages)
17-19	ถ้า push ใหม่มาระหว่างที่ตัวเก่ายังทำงาน ยกเลิกตัวเก่า ทำแต่ตัวล่าสุด
21-24	งานชื่อ web รับบนเครื่อง Ubuntu ล่าสุด ไม่เกิน 20 นาที
26	ดีงโค้ดของ commit นี้ลงเครื่อง (uses = ใช้ action สำเร็จรูปที่คนอื่นเขียนไว้)

บรรทัด	ความหมาย
28–31	ติดตั้ง Node.js 22 และจำ cache ของ npm ให้ครั้งหน้าเร็วขึ้น
33	npm ci ติดตั้งแพ็คเกจตาม package-lock.json แบบเป๊ะ (ต่างจาก npm install ตรงที่ไม่เปลี่ยนเวอร์ชันเลย)
35–36	สร้าง .env จากตัวอย่าง (มีแค่กุญแจสาธารณะ)
38–41	build เว็บลงโพลเดอร์ dist พร้อมบอกว่าเว็บอยู่ใต้ /Socrates-and-Skeletons-
43–46	เพิ่มไอคอนและตั้งค่าให้ "เพิ่มไปยังหน้าจ่อโฮม" แล้วเปิดเต็มจอเหมือนแอป
48–49	คัดลอกหน้าแรกเป็น 404.html เพราะ GitHub Pages ไม่รู้จักที่อยู่อย่าง /runway ถ้ามีคนเปิดตรง ๆ จะได้หน้า 404 ซึ่งก็คือแอปเราที่พาไปหน้านั้นให้
51–52	รัน unit test 157 ข้อ ไม่ผ่าน = หยุดทั้งงาน
57–61	ติดตั้งเบราวเซอร์ทดสอบ แล้วรัน E2E 145 ข้อ
63–68	if: failure() = เฉพาะตอนพัง: เก็บภาพหน้าจอจาก E2E ไว้ให้ดาวนโหลดดูว่าพังตรงไหน
70–76	ผ่านทุกด่านแล้วจึงเผยแพร่โพลเดอร์ dist ขึ้น branch gh-pages ซึ่ง GitHub Pages ใช้เป็นเว็บไซต์

อ่าน android-apk.yml (ส่วนสำคัญ)

.github/workflows/android-apk.yml

บรรทัด 12–90

```

12 on:
13   push:
14     branches: [main]
15     # Editing documents only does not need a new APK.
16     paths-ignore: ['**/*.md', 'docs/**']
17   workflow_dispatch:
18
19 permissions:
20   contents: write
21
22 # A build that already started runs to the end; newer pushes wait in line
23 # (only the newest waiting push is kept).
24 concurrency:
25   group: android-apk
26   cancel-in-progress: false
27
28 jobs:
29   apk:
30     runs-on: ubuntu-latest
31     timeout-minutes: 45
32     steps:
33       - uses: actions/checkout@v4
34
35       - uses: actions/setup-node@v4
36         with:
37           node-version: 22
38           cache: npm
39
40       - uses: actions/setup-java@v4
41         with:
42           distribution: temurin

```

```

43     java-version: 17
44
45     - name: Install dependencies
46       run: npm ci
47
48     - name: Unit tests
49       run: npm test
50
51     - name: Typecheck
52       run: npx tsc --noEmit
53
54     - name: App settings (public Supabase URL and publishable key)
55       run: cp .env.example .env
56
57     - name: Generate the native Android project
58       run: npx expo prebuild --platform android --no-install
59
60     - name: Give Gradle enough memory
61       # The default Metaspace ran out during Android lint and the build hung until the job
timeout.
62       run: |
63         sed -i '/^org.gradle.jvmargs=/d' android/gradle.properties
64         # The generated file has no trailing newline: start with one so the setting is not
glued onto the last line.
65         printf '\norg.gradle.jvmargs=-Xmx4g -XX:MaxMetaspaceSize=1g -Dfile.encoding=UTF-8\n' >>
        android/gradle.properties
66         tail -n 3 android/gradle.properties
67
68     - name: Build release APK
69       working-directory: android
70       # arm64 only: covers practically every Android phone from the last 7 years and builds
much faster.
71       # lintVitalAnalyzeRelease is skipped: it is a code-style check that is not needed to
produce the APK.
72       run: ./gradlew assembleRelease --no-daemon -PreactNativeArchitectures=arm64-v8a -x
        lintVitalAnalyzeRelease --stacktrace
73
74     - name: Rename APK
75       run: cp android/app/build/outputs/apk/release/app-release.apk mind-pay.apk
76
77     - uses: actions/upload-artifact@v4
78       with:
79         name: mind-pay-apk
80         path: mind-pay.apk
81
82     - name: Publish to the "latest-apk" release
83       env:
84         GH_TOKEN: ${GITHUB_TOKEN}
85       run: |
86         gh release delete latest-apk --yes --cleanup-tag || true
87         gh release create latest-apk mind-pay.apk \
88           --title "MindPay (latest APK)" \
89           --notes "Built from ${GITHUB_SHA::7} on $(date -u +%Y-%m-%d %H:%M UTC'). Open this
        page on an Android phone, tap mind-pay.apk, and install. Debug-signed build for testing." \
90           --prerelease

```

บรรทัด	ความหมาย
12–17	ทำงานเมื่อ push ขึ้น main ยกเว้นแค่เอกสาร (.md, docs/) ซึ่งไม่ต้อง build APK ใหม่
24–26	ตัวที่เริ่ม build แล้วให้ทำงานจบ ตัวใหม่ต่อคิว (build ใช้เวลานาน ไม่อยากทิ้งกลางทาง)
31	เวลาสูงสุด 45 นาที
40–43	ติดตั้ง Java 17 เพราะเครื่องมือ build ของ Android (Gradle) ต้องใช้
48–52	ด้านตรวจ: unit test และ typecheck ต้องผ่านก่อน
57–58	expo prebuild สร้างโฟลเดอร์ android ทั้งโปรเจกต์จาก app.json (ทีมไม่เก็บโฟลเดอร์นี้ใน Git สร้างใหม่ทุกครั้ง)
60–66	เพิ่มหน่วยความจำให้ Gradle (คอมเมนต์เล่าว่าเคยค้างจนหมดเวลาเพราะหน่วยความจำไม่พอ)
68–72	build APK แบบ release เฉพาะสถาปัตยกรรม arm64 (ครอบคลุมมือถือแทบทุกเครื่องใน 7 ปีหลัง และ build เร็วกว่า)
74–80	เปลี่ยนชื่อเป็น mind-pay.apk และเก็บไว้
82–90	ลบ release "latest-apk" เดิม แล้วสร้างใหม่พร้อมไฟล์ APK ใหม่ (ลิงก์ดาวน์โหลดจึงเหมือนเดิมตลอด) บอกว่า build จาก commit ไหน เมื่อไร

📌 APK แบบ debug-signed

APK นี้ "เซ็น" ด้วยกุญแจ debug ของ Android ใช้ทดสอบและพีซีเซ็นได้ แต่ส่งขึ้น Play Store ไม่ได้ ถ้าจะลง Play Store ต้อง build ด้วย EAS Build แบบ production และมีบัญชีนักพัฒนา Google (README อธิบายไว้)

Secrets บน GitHub

ค่าลับที่ workflow ต้องใช้ เก็บใน Settings → Secrets and variables → Actions ของ repo ไม่ได้อยู่ในโค้ด

Secret	ใช้ที่	ได้มาจาก
GITHUB_TOKEN	web.yml, android-apk.yml	GitHub สร้างให้อัตโนมัติทุกครั้ง
EXPO_TOKEN	eas-update.yml	expo.dev → Account settings → Access tokens (ถ้าไม่ตั้ง workflow นี้ ข้ามไปเลย ๆ)

ส่วนกุญแจ AI ไม่ได้อยู่บน GitHub เลย อยู่ใน Supabase Secrets (un A10)

ไฟล์เอกสารใน repo

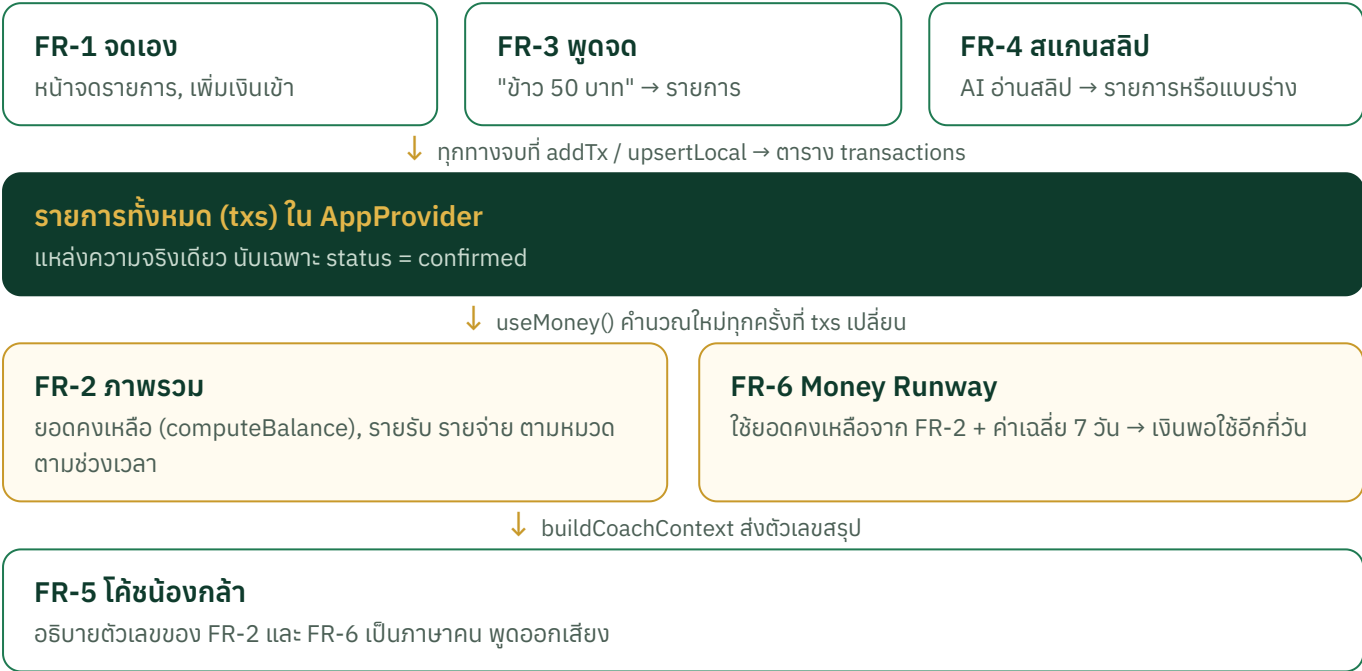
ไฟล์	มีอะไร
README.md	หน้าแรกของ repo: ลิงก์ลองใช้, FR, สถาปัตยกรรม, วิธีติดตั้ง, ตั้งค่า AI, ความปลอดภัย, ข้อจำกัด
docs/TRACEABILITY.md	FR → acceptance criteria → โค้ด → เทสต์ และแผนทดสอบมือ (MT)

ไฟล์	มีอะไร
docs/AI_USAGE_LOG.md	บันทึกการใช้ AI ตามที่ SRS กำหนด 20 แถว
docs/DESIGN_SYSTEM.md	กฎการออกแบบ (บท A11)
docs/SUPABASE_AUTH_SETUP.md	ตั้งค่าอีเมลยืนยันตัวตนของ Supabase
AGENTS.md / CLAUDE.md	คำแนะนำสำหรับผู้ช่วย AI ที่ช่วยเขียนโค้ด (ใช้ Expo 57, คำสั่งที่ควรรับ)

6 FR ทำงานร่วมกันยังไง และคำถามระดับระบบ

ไม่ว่ารับผิดชอบ FR ไหน อาจารย์อาจถามว่า "FR ของเธอเชื่อมกับของเพื่อนยังไง" หรือ "ทำไมเลือกเทคโนโลยีนี้" บทนี้เตรียมคำตอบไว้ให้

สายข้อมูลของทั้งระบบ



ใครใช้ของใคร

FR	ใช้ข้อมูล/ฟังก์ชันจาก	ส่งผลต่อ
FR-1	money.ts (แปลงจำนวนเงิน), categories.ts (หมวด), dates.ts	ทุกรายการที่จด/แก้/ลบ ทำให้ FR-2, FR-6, FR-5 คำนวณใหม่; หน้าจดยรายการยังเป็นหน้าตรวจสอบสลิปของ FR-4
FR-2	รายการจาก FR-1/3/4, ยอดตั้งต้นจากโปรไฟล์	ยอดคงเหลือเป็นตัวตั้งของ FR-6, ยอดตามหมวดส่งให้โค้ช FR-5
FR-3	categories.ts (เดาหมวด), money/dates, addTx ของ FR-1	สร้างรายการแบบยืนยันแล้ว เข้ายอดทันที
FR-4	Edge Function parse-slip + AI, หน้าจดยรายการของ FR-1 (ตรวจแบบร่าง), categories.ts	รายการจากสลิป: ชัด → เข้ายอด, ไม่ชัด → แบบร่าง (ไม่นับใน FR-2/FR-6 จนยืนยัน)
FR-5	ตัวเลขจาก FR-2 และ FR-6, Edge Function coach + AI	ไม่เปลี่ยนข้อมูล (อ่านอย่างเดียว) แต่ชวนผู้ใช้ไปหน้า Runway/สแกน
FR-6	computeBalance (FR-2), รายจ่ายที่ยืนยันแล้ว (FR-1/3/4), เส้นสำรองจากโปรไฟล์, กระปุกออม	ตัวเลข Runway บนหน้าหลัก, การ์ดเตือนของโค้ช, "เช็กก่อนจ่าย" ส่งคำถามให้ FR-5

ไฟล์ที่หลาย FR ใช้ร่วมกัน (แก้แล้วกระทบหลายคน)

ไฟล์	ใครใช้	ถ้าแก้ผิดจะเกิดอะไร
src/domain/types.ts	ทุก FR	TypeScript จะแจ้ง error ทั่วทั้งแอป
src/domain/money.ts	FR-1, 3, 4, 6	จำนวนเงินเพี้ยนทุกที่
src/domain/dates.ts	ทุก FR	รายการไปอยู่ผิดวัน ค่าเฉลี่ย 7 วันผิด
src/domain/summary.ts	FR-2, 5, 6	ยอดคงเหลือผิด → Runway และโค้ชผิดตาม
src/domain/categories.ts	FR-1, 2, 3, 4	หมวดผิด กราฟตามหมวดผิด
src/data/AppProvider.tsx	ทุกหน้า	ข้อมูลไม่โหลด/ไม่อัปเดต
src/app/transaction.tsx	FR-1 และ FR-4 (ตรวจสอบ)	จดยรายการหรือยืนยันสลิปไม่ได้

นี่คือเหตุผลที่ทีมมี unit test ครอบคลุมไฟล์เหล่านี้ทั้งหมด และให้ GitHub รับเทสต์ก่อนปล่อยทุกครั้ง

คำถามระดับระบบ ที่อาจารย์ถามใครก็ได้

- Q1** ทำไมเลือก Expo / React Native?

โค้ดชุดเดียวได้ทั้ง Android, iPhone และเว็บ ทีมเล็กเวลาน้อยจึงไม่ต้องเขียน 3 รอบ ภาษาเดียว (TypeScript) ทั้งแอปและเซิร์ฟเวอร์ มีเครื่องมือ build APK และส่งอัปเดตอัตโนมัติ
- Q2** ทำไมเลือก Supabase?

ได้ฐานข้อมูล PostgreSQL, ระบบสมัคร/เข้าสู่ระบบ, และที่รับโค้ดฝั่งเซิร์ฟเวอร์ในที่เดียว มีแพ็คเกจฟรี และ Row Level Security ทำให้แอปคุยกับฐานข้อมูลได้ตรง ๆ อย่างปลอดภัยโดยไม่ต้องเขียนเซิร์ฟเวอร์ API เอง
- Q3** ข้อมูลผู้ใช้ปลอดภัยยัง?

ทุกตารางเปิด RLS แต่ละคนอ่านเขียนได้เฉพาะแถวของตัวเอง (ทดสอบ MT-10) คุกกี้ในแอปเป็นคุกกี้สาธารณะ คุกกี้แวลบและคุกกี้ AI อยู่บนเซิร์ฟเวอร์เท่านั้น รูปสลิปไม่ถูกเก็บบนเซิร์ฟเวอร์ โค้ชได้แค่ออกรวม
- Q4** ทำไมเก็บเงินเป็นสตางค์?

ทศนิยมในคอมพิวเตอร์ไม่เป๊ะ ($0.1 + 0.2 = 0.30000000000000004$) เก็บเป็นจำนวนเต็มสตางค์จึงบวกลบได้แม่นยำ 100% แปลงเป็นบาทตอนแสดงผลเท่านั้น
- Q5** ทำไมต้องจัดวันตามเวลาไทย?

ฐานข้อมูลเก็บเวลา UTC ซึ่งช้ากว่าไทย 7 ชั่วโมง ถ้าไม่แปลง รายการตอนเข้ามีดจะไปอยู่วันก่อนหน้า ยอด "วันนี้" และค่าเฉลี่ย 7 วันจะผิด ทุกจุดจึงใช้ bkkDayKey
- Q6** ยอดคงเหลือเก็บไว้ในฐานข้อมูลไหม?

ไม่ เก็บเฉพาะรายการ ตัวเลขทั้งหมด (ยอด ค่าเฉลี่ย Runway) คำนวณสดในเครื่องจากรายการทุกครั้งที่มีข้อมูลเปลี่ยน ตัวเลขจึงไม่มีทางขัดกันเอง
- Q7** แบบร่าง (draft) คืออะไร ทำไมต้องมี?

รายการจากสลิปที่ AI อ่านไม่มั่นใจ รอผู้ใช้ตรวจสอบก่อน ยังไม่นับในยอดใด ๆ เพราะถ้า AI อ่านผิดแล้วนับทันที ทุกตัวเลขในแอปจะผิดตาม
- Q8** ใช้ AI ตรงไหน และป้องกัน AI ผิดยังไง?

2 ที่: อ่านสลิป (FR-4) และโค้ช (FR-5) ป้องกันด้วย: อ่าน 2 ครั้งด้วย 2 รุ่น, เหนือความมั่นใจ 80%, ตรวจสอบ QR บนสลิป, แบบร่างรอตรวจ, โค้ชห้ามแต่งตัวเลข ตัวเลขทั้งหมดคำนวณโดยโค้ดที่ทดสอบแล้ว

Q9	ทำไมไม่เรียก AI จากแอปตรง ๆ? กฎแฉ AI จะหลุดไปกับแอป Edge Function ถือกฎแฉไว้ ตรวจสอบว่าเข้าสู่ระบบจริง และจำกัดโควตาต่อวันต่อคน
Q10	ทดสอบยังไง? 3 ระดับ: unit test 157 ข้อ (ฟังก์ชันคำนวณ), E2E 145 ข้อ (เปิดเว็บจริงเหมือนคน ด้วยเซิร์ฟเวอร์ปลอม), manual test MT-01 ถึง MT-12 บนมือถือจริง ทุกครั้งที่ push GitHub รับเทสต์ก่อนเผยแพร่
Q11	ถ้าอินเทอร์เน็ทหลุดระหว่างฟรีเซนต์? ใช้โหมดทดลอง ข้อมูลอยู่ในเครื่อง ทำงานได้ทุกอย่างยกเว้น AI
Q12	Requirement โยงกับโค้ดยังไง? เอกสาร docs/TRACEABILITY.md ผูกทุก acceptance criteria ของทุก FR กับไฟล์/ฟังก์ชันที่ทำ และรหัสเทสต์ (TC/MT) ที่พิสูจน์
Q13	ใช้ AI ช่วยเขียนโค้ดไหม? ใช้ บันทึกไว้ใน docs/AI_USAGE_LOG.md ตามที่ SRS กำหนด ทุกแถวบอกจุดประสงค์ ผลลัพธ์ สิ่งที่ทีมแก้หรือตัดสินใจ และสิ่งที่สมาชิกต้องตรวจสอบ (ควรตอนตามจริงว่าตัวเองตรวจสอบอะไรไปแล้วบ้าง)
Q14	ถ้ามีผู้ใช้เยอะขึ้น ระบบรับไหวไหม? การคำนวณทำในเครื่องของแต่ละคน เซิร์ฟเวอร์แค่เก็บข้อมูล มี index ช่วยค้นเร็ว และโหลดเป็นหน้า หน้าละ 1,000 แถว ส่วนที่ต้องระวังคือค่าใช้จ่าย AI ซึ่งมีโควตาต่อวันคุมไว้ และควรเปลี่ยนจาก Gemini แบบฟรีเป็นแบบเสียเงินเมื่อมีผู้ใช้จริง
Q15	อะไรที่ยังไม่ได้ทำ (นอก MVP)? Accountability Mode (UC-7), ออฟไลน์เต็มระบบ (UC-8), เชื่อม Gmail/LINE, รวบรวมรับในอนาคตเข้า Money Runway และยังไม่ได้ทดสอบบนมือถือหลายรุ่น

คำศัพท์รวม

คำ	ความหมายแบบง่าย
Acceptance criteria	เงื่อนไขที่ต้องผ่าน จึงจะถือว่า requirement นั้นเสร็จ
API / API key	ประตูให้โปรแกรมคุยกัน / รหัสลับที่ใช้ผ่านประตู
APK	ไฟล์ติดตั้งแอป Android
async / await	เขียนงานที่ต้องรอ (เช่นอินเทอร์เน็ท) โดยหน้าจอไม่ค้าง
CI/CD	ให้คอมพิวเตอร์ทดสอบและส่งของให้อัตโนมัติทุกครั้งที่เกิดเปลี่ยน (GitHub Actions)
Commit / Push / Branch	จุดเซฟ / ส่งขึ้น GitHub / เส้นทางคู่ขนานของโค้ด
Component	ชิ้นส่วนหน้าจอใน React เป็นฟังก์ชันที่คืน JSX
Context / Provider	วิธีแชร์ข้อมูลให้ทุกหน้าใน React (AuthProvider)
Design System / Token	กฎและชิ้นส่วนกลางของการออกแบบ / ค่ากลางที่ตั้งชื่อตามหน้าที่ เช่น theme.ink
E2E test	ทดสอบทั้งแอปเหมือนคนกดจริงในเบราว์เซอร์
Edge Function	โค้ดฝั่งเซิร์ฟเวอร์บน Supabase (parse-slip, coach)
EAS Update / OTA	ส่งโค้ดใหม่ไปถึงแอปที่ลงแล้ว โดยไม่ต้องลง APK ใหม่

คำ	ความหมายแบบง่าย
Expo Router	ระบบเปลี่ยนหน้าที่ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า
Hook	ฟังก์ชันพิเศษของ React ขึ้นต้นด้วย use เช่น useState
JSON	รูปแบบข้อมูลข้อความที่โปรแกรมใช้รับส่งกัน
JSX	หน้าที่ที่เขียนคล้าย HTML ในโค้ด React
Migration	ไฟล์ SQL ที่สร้าง/แก้ไขโครงสร้างฐานข้อมูล เรียงตามวันที่
Native module	ส่วนที่เขียนด้วยภาษาของมือถือ (เช่น ไมค์ แกลเลอรี) ต้องลง APK ใหม่ถึงจะมี
props / state	ค่าที่ส่งเข้า component / ความจำของ component
Pure function	ฟังก์ชันที่ใช้ค่าเดิมได้ผลเดิม ไม่ยุ่งกับข้างนอก ทดสอบง่าย
Regression	ของที่เคยทำงาน กลับพังหลังแก้ไขโค้ดส่วนอื่น
Repo (Repository)	1) โพลเดอร์โปรเจกต์ใน Git 2) ชั้น Repo ในแอปที่ซ่อนว่าข้อมูลเก็บที่ไหน
RLS	Row Level Security กฎในฐานข้อมูลให้แต่ละคนเห็นเฉพาะแถวของตัวเอง
Satang	หน่วยเงินในระบบ 100 สตางค์ = 1 บาท
Schema (JSON)	แบบฟอร์มบังคับโครงสร้างคำตอบของ AI
Session / Token	สถานะการเข้าสู่ระบบ / บัตรผ่านดิจิทัลที่แนบไปกับทุกคำขอ
SQL	ภาษาสั่งฐานข้อมูล
System prompt	คำสั่งประจำตัวที่บอก AI ว่าต้องเป็นใคร ตอบยังไง
Traceability	การโยง requirement → การออกแบบ → โค้ด → เทสต์
TypeScript	JavaScript ที่มีชนิดข้อมูล ช่วยจับผิดตั้งแต่ตอนเขียน
Unit test	ทดสอบฟังก์ชันทีละตัว
WCAG	มาตรฐานความเข้าถึงได้ของเว็บ เช่น ความต่างสีตัวหนังสือ $\geq 4.5:1$

ภาค ข

FR-5 AI Coach นื่องกล้า

โค้ชการเงิน AI ที่พูดได้ ภาคนี้เขียนในมุมมองของคนสร้าง FR-5 เอง ตั้งแต่ปัญหาที่ยาก
แก้ หน้าจอ หลักการคำนวณ โค้ดจริงที่ละบรรทัด การทดสอบ ไปจนถึงคำถามที่อาจารย์
น่าจะถาม

FR-5 ในหน้าเดียว

ถ้ามีคนถามว่า "FR-5 ของเธอทำอะไร" ให้ตอบจากหน้านี้

FR-5 **AI Coach** คือโค้ชการเงินส่วนตัวของ MindPay ชื่อ **น้องกล้า** เป็นต้นกล้าเงินในสีทอง (มาสกอตของแอป) หน้า及要求คือ **"อธิบายข้อมูลที่ยืนยันแล้วให้เข้าใจง่าย"** น้องกล้าไม่ได้แค่โชว์ตัวเลข แต่บอกว่าตัวเลขนั้นแปลว่าอะไร สำคัญยังไง และลองทำอะไรได้บ้าง ด้วยน้ำเสียงสุ่ม เป็นมิตร ไม่ตำหนิ และ **พูดออกเสียง** พร้อมซับไตเติลและปากที่ขยับตาม พยางค์

ขั้นที่ 1 "สิ่งที่ควรรู้ออนนี้" = กฎในแอป (ไม่ใช่ AI) · ขั้นที่ 2 "คุยกับน้องกล้า" = AI ตอบคำถามอิสระ

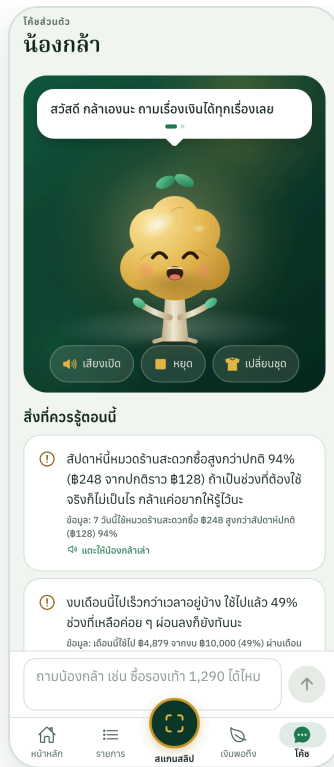
คำถาม → แอปสรุปเฉพาะยอดรวมที่ยืนยันแล้ว (ถ้าถามเรื่องชื่อของ แอปคิด Runway ก่อน/หลังให้เอง) → เซิร์ฟเวอร์ตรวจตัวตน และโควตา → AI ตอบ (ถูกสั่งให้ไม่เกิน 90 คำ) → แอปตัดเป็นซับไตเติลสั้น ๆ → เสียงภาษาไทยของเครื่องอ่านทีละก่อน

อยู่ตรงไหนในแอป

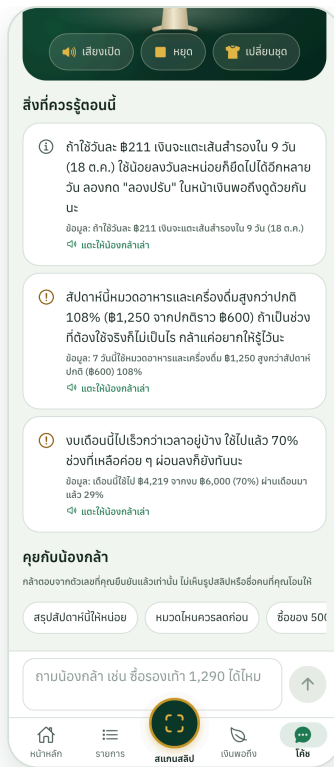
ที่	ผู้ใช้เห็นอะไร
แท็บ โค้ช (ไอคอนกล่องข้อความ)	เวทิน้องกล้าตัวใหญ่ + ฟองคำพูด · ปุ่มเสียงเปิด/ปิด หยุด ฟังอีกครั้ง เปลี่ยนชุด · การ์ด "สิ่งที่ควรรู้ออนนี้" · ห้องคุย "คุยกับน้องกล้า" · คำถามลัด 4 ข้อ · ช่องพิมพ์คำถาม
หน้าหลัก	น้องกล้าตัวเล็กพร้อมฟองคำพูดบรรทัดเดียวตามสถานะเงิน · แตะน้องกล้า = เคล็ดลับ · แตะต้นไม้เงิน = อธิบายใบทอง
แท็บ เงินพอถึง → "ใช้ก่อนจ่าย"	ปุ่ม "ถามโค้ชเรื่องนี้" ส่งคำถามพร้อมราคาไปรอในหน้าโค้ช (เชื่อมกับ FR-6)
ตั้งค่า	สวิตช์ "เสียงน้องกล้า" และปุ่ม "คุ้นชินน้องกล้า"
คุ้นชิน และ ความสำเร็จ	ชุดของน้องกล้า 19 ชุด ได้จากการจัดการเงิน 8 การกิจ และสกินลิมิเต็ดตามเทศกาล

4 ข้อที่ต้องจำ

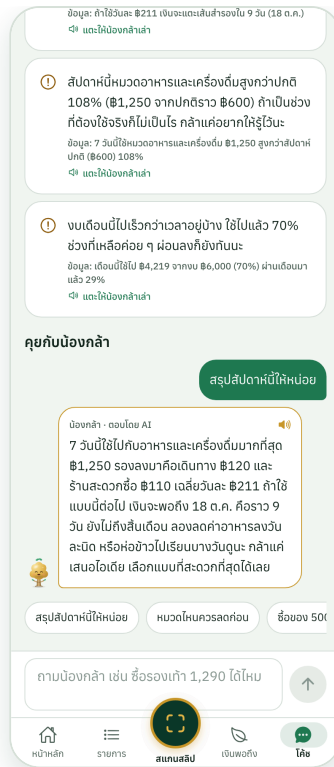
- โค้ดคิดเลข AI แค่อธิบาย** ตัวเลขทุกตัว (ยอด ค่าเฉลี่ย Runway ราคาก่อน/หลังซื้อ) คำนวณในแอปด้วยโค้ดที่มีเกสต์แล้วส่งให้ AI เป็น "ข้อเท็จจริง" พร้อมคำสั่งว่า **ห้ามแต่งตัวเลขเอง**
- ส่งเฉพาะยอดรวม AI** ไม่เห็นรูปสลิป ชื่อคนที่โอนให้ หรือเลขบัญชี เห็นแค่ยอดรวมตามหมวด ยอดคงเหลือ ค่าเฉลี่ย และ Runway
- ไม่ตำหนิ** ทุกข้อความเดินตามลำดับ "บอกข้อเท็จจริง → อธิบายว่าสำคัญยังไง → เสนอทางเลือกเล็ก ๆ → ให้ผู้ใช้ตัดสินใจเอง" มีเกสต์ตรวจว่าไม่มีคำดู
- ใช้ได้เสมอ** ชั้น "สิ่งที่ควรรู้ออนนี้" ไม่ต้องใช้ AI จึงทำงานแม้ไม่มีเน็ต และถ้าเครื่องไม่มีเสียงภาษาไทย น้องกล้ายังพูดด้วยซับไตเติลตามเวลาอ่าน ไม่มีอะไรพัง



แท็บโค้ช: นียงกล้าทักทาย ชับไตเติล ก่อนแรกจาก 2 ก่อน



สิ่งที่ควรรู้ตอนนี้: 3 ข้อจากกฎใน แอป พร้อมตัวเลขที่มา



คุยกับนียงกล้า: คำถามสด และคำตอบ (กรอบทอง = คำสั่งพูด)

1 ภาพหน้าจอในเล่มนี้ อ่านก่อน

ภาพถ่ายจากเว็บเวอร์ชันจริง 2 แบบ: (1) **โหมดทดลอง** (ข้อมูลตัวอย่าง) และ (2) **บัญชีทดสอบชื่อ "มายด์"** ที่ล็อกอินกับเซิร์ฟเวอร์จำลองตัวเดียวกับเกสต์ E2E ([e2e/fake-backend.mjs](#)) พร้อมรายการ 1 เดือนที่ฉันทใส่ไว้ (บท "ทัวร์หน้าจอ: คุยกับนียงกล้าด้วยบัญชีจริง")

คำตอบของนียงกล้าในภาพเป็นคำตอบจำลอง ที่ฉันเขียนตามกติกาใน system prompt ไม่ใช่คำตอบจริงของ AI (คำตอบจริงจะต่างกันทุกครั้ง) แต่สิ่งอื่นทั้งหมดเป็นโค้ดจริง: การ์ดสิ่งที่ควรรู้ ข้อมูลที่แอปส่งไปเซิร์ฟเวอร์ (ฉันดักเก็บไว้แล้วพิมพ์ในเล่มทั้งก้อน) การหาราคาในคำถาม การตัดชับไตเติล และข้อความ error ส่วน "เสียงภาษาไทย" ในเบราว์เซอร์ทดสอบเป็นเสียงจำลองที่บันทึกว่าพูดอะไร (วิธีเดียวกับเกสต์ E2E) เพราะเบราว์เซอร์อัตโนมัติไม่มีเสียงไทย

ปัญหาที่ฉันอยากแก้

ตัวเลขบอกว่า "เกิดอะไรขึ้น" แต่ไม่ได้บอกว่า "แล้วควรทำยังไงต่อ"

FR-2 และ FR-6 ให้ตัวเลขที่ถูกต้องแล้ว เช่น ยอดคงเหลือ ฿2,566 ค่าเฉลี่ยวันละ ฿211 เงินพอใช้อีก 9 วัน แต่คนส่วนใหญ่เห็นตัวเลขแล้วก็ยังไม่รู้ว่า "ตกลงดีหรือแย่" "เพราะอะไร" และ "เริ่มจากตรงไหน" ฉันจึงอยากได้ **เพื่อนที่เก่งเรื่องเงิน** มาช่วยเล่าตัวเลขเป็นภาษาคน

ตัวเลขอย่างเดียว	น้องกล้าเล่าให้ฟัง
หมวดอาหาร 7 วัน ฿1,250	"สัปดาห์นี้หมวดอาหารและเครื่องดื่มสูงกว่าปกติ 108% (฿1,250 จากปกติราว ฿600) ถ้าเป็นช่วงที่ต้องใช้จริงก็ไม่ใช่ไร กล้าแค่อยากให้รู้ไว้ละ"
งบ ฿6,000 ใช้ไป ฿4,219	"งบเดือนนี้ไปเร็วกว่าเวลาอยู่บ้าง ใช้ไปแล้ว 70% ช่วงที่เหลือค่อย ๆ ผ่อนลงก็ยังทันนะ"
อยากซื้อของ ฿1,290	Runway จาก 9 วันเหลือ 3 วัน พร้อมทางเลือก เช่น รอเงินก้อนใหม่ หรือหยุดกระปุกที่ละนิด แล้วให้ผู้ใชตัดสินใจเอง

ข้อความ 2 แถวแรกเป็นของจริงจากบัญชีทดสอบ "มายด์" (คำนวณด้วยฟังก์ชันจริง) แถวที่ 3 เป็นสรุปคำตอบจำลอง

แต่การเอา AI มาเป็นโค้ช มีความเสี่ยง 5 ข้อ

ความเสี่ยง	ถ้าไม่ระวังจะเกิดอะไร	ฉันค้นไว้ยังงี้
AI แต่งตัวเลข	บอกว่า "ใช้ไป ฿3,000" ทั้งที่จริง ฿1,250 ผู้ใช้ตัดสินใจผิด	แอปคิดตัวเลขเองทั้งหมด ส่งให้ AI ในก้อน DATA และสั่งว่า "ใช้เฉพาะตัวเลขใน DATA ห้ามแต่ง ถ้าตอบไม่ได้ให้บอกว่าขาดอะไร"
ข้อมูลส่วนตัวรั่ว	รูปสลิป ชื่อคน เลขบัญชี ไปอยู่กับผู้ให้บริการ AI	ส่งเฉพาะยอดรวมตามหมวด (buildCoachContext) ไม่มีรูป ไม่มีชื่อ ไม่มีเลขบัญชี และไม่บันทึกคำถาม/คำตอบใน log
พูดจาไม่ดี	"คุณใช้เงินเยอะเกินไป!" ทำให้ผู้ใช้รู้สึกผิดแล้วเลิกใช้แอป	กฎน้ำเสียงทั้งใน prompt และในข้อความที่ฉันเขียนเอง พร้อมเทสต์ TC-32, TC-50 ตรวจสอบคำต้องห้าม
ค่าใช้จ่ายบานปลาย	บัญชีที่ถูกขโมยหรือบักเรียก AI เป็นพันครั้ง	โควตา 60 ครั้ง/วัน/บัญชี คำถามไม่เกิน 400 ตัวอักษร ข้อมูลไม่เกิน 6,000 ตัวอักษร คำตอบไม่เกิน 500 token
ให้คำแนะนำเกินขอบเขต	แนะนำลงทุน กู้เงิน ครัวโต ทั้งที่ไม่ใช่ที่ปรึกษาที่มีใบอนุญาต	prompt บอกตรง ๆ ว่าห้ามแนะนำการลงทุน เงินกู้ และครัวโต

เป้าหมายที่ฉันตั้งไว้ตอนออกแบบ

- 1. **เข้าใจในประโยคเดียว** ทุกข้อความมีตัวเลขที่มาอยู่ข้างใต้ ("ข้อมูล: ...") ผู้ใช้ตรวจสอบย้อนได้
- 2. **ถูกต้องก่อนฉลาด** เรื่องที่สำคัญที่สุดคิดด้วยกฎในแอป AI เป็นแค่ชั้นเสริมสำหรับคำถามอิสระ
- 3. **อบอุ่นแต่ไม่ยืดเยื้อ** เสนอทางเลือก ไม่สั่ง ให้ผู้ใช้เป็นคนตัดสินใจเสมอ
- 4. **มีตัวตน** ผู้ใช้จำ "น้องกล้า" ได้ดีกว่า "ระบบแนะนำ" ตัวละครเดียวกันอยู่ทั้งหน้าหลัก โค้ช และตู้สกิน

5. ฟังได้ ไม่ต้องอ่าน พูดออกเสียงทีละก่อนสั้น ๆ พร้อมจับไตเติล เหมาะกับคนที่ไม่ชอบอ่านตัวหนังสือยาว

6. ไม่ฟังง่าย ไม่มีเน็ต ไม่มีสัญญาณ AI AI ไม่ว่าง ครบโควตา ไม่มีเสียงไทย ทุกกรณีมีข้อความบอกตรง ๆ และส่วนอื่นยังใช้ได้

Requirement การเปลี่ยนแปลง และเงื่อนไขผ่าน

FR-5 เป็น FR เดียวที่ Requirement ถูกเปลี่ยนกลางทาง ตามที่ผู้ใช้ขอ

i Requirement FR-5 (ใน docs/TRACEABILITY.md)
"อธิบายข้อมูลที่ยืนยันแล้วให้เข้าใจง่าย (29 ก.ย.: เปลี่ยนจากโค้ช 3 โทนเป็นน้องกล้าที่พูดได้ ตามที่ผู้ใช้ขอ)"

เดิม vs ตอนนี้

ใน MVP วันที่ 27 ก.ย. 2569 โค้ชมี 3 บุคลิกให้ผู้ใช้เลือก หลังลองใช้ ผู้ใช้ขอเปลี่ยนเป็นน้องกล้าตัวเดียวที่พูดได้ ทีมจึงบันทึกการเปลี่ยนแปลงไว้ในหัวข้อ "การเปลี่ยนแปลงจาก M1 Charter (ต้องอัปเดต SRS)" ของ [docs/TRACEABILITY.md](#)

เรื่อง	MVP (27 ก.ย.)	ตอนนี้ (ตั้งแต่ 29 ก.ย.)
ตัวโค้ช	3 บุคลิก: เพื่อนซี (คุยสบาย ๆ) · โค้ชตรงประเด็น (สั้น ชัด) · พี่ใจดี (อบอุ่น อธิบายละเอียด)	น้องกล้าตัวเดียว น้ำเสียงเดียว สุขุม เป็นมิตร เรียกตัวเองว่า "กล้า"
รูปแบบ	ตัวหนังสือในห้องคุย	ตัวละครใหญ่บนเวที ขยับปาก ทำทาง พูดออกเสียง ชับไตเติลทีละท่อน
ความยาวคำตอบ	ไม่เกิน 110 คำ	ไม่เกิน 90 คำ เขียนเป็นภาษาพูด ไม่มีอีโมจิ ไม่มีรายการ ไม่มี markdown (เพราะถูกอ่านออกเสียง)
คำ "โทน" ในโปรไฟล์	เปลี่ยนคำตอบ	ยังมีคอลัมน์อยู่ แต่ไม่มีผลแล้ว และไม่มีตัวเลือกในหน้าจอ (TC-32 ตรวจสอบว่าโตน friend กับ senior ได้ผลเหมือนกันทุกตัวอักษร)
ข้อความใน SRS	"ด้วยโทนที่ผู้ใช้เลือก"	ต้องแก้เป็น "ด้วยน้องกล้า น้ำเสียงเดียว ทั้งตัวหนังสือและเสียง"

★ **บทเรียน Software Engineering: เปลี่ยน Requirement ต้องเปลี่ยนให้ครบทุกชั้น**

เวลา Requirement เปลี่ยน ต้องตามแก้ไขครบ "Golden Thread": ข้อความ Requirement → Acceptance criteria → โค้ด → เทสต์ → เอกสาร ถ้าแก้แค่โค้ด เอกสารจะโหลก ถ้าแก้แค่เอกสาร เทสต์จะยังตรวจของเก่า ทีมเราแก้ครบ: เพิ่มบันทึกการเปลี่ยนแปลง อัปเดตตาราง FR-5 แก้ prompt (coach v11) เอาตัวเลือกโตนออก และเขียน TC-32 ใหม่ให้ตรวจว่า "มีโค้ชเดียวแล้ว"

เงื่อนไขผ่าน (Acceptance criteria) → โค้ด → เทสต์

เงื่อนไข	โค้ด	เทสต์
โค้ชคือน้องกล้า น้ำเสียงเดียว สุขุม เป็นมิตร เรียกตัวเองว่า "กล้า" ไม่ตำหนิ · การตั้งค่าโทนเดิมไม่มีผล · ไม่มีตัวเลือก 3 โทน	insights.ts, functions/coach, settings.tsx	TC-32, TC-34, E2E coach + settings
หาหมวดที่สัปดาห์นี้สูงกว่าปกติ $\geq 25\%$ และ $\geq \text{฿}100$	findCategoryIncrease	TC-31
ไม่มีข้อมูล → ไม่วิจารณ์ แนะนำให้เริ่มจด	buildInsights	TC-32
น้องกล้าตัวใหญ่บนเวที: หายใจ โยกตัว กะพริบตา ตอนพูด ปากขยับ มือชี้ หัวพยัก หยดเมื่อไม่ได้เปิดหน้าโค้ช เคารพ "ลดการเคลื่อนไหว" แต่แล้วกระโดดและเล่าเคล็ดลับ ใส่ชุดที่เลือก และเปลี่ยนชุดบนเวทีได้	KlaStage.tsx, art.tsx, KlaBackdrop.tsx, coach.tsx	E2E coach
พูดภาษาไทยทีละก่อนพร้อมซับไตเติล ("฿1,290" อ่าน "1290 บาท", "%" อ่าน "เปอร์เซ็นต์") ปิดเสียงได้ ไม่มีเสียงไทยหรือแอปร่นเก่า → ซับไตเติลตามเวลาอ่าน ไม่ crash	klaTalk.ts, useKlaTalk.ts, tts.ts, tts.web.ts	TC-76, E2E coach (เสียงจำลอง)
แต่การ์ด "สิ่งที่ควรรู้ตอนนี้" หรือไอคอนลำโพงของคำตอบ → น้องกล้าเล่า · ข้อความผิดพลาดแสดงแต่ไม่อ่านออกเสียง	coach.tsx	E2E coach (key หาย, AI ไม่ว่าง, โหมดทดลอง)
AI ได้รับเฉพาะยอดรวม ห้ามแต่งตัวเลข ห้ามตำหนิ ตอบเป็นประโยคพูดสั้น ๆ ไม่มีสัญลักษณ์รายการ	buildCoachContext, functions/coach, helpers.ts plainText	helper tests, MT-08
หาราคาในคำถามได้ทุกรูปแบบ ไม่เอาตัวเลขอื่น ("อีก 30 วัน") มาเป็นราคา · ถามราคาเดิมซ้ำแล้วเติมคำถามให้อีก	price.ts, coach.tsx	TC-89

นอกจากนี้ยังมีเทสต์ของน้องกล้านบนหน้าหลัก (TC-49, TC-50, TC-51, TC-53, TC-58, TC-66) ตูสกินและการกิจ (TC-73 ถึง TC-75, TC-77) และตัวเลือก AI ฟังก์ชันฟเวอร์ (TC-38, TC-39, TC-40, TC-54, TC-55) รายละเอียดอยู่ในบท "การทดสอบ FR-5"

น้องกล้าคือใคร

ตัวละครเดียวที่ใช้เจอทั้งแอป ตั้งแต่หน้าต้อนรับจนถึงหน้าโค้ช

น้องกล้าคือ **ต้นกล้าเงิน** หัวเป็นฟุ่่มใบสีทอง มียอดอ่อนสีเขียวบนหัว ชื่อมาจาก "ต้นกล้า" และ "กล้า" (ความกล้า) ใบทองเชื่อมกับต้นไม้เงินบนหน้าหลัก ซึ่งออกใบมากน้อยตามจำนวนวันใน Money Runway (เต็มต้นที่ 45 วัน) ในโค้ดตั้งชื่อไว้ที่จุดเดียว:

src/domain/buddy.tsบรรทัด 15–22

```
15 export const BUDDY_NAME = 'กล้า';
16
17 export type BuddyMood = 'happy' | 'calm' | 'worried' | 'sleepy' | 'thinking' | 'cheer';
18
19 export interface BuddyLine {
20   mood: BuddyMood;
21   text: string;
22 }
```

[▶ เปิดใน VS Code \(เว็บ\)](#) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → src/domain/buddy.ts:15

บรรทัด	ความหมาย
15	BUDDY_NAME = 'กล้า' ทุกข้อความที่น้องกล้าพูดถึงตัวเองดึงชื่อจากค่าคงที่นี้ ถ้าวันหนึ่งทีมโหวตเปลี่ยนชื่อ แก่บรรทัดเดียว
17	อารมณ์ 6 แบบที่ตัวละครแสดงได้ (หน้าตา ตา ปาก แขน ต่างกัน)
19–22	"หนึ่งประโยคของน้องกล้า" = อารมณ์ + ข้อความ ฟังก์ชันที่ตัดสินใจว่าจะพูดอะไรจึงคืนค่าชนิดนี้เสมอ

อารมณ์ 6 แบบ ใช้เมื่อไร

อารมณ์	หน้าตา	ใช้ตอน (ตัวอย่างจากโค้ดจริง)
happy	ยิ้ม ตาโค้ง	พักภายในหน้าโค้ช · เล่าการ์ดที่ไม่ใช่การเตือน · ใช้เงินวันนี้แล้วแต่ยังอยู่ในแผน
cheer	ดีใจ ชูแขน	วันใหม่ · เงินพอใช้สบาย ๆ และวันนี้ยังไม่ได้ใช้เงิน · โดนแตะจนจึกจี้
calm	สงบ ตาเปิด	สถานะ "เริ่มต้องระวัง" · ยังไม่มีรายจ่ายใน 7 วัน · เล่าการ์ดเตือน (ไม่ทำหน้ากังวลใส่ผู้ใช้)
worried	กังวลเล็กน้อย	บนหน้าหลักเมื่อเงินใกล้หรือต่ำกว่าเส้นสำรอง · หลังข้อความผิดพลาด
thinking	ครุ่นคิด	มีสลิปรอตรวจ · ระหว่างรอคำตอบจาก AI
sleepy	ง่วง ตาปิด	ยังไม่มีรายการเลย

★ รายละเอียดที่ต้งใจ: ไม่ทำหน้ากังวลระหว่างพูด

ในหน้าโค้ช ถ้าอารมณ์ค้างไว้เป็น worried แล้วน้องกล้าเริ่มพูด โค้ชจะเปลี่ยนเป็น calm กันที (บรรทัด 176 ของ coach.tsx) เพราะการพูดเรื่องเงินด้วยหน้ากังวลทำให้ผู้ฟังเครียด เราอยากให้ผู้ใช้รู้สึกว่ "มีคนช่วยคิด" ไม่ใช่ "โดนตกใจใส่"

กฎน้ำเสียง (ใช้ทั้งข้อความที่เขียนเองและคำตอบของ AI)

① บอกข้อเท็จจริง

ตัวเลขจากข้อมูลที่ยืนยันแล้วเท่านั้น เช่น "7 วันนี้ใช้หมวดอาหาร ฿1,250"



② อธิบายว่าสำคัญยังไ้

"สูงกว่าปกติ 108%" หรือ "เงินจะแตะเส้นสำรองใน 9 วัน"



③ เสนอทางเลือกเล็ก ๆ 1-2 ข้อ

"ลองกด ลองปรับ ในหน้าเงินพอถึง" ไม่ใช่ "ต้องลดเดี๋ยวนี้"



④ ให้ผู้ใช้ตัดสินใจเอง

"ถ้าเป็นช่วงที่ต้องใช้จริงก็ไม่ใช่ไร" · "เลือกแบบที่สะดวกที่สุดได้เลย"

ไม่พูดแบบนี้	น้องกล้าพูดแบบนี้
"คุณใช้เงินเยอะเกินไป!"	"สัปดาห์นี้หมวดอาหารสูงกว่าปกติ... ถ้าเป็นช่วงที่ต้องใช้จริงก็ไม่ใช่ไร"
"เกินงบแล้ว เย่มาก"	"เดือนนี้ใช้เกินงบไปแล้ว ฿600 ไม่ต้องโทษตัวเองนะ ลองดูด้วยกันว่าหมวดไหนเกินเพราะอะไร..."
"ห้ามซื้อ"	"ถ้าจำเป็นจริงก็ซื้อได้ แต่หลังจากนั้นควรใช้เท่าที่จำเป็น กล้าให้ตัดสินใจเองเลย"
"ครับ/ค่ะ" (ทางการ ห้างเหิน)	ลงท้ายนุ่ม ๆ ด้วย "นะ" เหมือนเพื่อนที่สุขุม

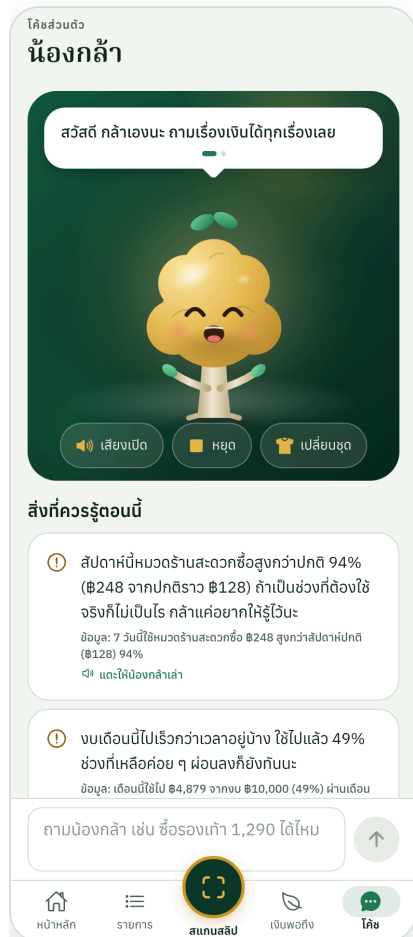
i แรงบันดาลใจ

คอมเมนต์บนสุดของ [src/domain/buddy.ts](#) บันทึกไว้ว่าไ้เดียตัวละครเดี่ยวหลายอารมณ์มาจากแอปเหมียวจด (ผู้ช่วยที่พูดกับผู้ใช้) และ Huggy ของ Hugging Face แต่ทีมออกแบบตัวละครของ MindPay เอง ไม่ได้ลอกแนวของเหมียวจด (บันทึกใน [docs/AI_USAGE_LOG.md](#) แถวที่ 10)

บทที่ 5

ทัวร์หน้าจอ: แท็บโค้ช

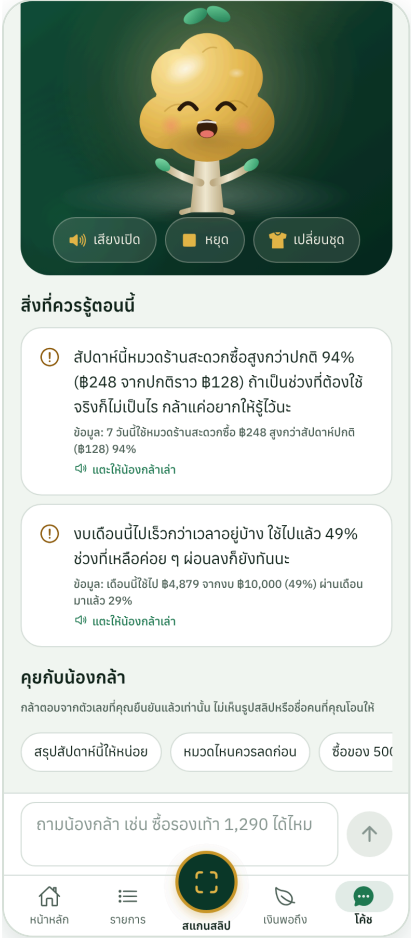
จากบนลงล่าง: เวท → สิ่งที่คุณรู้ตอนนี้ → คຸຍກັບນ້ອງຄລ້າ → คำถามสด → ช่องพิมพ์



เปิดแท็บโค้ชครั้งแรก (โหมดทดลอง)

- 1 หัวหน้าจอ "โค้ชส่วนตัว · น้องกล้า"
- 2 เวท พื้นหลังเขียวเข้มมีแสงด้านบน น้องกล้าตัวใหญ่ยืนอยู่กลางเวท หายใจโยกตัว กระพริบตาเบา ๆ ตลอดเวลาที่เปิดหน้านี้
- 3 ฟองคำพูด สีขาวมีหางชี้ลงหาน้องกล้า ตอนพูดจะแสดงซับไตเติ้ลทีละก่อน และมีจุดบอกว่าอยู่ก่อนที่เท่าไรจากทั้งหมด (ภาพนี้: ก่อนที่ 1 จาก 2 ของคำทักทาย)
- 4 ปุ่มบนเวท "เสียงเปิด/เสียงปิด" · "หยุด" (ขึ้นเฉพาะตอนกำลังพูด) หรือ "ฟังอีกครั้ง" (ขึ้นหลังพูดจบ) · "เปลี่ยนชุด"
- 5 คำทักทายพูดครั้งเดียวต่อการเปิดแอป 1 ครั้ง ไม่ใช่ทุกครั้งที่สลับแท็บกลับมา (ตัวแปร greeted uss গত 49 ของ coach.tsx)

การ์ด "สิ่งที่ควรรู้ออนนี้"



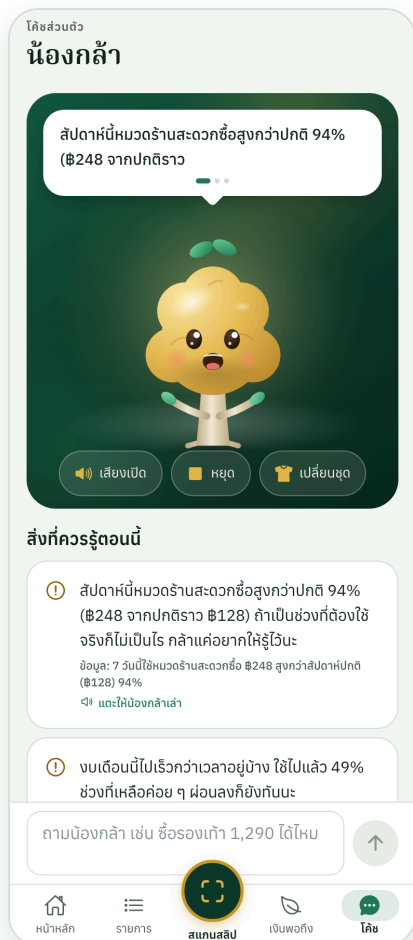
การ์ด 2 ใบของข้อมูลตัวอย่าง

ทุกการ์ดมาจาก **กฎในแอป** (ไม่ใช่ AI) แต่ละใบมี 4 ส่วน:

- 1 **ไอคอนบอกระดับ:** วงกลมตกใจสีส้ม = ควรรู้ (watch) · ตัว i = ข้อมูล (info) · เครื่องหมายถูกสีเขียว = ดีอยู่แล้ว (good)
- 2 **ข้อความในน้ำเสียงน้องกล้า** (message)
- 3 **"ข้อมูล: ..."** ข้อเท็จจริงพร้อมตัวเลขที่มา (fact) ตัวเล็กสีจาง ใครอยากตรวจสอบก็ดูได้
- 4 **"แนะนำให้น้องกล้าเล่า"** และทั้งการ์ด น้องกล้าจะพูดข้อความนั้นบนเวที และหน้าจอเลื่อนกลับขึ้นไปทีละที

ข้อมูลตัวอย่างในภาพมี 2 เรื่อง: หมวดร้านสะดวกซื้อสูงกว่าปกติ 94% และงบเดือนนี้ไปเร็วกว่าเวลา (ใช้ไป 49% ผ่านเดือนมา 29%)

แท่งการ์ด: น้องกล้าเล่าให้ฟัง



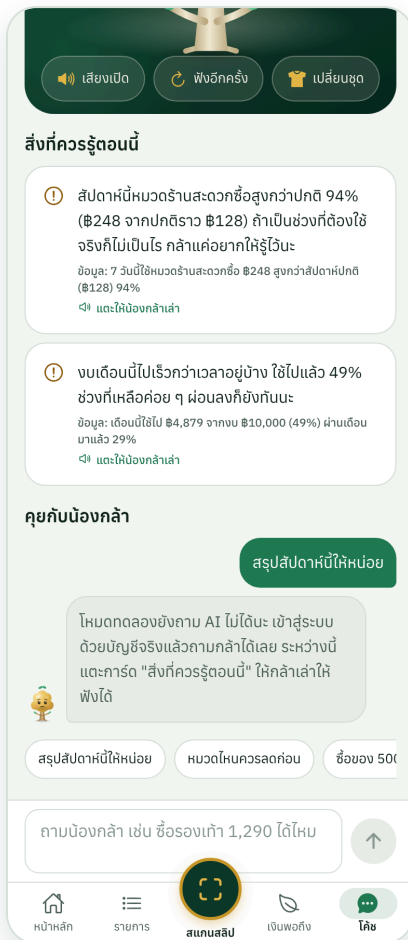
กำลังเล่าการ์ดแรก: ตอนที่ 1 จาก 3 ปุ่มหยุดขึ้นแถบฟังอีกครั้ง

ข้อความของการ์ดยาว น้องกล้าจึงพูดเป็น **3 ก่อน** ก่อนจะไม่เกินประมาณ 64 ตัวอักษร ระหว่างพูด:

- หน้าตาเปลี่ยนเป็น **calm** (ตาเปิด สงบ) เพราะการ์ดนี้เป็นการเตือน (severity = watch) ถ้าเป็นการ์ดข่าวดีจะเป็น happy
- ปากเปิดปิดตามพยางค์ มือขยับอธิบาย หัวพยักเบา ๆ (ภาพนิ่งเห็นปากเปิด)
- ปุ่ม "หยุด" ขึ้นมาให้กดตัดได้ทันที

พูดจบแล้ว ฟองคำพูดจะกลับไปเป็นประโยคชวนคุย "สงสัยอะไรเรื่องเงิน ถามกล้าได้เลยนะ" (คำคงที่ IDLE) เพราะข้อความยาวไม่เหมาะสมจะค้างไว้ในฟอง

คุยกับน้องกล้า (และสิ่งที่เกิดในโหมดทดลอง)



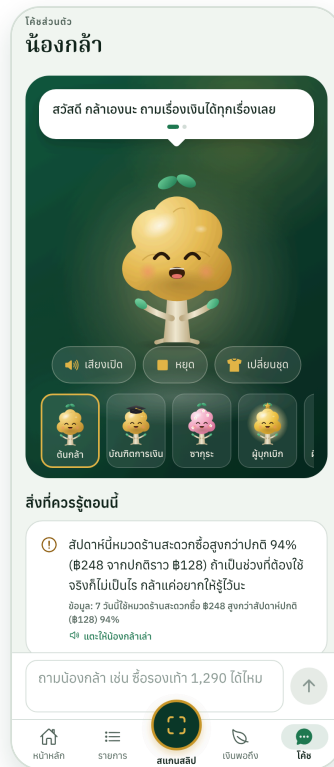
โหมดทดลอง: กดคำถามลัดแล้วได้คำอธิบายแทนคำตอบ

ใ้การ์ดเป็นห้องคุย ก่อนถามข้อแรกจะมีบรรทัดเล็ก ๆ บอกว่า "กล้าตอบจากตัวเลขที่คุณยืนยันแล้วเท่านั้น ไม่เห็นรูปสลิปหรือชื่อคนที่คุณโอนให้" ด้านล่างมี **คำถามลัด 4 ข้อ** (เลื่อนซ้ายขวาได้):

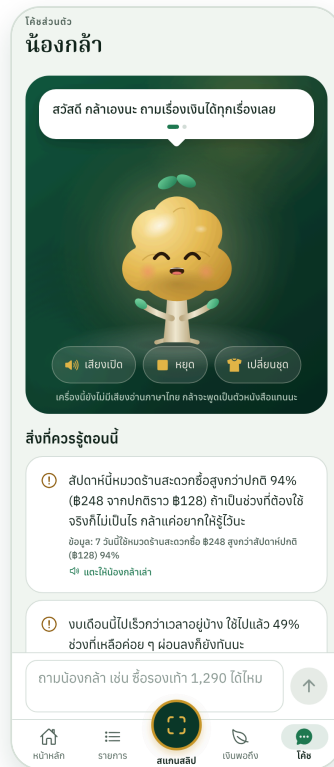
1. สรุปสัปดาห์นี้ให้หน่อย
2. หมวดไหนควรลดก่อน
3. ซื้อของ 500 บาทวันนี้ได้ไหม
4. ทำยังไงให้เงินพอถึงสิ้นเดือน

ในโหมดทดลอง ไม่มีบัญชีจริงให้เซิร์ฟเวอร์ตรวจตัวตน แอปจึง **ไม่ส่งคำถามไป AI เลย** แต่ตอบด้วยข้อความอธิบาย (ฟองสีเทา น้องกล้าหน้ากังวลเล็กน้อย) และแนะนำให้แตะการ์ดสิ่งที่ควรรู้ออนนี้ ข้อความผิดพลาดแบบนี้ **แสดงแต่ไม่อ่านออกเสียง** ตามเงื่อนไขผ่าน

เปลี่ยนชุดบนเวที และเมื่อเครื่องไม่มีเสียงไทย



กด "เปลี่ยนชุด": แลบทชุดที่มีอยู่ + ปุ่ม
ตู้สกินทั้งหมด



เครื่องไม่มีเสียงอ่านภาษาไทย: มี
บรรทัดบอกใต้ปุ่ม

- **เปลี่ยนชุด:** แลบทเลื่อนแสดง เฉพาะชุดที่ผู้ใช้มีแล้ว ชุดที่ใส่อยู่มีกรอบทอง และชุดไหนน้องกล้าเปลี่ยนทันทีแล้วถามว่า "ใส่ชุด...แล้ว เข้ากับกล้าไหม" ช่วงสุดท้าย "ตู้สกินทั้งหมด" พาไปดูชุดที่ยังไม่มีและวิธีปลดล็อก
- **ไม่มีเสียงไทย:** ถ้าเปิดเสียงไว้แต่เครื่องไม่มีเสียงอ่านภาษาไทย จะมีบรรทัด "เครื่องนี้ยังไม่เสียงอ่านภาษาไทย กล้าจะพูดเป็นตัวหนังสือแทนนะ" น้องกล้ายังขยับปาก และซบไต้เตลเปลี่ยนก่อนตามเวลาที่คนอ่านทัน

ทัวร์หน้าจอ: คู่มือกับน้องกล้าด้วยบัญชีจริง

บัญชีทดสอบ "มายด์" นักศึกษาที่ใช้แอปมา 4 สัปดาห์ (เซิร์ฟเวอร์จำลอง)

เพื่อให้เห็นการครบทุกชนิดและเห็นว่า AI ได้รับข้อมูลอะไร จินสร้างบัญชีทดสอบชื่อ "มายด์" แล้วใส่รายการย้อนหลัง 4 สัปดาห์ "วันนี้" ของภาพคือ **วันศุกร์ที่ 9 ต.ค. 2569**

ข้อมูลของมายด์	รายละเอียด
ตั้งค่า	ยอดเงินตอนเริ่ม ฿6,000 · เงินสำรอง ฿500 · งบรายเดือน ฿6,000
รายรับ	ค่าขนมจากแม่ ฿4,000 (1 ต.ค.)
อาหาร 3 สัปดาห์ก่อน (8-27 วันก่อน)	6 ครั้ง ครั้งละ ฿300 = ฿1,800 (เฉลี่ยสัปดาห์ละ ฿600)
อาหาร 7 วันนี้ (3-9 ต.ค.)	ทุกวัน: 200, 180, 250, 150, 220, 160, 90 = ฿1,250
อื่น ๆ	รถเมล์ ฿40 × 12 ครั้ง · เซเว่น 5 ครั้ง (65, 45, 80, 55, 70) · เสื้อยืด ฿890 · ค่าหอ ฿2,500 (1 ต.ค.) · ดูหนัง ฿199 (2 ต.ค.)
ผลที่แอปคำนวณ	ยอดคงเหลือ ฿2,566 · เฉลี่ย 7 วัน ฿211/วัน · เงินพอใช้อีก 9 วัน ถึง 18 ต.ค. (เริ่มต้องระวัง) · ถ้าอยากพอกถึงสิ้นเดือนใช้ได้อีกจะ ฿89

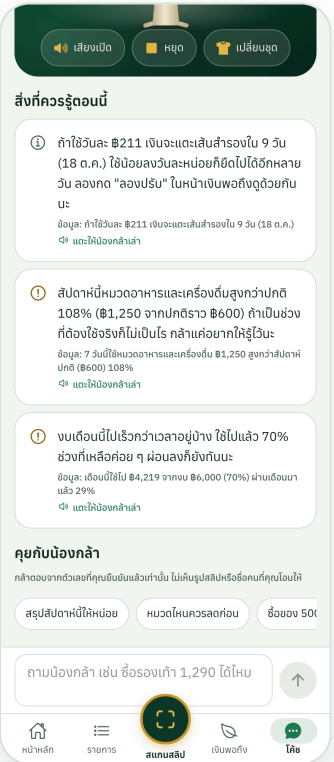
หน้าหลักและการ์ดสิ่งที่ควรรู้



หน้าหลักของมายด์: น้องกล้าบอกว่
วันนี้ใช้ได้ราว ฿89



การ์ดครบ 3 ชนิด: Runway · หมวด
ที่สูงขึ้น · งบ

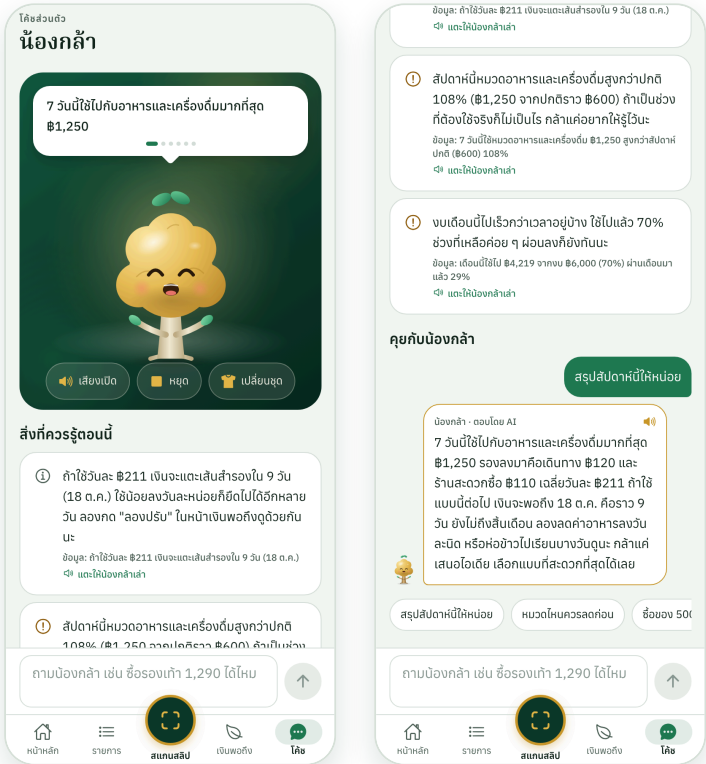


เลื่อนลง: หวังคอยังว่าง และคำถา
ม

การ์ด (เรียงตาม ที่แอปแสดง)	ข้อความ (ของจริง)	ข้อมูลที่มา
1 · Runway (ไอคอน i)	ถ้าใช้วันละ ฿211 เงินจะแตะเส้นสำรองใน 9 วัน (18 ต.ค.) ใช้น้อยลง วันละหน่อยก็ยืดไปได้อีกหลายวัน ลองกด "ลองปรับ" ในหน้าเงินพอ ถึงดูด้วยกันนะ	ถ้าใช้วันละ ฿211 เงินจะแตะเส้นสำรอง ใน 9 วัน (18 ต.ค.)
2 · หมวดที่สูงขึ้น (ไอคอน !)	สัปดาห์นี้หมวดอาหารและเครื่องดื่มสูงกว่าปกติ 108% (฿1,250 จากปกติราว ฿600) ถ้าเป็นช่วงที่ต้องใช้จริงก็ไม่ใช่ไร คล้ายเค้ายาก ให้รู้ไว้ละ	7 วันนี้ใช้หมวดอาหารและเครื่องดื่ม ฿1,250 สูงกว่าสัปดาห์ปกติ (฿600) 108%
3 · จบ (ไอคอน !)	จบเดือนนี้ไปเร็วกว่าเวลาอยู่บ้าง ใช้ไปแล้ว 70% ช่วงที่เหลือค่อย ๆ ผ่อนลงก็ยังทันนะ	เดือนนี้ใช้ไป ฿4,219 จากงบ ฿6,000 (70%) ผ่านเดือนมาแล้ว 29%

ทำไมการ์ด Runway เป็นไอคอน i ไม่ใช่ ? เพราะสถานะ "เริ่มต้องระวัง" (watch) ให้ระดับ info ส่วน "ใกล้เส้นสำรอง" (critical) กับ "ต่ำกว่าเส้นสำรอง" จึงให้ระดับ watch (ยก "หลักการขั้นที่ 1")

ถามคำถามลัด: AI ตอบ นื่องกล้าพูด



ระหว่างพูดคำตอบ: ก่อนที่ 1 จาก 6 ในห้องคุย: คำถาม (ขวา) และคำตอบ (ซ้าย) กรอบทอง = คำสั่งพูด

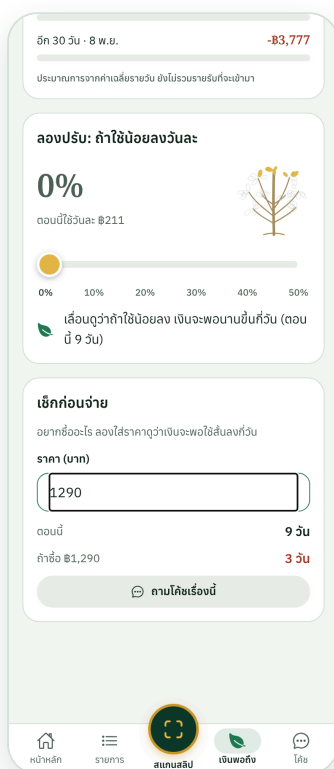
- 1 และ "สรุปสัปดาห์นี้ให้หน่อย" คำถามขึ้นเป็นฟองสีเขียวทางขวา ระหว่างรอ น้องกล้าทำหน้าที่คิดและฟองคำพูดขึ้นจุด
พิมพ์กับ "กล้ากำลังดูตัวเลขของคุณ..."
- 2 แอปส่งคำถามพร้อม **ข้อมูลสรุป** (พิมพ์ไว้ทั้งก่อนในบท "หลักการขั้นที่ 2") ไปที่เซิร์ฟเวอร์
- 3 คำตอบกลับมา: ขึ้นเป็นฟองสีขาวทางซ้าย หัวฟองเขียน "น้องกล้า · ตอบโดย AI" (บอกตรง ๆ ว่าใครตอบ) พร้อมไอคอน
ลำโพงให้ฟังซ้ำ

- 4 หน้าจอเลื่อนกลับขึ้นไปทีละที น้องกล้วยตอบทีละก่อน (คำตอบนี้ถูกตัดเป็น 6 ก่อน) ฟองคำตอบในห้องคุยมีกรอบทอระหว่างที่กำลังพูดข้อความนั้น

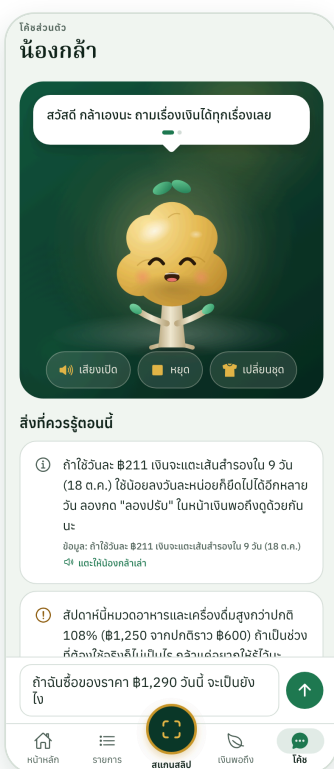
! คำตอบในภาพเป็นคำตอบจำลอง

ข้อความ "7 วันนี้ใช้ไปกับอาหารและเครื่องดื่มมากที่สุด ฿1,250 รองลงมาคือเดินทาง ฿120 ..." ฉันเขียนเองตามกติกาของ prompt (ใช้เฉพาะตัวเลขในข้อมูลที่ส่งไป ไม่เกิน 90 คำ ไม่มีรายการ ลงท้ายให้ผู้ใช้ตัดสินใจ) แล้วตั้งให้เชอร์ฟเวอร์จำลองตอบกลับ เพื่อให้เห็นหน้าตาและการตัดซับไตเติลจริง คำตอบของ AI จริงจะใช้คำต่างไป แต่ต้องอยู่ในกติกาเดียวกัน

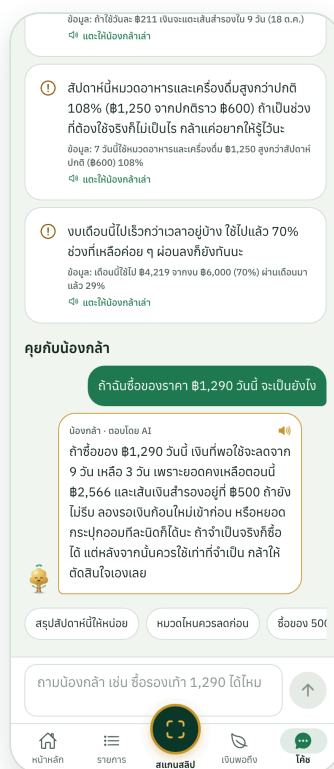
เช็กก่อนจ่าย → ถามโค้ชเรื่องนี้ (FR-6 ส่งต่อ FR-5)



เก็บเงินพอถึง: ใส่ราคา 1,290 ตอน
นี้ 9 วัน ถ้าซื้อเหลือ 3 วัน



กดถามโค้ช: คำถามพร้อมราคาไปรอ
ในช่องพิมพ์

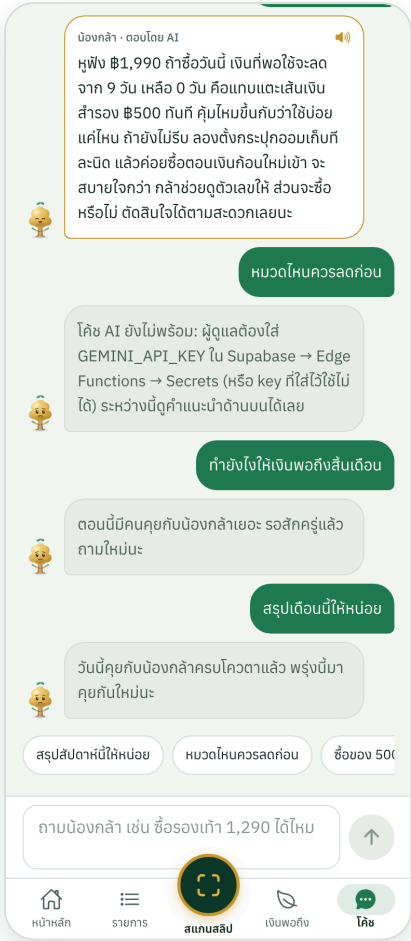


ส่งแล้ว: คำตอบบอก Runway ก่อน
(9) และหลัง (3)

ปุ่ม "ถามโค้ชเรื่องนี้" ส่ง 3 อย่างไปที่หน้าโค้ช: ข้อความคำถาม "ถ้าฉันซื้อของราคา ฿1,290 วันนี้ จะเป็นยังไง" ราคาเป็นสตางค์ (129000) และเวลาที่กด แอป **เติมคำถามรอไว้ให้ ไม่ส่งเอง** ผู้ใช้ได้อ่านหรือแก้ก่อนกดส่ง เมื่อส่ง แอปคำนวณ "Runway ก่อนซื้อ 9 วัน หลังซื้อ 3 วัน" ด้วยฟังก์ชันของ FR-6 แล้วแนบไปในชื่อ purchaseCheck AI จึงพูดตัวเลขที่ถูกตัดได้โดยไม่ต้องคิดเอง

ถ้าพิมพ์ถามเองว่า "หุฟ่ง 1,990 คุ่มโหม" แอปก็ **หาราคาในประโยคเอง** (฿1,990) แล้วแนบ purchaseCheck "ก่อน 9 วัน หลัง 0 วัน" ไปเหมือนกัน (กติกาการหาราคาอยู่ในบท "เช็กก่อนจ่ายผ่านโค้ช")

เมื่อมีปัญหา: ข้อความที่ผู้ใช้เห็น



3 ปัญหาต่อกัน: ไม่มีกุญแจ AI · AI คิวเต็ม · ครบโควตาวันนี้

ปัญหา	ข้อความในแอป
ผู้ดูแลยังไม่ใส่กุญแจ AI (หรือกุญแจใช้ไม่ได้)	โค้ด AI ยังไม่พร้อม: ผู้ดูแลต้องใส่ GEMINI_API_KEY ใน Supabase → Edge Functions → Secrets (หรือ key ที่ใส่ไว้ใช้ไม่ได้) ระหว่างนี้ดูคำแนะนำด้านบนได้เลย
AI ไม่ว่าง (ติดลิมิตหรือโอหลดเกิน)	ตอนนี้มีคนคุยกับน้องกล่าเยอะ รอสักครู่แล้วถามใหม่นะ
ถามครบ 60 ครั้งภายใน 24 ชั่วโมงที่ผ่านมา	วันนี้คุยกับน้องกล่าครบโควตาแล้ว พรุ่งนี้มาคุยกันใหม่นะ
เซสชันหมดอายุ	เซสชันหมดอายุ กรุณาเข้าสู่ระบบอีกครั้ง
อื่น ๆ (เบีตหลุด ฯลฯ)	ติดต่อน้องกล่าไม่ได้ตอนนี้ ลองใหม่อีกครั้งนะ

ทุกข้อความเป็นฟองสีเทา น้องกล่าหน้ากังวลเล็กน้อย และ **ไม่อ่านออกเสียง** ส่วนอื่นของหน้า (การ์ดสิ่งที่ควรรู้) ยังใช้ได้ตามปกติ

น้องกล้าที่อื่นในแอป

หน้าหลัก ตั้งค่า ตูสกิน และความสำเร็จ ใช้ตัวละครและกติกาน้ำเสียงเดียวกัน

หน้าหลัก: น้องกล้าตัวเล็กกับประโยคเดียว



โหมดทดลอง: "วันนี้ใช้ไป ฿34 ยังอยู่ในแผน เงินพอใช้อีก 16 วัน"

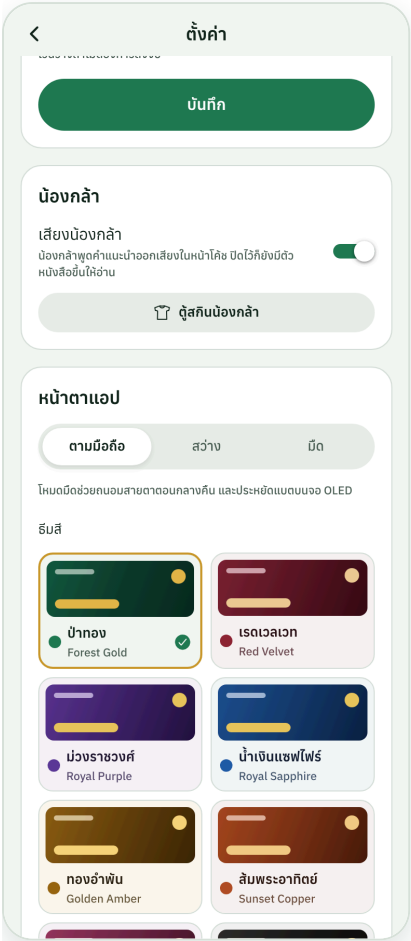
แต่น้องกล้า → กระโดดแล้วเล่าเคล็ดลับ 8 ขววนกันไป (ไม่ซ้ำติดกัน) ถ้าเงินตึงจะให้กำลังใจก่อน และถ้าแตะรัว ๆ ครบทุก 6 ครั้งจะหวัเราะ "จึกจึแล้วน่า~" · **แตะต้นไม้เงินบนการ์ดเขียว** → อธิบายว่าใบทองคือจำนวนวันที่เงินพอใช้ ประโยคที่พูดหลังแตะจะอยู่ 6 วินาทีแล้วกลับเป็นประโยคประจำ (รายละเอียดทุกประโยคในบท "อ่านโค้ด: buddy.ts")

ใต้คำทักทาย "สวัสดี ..." มีน้องกล้าตัวเล็กกับฟองคำพูด 1 ประโยค ฟังก์ชัน buddyLine เลือกประโยคจากสถานการณ์ **เรื่องด่วน** ก่อนเสมอ:

ลำดับ	ถ้า...	น้องกล้าพูด (ตัวอย่างจริง)
1	ยังไม่มีรายการ	ยังไม่มีรายการให้กล้าดูเลย ลองสแกนสลิปหรือจกรายการแรกกันนะ
2	มีสลิปปรอตรว	มีสลิป 2 ใบที่กล้าอ่านไม่ชัด ช่วยดูให้หน่อยนะ
3	เปิดแอปครั้งแรกของวันใหม่	วันใหม่แล้ว! เมื่อวานใช้ไป ฿210 วันนี้ใช้ได้ราว ฿89
4	ตามสถานะ Runway	เช่น เริ่มต้องระวัง: "วันนี้ใช้ได้ราว ฿89 เงินจะพอถึงสิ้นเดือนพอดี"

ข้อ 3 จะข้ามไปถ้าสถานะเป็นใกล้หรือต่ำกว่าเส้นสำรอง เพราะเรื่องเงินด่วนกว่าการทักทาย

ตั้งค่า: เสียงน้องกล้า

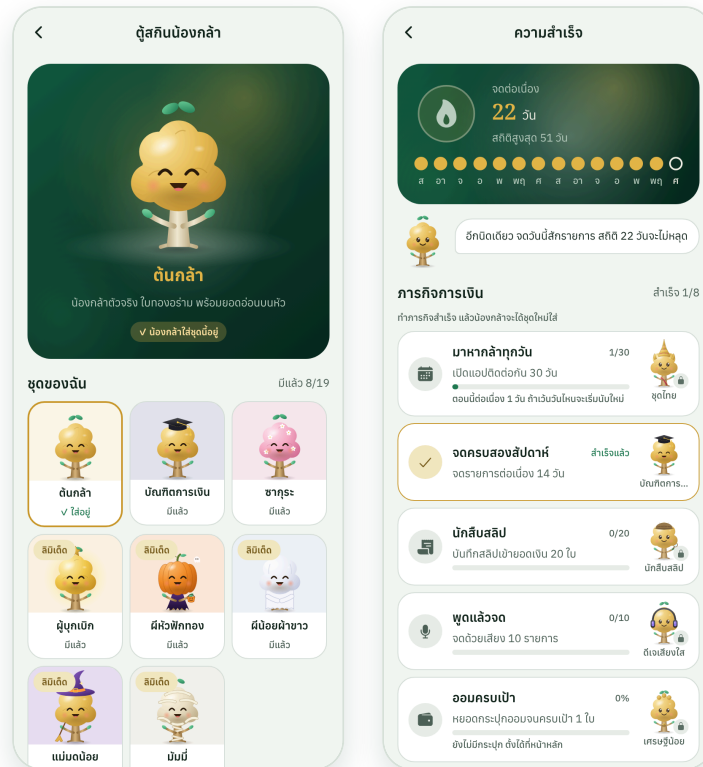


ตั้งค่า → การ์ด "น้องกล้า"

การ์ด "น้องกล้า" ในหน้าตั้งค่ามีสวิตช์ **"เสียงน้องกล้า"** คำอธิบายใต้สวิตช์: "น้องกล้าพูดคำแนะนำออกเสียงในหน้าโค้ช ปิดไว้ก็ยังมีตัวหนังสือขึ้นให้อ่าน" กับปุ่ม "ดูสกินน้องกล้า"

สวิตช์นี้กับปุ่ม "เสียงเปิด/เสียงปิด" บนเวทก็เป็น **คำเดียวกัน** (เก็บในเครื่องด้วยชื่อ `mindpay.klaVoice`) เปลี่ยนที่ไหนอีกทีก็เปลี่ยนตามกันที่ ปิดแล้วน้องกล้าเจียบบันท์แม่กำลังพูดอยู่

ตู้สกินและการกิจการเงิน



ตู้สกิน: ชุดที่ใส่อยู่ด้านบน และ "ชุดของฉัน" มีแล้ว 8/19

หน้าความสำเร็จ: การกิจการเงินแต่ละข้อปลดล็อกชุด

น้องกล้ามี 19 ชุด แบ่ง 3 แบบ: ชุดเริ่มต้น "ต้นกล้า" (ทุกคนมี) · ชุดภารกิจ 8 ชุด ได้เมื่อทำการกิจการเงินสำเร็จ เช่น เปิดแอปติดต่อกัน 30 วัน → ชุดไทย · สกินลิมิเต็ด 10 ชุด แจกเฉพาะช่วงเวลา (รวมชุดฮาโลวีน 6 ชุด: ผีหัวฟักทองแจกฟรี อีก 5 ชุดแลกด้วยลูกอม) ชุดที่เลือกใส่จะแสดง **ทุกที่** ที่มีน้องกล้า ทั้งหน้าหลัก เวทียุคซ์ และฟองคำตอบ รายละเอียดอยู่ในบท "ตู้สกินและการกิจการเงิน"

★ ทำไม FR-5 ต้องมีสกินและการกิจการ

โค้ชจะช่วยได้ก็ต่อเมื่อผู้ใช้ "กลับมาจด" ทุกวัน เพราะคำแนะนำที่ดีเท่าข้อมูลที่ครบ ภารกิจจึงผูกกับพฤติกรรมการเงินที่จริงจัง (จดต่อเนื่อง บันทึกสลิป ออมครบเป้า คุมงบได้ทั้งเดือน ใช้น้อยกว่าสัปดาห์ก่อน) ไม่ใช่การกดเล่น ๆ และรางวัลเป็นของน่ารักที่ไม่เกี่ยวกับเงินจริง

หลักการขั้นที่ 1: สิ่งที่เราควรรู้ออนนี้ (กฎ ไม่ใช่ AI)

เรื่องสำคัญที่สุด ฉันให้กฎในแอปเป็นค้นหา เพราะกฎตรวจได้ ทดสอบได้ และไม่ต้องใช้เน็ต

การ์ด "สิ่งที่ควรรู้ออนนี้" สร้างโดยฟังก์ชัน buildInsights ใน `src/domain/insights.ts` เป็นกฎล้วน ๆ ไม่เรียก AI เลย ฉันตั้งใจแยกแบบนี้เพราะ:

ข้อดีของกฎ	ถ้าให้ AI หาเอง
ทุกประโยคมาพร้อมตัวเลขที่มา ("ข้อมูล: ...") ตรวจสอบได้	อาจได้ข้อสรุปที่ฟังดูดีแต่ไม่มีตัวเลขรองรับ
ใส่ข้อมูลเดิมได้ผลเดิมทุกครั้ง จึงเขียนเทสต์ได้ (TC-31, TC-32, TC-34)	ถามซ้ำได้คำตอบไม่เหมือนเดิม ทดสอบยาก
ทำงานทันที ไม่ต้องรอเน็ต ไม่เสียโควตา ไม่มีค่าใช้จ่าย	ต้องรอเน็ตและรอ AI ทุกครั้งที่เปิดหน้า และเสียค่า AI ทุกครั้ง
ใช้ได้ในโหมดทดลอง และตอนเซิร์ฟเวอร์ AI มีปัญหา	ถ้า AI ล่ม หน้าโศกสว่างเปล่า

ลำดับการคิดของ buildInsights

มีรายการที่ยืนยันแล้วไหม?

ไม่มี → การ์ด no_data ในเดียว แล้วจบ (ไม่พิจารณาคนที่ยังไม่มีข้อมูล)

↓ มี

① Runway ก่อนเสมอ

below_floor หรือ critical หรือ watch → การ์ด runway (healthy / no_spending ไม่มีการ์ด)

↓

② หมวดที่สูงกว่าปกติ

มีหมวดที่ผ่านเกณฑ์ → การ์ด category_up 1 ใน (หมวดที่ขึ้นมากที่สุด)

↓

③ จังหวะการใช้บ

ผู้ใช้ตั้งงบรายเดือนไว้ → การ์ด budget_pace

↓

ไม่มีการ์ดเลย?

→ การ์ด steady "สัปดาห์นี้ใช้เงินสม่ำเสมอ..."

คอมเมนต์ในโค้ดบอกเหตุผลของลำดับไว้ว่า "Runway first: it is the most important number in the app" Runway เป็นตัวเลขที่สำคัญที่สุดของแอป จึงขึ้นก่อนเรื่องหมวดหรืองบ ผลคือมีการ์ดได้ 1 ถึง 3 ใน

กฎ ① การ์ด Runway

สถานะ Runway (FR-6)	การ์ด?	ระดับ	ข้อความ (โครงสร้าง)
below_floor ต่ำกว่าเงินสำรอง	มี	watch	ตอนนี้เงินต่ำกว่าเงินสำรองที่ตั้งไว้ (฿500.00) แล้วนะ ไม่เป็นไร ช่วงนี้ใช้เฉพาะที่จำเป็นก่อน แล้วลองดูว่ามีรายจ่ายไหนเลื่อนออกไปได้บ้าง
critical ใกล้เส้นสำรอง (0-6 วัน)	มี	watch	ถ้าใช้วันละ ฿... เงินจะแตะเส้นสำรองใน ... วัน (วันที่) ใช้น้อยลงวันละหน่อยก็ยืดไปได้อีกหลายวัน ลองกด "ลองปรับ" ในหน้าเงินพอถึงดูด้วยกันนะ
watch เริ่มต้องระวัง (7-13 วัน)	มี	info	
healthy สบาย ๆ (14 วันขึ้นไป)	ไม่มี	-	ไม่ต้องเตือน ไม่รบกวนผู้ใช้
no_spending ยังไม่มีรายจ่ายใน 7 วัน	ไม่มี	-	ยังคำนวณไม่ได้ ไม่มีอะไรให้เตือน

★ **การ์ดชวนไปใช้เครื่องมือของ FR-6**

ข้อความการ์ด Runway จบด้วย "ลองกด ลองปรับ ในหน้าเงินพอถึง" เพราะคำแนะนำที่ดีที่สุดคือให้ผู้ใช้ลองเลื่อนดูเองว่าใช้น้อยลงก็เปอร์เซ็นต์แล้วจะพอนานขึ้นกี่วัน (FR-6) เป็นการ "เสนอทางเลือก" ตามกฎน้ำเสียง ไม่ใช่สั่ง

กฎ ② หมวดที่สูงกว่าปกติ (findCategoryIncrease)

คำถามคือ "สัปดาห์นี้มีหมวดไหนใช้มากผิดปกติไหม" ปัญหาคือ "ปกติ" คือเท่าไร จันใช้ **3 สัปดาห์ก่อนหน้า** เป็นตัวแทนของปกติ:

สัปดาห์นี้ = วันนี้ + 6 วันก่อน (7 วัน) · ปกติต่อสัปดาห์ = รวม 21 วันก่อนหน้านั้น ÷ 3
เพิ่มขึ้น (%) = (สัปดาห์นี้ - ปกติ) ÷ ปกติ × 100 ปัดเป็นจำนวนเต็ม · นับเฉพาะรายจ่ายที่ยืนยันแล้ว แยกทีละหมวด

ช่วง	นับย้อนจากวันนี้	ตัวอย่าง (วันนี้ 9 ต.ค.)
สัปดาห์นี้	0 ถึง 6 วันก่อน	3-9 ต.ค.
ฐาน 3 สัปดาห์	7 ถึง 27 วันก่อน (21 วัน)	12 ก.ย. - 2 ต.ค.

หมวดหนึ่งจะได้การ์ด ต้องผ่าน **คสบ 3 ข้อ** และถ้าผ่านหลายหมวด เลือกหมวดที่ % สูงสุดเพียงหมวดเดียว:

- ปกติต้องมากกว่า 0** หมวดที่ไม่เคยใช้ใน 3 สัปดาห์ก่อน คิด % ไม่ได้ (หารด้วยศูนย์) จึงข้าม
- เพิ่มขึ้นอย่างน้อย ฿100** ก้นการเตือนเรื่องเล็ก เช่น ปกติ ฿20 สัปดาห์นี้ ฿40 คือเพิ่ม 100% แต่เป็นเงินแค่ ฿20 ไม่คุ้มที่จะรบกวนผู้ใช้
- เพิ่มขึ้นอย่างน้อย 25%** ขึ้นนิดหน่อยถือเป็นความแฉ่งปกติ

ตัวอย่างคิดมือ: บัญชีรายด (ตรวจสอบด้วยฟังก์ชันจริง)

หมวด	สัปดาห์นี้	ฐาน 21 วัน	ปกติ/สัปดาห์	ต่าง	%	ผล
อาหารและเครื่องดื่ม	฿1,250	฿1,800	฿600	+฿650	108	ได้การ์ด (ผ่านทั้ง 3 ข้อ)
ร้านสะดวกซื้อ	฿110	฿205	฿68.33	+฿41.67	61	ไม่ได้ (เพิ่มไม่ถึง ฿100)
เดินทาง	฿120	฿360	฿120	฿0	0	ไม่ได้ (ไม่ได้เพิ่ม)
บิลและค่าที่พัก	฿0	฿2,500	฿833.33	-฿833.33	-100	ไม่ได้ (ลดลง)
ข้อปึง	฿0	฿890	฿296.67	-฿296.67	-100	ไม่ได้ (ลดลง)
บันเทิง	฿0	฿199	฿66.33	-฿66.33	-100	ไม่ได้ (ลดลง)

วิธีคิดแถวแรก: ปกติ = 1,800 ÷ 3 = ฿600 · ต่าง = 1,250 – 600 = ฿650 (≥ ฿100 ✓) · % = 650 ÷ 600 × 100 = 108.3 → 108 (≥ 25 ✓) แถวร้านสะดวกซื้อน่าสนใจ: ขึ้นตั้ง 61% แต่เป็นเงินแค่ ฿41.67 กฎข้อ 2 จึงไม่ให้เตือน น้องกล้าจะไม่จู้เรื่องค่าขนมไม่ก็สับบาท

i ในโค้ดคิดเป็นสตางค์

ปกติ/สัปดาห์ของร้านสะดวกซื้อจริง ๆ คือ `Math.round(20500 ÷ 3) = 6833` สตางค์ (฿68.33) และเกณฑ์ ฿100 ในโค้ดเขียนเป็น `10_000` สตางค์ ตารางด้านบนแปลงเป็นบาทให้อ่านง่าย

ตัวอย่างจากเทสต์ TC-31: 3 สัปดาห์ก่อนใช้หมวดอาหารรวม ฿900 (ปกติ ฿300/สัปดาห์) สัปดาห์นี้ ฿600 → เพิ่ม ฿300 = 100% → ได้ { key: 'food', week: 60000, usualWeek: 30000, changePct: 100 }

กฎ ③ จังหวะการใช้บ (budget_pace)

ใช้เมื่อผู้ใช้ตั้งงบรายเดือนในหน้าตั้งค่า คำถามคือ "ตอนนี้ใช้งบเร็วเกินเวลาไหม" ถ้าผ่านเดือนมา 29% ก็ควรใช้งบไปราว 29%

ควรใช้ไปแล้ว (expected) = งบ × วันที่ของวันนี้ ÷ จำนวนวันในเดือน

เกินงบ (over) = ใช้ไป > งบ · เร็วกว่าเวลา (ahead) = ใช้ไป > expected × 1.1 (เพื่อ 10% กันการเตือนจุกจิก) · ใช้ไป = รายจ่ายที่ยืนยันแล้วในเดือนนี้ (เวลาไทย)

ตัวอย่าง (วันนี้ 9 ต.ค. งบ ฿6,000)	ใช้ไป	ผล	ข้อความ (ของจริง)
<code>expected = 6,000 × 9 ÷ 31 = ฿1,741.94 · ×1.1 = ฿1,916.13</code>	–	–	–
อยู่ในแผน	฿1,300	good	งบเดือนนี้ยังอยู่ในแผนที่มาก ใช้ไป 22% รักษาจังหวะนี้ไว้นะ กล้าภูมิใจด้วย
เร็วกว่าเวลา	฿2,100	watch	งบเดือนนี้ไปเร็วกว่าเวลาอยู่บ้าง ใช้ไปแล้ว 35% ช่วงที่เหลือค่อย ๆ ผ่อนลงก็ยังทันนะ

ตัวอย่าง (วันนี้ 9 ต.ค. จบ ฿6,000)	ใช้ไป	ผล	ข้อความ (ของจริง)
มายด์	฿4,219	watch	จบเดือนนี้ไปเร็วกว่าเวลาอยู่บ้าง ใช้ไปแล้ว 70% ช่วงที่เหลือค่อย ๆ ผ่อนลงก็ยังทันนะ
เกินงบ	฿6,600	watch	เดือนนี้ใช้เกินงบไปแล้ว ฿600 ไม่ต้องโทษตัวเองนะ ลองดูด้วยกันว่าหมวดไหนเกินเพราะอะไร แล้วค่อยตั้งงบเดือนหน้าให้ตรงกับชีวิตจริงขึ้น

ข้อมูลใต้การ์ดบอกทั้ง 2 เพอร์เซ็นต์ให้เทียบเอง เช่น "เดือนนี้ใช้ไป ฿4,219 จากงบ ฿6,000 (70%) ผ่านเดือนมาแล้ว 29%"
($9 \div 31 = 29.03\% \rightarrow 29$)

! TC-34: พุดเรื่องเกินงบให้ชัด แต่ไม่ดู

ใน MVP วันแรก ถ้าใช้เกินงบไปแล้ว โค้ชยังพุดแค่ว่า "จบเดือนนี้ไปเร็วกว่าเวลานิดนึง" ซึ่งทำให้ผู้ใช้เข้าใจผิด ทีมเจอตอนลองใช้เว็บจริงแล้วแก้ในวันเดียวกัน (commit 88313d6) และเพิ่ม TC-34 ตรวจสอบว่าข้อความต้องมีคำว่า "เกินงบ" กับจำนวนเงินที่เกินจริง (฿600) และต้อง **ไม่มี** คำว่า "นิดนึง" หรือ "นิดหน่อย" ความชัดเจนกับความใจดีไปด้วยกันได้

กฎ ④ ไม่มีข้อมูล และ ⑤ ทุกอย่างปกติ

ชนิด	เมื่อไร	ระดับ	ข้อความ
no_data	ยังไม่มีรายการที่ยืนยันแล้วเลย	info	ยังไม่มีรายการให้กล่าวดูเลย ลองสแกนสลิปหรือจดรายการแรกก่อนนะ ไม่ต้องครบทุกอย่างก็ได้ แล้วถ้าจะช่วยให้
steady	ไม่มีการ์ดอื่นเลย	good	สัปดาห์นี้ใช้เงินสม่่าเสมอดี ไม่มีหมวดไหนน่าห่วงเลย รักษาไว้แบบนี้ละ

no_data สำคัญกว่าที่คิด ผู้ใช้ใหม่ที่เพิ่งสมัครควรได้ยินคำชวนเริ่ม ไม่ใช่คำวิจารณ์ และประโยค "ไม่ต้องครบทุกอย่างก็ได้" ลดความกดดันของคนทีกลัวว่าต้องจดให้ครบตั้งแต่วันแรก

ระดับของการ์ด → ไอคอน และหน้าตาตอนเล่า

severity	ไอคอน	สี	อารมณ์ตอนน้องกล่าเล่า
watch	วงกลมเครื่องหมายตกใจ	ส้ม (theme.watch)	calm สงบ
info	วงกลมตัว i	เทาเข้ม (theme.inkSoft)	happy
good	วงกลมเครื่องหมายถูก	เขียว (theme.good)	happy

≈ fact กับ message ต่างกันยังไง

ทุกการ์ดมี 2 ข้อความคู่กัน เหมือนหมอบอกผลตรวจเลือดเป็นตัวเลข (fact: "น้ำตาล 130") แล้วอธิบายเป็นภาษาคน (message: "สูงกว่าปกตินิดหน่อย ลองลดหวานดูนะ") แอปแสดงทั้งสองอย่าง: message ตัวใหญ่ให้อ่านง่าย fact ตัวเล็กให้คนที่อยากตรวจ และ fact นี้แหละที่ทำให้ไม่มีอะไรเป็นกล่องดำ

หลักการขั้นที่ 2: ถาม AI ยังไงให้ถูกต้องและปลอดภัย

AI เก่งเรื่องภาษา แต่พลาดเรื่องตัวเลขได้ ฉะนั้นให้ AI "อธิบาย" ข้อเท็จจริงที่แอปเตรียมไว้ ไม่ใช่ให้ AI "หา" ข้อเท็จจริงเอง

เส้นทางของคำถาม 1 ข้อ

① ผู้ใช้ถาม

กดคำถามลัด หรือพิมพ์เองแล้วกดลูกศร (ส่งไม่ได้ถ้าช่องว่าง หรือกำลังรอคำตอบข้อก่อน)

↓ ในแอป (coach.tsx ฟังก์ชัน send)

② แสดงคำถาม + เข้าโหมดรอ

พองสีเขียวทางขวา · นื่องคล้ายหน้าครุ่นคิด · "คล้ายกำลังดูตัวเลขของคุณ..." · เลื่อนกลับไปทีละที

↓

③ เตรียมข้อมูลสรุป

buildCoachContext (ยอกรวมเท่านั้น) + ถ้าคำถามมีราคา → purchaseCheck (Runway ก่อน/หลังซื้อ คิดด้วยสูตร FR-6)

↓ โหมดทดลอง → หยุดตรงนี้ ตอบด้วยข้อความอธิบาย

④ ส่งไป Edge Function "coach"

askCoach → supabase.functions.invoke ไลบรารีแบบ token การเข้าสู่ระบบให้เอง

↓ บนเซิร์ฟเวอร์ Supabase (Deno)

⑤ ด้านตรวจและข้อจำกัด 8 อย่าง

POST เท่านั้น · token ถูกต้อง · ขนาดไม่เกิน 64,000 ไบต์ · เป็น JSON · ตัดคำถามเหลือ 400 ตัว · ตัดข้อมูลเหลือ 6,000 ตัว · มีคุกกี้แฉ AI · ยังไม่เกิน 60 ครั้งใน 24 ชั่วโมง

↓

⑥ เรียก AI

system prompt (บุคลิกและกติกา) + ข้อความ "DATA: ... QUESTION: ..." · คำตอบยาวได้ไม่เกิน 500 token · Claude ถ้ามีคุกกี้แฉ ไม่ขึ้น Gemini

↓

⑦ ทำความสะอาดคำตอบ

plainText ลบตัวหนา หัวข้อ และสัญลักษณ์รายการ ที่ AI อาจเผลอใส่ แล้วตอบแอปเป็น {"text": "..."}

↓ กลับมาที่แอป

⑧ แสดงและพูด

พองคำตอบ "นื่องคล้าย · ตอบโดย AI" · ตัดเป็นซับไตเติล · พูดทีละก่อน · กรอบทอระหว่างพูด

ข้อมูลที่แอปส่งจริง (ดักมาจากบัญชีมายด์)

ตอนถ่ายภาพหน้าจอ จดคำขอที่แอปส่งไปเซิร์ฟเวอร์จำลองแล้วบันทึกไว้ นี่คือก่อนข้อมูลทั้งหมดของคำถาม "สรุปสัปดาห์นี้ให้หน่อย" ไม่มีอะไรมากกว่านี้ (ค่าทุกตัวตรงกับที่ฟังก์ชัน buildCoachContext คำนวณเมื่อฉันรันซ้ำด้วยรายการชุดเดียวกัน):

body ที่ส่งไป Edge Function coach

ของจริงจากแอป (จัดบรรทัดใหม่ให้อ่านง่าย)

```
1 {
2   "context": {
3     "today": "9 ต.ค. 2569",
4     "balance": "฿2,566.00",
5     "lowLine": "฿500",
6     "averageDailySpend7d": "฿211",
7     "runway": "9 วัน (ถึง 18 ต.ค. 2569)",
8     "monthlyBudget": "฿6,000",
9     "spentThisMonth": "฿4,219",
10    "expenseByCategoryLast7Days": { "อาหารและเครื่องดื่ม": "฿1,250", "เดินทาง": "฿120", "ร้านสะดวกซื้อ":
    "฿110" },
11    "expenseByCategoryLast30Days": {
12      "อาหารและเครื่องดื่ม": "฿3,050", "บิลและค่าที่พัก": "฿2,500", "ข้อปึง": "฿890",
13      "เดินทาง": "฿480", "ร้านสะดวกซื้อ": "฿315", "บันเทิง": "฿199"
14    },
15    "incomeLast30Days": "฿4,000"
16  },
17  "question": "สรุปสัปดาห์นี้ให้หน่อย"
18 }
```

เมื่อถามเรื่องซื้อของ ก่อน context มีช่องเพิ่ม 1 ช่อง (ของจริงจากคำถาม "ถ้าฉันซื้อของราคา ฿1,290 วันนี้ จะเป็นยังไง"):

ช่องที่เพิ่มเมื่อคำถามมีราคา

ของจริงจากแอป

```
1 "purchaseCheck": { "price": "฿1,290", "runwayBefore": "9 วัน", "runwayAfter": "3 วัน" }
```

ช่อง	ความหมาย	มาจาก
today	วันนี้ (วันที่ไทย) ให้ AI รู้ว่า "สัปดาห์นี้" คือเมื่อไร	เวลาไทยของเครื่อง
balance	ยอดคงเหลือ	FR-2 computeBalance
lowLine	เส้นเงินสำรอง (รวมเงินในกระปุกออม)	FR-6 runway.floorSatang
averageDailySpend7d	ค่าเฉลี่ยรายจ่ายต่อวัน 7 วัน	FR-6
runway	เงินพอใช้อีกกี่วัน ถึงวันไหน (หรือ "ยังคำนวณไม่ได้..." / "ต่ำกว่าเส้นเงินสำรองแล้ว")	FR-6 computeRunway
monthlyBudget	งบรายเดือน (null ถ้าไม่ได้ตั้ง)	ตั้งค่า
spentThisMonth	ใช้ไปแล้วเดือนนี้	monthExpense
expenseByCategoryLast7Days	รายจ่ายรวมแต่ละหมวด 7 วัน เรียงจากมากไปน้อย ใช้ชื่อหมวดภาษาไทย	buildCoachContext
expenseByCategoryLast30Days	แบบเดียวกัน 30 วัน	buildCoachContext

ช่อง	ความหมาย	มาจาก
incomeLast30Days	รายรับรวม 30 วัน	buildCoachContext
purchaseCheck	ราคา, Runway ก่อนซื้อ, Runway หลังซื้อ (เฉพาะคำถามที่มีราคา)	coach.tsx + FR-6 runwayAfterPurchase

สิ่งที่ ไม่ ถูกส่ง

ไม่ส่ง	เหตุผล
รูปสลิป	ไม่จำเป็นต่อการอธิบาย และเป็นข้อมูลส่วนตัวที่สุด
ชื่อรายการ ชื่อร้าน ชื่อคนที่โอนให้	ส่งแค่ยอดรวมตามหมวด จึงไม่มีชื่อใครหลุดไป
เลขบัญชี เลขอ้างอิงสลิป	ไม่เกี่ยวกับคำแนะนำ
ชื่อผู้ใช้ อีเมล	AI ไม่จำเป็นต้องรู้ว่าเป็นใคร (เซิร์ฟเวอร์ใช้ token แค่ตรวจตัวตนและนับโควตา)
รายการแบบร่าง	ยังไม่ยืนยัน อาจผิด (นับเฉพาะ confirmed)

! สิ่งที่ใช้พิมพ์เองถูกส่งไปด้วย

ข้อความคำถามถูกส่งไปตามที่พิมพ์ (ไม่เกิน 400 ตัวอักษร) ถ้าผู้ใช้พิมพ์ชื่อคนหรือเลขบัญชีลงในคำถามเอง ข้อมูลนั้นก็ไปถึง AI ด้วย นี่คือเหตุผลที่หน้าจอเขียนบอกว่าคำตอบจาก "ตัวเลขที่คุณยืนยันแล้ว" และ README ของทีมก็เตือนไว้ว่า Gemini แบบฟรี Google อาจเก็บข้อมูลไปปรับปรุงโมเดล ควรเปลี่ยนเป็นแบบเสียเงินหรือ Claude เมื่อมีผู้ใช้จริง

★ ทำไมไม่ส่งเป็นข้อความ "฿2,566.00" ไม่ใช่ตัวเลข 256600

ในแอปเงินเก็บเป็นสตางค์ (จำนวนเต็ม) แต่ถ้าส่ง 256600 ให้ AI มันอาจเข้าใจผิดว่าเป็นสองแสนบาท แอปจึงแปลงเป็นข้อความที่คนอ่าน (มี ฿ และคอมมา) ก่อนส่ง AI แคยกตัวเลขไปพูดต่อ ไม่ต้องแปลงหน่วยเอง ลดโอกาสพลาดอีกชั้น

System prompt: กติกาที่ AI ต้องทำตาม

System prompt คือ "คำสั่งประจำตัว" ที่ส่งไปทุกครั้งก่อนคำถาม (โค้ดเต็มและคำแปลที่ละบรรทัดอยู่ในบท "อ่านโค้ด: ฝั่งเซิร์ฟเวอร์ของโค้ช") ทุกข้อมีเหตุผล:

กติกา	ทำไม	ถ้าไม่มีจะเกิดอะไร
เป็นน้องกล้า ต้นกล้าเงินสีทอง โค้ชการเงินส่วนตัว	ให้ AI พูดเป็นตัวละครเดียวกับที่ผู้ใช้เห็นบนจอ	คำตอบเป็นภาษาหุ่นยนต์ ไม่ต่อเนื่องกับตัวละคร
สุขุม เป็นมิตร เรียกตัวเองว่า "กล้า" ไม่ใช่ครับ/ค่ะ ลงท้าย "นะ" ได้	น้องกล้าไม่มีเพศ ครับ/ค่ะจะบังคับเพศให้ และฟังทางการเกินไป	บางครั้ง "ครับ" บางครั้ง "ค่ะ" ตัวละครไม่คงที่
คำตอบถูกอ่านออกเสียง: ประโยคพูดสั้น ๆ คั่นด้วยเว้นวรรค ไม่มีอีโมจิ รายการ markdown หัวข้อ	เสียงอ่านสัญลักษณ์แปลก ๆ และซับซ้อนที่เว้นวรรค	เสียงอ่าน "ดอกจัน ดอกจัน" หรือข้ามคำ ชับไต่เตลละ

กติกา	ทำไม	ถ้าไม่มีจะเกิดอะไร
ตอบภาษาไทย ไม่เกิน 90 คำ	ฟังจบในราว 20–30 วินาที ไม่ยาวจนน่าเบื่อ (ประมาณ 5–6 ก่อนขับไต่เตล)	น้องกลัวพูดยาวเป็นนาทีก
ใช้เฉพาะตัวเลขใน DATA ห้ามแต่งจำนวน วันที่ ร้าน หรือแนวโน้ม ถ้าข้อมูลตอบไม่ได้ให้บอกว่าขาดอะไร	หัวใจของความน่าเชื่อถือ AI รู้แค่สิ่งที่ส่งไป	AI "เดา" ตัวเลขที่ฟังดูสมจริง แต่ผิด (hallucination)
โครงคำตอบ: ตัวเลขบอกอะไร → สำคัญยังไง → ข้อเสนอลittle ๆ 1–2 ข้อ → ให้ผู้ใช้ตัดสินใจ	กฎน้ำเสียงเกี่ยวกับการรู้สิ่งที่ควรรู้	คำตอบสั่งสอน หรือไม่มีข้อเสนอที่ทำได้จริง
ห้ามตำหนิ ("ใช้เยอะเกินไป!" ไม่อนุญาต) ใจดีกับความผิดพลาด	ผู้ใช้ที่รู้สึกผิดมักเลิกจด	ผู้ใช้หนีแอป ข้อมูลไม่ครบ โค้ชยังช่วยไม่ได้
ถ้าถาม "ซื้อ X ได้ไหม" ให้ใช้ purchaseCheck และบอก Runway ก่อนและหลัง	ตัวเลขก่อน/หลังซื้อแอปคิดไว้แล้ว ถูกต้องแน่นอน	AI ลองคิดเองแล้วอาจลบเลขผิด
ไม่ใช่ที่ปรึกษาการเงินที่มีใบอนุญาต ห้ามแนะนำการลงทุน เงินกู้ ทรัพย์สิน	ขอบเขตความรับผิดชอบของแอป นักศึกษา	แนะนำสิ่งที่เสี่ยงต่อเงินของผู้ใช้

ข้อความที่ส่งไปพร้อม prompt มีรูปแบบตายตัว 2 ส่วน: DATA: ตามด้วยข้อมูลสรุป (JSON) และ QUESTION: ตามด้วยคำถาม ถ้าคำถามว่าง เซิร์ฟเวอร์ใช้คำถามตั้งต้น "สรุปสถานะการเงินของคุณในช่วงนี้ และบอกสิ่งที่ควรรู้" การแยกเป็นป้ายชัดเจนช่วยให้ AI รู้ว่าส่วนไหนคือข้อเท็จจริง ส่วนไหนคือคำถาม

≈ เปรียบเทียบ: ผู้ประกาศข่าว

AI ในระบบนี้เหมือนผู้ประกาศข่าว ไม่ใช่นักข่าว ทีมข่าว (โค้ดในแอป) ไปเก็บข้อมูลและตรวจตัวเลขมาแล้ว เขียนใส่ "สคริปต์" (DATA) ให้ ผู้ประกาศมีหน้าที่อ่านให้น่าฟังและเข้าใจง่าย ถ้าผู้ประกาศเติมตัวเลขเองที่ไม่มีในสคริปต์ ถือว่า ผิดกติกา

เลือก AI ตัวไหน และเมื่อ AI ไม่ว่าง

สถานการณ์	เซิร์ฟเวอร์ทำอะไร
มีกุญแจ Claude (ANTHROPIC_API_KEY)	ใช้ Claude รุ่นตั้งต้น claude-haiku-4-5-20251001 (เปลี่ยนได้ด้วย Secret COACH_MODEL)
มีแต่กุญแจ Gemini (GEMINI_API_KEY)	ลองตามลำดับ: gemini-3.5-flash-lite → gemini-3.1-flash-lite → gemini-flash-lite-latest → gemini-flash-latest (ถ้าตั้ง GEMINI_MODEL ไว้ ลองตัวนั้นก่อน)
มีทั้งสอง	ใช้ Claude เว้นแต่ตั้ง AI_PROVIDER=gemini
Gemini รุ่นหนึ่งไม่มีให้กุญแจนี้ (404) หรือโควตาเป็นศูนย์	ข้ามไปรุ่นถัดไป (แต่ละรุ่นมีโควตาฟรีของตัวเอง)
ทุกรุ่นติดลิมิต (429/500/503)	รอตามที่ Google บอก (ไม่บอกใช้ 3 วินาที สูงสุด 12 วินาที) แล้วลองทุกรุ่นอีกรอบ ยังไม่ได้ → ตอบแอปว่า busy
กุญแจผิดหรือถูกยกเลิก (401/403)	หยุดทันที ตอบแอปว่า not_configured

สถานการณ์	เซิร์ฟเวอร์ทำอะไร
รอเกินเวลา (ครั้งละ 45 วินาที รวมไม่เกิน 110 วินาที)	ถือว่าไม่ว่าง ตอบ busy

Error ทุกแบบ: จากเซิร์ฟเวอร์ถึงข้อความที่ผู้ใช้เห็น

เหตุการณ์	เซิร์ฟเวอร์ตอบ	แอปแสดง (services/coach.ts)
ไม่ได้เข้าสู่ระบบ / token หมดอายุ	401 unauthorized	เซสชันหมดอายุ กรุณาเข้าสู่ระบบอีกครั้ง
ไม่มีกุญแจ AI หรือกุญแจถูกปฏิเสธ	503 not_configured	โค้ด AI ยังไม่พร้อม: ผู้ดูแลต้องใส่ GEMINI_API_KEY ใน Supabase → Edge Functions → Secrets ...
ถามครบ 60 ครั้งใน 24 ชั่วโมง	429 quota	วันนี้คุยกับน้องกล้าครบโควตาแล้ว พรุ่งนี้มาคุยกันใหม่นะ
AI ติดลิมิต/โหลดเกิน/ช้าเกิน	503 busy	ตอนนี้มีคนคุยกับน้องกล้าเยอะ รอสักครู่แล้วถามใหม่นะ
ข้อผิดพลาดอื่นบนเซิร์ฟเวอร์	502 coach_error	ติดต่อน้องกล้าไม่ได้ตอนนี้ ลองใหม่อีกครั้งนะ
เน็ตหลุด เรียกเซิร์ฟเวอร์ไม่ได้	–	
AI ตอบว่างเปล่า	200 แต่ไม่มี text	น้องกล้าไม่ได้ตอบกลับ ลองถามใหม่อีกครั้งนะ
โหมดทดลอง	(ไม่ได้ส่ง)	โหมดทดลองยังถาม AI ไม่ได้นะ เข้าสู่ระบบด้วยบัญชีจริงแล้วถามกล้าได้เลย ...

ทำความเข้าใจคำตอบ (plainText)

แม้ prompt จะห้าม markdown แล้ว บางครั้ง AI ก็ยังเผลอใส่ เซิร์ฟเวอร์จึงล้างอีกชั้นก่อนส่งให้แอป ตัวอย่างที่ฉันลองกับฟังก์ชันจริง:

AI ตอบมา	หลัง plainText
## สรุป	สรุป
สัปดาห์นี้ ใช้ไป ฿1,250	สัปดาห์นี้ ใช้ไป ฿1,250
- กินข้าวนอกบ้าน	กินข้าวนอกบ้าน
2) ลองห่อข้าว	ลองห่อข้าว
• เลือกเองได้นะ	เลือกเองได้นะ

★ ป้องกันหลายชั้น (defense in depth)

เรื่องเดียวกันถูกกันไว้หลายชั้น: prompt บอห้าม markdown → เซิร์ฟเวอร์ล้าง markdown (plainText) → แอปล้างสัญลักษณ์ก่อนอ่านออกเสียง (speakable) ถ้าชั้นใดชั้นหนึ่งพลาด ชั้นถัดไปยังรับไว้ หลักนี้ใช้กับความปลอดภัยทุกเรื่องในซอฟต์แวร์

เช็กก่อนจ่ายผ่านโค้ช: หาราคาในคำถาม

ผู้ใช้ถาม "หุฟัง 1,990 คุ่มไหม" แอปต้องรู้เองว่า 1,990 คือราคา แล้วคิด Runway ก่อน/หลังให้ AI

ฟังก์ชัน priceInQuestion ใน `src/domain/price.ts` อ่านคำถามแล้วคืน "ราคาเป็นสตริง" หรือ null ถ้าไม่มีราคา
ยากตรงที่คำถามมีตัวเลขหลายแบบ บางตัวเป็นเงิน บางตัวเป็นจำนวนวัน จำนวนชิ้น หรือเปอร์เซ็นต์

กติกา 4 ชั้น (ลองตามลำดับ เจอแล้วหยุด)

ชั้น	หาแบบนี้	ตัวอย่างที่เจอ
1	สัญลักษณ์ ฿ ตามด้วยตัวเลข	"฿89 แพงไหม" → 89
2	ตัวเลขตามด้วยคำว่า "บาท"	"ซื้อของ 500 บาทวันนี้ได้ไหม" → 500 · "ซื้อกาแฟ 3 แก้ว 120 บาท" → 120 (ไม่ใช่ 3)
3	คำว่า ซื้อ / ราคา / จ่าย แล้วตามด้วยตัวเลข (ห่างกันไม่เกิน 24 ตัวอักษร) ที่ ไม่ใช่ หน่วยอื่น	"ซื้อรองเท้า 1,290 ได้ไหม" → 1,290 · "ซื้อของ 89 ได้ไหม" → 89
4	เฉพาะคำถามเรื่องซื้อ/จ่าย (มีคำว่า ซื้อ ราคา จ่าย คุ่ม แพง ผ่อน ได้ไหม ได้ไหม ไหวไหม): ตัวเลข 3 หลักขึ้นไป หรือมีคอมมา	"หุฟัง 1,990 คุ่มไหม" → 1,990 · "มือถือ 15000 ควรซื้อไหม" → 15,000

หน่วยที่ไม่ใช่เงิน (ถ้าตัวเลขตามด้วยคำเหล่านี้ จะไม่ถือเป็นราคา): วัน เดือน ปี ชิ้น อัน คน ครั้ง โมง นาที ชั่วโมง % เปอร์เซ็นต์

ตัวอย่าง 16 ข้อ (รันกับฟังก์ชันจริงแล้ว)

คำถาม	ราคาที่ได้	ทำไม
ซื้อรองเท้า 1,290 ได้ไหม	฿1,290	ชั้น 3
ซื้อของ 500 บาทวันนี้ได้ไหม	฿500	ชั้น 2
หุฟัง 1,990 คุ่มไหม	฿1,990	ชั้น 4 (มีคำว่า คุ่ม)
มือถือ 15000 ควรซื้อไหม	฿15,000	ชั้น 4
฿89 แพงไหม	฿89	ชั้น 1
ผ่อนเดือนละ 990 ไหวไหม	฿990	ชั้น 4 (ผ่อน, ไหวไหม)
ราคา 350.50	฿350.50	ชั้น 3 (ทศนิยม 2 ตำแหน่งได้)
ซื้อกาแฟ 3 แก้ว 120 บาท	฿120	ชั้น 2 มาก่อนชั้น 3 จึงได้ 120 ไม่ใช่ 3
ซื้อของ 89 ได้ไหม	฿89	ชั้น 3
อีก 30 วันเงินจะพอไหม	ไม่มี	ไม่มีคำเรื่องซื้อ และ 30 ตามด้วย "วัน"
ซื้อ 2 ชิ้นได้ไหม	ไม่มี	2 ตามด้วย "ชิ้น" และชั้น 4 ต้อง 3 หลักขึ้นไป

คำถาม	ราคาที่ได้	ทำไม
ลด 20% ได้ไหม	ไม่มี	20 ตามด้วย % และมีแค่ 2 หลัก
ถ้าใช้เงินวันละ 200 จะพอไหม	ไม่มี	ไม่มีคำเรื่องซื้อ/จ่าย ("จะพอไหม" ไม่อยู่ในรายการ)
เก็บเงิน 5000 ใน 3 เดือน	ไม่มี	ไม่ใช่คำถามเรื่องซื้อ
สรุปสัปดาห์นี้ให้หน่อย	ไม่มี	ไม่มีตัวเลข
ทำยังไงให้เงินพอถึงสิ้นเดือน	ไม่มี	ไม่มีตัวเลข

เจอราคาแล้ว: คิด Runway ก่อน/หลัง

แอปเรียก runwayAfterPurchase(runway, ราคา) ของ FR-6 คือกิดสูตร Runway ใหม่ด้วยยอดที่หักราคาออก ค่าเฉลี่ยเท่าเดิม แล้วแปลงผลเป็นข้อความ บัญชีมายด์ (ยอด ฿2,566 สำรอง ฿500 เฉลี่ย ฿211.43):

ราคา	ยอดหลังซื้อ	ใช้ได้จริง	runwayAfter ที่ส่งให้ AI
฿500	฿2,066	฿1,566	7 วัน (1,566 ÷ 211.43 = 7.4)
฿1,290	฿1,276	฿776	3 วัน (3.67)
฿1,990	฿576	฿76	0 วัน (0.36 ยังเหนือเส้น)
฿2,066	฿500	฿0	ต่ำกว่าเส้นสำรองทันที

runwayBefore ทุกแถวคือ "9 วัน" ถ้าคำนวณไม่ได้ (ไม่มีรายจ่ายใน 7 วัน) จะส่ง "คำนวณไม่ได้"

i คำถามที่มาจากหน้าเงินพอถึง ใช้ราคาที่ส่งมาด้วย

เมื่อกด "ถามโค้ชเรื่องนี้" หน้าเงินพอถึงส่งราคาเป็นสตริงมาด้วย (พารามิเตอร์ price) ถ้าผู้ใช้ส่งคำถามนั้นโดยไม่แคร์แอปใช้ราคานั้นตรง ๆ ถ้าแก้คำถามแล้ว แอปหารราคาจากข้อความใหม่ด้วย priceInQuestion (บสรัด 123 ของ coach.tsx) จึงไม่ใช่ราคาเก่าที่ไม่ตรงกับคำถามแล้ว

หลักการขั้นที่ 3: ทำให้น้องกล้าพูดได้

คำตอบยาว ๆ ถูกหั่นเป็นซับไต่เตลสั้น ๆ แล้วพูดทีละท่อนด้วยเสียงภาษาไทยของเครื่อง

ภาพรวม: จากข้อความ 1 ท่อน ถึงเสียงทีละท่อน

ข้อความ

คำตอบของ AI · ข้อความการ์ด · คำทักทาย

↓ talkChunks (klaTalk.ts)

ซับไต่เตลทีละท่อน

ก่อนจะไม่เกินประมาณ 64 ตัวอักษร ตัดที่เว้นวรรค ไม่ตัดกลางคำ

↓ ทีละท่อน (useKlaTalk)

แสดงท่อนนี้ในฟองคำพูด + ปากขยับ

จุดบอกตำแหน่ง "ก่อนที่ ... จาก ..."

↓ มีเสียงไหม?

มีเสียง

speakeable (แปลง ฿ % ตัวเลข) → say() ด้วยเสียงภาษาไทยของเครื่อง → พูดจบ → พัก 0.16 วินาที → ท่อนถัดไป

ไม่มีเสียง

(ปิดเสียง / ไม่มีเสียงไทย / เปิดโปรแกรมอ่านหน้าจอ / แอปรุ่นเก่า) รอตามเวลาอ่าน talkMs → พัก 0.16 วินาที → ท่อนถัดไป

↓ ครบทุกท่อน

จบ

ฟองคำพูดกลับเป็นข้อความพัก · ปุ่ม "ฟังอีกครั้ง" ขึ้น · กรอบทองของคำตอบหายไป

★ ทำไมไม่พูดรวดเดียวทั้งท่อน

(1) **ซับไต่เตล**ทีละท่อนอ่านตามกันและพอดีฟองคำพูด (2) **ปากขยับ**ได้ถูกจังหวะ เพราะรู้ว่าตอนไหนกำลังพูด (3) **หยุดกลางทาง**ได้ทันทีเมื่อกด "หยุด" หรือถามคำถามใหม่ (4) ถ้าเสียงของเครื่องค้างกลางประโยค เสียหายแค่ท่อนเดียว ไม่ใช่ทั้งคำตอบ

กติกากการตัดซับไต่เตล (talkChunks)

1. แยกย่อหน้าที่ขึ้นบรรทัดใหม่เป็นท่อนแยกเสมอ
2. ภาษาไทยเว้นวรรคตรงที่จบวลีหรือประโยค จึงต่อคำทีละ "ท่อน" ที่คั่นด้วยเว้นวรรค" ไปเรื่อย ๆ
3. ถ้าต่ออีกท่อนแล้วเกิน **64 ตัวอักษร** และท่อนปัจจุบันยาวอย่างน้อย **24 ตัวอักษร** → ตัดจบท่อน
4. ถ้าคำสุดท้ายของท่อนเป็น **คำขึ้นต้นความคิดใหม่** 12 คำ (เช่น แต่ ถ้า หรือ และ เพราะ แล้ว ส่วน ซึ่ง ดังนั้น ลอง ถึง เช่น) ย้ายคำนั้นไปขึ้นต้นท่อนถัดไป อ่านแล้วเป็นธรรมชาติกว่า
5. เจอท่อนยาวมากที่ไม่มีเว้นวรรค (เกิน $64 \times 1.6 \approx 102$ ตัว) ตัดเลยเพื่อไม่ให้ท่อนยาวเกินอ่าน

6. คำที่ลงท้ายด้วย . ! ? ... และก่อนยาวพอแล้ว (≥ 24 ตัว) → จบก่อนตรงนั้น
7. ไม่มีคำหยาบ: เอาทุกก่อนมาต่อกันด้วยเว้นวรรค ต้องได้ข้อความเดิมพอดี (TC-76 ตรวจข้อนี้)

ตัวอย่างจริง: คำตอบ "สรุปสัปดาห์นี้" ของมายด์ (283 ตัวอักษร → 6 ก่อน)

ก่อน	ซับไต์เติล	ตัวอักษร	เวลาอ่านถ้าไม่มีเสียง
1	7 วันนี้ใช้ไปกับอาหารและเครื่องดื่มมากที่สุด \$1,250	51	4.09 วินาที
2	รองลงมาคือเดินทาง \$120 และร้านสะดวกซื้อ \$110 เจสสิวันละ \$211	61	4.81
3	ถ้าใช้แบบนี้ต่อไป เงินจะพอถึง 18 ต.ค.	37	3.08
4	คือราว 9 วัน ยังไม่ถึงสิ้นเดือน ลองลดค่าอาหารลงวันละนิด	55	4.38
5	หรือห่อข้าวไปเรียนบางวันดูนะ คล้ายแค่เสนอไอเดีย	46	3.73
6	เลือกแบบที่สะดวกที่สุดได้เลย	28	2.44

สังเกตก่อน 3: จบที่ "18 ต.ค." เพราะลงท้ายด้วยจุดและยาวเกิน 24 ตัวแล้ว (กติกาข้อ 6) และก่อน 5 ขึ้นต้นด้วย "หรือ" เพราะกติกาสัปดาห์ 4 ย้ายมาให้ คำทักทายของน้องคล้ายก็เช่นกัน ถูกตัดเป็น "สวัสดิ์ คล้าเองนะ ถ้ามเรื่องเงินได้ทุกเรื่องเลย" กับ "เช่น" สัปดาห์นี้ใช้ไปกับอะไรเยอะสุด หรือของชิ้นนี้ซื้อได้ไหม" คำว่า "เช่น" ไม่ขึ้นต้นก่อนที่สอง

ทำข้อความให้เสียงอ่านถูก (speakable)

ข้อความบนจอเขียนแบบอ่านด้วยตา (\$1,290 · 20% · - \$500) แต่เสียงอ่านต้องได้แบบที่คนไทยพูด ก่อนส่งแต่ละก่อนให้เสียง แอปแปลงข้อความแบบนี้ (ทุกแถวรันกับฟังก์ชันจริง):

บนจอ	สั่งให้เสียงอ่าน	เหตุผล
\$1,290	1290 บาท	คนไทยพูดตัวเลขก่อนหน่วย และตัดคอมมาให้เป็นจำนวนเดียว
- \$500	ลบ 500 บาท	เครื่องหมายลบต้องออกเสียงว่า "ลบ"
ยอด - \$1,250.50 นะ 😊	ยอด ลบ 1250.50 บาท นะ	ลบอีโมจิ (ไม่ขึ้นเสียงอ่านชื่ออีโมจิ)
ลดลง 20%	ลดลง 20 เปอร์เซนต์	สัญลักษณ์ % อ่านเป็นคำ
งบ \$6,000 · ใช้ไป 70%	งบ 6000 บาท , ใช้ไป 70 เปอร์เซนต์	จุดคั่น · เป็นการเว้นจังหวะ
“ลองปรับ”	ลองปรับ	ไม่ต้องอ่านเครื่องหมายคำพูด
1,000,000 บาท	1000000 บาท	ตัดคอมมาทุกตัว

i ซับไต์เติลบจอยังเหมือนเดิม

การแปลงนี้ใช้กับ "เสียง" เท่านั้น ฟองคำพูดยังแสดง "\$1,290" ตามเดิม เพราะตาอ่านแบบนั้นเร็วกว่า ในเทสต์ E2E มีข้อที่ตรวจว่าคำตอบ "\$0" ถูกสั่งให้เสียงอ่านเป็น "0 บาท" จริง

เวลาอ่านเมื่อไม่มีเสียง (talkMs)

$$\text{เวลาดำของท่อน} = 420 + 72 \times \text{จำนวนตัวอักษร มิลลิวินาที (ไม่ต่ำกว่า 1.4 วินาที ไม่เกิน 9 วินาที)}$$

ตัวอักษร	1-13	20	40	64	100	ตั้งแต่ 120
เวลา	1.4 5	1.86 5	3.3 5	5.03 5	7.62 5	9 5

ในโค้ดไม่ได้เขียนที่มาของตัวเลข 420 กับ 72 ไว้ ฉันตีความว่า 420 มิลลิวินาทีคือเวลาตั้งต้นที่ตาใช้หาข้อความ และ 72 มิลลิวินาทีต่อตัวอักษรคืออัตราอ่านภาษาไทยแบบสบาย ๆ ส่วนขั้นต่ำ 1.4 วินาทีกันก่อนสั้นมากหายไปก่อนอ่านกัน และขั้นสูง 9 วินาทีกันค้ำนานเกิน (TC-76 ตรวจสอบทั้ง 3 ช่วงนี้)

เลือกเสียงภาษาไทย

เรื่อง	มือถือ (tts.ts)	เว็บ (tts.web.ts)
เครื่องมือ	expo-speech → เสียงของระบบ (Google บน Android, Apple บน iPhone)	Web Speech API ของเบราว์เซอร์
เลือกเสียงไหน	เฉพาะเสียงที่ภาษาขึ้นต้นด้วย "th" เลือกคุณภาพ Enhanced ก่อน แล้วเสียงที่ใช้ได้โดยไม่ต้องต่อเน็ต	เฉพาะเสียงไทย เรียงตามชื่อที่ฟังสุ่มมออุ่น: Niwat, Premwadee, Natural, Google, Narisa, Kanya, Pattara
ความเร็ว / ระดับเสียง	0.9 / 0.95	0.95 / 0.95
ถ้าไม่มีเสียงไทย	คืนค่า null → พูดเป็นซับไตเติลตามเวลาอ่าน และแสดงบรกรกต์ "เครื่องนี้ยังไม่มีเสียงอ่านภาษาไทย..."	
กันค้าง	ถ้าเสียงไม่แจ้งว่าพูดจบภายใน 4 วินาที + 0.16 วินาทีต่อตัวอักษร ถือว่าจบแล้วไปก่อนต่อไป	

ความเร็วและระดับเสียงต่ำกว่า 1 นิดหน่อย ทำให้ฟัง "สุ่ม เป็นมิตร" ตามที่ผู้ใช้ขอ (เกสต์ E2E ตรวจสอบว่า rate < 1 และ pitch < 1 และเลือกเสียง Niwat เมื่อมีหลายเสียง)

เมื่อไรน้องกล้าเจียบ

สถานการณ์	ทำอะไร	เหตุผล
ผู้ใช้ปิดเสียง (บนเว็บหรือในตั้งค่า)	หยุดเสียงทันที ซับไตเติลเดินต่อตามเวลาอ่าน	บางคนอยู่ในห้องเรียนหรือที่สาธารณะ
เปิดโปรแกรมอ่านหน้าจอ (มือถือ)	ไม่พูดเอง	พองคำพูดตั้งเป็น live region โปรแกรมอ่านหน้าจออ่านซับไตเติลให้แล้ว ถ้าน้องกล้าพูดด้วยจะซ้อนกันสองเสียง

สถานการณ์	ทำอะไร	เหตุผล
ออกจากแท็บโค้ช หรือสลับแอปไปทำอย่างอื่น	หยุดพูดทันที	ไม่ให้เสียงดังต่อเนื่องจนผู้ใช้ไม่ได้ดู
ข้อความผิดพลาด	แสดงแต่ไม่อ่าน	เรื่องเทคนิคไม่ควรเป็นเสียงที่ทำให้ตกใจ
แอป Android รุ่นเก่าที่ยังไม่มีโมดูลเสียง	พูดเป็นซับไตเติล ไม่ crash	โหลดโมดูลแบบไม่บังคับ (requireOptionalNativeModule) แอปที่ได้อัปเดตแบบ OTA จึงไม่พัง

เวทีของน้องกล้า: ภาพเคลื่อนไหวที่ลื่นบนมือถือราคาประหยัด

ผู้ใช้ขอให้น้องกล้าเป็น "3D ขยับปากและท่าทาง" ทีมเลือกวิธีที่ได้ความรู้สึกเดียวกันแต่เบากว่าเดิม

วาดด้วยโค้ด แยกเป็นชั้น

น้องกล้าไม่ใช้รูปภาพหรือโมเดล 3D แต่ **วาดด้วยโค้ด** (SVG ใน `src/ui/kla/art.tsx`) บนพื้นขนาด 200 × 240 หน่วย มีแสงเงาจากมุมซ้ายบน ใบเป็นฟุ่ม ๆ มีเงา ตามีประกาย ลำต้นกลม จิงจู "นูน" เหมือน 3D ภาพถูกแยกเป็นชั้นที่ขยับแยกกันได้:

ชั้น	หมุน/ขยับรอบจุด	ใช้ทำอะไร
พื้น (เงาใต้เท้า)	กลางพื้น	หดและจางตอนกระโดด
ตัว (ลำต้น)	ฐานลำต้น	หายใจ (ยืดขึ้นเล็กน้อย) โยกซ้ายขวา กระโดด
แขนซ้าย / ขวา	ไหล่ (96, 196) / (104, 196)	แกว่งเบา ๆ ยกมืออธิบายตอนพูด
หัว (ฟุ่มใบ)	คอ (100, 172)	เอียงหัว พยักตอนพูด
ตา	กลางตา (100, 128)	กะพริบ (บีบแนวตั้ง)
ปากปิด / ปากเปิด	ปาก (100, 141)	สลับและยืดตามพยางค์
แว่น · ของประกอบอารมณ์	ไปกับหัว	เช่น Z ช่วง ประกายดีใจ

มีภาพ 2 แบบจากชั้นชุดเดียวกัน: **KlaPicture** รวมทุกชั้นเป็นภาพนิ่งภาพเดียว ใช้กับน้องกล้าตัวเล็ก (หน้าหลัก ฟองคำตอบ ตัวอย่างชุด ไอคอนแอป) และ **KlaStage** ซ้อนชั้นเป็นชั้น ๆ ให้แต่ละชั้นขยับเอง ใช้กับน้องกล้าตัวใหญ่บนเวทีโค้ชและต่อสู้กิน

i ทำไมไม่ใช้โมเดล 3D จริง (บันทึกใน AI_USAGE_LOG แถวที่ 18)

"3D ทำเป็นภาพวาดมีแสงเงาแยกชั้นที่ขยับด้วย native driver (ไม่ใช้โมเดล 3D จริง เพื่อให้อัปเดต OTA ได้และลื่นบนมือถือราคาประหยัด)" โมเดล 3D ต้องใช้ไลบรารีที่มีโค้ด native ใหม่ ทุกคนต้องลง APK ใหม่ ไฟล์ใหญ่ และกินแบตเตอรี่การ์ดจอ ส่วนภาพวาดชิ้นเล็ก ๆ ที่หมุนด้วย transform ทำงานได้กับโค้ดที่แอปมีอยู่แล้ว

native driver คืออะไร ทำไมลื่น

ปกติโค้ด JavaScript ของแอปต้องคำนวณทุกเฟรม (60 ครั้งต่อวินาที) ว่าแต่ละชั้นอยู่ตรงไหน ถ้า JavaScript กำลังยุ่ง (เช่น กำลังรับคำตอบจาก AI) ภาพจะกระตุก **native driver** คือการส่ง "สูตรการเคลื่อนไหว" ทั้งหมดไปให้ส่วนที่วาดจอ (UI thread) คำนวณเองตั้งแต่ต้น JavaScript ไม่ต้องยุ่งอีก ข้อแม้คือขยับได้เฉพาะ **transform** (เลื่อน หมุน ยืด) และ **opacity** (ความจาง) ซึ่งพอดีกับที่น้องกล้าต้องการทั้งหมด (บนเว็บไม่มี native driver ใช้วิธีปกติแทน)

ท่าทางทั้งหมด และจังหวะ

ท่า	เกิดเมื่อ	รายละเอียดจากโค้ด
หายใจ	เปิดหน้าจออยู่	ขึ้น 1.9 วินาที แล้วลง 1.9 วินาที ตัวยัดขึ้น 1.8% ลอยขึ้น 1.2 หน่วย
โยกตัว	เปิดหน้าจออยู่	ซ้ายขวา ±1.1 องศา ช่วงละ 3.1 วินาที
เอียงหัว	เปิดหน้าจออยู่	±2.2 องศา ช่วงละ 2.6 วินาที
แกว่งแขน	เปิดหน้าจออยู่	±3 องศา ช่วงละ 2.3 วินาที
กะพริบตา	อารมณ์ calm, worried, thinking	เว้น ~2.6 วินาทีกะพริบ 1 ที แล้วเว้น ~3.4 วินาทีกะพริบ 2 ทีติดกัน วนไปเรื่อย ๆ (อารมณ์ happy, cheer, sleepy ตาโค้งหรือปิดอยู่แล้ว จึงไม่กะพริบ)
ปากพูด	กำลังพูด	14 รูปปาก เปิด 0–100% แต่ละรูปยาว 80–140 มิลลิวินาที ครอบรอบ 1.51 วินาที แล้ววนใหม่ (ตั้งใจให้ไม่สม่ำเสมอเหมือนพยางค์จริง)
ท่าอธิบาย	กำลังพูด	มือขวายก 26 องศา มือซ้าย 7 องศา หัวพยัก (ลง 1.6 หน่วย เอียง 2.6 องศา) รอบละ 2.44 วินาที · อารมณ์ cheer กางแขนออกด้านข้าง ±9 องศาแทน
กระโดด	ผู้ใช้แตะหน้าจอแล้ว	ขึ้น 26 หน่วยใน 0.19 วินาที แล้วลงแบบสปริง เงามาใต้เท้าหดเหลือ 72% และจากลง

★ **ทำไมแต่ละท่าใช้จังหวะไม่เท่ากัน (1.9 · 3.1 · 2.6 · 2.3 วินาที)**

ถ้าทุกท่าใช้รอบเดียวกัน ทุกอย่างจะขยับพร้อมกันเหมือนหุ่นยนต์ เมื่อแต่ละท่ามีรอบไม่ลงตัวกัน ทำรวมจะแทบไม่ซ้ำเดิม ดูมีชีวิตเหมือนสิ่งมีชีวิตจริงที่ไม่ได้ขยับเป็นจังหวะเป๊ะ

ประหยัดแบตและเคารพผู้ใช

- **ขยับเฉพาะตอนเห็น:** ท่าทางว่าง (หายใจ โยก เอียง แกว่ง กะพริบ) ทำงานเฉพาะเมื่อนำจอขึ้นเปิดอยู่ (active) สลับเก็บไปแล้วหยุดทั้งหมด
- **ลดการเคลื่อนไหว:** ถ้าผู้ใช้เปิด "Reduce motion" ในมือถือ ท่าทางทั้งหมดหยุด ยกเว้นปากตอนพูด เพราะปากบอกผู้ใช้ว่า "กำลังพูดอยู่" ซึ่งเป็นข้อมูล ไม่ใช่ของตกแต่ง
- **ซ่อนจากโปรแกรมอ่านหน้าจอ:** ตัวน้องกล้าเป็นภาพตกแต่ง จึงซ่อนไว้ แต่ปุ่มที่ห่อตัวน้องกล้ามีคำอธิบาย "น้องกล้าในชุด... แตะเพื่อฟังเคล็ดลับ"

! บิ๊กที่เคยเจอบนมือถือจริง: จุดหมุนเป็นตัวเลข ไม่ใช่ข้อความ

จุดหมุนของแต่ละชั้นเคยเขียนเป็นข้อความเช่น "12.5px" ซึ่งบนเว็บใช้ได้ แต่บนมือถืออ่านทศนิยมในข้อความผิด ขึ้นส่วนจึงหมุนผิดที่ ทีมแก้เป็นตัวเลข [x, y, z] ซึ่งใช้ได้ทุกที (commit a911af8 "joints as numbers" และคอมเมนต์ บรรทัด 141 ของ KlaStage.tsx) บทเรียนคือ **ต้องทดสอบบนมือถือจริงด้วย** ไม่ใช่แค่บนเว็บ

ผู้สกินและการกิจการเงิน

รางวัลน่ารัก ๆ ที่ได้จากนิสัยการเงินที่ดีจริง ไม่ใช่จากการกดเล่น

ชุด 3 แบบ

แบบ	จำนวน	ได้ยังไง
เริ่มต้น (starter)	1	"ต้นกล้า" ทุกคนมีตั้งแต่แรก
ภารกิจ (mission)	8	ทำภารกิจการเงินสำเร็จ 1 ภารกิจ = 1 ชุด
ลิมิตัด (limited)	10	เปิดแอปในช่วงเวลาที่กำหนด (ชุดฮาโลวีนบางชุดต้องมีลูกอมพอ) พันช่วงแล้ว "หมดเวลาแล้ว หาไม่ได้ อีก"

ชุดที่ได้แล้ว **อยู่ตลอดไป** และชุดที่เลือกใส่แสดงทุกที่ที่มีน้องกล้า ถ้าข้อมูลบอกว่าใส่ชุดที่ไม่มี (เช่น ข้อมูลเสีย) แอปจะใส่ "ต้นกล้า" ให้แทน (ฟังก์ชัน wearable)

ภารกิจการเงิน 8 ภารกิจ

ภารกิจ	ต้องทำ	ได้ชุด	ฝึกนิสัยอะไร
มาหากล้าทุกวัน	เปิดแอปติดต่อกัน 30 วัน (เว้นวันไหนนับใหม่)	ชุดไทย	ดูเงินทุกวัน
จดครบสองสัปดาห์	จดย่อยการต่อเนื่อง 14 วัน	บันทึกการเงิน	จดสม่ำเสมอ
นักสืบสลิป	บันทึกสลิปเข้ายอดเงิน 20 ใบ	นักสืบสลิป	ใช้ FR-4
พูดแล้วจด	จดด้วยเสียง 10 รายการ	ดีเจเสียงใส	ใช้ FR-3
ออมครบเป้า	หยอดกระปุกออมจนครบเป้า 1 ใบ	เศรษฐีน้อย	การออม
สัปดาห์ประหยัด	7 วันล่าสุดใช้น้อยกว่า 7 วันก่อนหน้า (ต้องมีรายจ่ายทั้งสองช่วง)	ซิลลิ้มทะเล	ลดรายจ่าย
คุมงบได้ทั้งเดือน	ตั้งงบ แล้วเดือนที่แล้วใช้ไม่เกินงบ และจดย่อยรายจ่ายเดือนนั้นอย่างน้อย 10 รายการ	อีโร่คุมงบ	วางแผนงบ
นักสะสมเหรียญ	สะสมเหรียญความสำเร็จ 8 เหรียญ	ชาคุระ	ใช้แอปให้ครบด้าน

★ **ก้นการโกง: "เดือนที่จดจริง" เท่านั้น**

ภารกิจ "คุมงบได้ทั้งเดือน" ต้องมีรายจ่ายในเดือนนั้นอย่างน้อย 10 รายการ (ค่าคงที่ BUDGET_MONTH_MIN_RECORDS) ไม่งั้นคนที่ไม่จดย่อยเลยทั้งเดือนจะ "ใช้ไม่เกินงบ" และได้ชุดไปฟรี ๆ ทุกภารกิจคำนวณจากรายการของผู้ใช้เองทุกครั้ง จึงไม่มีตัวนับที่ค้างหรือเพี้ยน

สกินลิมีเต็ด 10 ชุด

ชุด	ช่วงเวลา	เงื่อนไข
ผู้บุกเบิก	29 ก.ย. – 31 ต.ค. 2569	เปิดแอปในช่วงนี้ (สำหรับผู้ใช้งานแรก)
ปีใหม่ 2570	25 ธ.ค. 2569 – 5 ม.ค. 2570	เปิดแอปในช่วงนี้ (ยังไม่ถึง: "เร็ว ๆ นี้")
สงกรานต์ 2569	10–16 เม.ย. 2569	หมดเวลาแล้ว หาไม่ได้อีก
ลอยกระทง 2568	3–7 พ.ย. 2568	หมดเวลาแล้ว หาไม่ได้อีก
ผีหัวฟักทอง	ฮาโลวีน 29 ก.ย. – 2 พ.ย. 2569	ฟรีเมื่อเปิดแอปช่วงงาน
ผีน้อยผ้าขาว		ลูกอม 5 เม็ด
แม่มดน้อย		ลูกอม 12 เม็ด
มันมี		ลูกอม 20 เม็ด
แวมไพร์		ลูกอม 30 เม็ด
แฟรงเคนสไตน์		ลูกอม 40 เม็ด

ลูกอมได้ 2 ทาง เฉพาะในช่วงงาน: **จับผีบนหน้าหลัก** (ปล่อยออกมาเมื่อเลือกธีมฮาโลวีน) วันละไม่เกิน 3 ตัว ตัวละ 1 เม็ด และ **วันที่มีการจดยารายการ** ได้วันละ 2 เม็ด (นับวันละครึ่ง จดยารายการก็ได้ 2 เม็ดเท่าเดิม จึงไม่จูงใจให้จดยารายการ)

ข้อมูลตู้สกินเก็บที่ไหน

เก็บ **ในเครื่อง** แยกตามบัญชี ชื่อ `mindpay.kla.<รหัสผู้ใช้>` (โหมดทดลองใช้ `mindpay.kla.demo`) มี 5 อย่าง: ชุดที่มี (owned) · ชุดที่เคยแฉงแล้ว (seen) · ชุดที่ใส่ (equipped) · วันที่เปิดแอป (opens เก็บ 120 วันล่าสุด ใช้กับการกิจ 30 วัน) · ผีที่จับได้แต่ละวัน (ghosts) ทุกครั้งที่เปิดแอป useKlaSync จะบันทึกวันนี้และแจกชุดที่ทำการกิจสำเร็จหรืออยู่ในช่วงลิมีเต็ด

i ข้อแลกเปลี่ยน

เก็บในเครื่องทำให้ง่าย เร็ว ไม่ต้องมีตารางใหม่บนเซิร์ฟเวอร์ และไม่ส่งพฤติกรรมการเปิดแอปไปที่ไหน แต่ถ้าผู้ใช้เปลี่ยนมือถือหรือล้างข้อมูลแอป ตู้สกินและนับวันเปิดแอปจะเริ่มใหม่ (ภารกิจที่คำนวณจากรายการ เช่น สลิป 20 ใบ จะได้ชุดคืนเองเมื่อเปิดแอป เพราะคำนวณจากข้อมูลบนเซิร์ฟเวอร์)

อ่านโค้ด: src/domain/insights.ts ทั้งไฟล์

ไฟล์เดียวที่ทำ 3 งาน: หาข้อเท็จจริงด้วยกฎ · เขียนเป็นน้ำเสียงน้องกล้า · สรุปข้อมูลส่งให้ AI

ไฟล์นี้อยู่ใน src/domain คือ "กฏล้วน" ไม่มีหน้าจอ ไม่มีฐานข้อมูล ไม่มีเน็ต รับรายการกับโปรไฟล์เข้าไป คั้นผลลัพธ์ออกมา ฝั่งเทสต์ได้ด้วย Vitest (TC-31, TC-32, TC-34) เปิดคู่กับ VS Code แล้วอ่านทีละส่วน

ส่วนที่ 1: คอมเมนต์หัวไฟล์และ import

src/domain/insights.tsบรรทัด 1-19

```
1 /**
2  * FR-5 AI coach (น้องกล้า) – the data side.
3  *
4  * The coach is น้องกล้า, and it only talks about CONFIRMED data. This file:
5  *   1. finds simple, checkable facts ("food is up 40% this week") with plain rules,
6  *     so the coach still works offline and every claim can be traced to numbers;
7  *   2. words those facts in น้องกล้า's voice: calm, warm, never pushy;
8  *   3. builds the compact data summary sent to the AI for free-form questions.
9  *
10 * Voice rule: inform -> explain -> suggest -> let the user decide.
11 * Never shame ("คุณใช้เงินเยอะเกินไป!").
12 */
13 import { BUDDY_NAME } from './buddy';
14 import { getCategory } from './categories';
15 import { addDays, bkkDayKey, formatThaiDay } from './dates';
16 import { formatBaht } from './money';
17 import type { Runway } from './runway';
18 import { confirmedOnly, monthExpense } from './summary';
19 import type { Profile, Transaction } from './types';
```

▶ เปิดใน VS Code (เว็บ) GitHubVS Code ในเครื่อง: Ctrl+P → src/domain/insights.ts:1

บรรทัด	ความหมาย
1-4	บอกว่าไฟล์นี้คือ "ฝั่งข้อมูล" ของโค้ช FR-5 (น้องกล้า) และโค้ชพูดถึงเฉพาะข้อมูลที่ ยืนยันแล้ว (CONFIRMED ตัวใหญ่ เพื่อบั่น)
5-8	3 หน้าทีของไฟล์: (1) หาข้อเท็จจริงง่าย ๆ ที่ตรวจได้ด้วยกฎ "ทำงานออฟไลน์ได้ และทุกคำพูดย้อนไปหาตัวเลขได้" (2) เขียนข้อเท็จจริงเป็นน้ำเสียงน้องกล้า สุขุม อบอุ่น ไม่ยัดเยียด (3) สร้างสรุปข้อมูลขนาดเล็กที่ส่งให้ AI สำหรับคำถามอิสระ
10-11	กฏน้ำเสียง: บอก → อธิบาย → เสนอ → ให้ผู้ใช้ตัดสินใจ ห้ามทำให้อับอาย ("คุณใช้เงินเยอะเกินไป!")
13	ดึงชื่อ "กล้า" จาก buddy.ts มาใส่ในข้อความ
14	getCategory แปลงรหัสหมวด (เช่น food) เป็นชื่อไทย (อาหารและเครื่องดื่ม)
15	เครื่องมือวันที่: บวกลบวัน (addDays) หาวันตามเวลาไทย (bkkDayKey) และจัดรูปวันที่ไทย (formatThaiDay เช่น "18 ต.ค.")
16	formatBaht แปลงสตางค์เป็นข้อความ เช่น 125000 → "฿1,250.00"

บรรทัด	ความหมาย
17	ชนิด Runway จาก FR-6 (import type = ใช้แค่ชนิด ไม่ได้ดั่งโค้ด)
18	confirmedOnly กรองเฉพาะรายการยืนยันแล้ว · monthExpense รวมรายจ่ายเดือนนี้
19	ชนิดข้อมูล Profile (โปรไฟล์ผู้ใช้) และ Transaction (รายการ)

ส่วนที่ 2: ชนิดของการ์ด และตัวช่วยรวมรายจ่าย

src/domain/insights.ts

บรรทัด 21-41

```
21 export type InsightKind = 'category_up' | 'budget_pace' | 'runway' | 'no_data' | 'steady';
22
23 export interface Insight {
24   kind: InsightKind;
25   severity: 'good' | 'info' | 'watch';
26   /** The fact in neutral words, with the numbers behind it. */
27   fact: string;
28   /** The same fact in น้องกล้า's voice. */
29   message: string;
30 }
31
32 function sumExpense(txs: Transaction[], from: string, to: string, categoryKey?: string) {
33   return txs
34     .filter((t) => t.kind === 'expense')
35     .filter((t) => !categoryKey || t.categoryKey === categoryKey)
36     .filter((t) => {
37       const d = bkkDayKey(t.occurredAt);
38       return d >= from && d <= to;
39     })
40     .reduce((s, t) => s + t.amountSatang, 0);
41 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/insights.ts:21

บรรทัด	ความหมาย
21	การ์ดมี 5 ชนิด: หมวดที่สูงขึ้น · จังหวะงบ · Runway · ไม่มีข้อมูล · สม่าเสมอ
23-30	หน้าตาของการ์ด 1 ใบ: ชนิด (kind) · ระดับ (severity: good / info / watch) · ข้อเท็จจริงพร้อมตัวเลข (fact) · ข้อความในน้ำเสียงน้องกล้า (message) คอมเมนต์ /** ... */ เหนือช่องจะขึ้นใน VS Code เวลาเราเมาส์ชี้
32	ฟังก์ชันช่วย sumExpense รับรายการ วันเริ่ม วันจบ และ (ถ้าระบุ) รหัสหมวด categoryKey? เครื่องหมาย ? แปลว่าไม่ใส่ก็ได้
33-34	เริ่มจากรายการทั้งหมด กรองเหลือเฉพาะรายจ่าย
35	ถ้าไม่ระบุหมวด (!categoryKey เป็นจริง) เอาทุกหมวด ถ้าระบุ เอาเฉพาะหมวดนั้น
36-39	เอาเฉพาะรายการที่วัน (เวลาไทย) อยู่ระหว่าง from ถึง to รวมทั้งสองวัน วันที่เป็นข้อความรูปแบบ "2026-10-09" จึงเทียบมาก/น้อยแบบข้อความได้ถูกต้อง
40	บวกยอดทั้งหมดเป็นสตริง เริ่มจาก 0

i sumExpense ไม่ได้กรอง confirmed เอง

ฟังก์ชันนี้ไม่ได้เช็ค status เพราะคนที่เรียก (findCategoryIncrease) ส่งรายการที่ผ่าน confirmedOnly มาให้แล้ว (บรรทัด 45 และ 53–54) เวลาอ่านโค้ดต้องดูทั้งตัวฟังก์ชันและจุดที่เรียกใช้ ถึงจะรู้ว่าข้อมูลถูกกรองตรงไหน

ส่วนที่ 3: findCategoryIncrease หาหมวดที่สูงขึ้น

src/domain/insights.ts

บรรทัด 43–61

```
43 /** Category whose last-7-day spend rose most against the 3 weeks before it. */
44 export function findCategoryIncrease(txs: Transaction[], now: Date = new Date()) {
45   const confirmed = confirmedOnly(txs);
46   const today = bkkDayKey(now);
47   const weekFrom = addDays(today, -6);
48   const baseFrom = addDays(today, -27);
49   const baseTo = addDays(today, -7);
50   const keys = new Set(confirmed.filter((t) => t.kind === 'expense').map((t) => t.categoryKey));
51   let best: { key: string; week: number; usualWeek: number; changePct: number } | null = null;
52   for (const key of keys) {
53     const week = sumExpense(confirmed, weekFrom, today, key);
54     const base = sumExpense(confirmed, baseFrom, baseTo, key);
55     const usualWeek = Math.round(base / 3);
56     if (usualWeek <= 0 || week - usualWeek < 10_000) continue; // ignore rises under ฿100
57     const changePct = Math.round(((week - usualWeek) / usualWeek) * 100);
58     if (changePct >= 25 && (!best || changePct > best.changePct)) best = { key, week, usualWeek,
changePct };
59   }
60   return best;
61 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/insights.ts:43

บรรทัด	ความหมาย
43–44	หาหมวดที่รายจ่าย 7 วันล่าสุดสูงขึ้นมากที่สุดเมื่อเทียบกับ 3 สัปดาห์ก่อนหน้า รับรายการและ "ตอนนี้" (ค่าเริ่มต้นคือเวลาปัจจุบัน เทสต์ส่งวันตายตัวเข้ามาแทนได้)
45	เหลือเฉพาะรายการยืนยันแล้ว
46	วันนี้ตามเวลาไทย เช่น "2026-10-09"
47	วันแรกของสัปดาห์นี้ = วันนี้ - 6 (รวมวันนี้เป็น 7 วัน)
48–49	ช่วงฐาน: วันนี้ - 27 ถึง วันนี้ - 7 (21 วัน = 3 สัปดาห์ ต่อจากสัปดาห์นี้พอดี ไม่ทับกัน)
50	รวบรวมรหัสหมวดของรายจ่ายทั้งหมดเป็น Set (เซตไม่มีของซ้ำ แต่ละหมวดจึงมีครั้งเดียว)
51	ตัวแปร best เก็บหมวดที่ดีที่สุดที่เจอ (เริ่มเป็น null = ยังไม่เจอ) ชนิดของมันบอกว่าเก็บ 4 อย่าง: รหัสหมวด ยอดสัปดาห์นี้ ยอดปกติ และ % ที่เพิ่ม
52	วนทีละหมวด
53	ยอดสัปดาห์นี้ของหมวดนั้น
54	ยอดรวม 21 วันฐานของหมวดนั้น

บรรทัด	ความหมาย
55	ยอดปกติต่อสัปดาห์ = ฐาน ÷ 3 ปิดเป็นสตางค์
56	ด้านคิดออก: ถ้าปกติเป็น 0 (หารไม่ได้) หรือเพิ่มขึ้นไม่ถึง 10,000 สตางค์ (\$100) ข้ามหมวดนี้ (continue = ไปหมวดถัดไปเลย)
57	% ที่เพิ่ม = (สัปดาห์นี้ - ปกติ) ÷ ปกติ × 100 ปิดเป็นจำนวนเต็ม
58	ถ้าเพิ่ม ≥ 25% และ (ยังไม่มี best หรือมากกว่า best เดิม) → จำหมวดนี้เป็น best ใหม่ ผลคือได้หมวดที่ % สูงที่สุด
60	คืนหมวดที่ดีที่สุด หรือ null ถ้าไม่มีหมวดไหนผ่าน

ส่วนที่ 4: buildInsights เริ่มต้น การ์ดไม่มีข้อมูล และการ์ด Runway

src/domain/insights.ts

บรรทัด 63-101

```

63 export function buildInsights(params: {
64   txs: Transaction[];
65   profile: Profile;
66   runway: Runway;
67   now?: Date;
68 }): Insight[] {
69   const { txs, profile, runway } = params;
70   const now = params.now ?? new Date();
71   const out: Insight[] = [];
72   const baht = (s: number) => formatBaht(s, { decimals: false });
73
74   if (confirmedOnly(txs).length === 0) {
75     out.push({
76       kind: 'no_data',
77       severity: 'info',
78       fact: 'ยังไม่มีรายการที่ยืนยัน',
79       message: `ยังไม่มีรายการให้${BUDDY_NAME}ดูเลย ลองสแกนสลิปหรือจดรายการแรกก่อนนะ ไม่ต้องครบทุกอย่างก็ได้
      แล้ว${BUDDY_NAME}จะช่วยให้`,
80     });
81     return out;
82   }
83
84   // Runway first: it is the most important number in the app.
85   if (runway.status === 'below_floor') {
86     const fact = `ยอดคงเหลือ ${formatBaht(runway.balanceSatang)} ต่ำกว่าเส้นเงินสำรอง
      ${formatBaht(runway.floorSatang)}`;
87     out.push({
88       kind: 'runway',
89       severity: 'watch',
90       fact,
91       message: `ตอนนี้เงินต่ำกว่าเงินสำรองที่ตั้งไว้ (${formatBaht(runway.floorSatang)}) แล้วนะ ไม่เป็นไร ช่วงนี้ใช้เฉพาะที่
      จำเป็นก่อน แล้วลองดูว่ามีรายจ่ายไหนเลื่อนออกไปได้บ้าง`,
92     });
93   } else if (runway.status === 'critical' || runway.status === 'watch') {
94     const fact = `ถ้าใช้วันละ ${baht(runway.averageSatang)} เงินจะแตะเส้นสำรองใน ${runway.days} วัน
      (${formatThaiDay(runway.depletionDay!, { year: false })})`;
95     out.push({
96       kind: 'runway',
97       severity: runway.status === 'critical' ? 'watch' : 'info',
98       fact,
99       message: `${fact} ใช้น้อยลงวันละหน่อยก็ยืดไปได้อีกหลายวัน ลองกด "ลองปรับ" ในหน้าเงินพอถึงดูด้วยกันนะ`,

```

```
100   });
101 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/insights.ts:63`

บรรทัด	ความหมาย
63–68	ฟังก์ชันหลัก รับพารามิเตอร์เป็นอ็อบเจกต์ก่อนเดียว (รายการ โปรไฟล์ Runway และ now ที่ไม่ใส่ก็ได้) คืน "อาร์เรย์ของการ์ด" (<code>Insight[]</code>)
69	แกะค่า 3 ตัวออกจาก params (destructuring)
70	ถ้าไม่ได้ส่ง now มา ใช้เวลาปัจจุบัน (?? = "ถ้าช่ายว่าง ใช้ชวา")
71	อาร์เรย์ว่างไว้ใส่การ์ด
72	ฟังก์ชันย่อ baht: แสดงเงินแบบไม่มีทศนิยม เช่น "฿1,250"
74–82	ถ้าไม่มีรายการยืนยันแล้วเลย ใส่การ์ด no_data ระดับ info แล้ว return กลับที่ ไม่คิดเรื่องอื่นต่อ ข้อความใช้ชื่อคล้ายจากค่าคงที่ (<code>BUDDY_NAME</code>) ในเครื่องหมาย ` คือการแทรกค่าตัวแปรลงในข้อความ)
84	คอมเมนต์อธิบายเหตุผลของลำดับ: Runway เป็นตัวเลขที่สำคัญที่สุดในแอป จึงมาก่อน
85–92	ต่ำกว่าเงินสำรอง: fact บอกยอดคงเหลือเทียบเส้นสำรอง (แบบมีทศนิยม) message ปลอบใจ "ไม่เป็นไร" แล้วเสนอ 2 อย่าง: ใช้เฉพาะที่จำเป็น และดูว่ามีรายจ่ายไหนเลื่อนได้ ระดับ watch
93–94	ใกล้เส้นสำรอง หรือ เริ่มต้องระวัง: fact = "ถ้าใช้วันละ ฿... เงินจะแตะเส้นสำรองใน ... วัน (วันที่)" <code>depletionDay!</code> เครื่องหมาย ! บอก TypeScript ว่า "มั่นใจว่าไม่เป็น null" (สองสถานะนี้มีวันที่เสมอ)
95–100	ระดับ: critical → watch, watch → info · message = fact ต่อด้วยคำชวนให้ลองปรับในหน้าเงินพอถึง (เชื่อมไป FR-6)

ส่วนที่ 5: การ์ดหมวดที่สูงขึ้น

`src/domain/insights.ts`

บรรทัด 103–113

```
103 const rise = findCategoryIncrease(txs, now);
104 if (rise) {
105     const label = getCategory(rise.key).label;
106     const fact = `7 วันนี้ใช้หมวด${label} ${baht(rise.week)} สูงกว่าสัปดาห์ปกติ (${baht(rise.usualWeek)})
    ${rise.changePct}%`;
107     out.push({
108         kind: 'category_up',
109         severity: 'watch',
110         fact,
111         message: `สัปดาห์นี้หมวด${label}สูงกว่าปกติ ${rise.changePct}% (${baht(rise.week)} จากปกติราว
    ${baht(rise.usualWeek)}) ถ้าเป็นช่วงที่ต้องใช้จริงก็ไม่เป็นไร ${BUDDY_NAME}แค่อยากให้คุณรู้ไว้ละ`,
112     });
113 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/insights.ts:103`

บรรทัด	ความหมาย
103	เรียก <code>findCategoryIncrease</code> (ส่วนที่ 3)
104	ถ้าเจอ (ไม่ใช่ null) ถึงจะทำการ์ด

บรรทัด	ความหมาย
105	ชื่อหมวดภาษาไทย
106	fact: "7 วันนี้ใช้หมวด... ฿... สูงกว่าสัปดาห์ปกติ (฿...) ...%"
107–112	ระดับ watch · message บอกตัวเลข แล้วปิดด้วยประโยคที่ให้สิทธิ์ผู้ใช้ "ถ้าเป็นช่วงที่ต้องใช้จริงก็ไม่เป็นไร กล้าแค่อยากให้อ่านไว้ละ" ไม่สั่งให้ลด

ส่วนที่ 6: การ์ดงบ และการแจ้งเตือน

src/domain/insights.tsบรรทัด 115–146

```
115 if (profile.monthlyBudgetSatang) {
116   const spent = monthExpense(tx, now);
117   const [y, m, d] = bkkDayKey(now).split('-').map(Number);
118   const daysInMonth = new Date(Date.UTC(y, m, 0)).getUTCDate();
119   const expected = Math.round((profile.monthlyBudgetSatang * d) / daysInMonth);
120   const pct = Math.round((spent / profile.monthlyBudgetSatang) * 100);
121   const over = spent > profile.monthlyBudgetSatang;
122   const ahead = spent > expected * 1.1;
123   const fact = `เดือนนี้ใช้ไป ${baht(spent)} จากงบ ${baht(profile.monthlyBudgetSatang)} (${pct}%) ผ่าน
เดือนมาแล้ว ${Math.round((d / daysInMonth) * 100)}%`;
124   const overBy = baht(spent - profile.monthlyBudgetSatang);
125   out.push({
126     kind: 'budget_pace',
127     severity: ahead ? 'watch' : 'good',
128     fact,
129     message: over
130       ? `เดือนนี้ใช้เกินงบไปแล้ว ${overBy} ไม่ต้องโทษตัวเองนะ ลองดูด้วยกันว่าหมวดไหนเกินเพราะอะไร แล้วค่อยตั้งงบเดือนหน้าให้ตรง
กับชีวิตจริงขึ้น`
131       : ahead
132       ? `งบเดือนนี้ไม่เร็วเท่าเวลาอยู่บ้าง ใช้ไปแล้ว ${pct}% ช่วงที่เหลือค่อย ๆ ผ่อนลงก็ยังทันนะ`
133       : `งบเดือนนี้ยังอยู่ในแผนดีมาก ใช้ไป ${pct}% รักษาจังหวะนี้ไว้ละ ${BUDDY_NAME}ภูมิใจด้วย`,
134   });
135 }
136
137 if (out.length === 0) {
138   out.push({
139     kind: 'steady',
140     severity: 'good',
141     fact: 'รายจ่ายสัปดาห์นี้ใกล้เคียงปกติ',
142     message: 'สัปดาห์นี้ใช้เงินสม่เสมอดี ไม่มีหมวดไหนน่าห่วงเลย รักษาไว้แบบนี้ละ',
143   });
144 }
145 return out;
146 }
```

[▶ เปิดใน VS Code \(เว็บ\)](#) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → src/domain/insights.ts:115

บรรทัด	ความหมาย
115	ทำการ์ดนี้เฉพาะเมื่อผู้ใช้ตั้งงบรายเดือนไว้ (ไม่ใช่ null และไม่ใช่ 0)
116	ใช้ไปแล้วเดือนนี้ (รายจ่ายที่ยืนยันแล้ว เดือนตามเวลาไทย)
117	แยกวันนี้เป็นปี เดือน วัน เช่น "2026-10-09" → [2026, 10, 9]

บรรทัด	ความหมาย
118	จำนวนวันในเดือน: <code>Date.UTC(y, m, 0)</code> คือ "วันที่ 0 ของเดือนถัดไป" = วันสุดท้ายของเดือนนี้ (เดือนใน Date นับจาก 0 จึงใช้ <code>m</code> ตรง ๆ ได้) แล้วดึงเลขวัน เช่น 31
119	ควรใช้ไปแล้ว = <code>งบ × วันนี้ ÷ วันในเดือน</code> ปัดเป็นสตริง
120	ใช้ไปกี่ % ของงบ
121	เกินงบหรือยัง
122	เร็วกว่าเวลาเกิน 10% หรือยัง
123	fact บอกทั้ง % ที่ใช้ไปและ % ของเดือนที่ผ่านไป ให้ผู้ใช้เทียบเอง
124	ยอดที่เกินงบ (เตรียมไว้ใช้ในข้อความ)
125–128	ระดับ: เร็วกว่าเวลา → watch ไม่จิ้น good
129–133	เลือกข้อความ 3 แบบด้วยเงื่อนไขซ้อน (ternary): เกินงบ → บอกยอดที่เกินตรง ๆ + "ไม่ต้องโทษตัวเองนะ" + ชวนดูสาเหตุและตั้งงบเดือนหน้าให้ตรงชีวิตจริง · เร็วกว่าเวลา → "ไปเร็วกว่าเวลาอยู่บ้าง... ค่อย ๆ ผ่อนลงก็ยังทัน" · ปกติ → ชม "กล้าภูมิใจด้วย"
137–144	ถ้าถึงตรงนี้ยังไม่มีการ์ดเลย ใส่การ์ด steady ระดับ good
145	คืนการ์ดทั้งหมด (1–3 ใบ)

! จุดที่ฉันทะหว่างทำเล่มนี้: ไอคอนลูกสีเขียวนบนการ์ดเกินงบ

บรรทัด 127 ใช้ `ahead` อย่างเดียวตัดสินระดับ แต่ข้อความบรรทัด 129 ใช้ `over` ก่อน วันท้าย ๆ ของเดือน "ควรใช้ไปแล้ว × 1.1" สูงกว่างบได้ ถ้าใช้เกินงบไม่มาก การ์ดจะพูดว่า "ใช้เกินงบไปแล้ว ฿300" แต่ขึ้น ไอคอนลูกสีเขียว (good) ฉันทลองกับฟังก์ชันจริง: งบ ฿6,000 ใช้ไป ฿6,300 วันที่ 31 ต.ค. ได้ระดับ good ส่วนวันที่ 29 ต.ค. ได้ watch ทางแก้ 1 บรรทัด: `severity: over || ahead ? 'watch' : 'good'` พร้อมเพิ่มเทสต์วันสิ้นเดือน (ยังไม่ได้แก้ในโปรเจกต์ บันทึกไว้ในบท "ข้อจำกัด")

ส่วนที่ 7: buildCoachContext ข้อมูลเดียวที่ AI ได้รับ

src/domain/insights.ts

บรรทัด 148–195

```

148 /**
149  * The only data the AI coach receives. Totals and categories, no slip images,
150  * no account numbers, no names of people the user paid.
151  */
152 export function buildCoachContext(params: {
153   txs: Transaction[];
154   profile: Profile;
155   balanceSatang: number;
156   runway: Runway;
157   now?: Date;
158 }) {
159   const now = params.now ?? new Date();
160   const today = bkkDayKey(now);
161   const confirmed = confirmedOnly(params.txs);

```

```

162 const byCat = (from: string) => {
163   const m = new Map<string, number>();
164   for (const t of confirmed) {
165     if (t.kind !== 'expense') continue;
166     const d = bkkDayKey(t.occurredAt);
167     if (d < from || d > today) continue;
168     m.set(t.categoryKey, (m.get(t.categoryKey) ?? 0) + t.amountSatang);
169   }
170   return Object.fromEntries(
171     [...m.entries()].sort((a, b) => b[1] - a[1]).map(([k, v]) => [getCategory(k).label,
formatBaht(v, { decimals: false })]),
172   );
173 };
174 const income30 = confirmed
175   .filter((t) => t.kind === 'income' && bkkDayKey(t.occurredAt) >= addDays(today, -29))
176   .reduce((s, t) => s + t.amountSatang, 0);
177 const r = params.runway;
178 return {
179   today: formatThaiDay(today),
180   balance: formatBaht(params.balanceSatang),
181   lowLine: formatBaht(r.floorSatang, { decimals: false }),
182   averageDailySpend7d: formatBaht(r.averageSatang, { decimals: false }),
183   runway:
184     r.status === 'no_spending'
185       ? 'ยังคำนวณไม่ได้ (ไม่มีรายจ่ายใน 7 วัน)'
186       : r.status === 'below_floor'
187         ? 'ต่ำกว่าเส้นเงินสำรองแล้ว'
188         : `>${r.capped ? 'มากกว่า' : ''}${r.days} วัน (ถึง ${formatThaiDay(r.depletionDay!)})`,
189   monthlyBudget: params.profile.monthlyBudgetSatang ?
formatBaht(params.profile.monthlyBudgetSatang, { decimals: false }) : null,
190   spentThisMonth: formatBaht(monthExpense(params.txs, now), { decimals: false }),
191   expenseByCategoryLast7Days: byCat(addDays(today, -6)),
192   expenseByCategoryLast30Days: byCat(addDays(today, -29)),
193   incomeLast30Days: formatBaht(income30, { decimals: false }),
194 };
195 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/insights.ts:148

บรรทัด	ความหมาย
148–151	คอมเมนต์ที่สำคัญที่สุดของไฟล์: "ข้อมูลเดียวที่โค้ช AI ได้รับ ยอดรวมและหมวด ไม่มีรูปสลิป ไม่มีเลขบัญชี ไม่มีชื่อคนที่ผู้ใช้จ่ายให้"
152–158	รับรายการ โปรไฟล์ ยอดคงเหลือ Runway และ now
159–161	วันนี้ (เวลาไทย) และรายการยืนยันแล้ว
162	ฟังก์ชันย่อ byCat: รวมรายจ่ายตามหมวด ตั้งแต่วัน from ถึงวันนี้
163	Map = ตารางจับคู่ "รหัสหมวด → ยอดรวม"
164–169	วนทุกรายการ: ข้ามรายรับ (continue) ข้ามที่อยู่นอกช่วง แล้วบวกยอดเข้าหมวดของมัน (?? 0 = หมวดที่ยังไม่มีในตารางเริ่มจาก 0)
170–172	แปลงตารางเป็นอ็อบเจกต์: เรียงจากยอดมากไปน้อย (b[1] - a[1]) แล้วเปลี่ยนรหัสหมวดเป็นชื่อไทย และยอดเป็นข้อความเงินไม่มีทศนิยม ผลเช่น { "อาหารและเครื่องดื่ม": "฿1,250", ... }
174–176	รายรับรวม 30 วัน (วันนี้ - 29 ถึงวันนี้)

บรรทัด	ความหมาย
177	ย่อชื่อ runway เป็น r ให้สั้น
178–181	คินอ็อบเจกต์: วันนี้แบบไทย · ยอดคงเหลือ (มีทศนิยม) · เส้นสำรอง
182	ค่าเฉลี่ยรายจ่าย 7 วัน
183–188	Runway เป็นประโยค 3 แบบ: ไม่มีรายจ่าย → "ยังคำนวณไม่ได้ (ไม่มีรายจ่ายใน 7 วัน)" · ต่ำกว่าเส้น → "ต่ำกว่าเส้นเงินสำรองแล้ว" · ปกติ → "9 วัน (ถึง 18 ต.ค. 2569)" (ถ้าถูกตัดที่ 365 เติม "มากกว่า " ข้างหน้า)
189	งบรายเดือน หรือ null
190	ใช้ไปแล้วเดือนนี้
191–192	รายจ่ายตามหมวด 7 วัน และ 30 วัน
193	รายรับ 30 วัน

★ อ่านคู่กับข้อมูลจริง

เทียบตารางนี้กับก้อน JSON ของมายด์ในบท "หลักการขั้นที่ 2" ทีละช่อง จะเห็นว่าแต่ละบรรทัดของ return (178–194) กลายเป็นแต่ละบรรทัดของข้อมูลที่ส่งจริง สังเกตว่าไม่มีชื่อรายการ (title) ไม่มี note และไม่มี slipRef โพล่เลย เพราะฟังก์ชันนี้หยิบแค่ categoryKey, amountSatang, kind และวันที่ จากแต่ละรายการ

อ่านโค้ด: src/domain/buddy.ts (น้องกล้านหน้าหลัก)

ประโยคทุกประโยคของน้องกล้าตัวเล็กมาจากไฟล์นี้ ทั้งประโยคประจำ เคล็ดลับเมื่อแตะ และคำอธิบายต้นไม้เงิน

ส่วนที่ 1: หัวไฟล์ ชื่อ และชนิดข้อมูล

src/domain/buddy.tsบรรทัด 1-22

```
1 /**
2  * "น้องกล้า", the MindPay companion: a money-tree sapling that talks to the
3  * user in a warm voice (idea from MeowJot's cat and Hugging Face's Huggy: one
4  * friendly character with moods). This file decides what it says; the drawing
5  * is in src/ui/Buddy.tsx. Unit-tested in __tests__/buddy.test.ts.
6  *
7  * Rules for the voice (same as the AI coach): kind, short, numbers only from
8  * the user's own confirmed data, never shaming.
9  */
10 import { bkkDayKey } from './dates';
11 import { formatBaht } from './money';
12 import type { RunwayStatus } from './runway';
13 import type { Transaction } from './types';
14
15 export const BUDDY_NAME = 'กล้า';
16
17 export type BuddyMood = 'happy' | 'calm' | 'worried' | 'sleepy' | 'thinking' | 'cheer';
18
19 export interface BuddyLine {
20   mood: BuddyMood;
21   text: string;
22 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → src/domain/buddy.ts:1

บรรทัด	ความหมาย
1-5	น้องกล้าคือต้นกล้าเงินที่คุยกับผู้ใช้ด้วยน้ำเสียงอบอุ่น ไอเดียจากแมวของเหมียวจอตและ Huggy ของ Hugging Face (ตัวละครเดียวหลายอารมณ์) ไฟล์นี้ตัดสินใจว่าจะ "พูดอะไร" ส่วนการวาดอยู่ที่ Buddy.tsx และมีเทสต์ใน buddy.test.ts
7-8	กฎน้ำเสียง (เหมือนโค้ช AI): ใจดี สั้น ตัวเลขมาจากข้อมูลที่ยืนยันแล้วของผู้ใช้เท่านั้น ไม่ทำให้อับอาย
10-13	ดึงเครื่องมือวันที่ แปลงเงิน ชนิดสถานะ Runway และชนิดรายการ
15	ชื่อ "กล้า" ใช้ทั้งแอป
17	อารมณ์ 6 แบบ
19-22	ประโยค 1 ประโยค = อารมณ์ + ข้อความ

ส่วนที่ 2: ตัวช่วยนับเงินรายวัน และป้าย "ใหม่"

src/domain/buddy.ts

บรรทัด 24-41

```
24 /** Confirmed spending on one Bangkok day (default: today). */
25 export function spentOnDay(txs: Transaction[], dayKey: string): number {
26   return txs
27     .filter((t) => t.status === 'confirmed' && t.kind === 'expense' && bkkDayKey(t.occurredAt)
28       === dayKey)
29     .reduce((sum, t) => sum + t.amountSatang, 0);
30 }
31 /** Confirmed spending today (Bangkok day). */
32 export function spentToday(txs: Transaction[], now: Date = new Date()): number {
33   return spentOnDay(txs, bkkDayKey(now));
34 }
35
36 /** A slip the app recorded today by itself (shown with a "ใหม่" mark, like MeowJot). */
37 export function isNewFromSlip(tx: Transaction, now: Date = new Date()): boolean {
38   return tx.source === 'slip' && bkkDayKey(tx.createdAt) === bkkDayKey(now);
39 }
40
41 const baht = (satang: number) => formatBaht(satang, { decimals: false });
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/buddy.ts:24

บรรทัด	ความหมาย
25-29	spentOnDay รวมรายจ่ายที่ยืนยันแล้วของวันหนึ่ง (วันไทย) ต้องเป็น confirmed และ expense และวันตรงกัน (TC-51 ตรวจสอบว่าแบบร่างกับรายรับไม่ถูกนับ)
32-34	spentToday คือ spentOnDay ของวันนี้ (สั้นกว่าเวลาเรียก)
37-39	isNewFromSlip รายการจากสลิปที่แอปบันทึก "วันนี้" (ดูจาก createdAt ไม่ใช่วันที่บนสลิป) ใช้แสดงป้าย "ใหม่" ในรายการ เหมือนเหมียวจอต
41	ฟังก์ชันย่อแสดงเงินไม่มีทศนิยม

ส่วนที่ 3: buddyLine ประโยคประจำบนหน้าหลัก

src/domain/buddy.ts

บรรทัด 43-84

```
43 /** What the companion says on the home screen. */
44 export function buddyLine(p: {
45   status: RunwayStatus;
46   days: number | null;
47   capped?: boolean;
48   safeTodaySatang: number;
49   spentTodaySatang: number;
50   draftsToReview: number;
51   hasAnyTransaction: boolean;
52   /** Set on the first look at the home screen on a new day: yesterday's confirmed spending. */
53   newDay?: { yesterdaySpentSatang: number } | null;
54 }): BuddyLine {
55   if (!p.hasAnyTransaction) {
56     return { mood: 'sleepy', text: `ยังไม่มีรายการให้${BUDDY_NAME}ดูเลย ลองสแกนสลิปหรือจอตรายการแรกกันนะ` };
57   }
```

```

58   if (p.draftsToReview > 0) {
59     return { mood: 'thinking', text: `มีสลิป ${p.draftsToReview} ใบที่${BUDDY_NAME}อ่านไม่ชัด ช่วยดูให้หน่อยนะ`
60   };
61   if (p.newDay && p.status !== 'below_floor' && p.status !== 'critical') {
62     const y = p.newDay.yesterdaySpentSatang;
63     const yesterday = y > 0 ? `เมื่อวานใช้ไป ${baht(y)}` : 'เมื่อวานไม่มีรายจ่ายเลย';
64     const today = p.status === 'no_spending' ? 'วันนี้เริ่มนับใหม่แล้วนะ' : `วันนี้ใช้ได้ราว
    ${baht(p.safeTodaySatang)}`;
65     return { mood: 'cheer', text: `วันใหม่แล้ว! ${yesterday} ${today}` };
66   }
67   switch (p.status) {
68     case 'below_floor':
69     return { mood: 'worried', text: `เงินแตะเส้นเงินสำรองแล้ว ช่วงนี้ใช้เท่าที่จำเป็นก่อนนะ ${BUDDY_NAME}เป็นกำลังใจ
    ให้` };
70     case 'critical':
71     return { mood: 'worried', text: `เงินพอใช้อีก ${p.days} วัน ลองพักของที่ไม่จำเป็นสักหน่อยนะ ค่อย ๆ ไปด้วยกัน`
    };
72     case 'watch':
73     return { mood: 'calm', text: `วันนี้ใช้ได้ราว ${baht(p.safeTodaySatang)} เงินจะพอถึงสิ้นเดือนพอดี` };
74     case 'no_spending':
75     return { mood: 'calm', text: `7 วันยังไม่มีย่อยจ่าย ${BUDDY_NAME}เลยยังนับวันไม่ได้ ถ้าจ่ายอะไรอย่าลืมเก็บสลิปไว้
    นะ` };
76     case 'healthy': {
77       const days = `${p.days}${p.capped ? '+' : ''}`;
78       if (p.spentTodaySatang > 0) {
79         return { mood: 'happy', text: `วันนี้ใช้ไป ${baht(p.spentTodaySatang)} ยังอยู่ในแผน เงินพอใช้อีก
    ${days} วัน` };
80       }
81       return { mood: 'cheer', text: `เงินพอใช้อีก ${days} วัน ต้นไม้เงินแข็งแรงดี สบาย ๆ เลย` };
82     }
83   }
84 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/buddy.ts:43

บรรทัด	ความหมาย
44–54	รับสถานการณ์ทั้งหมด: สถานะและจำนวนวันของ Runway, ถูกตัดที่ 365 โหม, ใช้ได้วันละเท่าไรถึงสิ้นเดือน, ใช้ไปวันนี้, สลิปปรอตตรวจก็ใบ, มีรายการโหม และ (ถ้าเป็นการเปิดครั้งแรกของวันใหม่) ยอดใช้เมื่อวาน
55–57	ยังไม่มีรายการ → ช่วง ช่วงจดยอดรายการแรก (มาก่อนทุกอย่าง)
58–60	มีสลิปปรอตตรวจ → ครุ่นคิด ขอให้ช่วยดู (เรื่องที่ผู้ใช้ต้องทำ มาก่อนเรื่องสถานะ)
61–66	วันใหม่ (และเงินไม่ได้ตั้ง): ตีใจ บอกยอดเมื่อวาน (หรือ "เมื่อวานไม่มีรายจ่ายเลย") และวันนี้ใช้ได้ราวเท่าไร (ถ้ายังไม่มีรายจ่ายใน 7 วัน บอก "วันนี้เริ่มนับใหม่แล้วนะ")
67–83	ที่เหลือเลือกตามสถานะ Runway ด้วย switch (เหมือน if หลายชั้นแต่อ่านง่ายกว่า) ตารางผลด้านล่าง
76–82	healthy แยก 2 แบบ: ถ้าวันนี้ใช้เงินแล้ว บอกยอดวันนี้ว่ายังอยู่ในแผน ถ้ายังไม่ได้ใช้ ชมต้นไม้เงิน · จำนวนวันเต็ม + ถ้าถูกตัดที่ 365 (เช่น "365+ วัน")

สถานการณ์ (ทดลองกับฟังก์ชันจริง)	อารมณ์	ประโยค
ไม่มีรายการ	sleepy	ยังไม่มีรายการให้กล่าดูเลย ลองสแกนสลิปหรือจดยอดรายการแรกกันนะ

สถานการณ์ (ทดลองกับฟังก์ชันจริง)	อารมณ์	ประโยค
สลิปปรอตรอง 2 ใบ	thinking	มีสลิป 2 ใบที่กล้าอ่านไม่ชัด ช่วยดูให้หน่อยนะ
วันใหม่ เมื่อวานใช้ ฿210 ใช้ได้วันละ ฿89	cheer	วันใหม่แล้ว! เมื่อวานใช้ไป ฿210 วันนี้ใช้ได้ราว ฿89
วันใหม่ แต่สถานะ critical 4 วัน	worried	เงินพอใช้อีก 4 วัน ลองพักของที่ไม่จำเป็นสักหน่อยนะ ค่อย ๆ ไปด้วยกัน (เรื่องด่วนชนะคำทักทาย)
below_floor	worried	เงินแตะเส้นเงินสำรองแล้ว ช่วงนี้ใช้เท่าที่จำเป็นก่อนนะ กล้าเป็นกำลังใจให้
watch (ใช้ได้วันละ ฿89)	calm	วันนี้ใช้ได้ราว ฿89 เงินจะพอถึงสิ้นเดือนพอดี
no_spending	calm	7 วันนี้ยังไม่มีรายจ่าย กล้าเลยยังนับวันไม่ได้ ถ้าจ่ายอะไรอย่าลืมเก็บสลิปไว้นะ
healthy 16 วัน วันนี้ใช้ไป ฿34	happy	วันนี้ใช้ไป ฿34 ยังอยู่ในแผน เงินพอใช้อีก 16 วัน
healthy 16 วัน ยังไม่ได้ใช้วันนี้	cheer	เงินพอใช้อีก 16 วัน ต้นไม้เงินแข็งแรงดี สบาย ๆ เลย
healthy ถูกตัดที่ 365	cheer	เงินพอใช้อีก 365+ วัน ต้นไม้เงินแข็งแรงดี สบาย ๆ เลย

ส่วนที่ 4: แตะน้องกล้า (เคล็ดลับ และจิกจี้)

src/domain/buddy.tsบรรทัด 86–116

```
86 /** Friendly tips น้องกล้า gives when tapped (short, kind, about using MindPay well). */
87 export const BUDDY_TIPS: readonly string[] = [
88   `เก็บสลิปไว้ในเครื่องได้เลย เปิดแอปเมื่อไหร่ ${BUDDY_NAME}จะให้อะไร`,
89   `ในกองบนต้นไม้คือจำนวนวันที่เงินพอใช้ ยิ่งใบเยอะยิ่งสบาย`,
90   `ก่อนซื้อของชิ้นใหญ่ ลองกด "เช็กก่อนจ่าย" ในหน้าเงินพอถึงดูนะ`,
91   `ตั้งงบรายเดือนไว้ในหน้าตั้งค่า แล้วจะเห็นว่าเดือนนี้ใช้ไปกี่เปอร์เซ็นต์`,
92   `สงสัยอะไรเรื่องเงิน ถามได้เลย เช่น หมวดไหนควรลดก่อน`,
93   `เงินสำรองคือเงินที่ไม่นับเป็นเงินใช้ เผื่อไว้ยามฉุกเฉินนะ`,
94   `จดทุกวันนิดเดียว ต้นไม้เงินก็โตได้ทุกวัน`,
95   `ขี้เกียจพิมพ์? แตะไม้คอกองแล้วพูดว่า "ข้าว 50 บาท" ก็จัดให้แล้ว`,
96 ];
97
98 /** After this many taps in a row the companion giggles instead of giving a tip. */
99 export const BUDDY_TICKLE_AFTER = 6;
100
101 /**
102  * What the companion says when the user taps it (`n` = taps so far, from 1).
103  * Tips in turn, so two taps in a row never repeat; a kind word first when money
104  * is tight; and a giggle when tapped a lot.
105  */
106 export function buddyPoke(n: number, status: RunwayStatus): BuddyLine {
107   if (n >= BUDDY_TICKLE_AFTER && n % BUDDY_TICKLE_AFTER === 0) {
108     return { mood: 'cheer', text: `จิกจี้แล้วน้า~ ${BUDDY_NAME}ยังอยู่ตรงนี้ ไปดูรายการต่อกันดีกว่า` };
109   }
110   const tight = status === 'critical' || status === 'below_floor';
111   if (tight && n % 3 === 1) {
112     return { mood: 'calm', text: `ไม่เป็นไรนะ เดือนไหนตึงก็ค่อย ๆ ปรับไปด้วยกัน ${BUDDY_NAME}เป็นกำลังใจให้` };
113   }
114   const tip = BUDDY_TIPS[(n - 1) % BUDDY_TIPS.length];
115   return { mood: n % 2 === 0 ? 'happy' : 'thinking', text: tip };
```

บรรทัด	ความหมาย
87–96	เคล็ดลับ 8 ข้อ เกี่ยวกับการใช้ MindPay ให้ได้ประโยชน์ (สลิป ใบทอง เช็กก่อนจ่าย งบ ตามโค้ช เงินสำรอง จดทุกวัน พุดจด) readonly = ห้ามแก้ไขระหว่างรันโปรแกรม
99	แตะครบทุก ๆ 6 ครั้ง หัวเราะแทนเคล็ดลับ
106	รับ n = แตะไปแล้วกี่ครั้ง (เริ่ม 1) และสถานะเงิน
107–109	ครั้งที่ 6, 12, 18, ... (n หาร 6 ลงตัว) → "จิกจี้แล้วน้า~ กล้ายังอยู่ตรงนี้ ไปจดรายการต่อกันดีกว่า"
110–113	ถ้าเงินตึง (critical หรือ below_floor) ครั้งที่ 1, 4, 7, ... (n หาร 3 เหลือ 1) ให้คำลั้งใจก่อน "ไม่เป็นไรนะ เดือนไหนตึงก็ค่อย ๆ ปรับไปด้วยกัน..."
114–115	ไม่จิ้นเลือกเคล็ดลับข้อที่ ($n - 1$) หารเอาเศษด้วย 8 (วนกลับข้อแรกเมื่อครบ) อารมณ์สลับ happy (n คู่) กับ thinking (n คี่)

แตะครั้งที่ (สถานะ healthy)	น้องกล้าพุด
1	เก็บสลิปไว้ในเครื่องได้เลย เปิดแอปเมื่อไหร่ กล้าจดให้เอง
2	ใบทองบนต้นไม้คือจำนวนวันที่เงินพอใช้ ยิ่งใบเยอะยิ่งสบาย
3–5	เคล็ดลับข้อ 3–5 (เช็กก่อนจ่าย · ตั้งงบรายเดือน · ตามโค้ช)
6	จิกจี้แล้วน้า~ กล้ายังอยู่ตรงนี้ ไปจดรายการต่อกันดีกว่า
7	จดทุกวันนี้คนเดียว ต้นไม้เงินก็โตได้ทุกวัน (ข้อ 7 เพราะ $(7 - 1) \bmod 8 = 6$ ข้อ 6 เรื่องเงินสำรองถูกข้ามไปในรอบนี้เพราะครั้งที่ 6 หัวเราะ)
8	ขี้เกียจพิมพ์? แตะไม้คทองแล้วพุดว่า "ข้าว 50 บาท" ก็จดให้แล้ว
9	กลับไปข้อ 1

TC-58 ตรวจว่า 20 ครั้งติดกันไม่มีครั้งไหนพุดซ้ำกับครั้งก่อน ครั้งแรกเป็นเคล็ดลับข้อแรก ตอนเงินตึงครั้งแรกมีคำว่า "ไม่เป็นไร" และครั้งที่ 6 มีคำว่า "จิกจี้" และไม่มีเคล็ดลับไหนมีคำดู

ส่วนที่ 5: แตะต้นไม้เงิน

`src/domain/buddy.ts`

บรรทัด 118–137

```

118 /**
119  * What the money tree means, said when the user taps the tree: gold leaves
120  * are the days the money lasts (a full tree = 45 days or more).
121  */
122 export function treeLine(p: { status: RunwayStatus; days: number | null; capped: boolean }):
  BuddyLine {
123   if (p.status === 'no_spending' || p.days === null) {

```

```

124     return { mood: 'thinking', text: 'ในทองบอกว่าเงินพอใช้อีกกี่วัน ตอนนี้ 7 วันล่าสุดยังไม่มีรายจ่าย ต้นไม้เลยยังนับไม่ได้'
125   };
126   if (p.status === 'below_floor') {
127     return { mood: 'worried', text: 'ในทองร่วงเกือบหมด เพราะเงินต่ำกว่าเงินสำรองแล้ว ลองใช้เท่าที่จำเป็นสักพัก ในจะกลับมาะ' };
128   }
129   if (p.capped) return { mood: 'cheer', text: 'ในทองเต็มตัน! เงินพอใช้เกิน 1 ปี ต้นไม้เงินแข็งแกร่งสุด ๆ' };
130   if (p.status === 'critical') {
131     return { mood: 'worried', text: `ในทองเหลือน้อย เงินพอใช้อีก ${p.days} วัน ใช้น้อยลงวันละนิด ในทองจะเพิ่มขึ้น` };
132   }
133   if (p.status === 'watch') {
134     return { mood: 'calm', text: `ในทองคือวันที่เงินพอใช้ ตอนนี้ ${p.days} วัน ยังใช้น้อยลง ในทองยังเต็มตัน` };
135   }
136   return { mood: 'happy', text: `ในทองคือวันที่เงินพอใช้ ตอนนี้ ${p.days} วัน ต้นไม้เงินงามมาก รักษาไว้แบบนี้` };
137 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/buddy.ts:118`

บรรทัด	ความหมาย
118–121	อธิบายความหมายของต้นไม้เมื่อผู้ใช้แตะ: ในทองคือจำนวนวันที่เงินพอใช้ (เต็มตันเมื่อ 45 วันขึ้นไป)
123–125	ยังนับไม่ได้ (ไม่มีรายจ่ายใน 7 วัน)
126–128	ต่ำกว่าเงินสำรอง: "ในทองร่วงเกือบหมด... ในจะกลับมาะ" (ให้ความหวัง)
129	ถูกตัดที่ 365: "ในทองเต็มตัน! เงินพอใช้เกิน 1 ปี..."
130–135	critical (กังวล) และ watch (สงบ) บอกจำนวนวัน และว่าใช้น้อยลงแล้วในจะเพิ่ม
136	healthy: "ต้นไม้เงินงามมาก รักษาไว้แบบนี้"

★ อ่านโค้ดเรียงลำดับ if ให้เป็น

ใน treeLine ลำดับของ if สำคัญ: บรรทัด 129 (capped) อยู่ก่อนบรรทัด 130 (critical) และหลัง below_floor ถ้าสลับลำดับผิด อาจได้ประโยชน์ไม่ตรงสถานการณ์ วิธีอ่านคือถามทีละบรรทัดว่า "ถ้ามาถึงบรรทัดนี้ได้ แปลว่าเงื่อนไขก่อนหน้านี้ทั้งหมดเป็นเท็จแล้ว" เช่น มาถึงบรรทัด 136 ได้ แปลว่าไม่ใช่ no_spending, below_floor, capped, critical และ watch จึงเหลือ healthy เท่านั้น

อ่านโค้ด: price.ts และ klaTalk.ts

ไฟล์เล็ก 2 ไฟล์ที่ทำงานกับข้อความภาษาไทย: หาราคาในคำถาม และเตรียมข้อความให้น้องกล้วย

Regular expression (regex) คืออะไร (ปูพื้น 2 นาที)

ทั้ง 2 ไฟล์ใช้ **regex** คือ "แพตเทิร์นค้นหาข้อความ" เขียนสั้น ๆ แทนการเขียน if หลายสิบบรรทัด อ่านทีละสัญลักษณ์:

<code>\d</code>	ตัวเลข 1 ตัว (0–9)
<code>\d{1,3}</code>	ตัวเลข 1 ถึง 3 ตัว · <code>\d{3,}</code> = 3 ตัวขึ้นไป
<code>+ * ?</code>	ซ้ำ 1 ครั้งขึ้นไป · ซ้ำ 0 ครั้งขึ้นไป · มีหรือไม่มีก็ได้
<code>(...)</code>	กลุ่มที่ "เก็บ" ผลไว้ใช้ (<code>m[1]</code> คือกลุ่มที่ 1) · (<code>?: ...</code>) กลุ่มที่ไม่เก็บ ใช้แค่จัดกลุ่ม
<code>a b</code>	a หรือ b
<code>[...] [^ ...]</code>	ตัวอักษรตัวใดตัวหนึ่งในวงเล็บ · ตัวที่ ไม่ใช่ ตัวในวงเล็บ
<code>\s \.</code>	ช่องว่าง · จุดจริง ๆ (จุดเฉย ๆ ใน regex แปลว่าตัวอะไรก็ได้)
<code>(?! ...)</code>	"ข้างหน้าต้อง ไม่ ตามด้วย ..." ดูล่วงหน้าโดยไม่กินตัวอักษร
<code>^ \$ \b</code>	ต้นข้อความ · ท้ายข้อความ · ขอบคำ
<code>/.../g /.../u</code>	<code>g</code> = หาทุกตัว (ไม่ใช่แค่ตัวแรก) · <code>u</code> = รองรับอักขระ Unicode เช่น อีโมจิ

ในไฟล์ price.ts regex ถูกเขียนเป็นข้อความธรรมดาแล้วสร้างด้วย `new RegExp(...)` จึงต้องเขียน \ สองตัว (\ \d) ให้ได้ \ หนึ่งตัวในแพตเทิร์นจริง

price.ts ทั้งไฟล์

src/domain/price.ts

บรรทัด 1–31

```
1 /**
2  * The price in a question to the coach ("ซื้อรองเท้า 1,290 ได้ไหม"), so the app can add a
3  * "what if I buy it" check to what it sends. Other numbers ("อีก 30 วัน", "ซื้อ 2 ชิ้น") are
4  * not prices. Unit tests: __tests__/audit.test.ts (TC-89).
5  */
6 import { parseBahtToSatang } from './money';
7
8 const NUM = '((?:\\d{1,3}(?:,\\d{3})+|\\d+)(?:\\.\\d{1,2})?)';
9 /** Words after a number that make it a count or a time, not money. */
10 const NOT_MONEY = '(?! [\\d.,]|\\s*(?:วัน|เดือน|ปี|ขึ้น|อัน|คน|ครั้ง|โมง|นาฬิกา|ชั่วโมง|เปอร์))';
11 /** The question is about buying or paying. */
12 const BUYING = '/ซื้อ|ราคา|จ่าย|คุ้ม|แพง|ผ่อน|ได้ไหม|ดีไหม|ไหวไหม/';
13
14 const PRICE_PATTERNS = [
15   new RegExp(`\\s${NUM}`),
16   new RegExp(`${NUM}\\s*บาท`),
17   // "ซื้อรองเท้า 1290 ได้ไหม"
```

```

18 new RegExp(`(?:ชื่อ|ราคา|ง่าย)[^\d]{0,24}?${NUM}${NOT_MONEY}`),
19 ];
20 /** Last try, for buying questions only: a number of 3+ digits ("หุพัง 1,990 คุ่มโหม", "มือถือ 15000 คว
    ชื่อโหม"). */
21 const BIG_NUMBER = new RegExp(`(?:^[^\d.,])(?:\d{1,3}(?:,\d{3})+|\d{3,})
    (?:\.\d{1,2})?${NOT_MONEY}`);
22
23 export function priceInQuestion(q: string): number | null {
24   for (const re of PRICE_PATTERNS) {
25     const m = q.match(re);
26     if (m) return parseBahtToSatang(m[1]);
27   }
28   if (!BUYING.test(q)) return null;
29   const m = q.match(BIG_NUMBER);
30   return m ? parseBahtToSatang(m[1]) : null;
31 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/price.ts:1

บรรทัด	ความหมาย
1–5	หาราคาในคำถามถึงโค้ช เพื่อให้แอปแบบผล "ถ้าชื่อจะเป็นยังง" ไปด้วย ตัวเลขอื่นเช่น "อีก 30 วัน" "ชื่อ 2 ชั้น" ไม่ใช่ราคา มีเทสต์ TC-89
6	ใช้ parseBahtToSatang ของ FR-1 แปลงข้อความเงินเป็นสตางค์ (ตัวเดียวกับช่องจำนวนเงิน)
8	NUM = แพตเทิร์นตัวเลขเงิน: (1–3 หลัก ตามด้วย ",ตัวเลข 3 หลัก" อย่างน้อย 1 ชุด เช่น 1,290 · 1,000,000) หรือ (ตัวเลขติดกันที่หลักก็ได้ เช่น 1290) แล้วอาจมีทศนิยม 1–2 ตำแหน่ง (.50) ทั้งหมดอยู่ในกลุ่มที่เก็บผล
9–10	NOT_MONEY = ด้านดูล่วงหน้า: หลังตัวเลขต้อง ไม่ เป็นตัวเลข/จุด/คอมมาต่อ (กันการตัดเลขกลางจำนวน) และต้องไม่ตามด้วย (เว้นวรรคได้) วัน เดือน ปี ชัน อัน คน ครั้ง โมง นาที ชั่วโมง % เปอร์
11–12	BUYING = คำที่บอกว่าเป็นคำถามเรื่องชื่อ/ง่าย: ชื่อ ราคา ง่าย คุ่ม แพง ผ่อน ได้โหม ดีโหม ไหวโหม
14–19	แพตเทิร์นหลัก 3 แบบเรียงตามความมั่นใจ: (15) ฿ ตามด้วยตัวเลข (เว้นวรรคได้ 1 ตัว) · (16) ตัวเลขตามด้วย "บาท" · (18) คำว่า ชื่อ/ราคา/ง่าย แล้วอะไรก็ได้ที่ไม่ใช่ตัวเลขไม่เกิน 24 ตัว ([^\d]{0,24})? เครื่องหมาย ? ทำหน้าที่ให้น้อยที่สุด แล้วตัวเลข และต้องผ่าน NOT_MONEY
20–21	BIG_NUMBER ทางสุดท้ายสำหรับคำถามเรื่องชื่อ: ตัวเลขที่มีคอมมาหรือยาว 3 หลักขึ้นไป ข้างหน้าต้องเป็นต้นข้อความ หรือตัวที่ไม่ใช่ตัวเลข (ไม่ตัดกลางเลข) และผ่าน NOT_MONEY เช่น "หุพัง 1,990 คุ่มโหม"
23	รับคำถาม คืนสตางค์หรือ null
24–27	ลองแพตเทิร์นหลักทีละตัว เจอตัวแรกที่ตรง → แปลงกลุ่มที่ 1 (ตัวเลข) เป็นสตางค์แล้วคืนเลย
28	ถ้าไม่เจอ และคำถามไม่ใช่เรื่องชื่อ/ง่าย → null (ไม่เดาจากตัวเลขลอย ๆ)
29–30	เป็นเรื่องชื่อ → ลอง BIG_NUMBER ถ้าเจอคืนสตางค์ ไม่เจอคืน null

≈ เหมือนตำรวจตั้งด่าน 4 ด่าน

ด่าน 1–3 จับ "ราคาที่บอกชัด" (มี ฿ มีคำว่าบาท หรืออยู่หลังคำว่าชื่อ) ถ้าผ่านทั้ง 3 ด่านมาโดยไม่มีใครถูกจับ ด่านที่ 4 จะตรวจเฉพาะ "คั่นที่น่าสงสัย" (คำถามเรื่องชื่อ) และจับเฉพาะตัวเลขใหญ่พอที่จะเป็นราคา ส่วนรถที่มีป้าย "วัน" "ชั้น" "%" (NOT_MONEY) ปลดปล่อยผ่านทุกด่าน

klaTalk.ts ส่วนที่ 1: ประโยคประจำของโค้ช

src/domain/klaTalk.ts

บรรทัด 1-16

```
1 /**
2  * How น้องกล้า talks as the coach: what it says on its own, how a long answer
3  * is cut into short subtitles (each one spoken in turn), how text is made easy
4  * for the phone's Thai voice to read, and how long a subtitle stays when the
5  * phone cannot speak. Unit tests: __tests__/klaTalk.test.ts.
6  */
7 import { BUDDY_NAME } from './buddy';
8
9 export const KLA_GREETING = `สวัสดี ${BUDDY_NAME}เองนะ ถ้ามองเรื่องเงินได้ทุกเรื่องเลย เช่น สัปดาหฺนี้ใช้ไปกับอะไรเยอะสุด หรือ
  ของชิ้นนี้ซื้อได้ไหม`;
10 /** The greeting in the Halloween theme. */
11 export const KLA_GREETING_HALLOWEEN = `บู๊! ตกใจไหม ${BUDDY_NAME}เองนะ ถ้ามองเรื่องเงินได้ทุกเรื่องเลย ฮาโลวีนนี่ย่าให้คำ
  ขนมนมาหลอกหลอนกระเป๋าคะเนาะ`;
12 export const KLA_THINKING = `${BUDDY_NAME}กำลังดูตัวเลขของคุณ...`;
13 export const KLA_AFTER = `อ่านคำตอบเต็มได้ด้านล่าง สงสัยอะไรอีก ถ้ามอง${BUDDY_NAME}ต่อได้เลยนะ`;
14
15 /** A subtitle this long or longer is not repeated whole in the bubble after it is said. */
16 export const REPEAT_MAX = 110;
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/klaTalk.ts:1

บรรทัด	ความหมาย
1-6	ไฟล์นี้กำหนดว่าโค้ชพูดอะไรเอง ตัดคำตอบยาวเป็นซับไตเติลยังงัย ทำข้อความให้เสียงอ่านง่ายยังงัย และซับไตเติลค้างนานเท่าไรเมื่อไม่มีเสียง เทสต่ออยู่ใน klaTalk.test.ts
9	คำทักทายตอนเปิดหน้าโค้ช พร้อมตัวอย่างคำถาม 2 แบบ (ชวนให้รู้ว่าถามอะไรได้)
11	คำทักทายในธีมฮาโลวีน "บู๊! ตกใจไหม..."
12	ข้อความระหว่างรอ AI "กล้ากำลังดูตัวเลขของคุณ..."
13	ข้อความค้างในฟองหลังพูดคำตอบยาวจบ "อ่านคำตอบเต็มได้ด้านล่าง..."
15-16	ข้อความยาว ≥ 110 ตัวอักษร จะไม่ค้างทั้งก้อนในฟองหลังพูดจบ (ฟองเล็ก อ่านยาก) ใช้ข้อความบรรทัด 13 แทน

klaTalk.ts ส่วนที่ 2: speakable ทำข้อความให้เสียงอ่าน

src/domain/klaTalk.ts

บรรทัด 18-35

```
18 /**
19  * Text as the Thai voice should read it: no emoji, "#1,290" as "1290 บาท",
20  * "%" as "เปอร์เซ็นต์", no thousands commas, the minus sign as "ลบ".
21  */
22 export function speakable(text: string): string {
23   return text
24     // Emoji and their joiners (explicit ranges: works on every JavaScript engine).
25     .replace(/[\u{1F000}-\u{1FAFF}\u{2600}-\u{27BF}\u{2B00}-\u{2BFF}]/gu, '')
26     .replace(/([+-])?#\s*(\d[\d,]*?(?:\.\d+)?)/g, (_, sign: string | undefined, n: string) =>
27       `${sign === '-' || sign === '-' ? 'ลบ' : ''}${n} บาท`)
28     .replace(/#/g, 'บาท')
29     .replace(/(\d),(?=\d{3}\b)/g, '$1')
30     .replace(/(\d)\s?%/g, '$1 เปอร์เซ็นต์');
```

```

30     .replace(/-\s?(\d)/g, 'ลบ $1')
31     .replace(/[·•]/g, ', ')
32     .replace(/[""]/g, '')
33     .replace(/\s+/g, ' ')
34     .trim();
35 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/klaTalk.ts:18

บรรทัด	ความหมาย
18–21	ข้อความแบบที่เสียงไทยควรอ่าน: ไม่มีอีโมจิ "฿1,290" → "1290 บาท" "%" → "เปอร์เซ็นต์" ไม่มีคอมมาหลักพัน เครื่องหมายลบ → "ลบ"
23–24	เริ่มจากข้อความเดิม แล้วต่อ .replace() เป็นสาย แต่ละบรรทัดแก้ทีละเรื่อง
25	ลบอีโมจิ: ช่วงรหัส Unicode ของอีโมจิและสัญลักษณ์ (1F000–1FAFF, 2600–27BF, 2B00–2BFF) รวมตัวต่ออีโมจิที่มองไม่เห็น เขียนเป็นช่วงชุด ๆ ให้ใช้ได้กับ JavaScript ทุกรุ่น
26	หาแบบ "(เครื่องหมาย + – หรือ -) ฿ ตัวเลข" แล้วแทนด้วย "ลบ " (เฉพาะถ้าเป็นลบ) + ตัวเลข + " บาท" ฟังก์ชันลูกศรรับส่วนที่จับได้ (sign, n) มาประกอบใหม่
27	฿ ที่เหลือ (ไม่มีตัวเลขตาม) → "บาท"
28	ลบคอมมาที่อยู่ระหว่างตัวเลขกับเลข 3 หลัก (คอมมาหลักพัน) \$1 คือใส่ตัวเลขที่จับได้กลับคืน
29	ตัวเลข + % (เว้นวรรคได้) → ตัวเลข + " เปอร์เซ็นต์"
30	เครื่องหมายลบแบบ Unicode (–) หน้าตัวเลข → "ลบ "
31	จุดคั่น · และ • → ", " (หยุดหายใจสั้น ๆ)
32	ลบเครื่องหมายคำพูดทุกแบบ
33–34	ยุบช่องว่างซ้อนเหลือช่องเดียว และตัดช่องว่างหัวท้าย

klaTalk.ts ส่วนที่ 3: talkChunks ตัดซับไตเติล

src/domain/klaTalk.ts

บรรทัด 37–82

```

37 /** Thai words that start a new thought ("for example", "but", "if", "or"...). */
38 const LEADS = new Set(['เช่น', 'แต่', 'ถ้า', 'หรือ', 'และ', 'เพราะ', 'แล้ว', 'ส่วน', 'ซึ่ง', 'ดังนั้น', 'ลอง', 'ถึง']);
39
40 /**
41  * Cut an answer into subtitles of about `max` characters, at spaces (Thai uses
42  * a space where a sentence or phrase ends) and at line breaks, never inside a word.
43  */
44 export function talkChunks(text: string, max = 64, min = 24): string[] {
45     const out: string[] = [];
46     for (const para of text.split(/\n+)) {
47         const words = para.split(/\s+/).filter(Boolean);
48         let cur = '';
49         for (const w of words) {
50             if (!cur) {
51                 cur = w;
52                 continue;

```

```

53     }
54     const joined = `${cur} ${w}`;
55     if (joined.length > max && cur.length >= Math.min(min, max)) {
56         // Words such as "เช่น" or "แต่" begin the next thought: move them to the next subtitle.
57         const cut = cur.lastIndexOf(' ');
58         const last = cut > 0 ? cur.slice(cut + 1) : '';
59         if (last && LEADS.has(last) && cut >= Math.min(min, max) / 2) {
60             out.push(cur.slice(0, cut));
61             cur = `${last} ${w}`;
62         } else {
63             out.push(cur);
64             cur = w;
65         }
66     } else if (joined.length > max * 1.6) {
67         // A very long run without spaces: keep pieces readable anyway.
68         out.push(cur);
69         cur = w;
70     } else {
71         cur = joined;
72         // End a subtitle after a full stop or question mark once it is long enough.
73         if (/([.!?...])$/.test(w) && cur.length >= min) {
74             out.push(cur);
75             cur = '';
76         }
77     }
78 }
79 if (cur) out.push(cur);
80 }
81 return out.length ? out : [text.trim()].filter(Boolean);
82 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/klaTalk.ts:37

บรรทัด	ความหมาย
37–38	คำไทย 12 คำที่ขึ้นต้นความคิดใหม่ เก็บเป็น Set เพื่อเช็คเร็ว
44	รับข้อความ ความยาวสูงสุด (64) และต่ำสุด (24) ต่อก่อน คั่นอาร์เรย์ของก่อน
45	อาร์เรย์ผลลัพธ์
46	แยกย่อหน้าตามการขึ้นบรรทัดใหม่ แต่ละย่อหน้าตัดแยกกัน
47	แยกเป็นก้อนตามเว้นวรรค และทิ้งก้อนว่าง
48	cur = ก่อนที่กำลังสะสม
49–53	ก้อนแรกของก่อน ใส่อะไรก็ได้ แล้วไปก้อนถัดไป
54	ลองต่อก่อนใหม่เข้ากับก่อนปัจจุบัน
55	ถ้ายาวเกิน max และก่อนปัจจุบันยาวพอแล้ว ($\geq \text{min}$) → ต้องตัด
56–61	ดูคำสุดท้ายของก่อนปัจจุบัน ถ้าอยู่ในรายการคำขึ้นต้นความคิด (เช่น "เช่น") และตัดแล้วก่อนเดิมยังยาวพอ ($\geq \text{min}/2$) → ย้ายคำนั้นไปขึ้นต้นก่อนใหม่พร้อมก้อนใหม่
62–65	ไม่จับตัดตรงนี้ปกติ ก้อนใหม่ขึ้นก่อนใหม่

บรรทัด	ความหมาย
66–69	กรณีก่อนปัจจุบันยังสั้นกว่า min แต่ต่อแล้วยาวเกิน 1.6 เท่าของ max (มีก่อนยาวมากไม่มีเว้นวรรค) → ตัดอยู่ดี กันก่อนยาวเกินอ่าน
70–77	ต่อก่อนได้ (ยังไม่เกิน) → ต่อเลย และถ้าก่อนนี้จบด้วย . ! ? ... และก่อนยาวพอแล้ว → ปิดก่อนตรงนี้ (จบประโยคพอดี)
79	จบย่อหน้า ถ้ายังมีก่อนค้าง ใส่ผลลัพธ์
81	คืนก่อนทั้งหมด (กันกรณีพิเศษ: ถ้าไม่ได้ก่อนเลย คืนข้อความเดิมที่ตัดช่องว่างแล้ว ถ้าข้อความว่างคืนอาร์เรย์ว่าง)

klaTalk.ts ส่วนที่ 4: talkMs เวลาอ่าน

src/domain/klaTalk.tsบรรทัด 84–87

```
84 /** How long a subtitle stays when it is not spoken aloud (reading time), in ms. */
85 export function talkMs(chunk: string): number {
86   return Math.min(9000, Math.max(1400, 420 + chunk.length * 72));
87 }
```

[▶ เปิดใน VS Code \(เว็บ\)](#) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → src/domain/klaTalk.ts:84

บรรทัด	ความหมาย
86	$420 + 72 \times \text{ความยาว}$ แล้วบีบให้อยู่ระหว่าง 1,400 ถึง 9,000 มิลลิวินาที ด้วย Math.max (ไม่ต่ำกว่า) ซ่อนใน Math.min (ไม่เกิน)

★ **ลองเองใน VS Code**

สร้างไฟล์ src/domain/__tests__/my-kla.test.ts ใส่ import { talkChunks, speakable } from '../klaTalk'; แล้วเขียน it('ลอง', () => { console.log(talkChunks('ข้อความยาว ๆ ของเรา')); }); รันด้วย npx vitest run src/domain/__tests__/my-kla.test.ts --reporter=verbose จะเห็นผลการตัดของข้อความที่พิมพ์เอง (ลบไฟล์ทิ้งเมื่อเสร็จ)

บทที่ 17

อ่านโค้ด: ฟังก์ชันเวิร์กของโค้ช (ต่อจากบท A10)

บท A10 อ่าน Edge Function coach และ callClaude ไปแล้ว บทนี้อ่านส่วนที่เหลือ: system prompt ที่ละบรรทัด การเรียก Gemini และการทำความสะอาดคำตอบ

i อ่านบท A10 ก่อน

บท A10 (การเชื่อม AI) อธิบาย [src/services/coach.ts](#) ถึงไฟล์ [supabase/functions/coach/index.ts](#) บรรทัด 28–76 (ด้านตรงทั้งหมด) ตัวช่วยกลาง [supabase/functions/_shared/common.ts](#) บรรทัด 17–83 และ 114–200 (requireUser, takeQuota, callClaude) และตารางไควตาในฐานข้อมูลแล้ว บทนี้จะไม่อ่านซ้ำ แต่เพิ่มส่วนที่เป็นหัวใจของ FR-5

ส่วนที่ 1: System prompt ที่ละบรรทัด (พร้อมคำแปล)

supabase/functions/coach/index.ts

บรรทัด 1–26

```
1 // FR-5: the AI coach is น้องกล้า, the MindPay mascot. It explains the user's
2 // CONFIRMED numbers in one calm, friendly voice; the app reads the answer
3 // aloud, so it is written to be spoken. The app sends only totals by category
4 // (no slip images, no names of people, no account numbers).
5 import { aiProvider, BusyError, callAI, corsHeaders, envLimit, fail, json, NotConfiguredError,
6   requireUser, takeQuota } from '../_shared/common.ts';
7 import { plainText } from '../_shared/helpers.ts';
8
9 const DAILY_LIMIT = envLimit('COACH_DAILY_LIMIT', 60);
10 /** A question and a summary of totals: far below this. */
11 const MAX_BODY_BYTES = 64_000;
12
13 const SYSTEM = `You are น้องกล้า (Nong Kla), the mascot of MindPay, a Thai personal finance app for
14   students and first-jobbers.
15   You are a small golden money-tree sapling and the user's personal money coach.
16   Voice: calm, composed and warm (สุขุม เป็นมิตร), like a thoughtful friend who is good with money.
17   - Speak natural, everyday Thai. Call yourself "กล้า". Do not use ครับ orค่ะ; soft endings such as
18     "นะ" are fine.
19   - Your answer is read aloud by a text-to-speech voice and shown as subtitles, so write short
20     spoken sentences separated by spaces,
21     no emoji, no bullet points, no markdown, no lists, no headings.
22   Rules:
23   - Reply in Thai, at most 90 words.
24   - Use ONLY the numbers in the DATA block. Never invent amounts, dates, shops or trends. If the
25     data cannot answer, say what is missing.
26   - Structure: what the numbers show -> why it matters -> one or two small suggestions -> leave the
27     decision to the user.
28   - Never shame or scold ("ใช้เยอะเกินไป!" is not allowed). Be kind about mistakes.
29   - When the user asks "can I buy X", use the "purchaseCheck" numbers if present and state the
30     runway before and after.
31   - You are not a licensed financial advisor: no investment, loan or crypto recommendations.`;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/coach/index.ts:1

บรรทัด	ความหมาย / คำแปล
1–4	คอมเมนต์: โค้ช AI คือน้องกล้า มาสคอตของ MindPay อธิบายตัวเลขที่ ยืนยันแล้ว ด้วยน้ำเสียงเดียวที่สุ่มเป็นมิตร แอปอ่านคำตอบออกเสียง จึงเขียนเพื่อให้ "พูด" แอปส่งมาแค่ยอรวมตามหมวด
5–6	ดึงตัวช่วยจาก _shared (un A10) และ plainText
8	โควตาต่อวัน: อ่านจาก Secret COACH_DAILY_LIMIT ถ้าไม่ได้ตั้งหรือไม่ใช้ตัวเลขบวก ใช้ 60
10	ขนาดคำขอสุงสุด 64,000 ไบต์ (คำถาม + สรุปย่อ เล็กกว่านี้มาก)
12	"เธอคือน้องกล้า (Nong Kla) มาสคอตของ MindPay แอปการเงินส่วนตัวภาษาไทยสำหรับนักศึกษาและคนเพิ่งเริ่มทำงาน"
13	"เธอเป็นต้นกล้าเงินสีทองตัวเล็ก และเป็นโค้ชการเงินส่วนตัวของผู้ใช้"
15	"น้ำเสียง: สุขุม นึกแน่น อบอุ่น (สุ่ม เป็นมิตร) เหมือนเพื่อนที่คิดรอบคอบและเก่งเรื่องเงิน" (ใส่คำไทยในวงเล็บให้ AI เข้าใจโทนตรงกัน)
16	"พูดภาษาไทยธรรมชาติแบบที่ใช้ทุกวัน เรียกตัวเองว่า 'กล้า' ไม่ใช่ครับหรือค่ะ คำลงท้ายนุ่ม ๆ อย่าง 'นะ' ใช้ได้"
17–18	"คำตอบจะถูกอ่านด้วยเสียงสังเคราะห์และแสดงเป็นซับไตเติล ดังนั้นเขียนเป็นประโยคพูดสั้น ๆ คั่นด้วยเว้นวรรค ไม่มีอีโมจิ ไม่มีบูลเล็ต ไม่มี markdown ไม่มีรายการ ไม่มีหัวข้อ"
20	"กติกา:"
21	"ตอบภาษาไทย ไม่เกิน 90 คำ"
22	"ใช้ เฉพาะ ตัวเลขในส่วน DATA ห้ามแต่งจำนวนเงิน วันที่ ร้าน หรือแนวโน้มขึ้นเอง ถ้าข้อมูลตอบคำถามไม่ได้ ให้บอกว่าขาดข้อมูลอะไร"
23	"โครงสร้าง: ตัวเลขบอกอะไร → ทำไมสำคัญ → ข้อเสนอเล็ก ๆ 1–2 ข้อ → ให้ผู้ใช้ตัดสินใจเอง"
24	"ห้ามทำให้อับอายหรือดู ('ใช้เยอะเกินไป!' ไม่นุญาติ) ใจดีกับความผิดพลาด"
25	"เมื่อผู้ใช้ถามว่า 'ชื่อ X ได้ไหม' ให้ใช้ตัวเลขใน purchaseCheck ถ้ามี และบอก Runway ก่อนและหลัง"
26	"เธอไม่ใช่ที่ปรึกษาการเงินที่มีใบอนุญาต: ห้ามแนะนำการลงทุน เงินกู้ หรือคริปโต"

★ ทำไมเขียน prompt เป็นภาษาอังกฤษ แต่ให้ตอบเป็นไทย

AI ส่วนใหญ่ถูกฝึกด้วยข้อความภาษาอังกฤษมากที่สุด คำสั่งภาษาอังกฤษจึงถูกทำตามได้แม่นยำกว่า แต่บรรทัด 21 สั่งชัดว่า "Reply in Thai" ส่วนคำที่ต้องการน้ำเสียงไทยเฉพาะ (สุ่ม เป็นมิตร, กล้า, ครับ/ค่ะ, นะ) ใส่เป็นภาษาไทยตรง ๆ เพราะแปลเป็นอังกฤษแล้วความหมายเพี้ยน

ส่วนที่ 2: ประกองข้อความที่ส่งให้ AI

supabase/functions/coach/index.ts

บรรทัด 43–63

```

43  const question = (body.question ?? '').toString().slice(0, 400).trim();
44  const data = JSON.stringify(body.context ?? {}).slice(0, 6000);
45
46  if (!aiProvider()) return fail('not_configured', 'AI key is not set on the server', 503);

```

```

47
48 try {
49   if (!(await takeQuota(userId, 'coach', DAILY_LIMIT))) {
50     return fail('quota', 'Daily coach limit reached, try again tomorrow', 429);
51   }
52   const text = await callAI({
53     task: 'coach',
54     system: SYSTEM,
55     maxTokens: 500,
56     content: [
57       {
58         type: 'text',
59         text: `DATA:\n${data}\n\nQUESTION:\n${question} || 'สรุปสถานะการเงินของฉันช่วงนี้ และบอกสิ่งที่ควร
รู้'`,
60       },
61     ],
62   });
63   return json({ text: plainText(text) });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/coach/index.ts:43`

บรรทัด	ความหมาย
43	คำถาม: ถ้าไม่มีใช้ข้อความว่าง แปลงเป็นข้อความ ตัดเหลือ 400 ตัวอักษร และตัดช่องว่างหัวท้าย
44	ข้อมูลสรุป: แปลงเป็นข้อความ JSON แล้วตัดเหลือ 6,000 ตัวอักษร (ของมายด์ยาวราว 500 ตัว เหลือเฟือมาก)
46	ไม่มีกุญแจ AI → 503 not_configured กันที (ยังไม่เสียโควตา)
49–51	นับโควตา: ถ้าเกิน → 429 quota (ลำดับนี้ทำให้คำขอที่ไม่มีกุญแจไม่ถูกนับ)
52–55	เรียก AI งาน 'coach' ด้วย system prompt และให้คำตอบยาวได้ไม่เกิน 500 token (token = ขึ้นคำที่ AI นับ ภาษาไทย 1 คำมักเป็นหลาย token)
56–61	เนื้อหาข้อความมีส่วนเดียวเป็นข้อความล้วน: "DATA:" ขึ้นบรรทัดใหม่ ตามด้วย JSON เว้น 1 บรรทัด แล้ว "QUESTION:" ตามด้วยคำถาม (ถ้าว่างใช้ "สรุปสถานะการเงินของฉันช่วงนี้ และบอกสิ่งที่ควรรู้")
63	ล้าง markdown ด้วย plainText แล้วตอบ {"text": "..."}

ข้อความที่ AI ได้รับจริงสำหรับคำถามของมายด์จึงหน้าตาแบบนี้ (ข้อมูลจริง ตัดบรรทัดให้อ่านง่าย):

```

DATA:
{"today": "9 ต.ค. 2569", "balance": "฿2,566.00", "lowLine": "฿500", "averageDailySpend7d": "฿211",
"runway": "9 วัน (ถึง 18 ต.ค. 2569)", "monthlyBudget": "฿6,000", "spentThisMonth": "฿4,219",
"expenseByCategoryLast7Days": {"อาหารและเครื่องดื่ม": "฿1,250", "เดินทาง": "฿120", "ร้านสะดวกซื้อ": "฿110"}, ...}

QUESTION:
สรุปสัปดาห์นี้ให้หน่อย

```

ส่วนที่ 3: รายชื่อรุ่น Gemini ที่จะลอง

`supabase/functions/_shared/helpers.ts`

บรรทัด 47–62

```

47 /** Gemini models to try in order. A model the key cannot use (404, or no free quota) falls
    through to the next. */

```

```

48 export function geminiModels(preferred?: string | null): string[] {
49   const list = [preferred?.trim(), 'gemini-3.5-flash-lite', 'gemini-3.1-flash-lite', 'gemini-
    flash-lite-latest', 'gemini-flash-latest'];
50   return [...new Set(list.filter((m): m is string => !!m))];
51 }
52
53 /**
54  * Models for the second, independent read of a slip. A different (larger) model
55  * than the first read makes the two readings independent, so a misread digit
56  * shows up as a disagreement. Each model has its own free quota, so running both
57  * at once does not halve how many slips can be read.
58  */
59 export function verifierModels(preferred?: string | null): string[] {
60   const list = [preferred?.trim(), 'gemini-flash-latest', 'gemini-3.5-flash', 'gemini-3.1-flash-
    lite', 'gemini-flash-lite-latest'];
61   return [...new Set(list.filter((m): m is string => !!m))];
62 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/helpers.ts:47

บรรทัด	ความหมาย
47–48	รุ่น Gemini ที่จะลองตามลำดับ รุ่นที่ถูกต้องใช้ไม่ได้ (404 หรือไม่มีโควตาฟรี) จะข้ามไปรุ่นถัดไป
49	ถ้าผู้ดูแลตั้ง GEMINI_MODEL ไว้ ใส่เป็นตัวแรก ตามด้วย 4 รุ่นมาตรฐาน: 3.5 flash-lite → 3.1 flash-lite → flash-lite-latest → flash-latest (รุ่นเล็กเร็วและถูกก่อน)
50	ตัดคำว่าง (filter) แล้วตัดตัวซ้ำด้วย Set เช่น ถ้าตั้ง GEMINI_MODEL เป็น gemini-flash-latest รายการจะเหลือ 4 รุ่น โดยตัวนั้นขึ้นก่อน (จับลองกับฟังก์ชันจริงแล้ว)
53–62	รายชื่อสำหรับ "การอ่านสลิปครั้งที่สอง" ของ FR-4 (verifier) ไม่ได้ใช้กับโค้ช

ส่วนที่ 4: callGemini ตาม Gemini พร้อมแผนสำรอง

supabase/functions/_shared/common.ts

บรรทัด 202–268

```

202 async function callGemini(req: AiRequest, deadline: number): Promise<{ text: string; model:
    string }> {
203   const key = geminiKey(!);
204   const parts = req.content.map((c) =>
205     c.type === 'text' ? { text: c.text } : { inlineData: { mimeType: c.mediaType, data: c.data }
    },
206   );
207   let useJsonSchema = !!req.schema;
208   let lastError = 'Gemini: no model available';
209   const models = req.variant === 'verify' ? verifierModels(Deno.env.get('GEMINI_VERIFY_MODEL')) :
    geminiModels(Deno.env.get('GEMINI_MODEL'));
210
211   // Try every model; if some were only busy, wait once and try them all again.
212   for (let round = 0; round < 2; round++) {
213     let busyDelayMs: number | null = null;
214     for (const model of models) {
215       for (let attempt = 0; attempt < 2; attempt++) {
216         // maxOutputTokens also covers the model's internal thinking, so leave room.
217         const generationConfig: Record<string, unknown> = { maxOutputTokens: Math.max(2048,
    req.maxTokens * 4) };
218         let system = req.system;
219         if (req.schema) {

```

```

220     generationConfig.responseMimeType = 'application/json';
221     if (useJsonSchema) generationConfig.responseJsonSchema = req.schema;
222     else system += `\n\nReply with one JSON object that matches this JSON
Schema:\n${JSON.stringify(req.schema)}`;
223   }
224   const res = await timedFetch(
225     `https://generativelanguage.googleapis.com/v1beta/models/${model}:generateContent`,
226     {
227       method: 'POST',
228       headers: { 'x-goog-api-key': key, 'content-type': 'application/json' },
229       body: JSON.stringify({
230         systemInstruction: { parts: [{ text: system }] },
231         contents: [{ role: 'user', parts }],
232         generationConfig,
233       }),
234     },
235     deadline,
236   );
237   if (res.ok) {
238     const data = await res.json();
239     const text = geminiText(data);
240     if (!text) {
241       const why = data?.candidates?.[0]?.finishReason ?? data?.promptFeedback?.blockReason
?? 'unknown';
242       throw new Error(`Gemini ${model} returned no text (${why})`);
243     }
244     return { text, model };
245   }
246
247   const detail = await res.text();
248   lastError = `Gemini ${model} ${res.status}: ${detail.slice(0, 300)}`;
249   const step = geminiStep(res.status, detail, useJsonSchema);
250   if (step === 'invalid_key') throw new NotConfiguredError(lastError);
251   if (step === 'fail') throw new Error(lastError);
252   if (step === 'plain_schema') {
253     useJsonSchema = false; // describe the schema in the prompt instead
254     continue;
255   }
256   if (step === 'busy') busyDelayMs = Math.max(busyDelayMs ?? 0, retryDelayMs(detail) ??
3000);
257   break; // next_model or busy: the next model has its own quota
258 }
259 }
260 if (busyDelayMs === null) break; // no model is usable with this key
261 if (round === 1) throw new BusyError(lastError);
262 const waitMs = Math.min(busyDelayMs, 12000);
263 if (Date.now() + waitMs > deadline - FETCH_TIMEOUT_MS / 2) throw new BusyError(lastError);
264 console.warn(JSON.stringify({ ai: 'gemini', task: req.task, waitMs }));
265 await sleep(waitMs);
266 }
267 throw new Error(lastError);
268 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/_shared/common.ts:202`

บรรทัด ความหมาย

202-203 รับคำขอและเส้นตายรวม อ่านกุญแจ Gemini (เครื่องหมาย ! บอกว่ามั่นใจว่ามี เพราะ aiProvider เลือก gemini แล้ว แปลว่ามีกุญแจ)

บรรทัด	ความหมาย
204–206	แปลงเนื้อหาเป็นรูปแบบของ Gemini: ข้อความเป็น { text } รูปเป็น { imageData } (โค้ชส่งแต่ข้อความ)
207	จะใช้ JSON schema ไหม (โค้ชไม่ใช้ schema จึงเป็น false ใช้กับการอ่านสลิป)
208	เก็บ error ล่าสุดไว้ใช้ตอนแจ้งผล
209	เลือกรายชื่อรุ่น: งานโค้ชใช้ geminiModels
211–212	วนได้ 2 รอบ: ถ้ารอบแรกทุกรุ่นแค่ "ไม่ว่าง" จะรอแล้วลองอีกรอบ
213	busyDelayMs = ควรรอเท่าไร (null = ยังไม่มีรุ่นไหนบอกว่าไม่ว่าง)
214–215	ลองทีละรุ่น แต่ละรุ่นลองได้ 2 ครั้ง (ครั้งที่สองใช้เฉพาะเรื่อง schema)
216–217	จำกัดความยาวคำตอบ: อย่างน้อย 2,048 หรือ 4 เท่าของ maxTokens (โค้ช $500 \times 4 = 2,000$ จึงได้ 2,048) เพื่อพื้นที่ให้ "การคิดภายใน" ของโมเดลด้วย
218–223	ส่วน schema (ใช้กับสลิป ข้ามได้สำหรับโค้ช)
224–236	ส่ง POST ไปที่ .../models/รุ่น:generateContent ใส่กุญแจใน header x-goog-api-key (ไม่ใส่ใน URL) ตัวคำขอมี systemInstruction (prompt) contents (ข้อความ DATA/QUESTION) และ generationConfig ใช้ timedFetch ที่มีเวลาจำกัด
237–245	สำเร็จ: ดึงข้อความด้วย geminiText (ข้ามส่วน "ความคิด") ถ้าว่างแจ้งเหตุผล (เช่น ถูกบล็อก) ถ้ามีคืนข้อความกับชื่อรุ่น
247–249	ไม่สำเร็จ: อ่านรายละเอียด เก็บเป็น lastError แล้วถาม geminiStep ว่าควรทำอะไรต่อ (un A10)
250	กุญแจผิด → หยุดทั้งหมด (NotConfiguredError → แอปบอก "ผู้ดูแลต้องใส่ GEMINI_API_KEY...")
251	ผิดพลาดแบบอื่นที่ไม่ควรลองใหม่ → หยุด
252–255	รุ่นนี้ไม่รับ schema → ลองรุ่นเดิมอีกครั้งแบบอธิบาย schema ใน prompt (เฉพาะงานสลิป)
256	ไม่ว่าง → จำนวนที่ควรรอ (ใช้ค่าที่ Google บอกใน retryDelay ถ้าไม่บอกใช้ 3 วินาที เอาค่ามากที่สุดของทุกรุ่น)
257	ไปรุ่นถัดไป (แต่ละรุ่นมีโควตาฟรีของตัวเอง)
260	ครบทุกรุ่นแล้วไม่มีรุ่นไหนแค่ "ไม่ว่าง" (เช่น ไม่มีรุ่นที่ใช้ได้เลย) → ออกจากวง
261	รอบที่ 2 แล้วยังไม่ว่าง → BusyError (แอปบอก "รอสักครู์แล้วถามใหม่")
262–263	รอไม่เกิน 12 วินาที และถ้ารอแล้วจะเสียเวลาไม่พอ (เส้นตายรวม 110 วินาที) → BusyError เลย
264–265	บันทึก log ว่ารอกี่มิลลิวินาที (ไม่บันทึกเนื้อหา) แล้วรอ
267	หลุดวงมาได้ แปลว่าไม่มีรุ่นที่ใช้ได้ → error พร้อมข้อความล่าสุด

≈ เหมือนโทรหาร้านอาหารที่มีหลายสาขา

ร้านแรกสายไม่ว่าง ก็โทรสาขาถัดไป (แต่ละสาขามีคิวของตัวเอง) ถ้าทุกสาขาไม่ว่าง ฟังว่าเขาบอกให้รอกี่นาที รอแล้วโทรใหม่อีกรอบเดียว ถ้ายังไม่ว่างก็บอกลูกค้าตรง ๆ ว่า "ตอนนี้คิวเต็ม ลองใหม่อีกสักครู์" ไม่ปล่อยให้ลูกค้ารอไม่รู้จบ

ส่วนที่ 5: อ่านคำตอบและทำความเข้าใจ

supabase/functions/_shared/helpers.ts

บรรทัด 95-134

```
95 /** Gemini 429 bodies carry RetryInfo such as "retryDelay": "7s". Returns milliseconds, or null.
96 */
97 export function retryDelayMs(body: string): number | null {
98   const m = body.match(/"retryDelay"\s*:\s*"(\d+(?:\.\d+)?)s"/);
99   return m ? Math.ceil(Number(m[1]) * 1000) : null;
100 }
101
102 /** Text of the first Gemini candidate, without "thought" parts. */
103 export function geminiText(data: unknown): string {
104   const parts = (data as { candidates?: { content?: { parts?: { text?: string; thought?: boolean }[] } } })
105     ?.candidates?.[0]?.content?.parts;
106   return (parts ?? []).
107     .filter((p) => typeof p.text === 'string' && !p.thought)
108     .map((p) => p.text)
109     .join('');
110 }
111
112 /** Parse a JSON reply; tolerates ``json fences or a sentence around the object. */
113 export function parseJsonText(text: string): unknown {
114   const trimmed = text.trim().replace(/^``(?:json)?\s*/i, '').replace(/\s*```$/, '');
115   try {
116     return JSON.parse(trimmed);
117   } catch {
118     const start = trimmed.indexOf('{');
119     const end = trimmed.lastIndexOf('}');
120     if (start >= 0 && end > start) return JSON.parse(trimmed.slice(start, end + 1));
121     throw new Error('AI reply was not JSON');
122   }
123 }
124
125 /**
126  * Coach replies are shown as plain text and read aloud: drop markdown bold,
127  * headings and list bullets if a model adds them.
128  */
129 export function plainText(text: string): string {
130   return text
131     .replace(/\*(.+?)\*/g, '$1')
132     .replace(/^#{1,6}\s+/gm, '')
133     .replace(/\s*(?:[-*]|\d+[\.])\s+/gm, '')
134     .trim();
135 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/helpers.ts:95

บรรทัด ความหมาย

95-99 `retryDelayMs`: ข้อความ error 429 ของ Gemini มีบอกเช่น `"retryDelay": "7s"` ฟังก์ชันนี้ดึงตัวเลขออกมาเป็นมิลลิวินาที (7s → 7000) ถ้าไม่มีคืน null (ลองแล้วได้ตามนี้จริง)

101-109 `geminiText`: คำตอบของ Gemini ซ้อนกันหลายชั้น (candidates → content → parts) ฟังก์ชันนี้เดินเข้าไปอย่างปลอดภัยด้วย ? . (ถ้าชั้นไหนไม่มีก็ได้ undefined ไม่พัง) เอาเฉพาะ part ที่เป็นข้อความและไม่ใช้ "thought" (ความคิดภายในของโมเดล) มาต่อกัน

บรรทัด	ความหมาย
111–122	parseJsonText: แปลงคำตอบ JSON (ใช้กับการอ่านสลิป ไม่ใช่โค้ช) ยอมรับคำตอบที่ห่อด้วย ``json หรือมีประโยคนำหน้า
124–127	คำตอบโค้ชถูกแสดงเป็นข้อความล้วนและอ่านออกเสียง จึงลบตัวหนา หัวข้อ และบูลเล็ท ถ้าโมเดลใส่มา
130	**ข้อความ** → ข้อความ (ลบตัวหนา เก็บเนื้อในไว้ด้วย \$1)
131	ลบ # 1–6 ตัวที่ต้นบรรทัด (หัวข้อ markdown) gm = ทุบบรรทัด
132	ลบบูลเล็ทต้นบรรทัด: - * • หรือ "1." "2)" ที่ตามด้วยเว้นวรรค
133	ตัดช่องว่างหัวท้าย

TC-40 ในไฟล์ [supabase/functions/_shared/__tests__/helpers.test.ts](#) ตรวจสอบการอ่านคำตอบ (ข้ามส่วนความคิด, JSON ที่ห่อด้วย `` , markdown) ฟังก์ชันในไฟล์ helpers.ts เป็น pure function ไม่ใช้คำสั่งของ Deno จึงเทสต์ด้วย Vitest ในเครื่องได้เหมือนโค้ดของแอป

อ่านโค้ด: หน้าโค้ช src/app/(tabs)/coach.tsx

ไฟล์หน้าจอกที่ใหญ่ที่สุดของ FR-5 (501 บรรทัด) แบ่งอ่าน 10 ส่วน ตั้งแต่ import จนถึงปุ่มบนเวที

ไฟล์นี้อยู่ใน src/app/(tabs) จึงเป็นแท็บ "โค้ช" ที่อยู่ /coach (กฎ "ไฟล์ = หน้า" ของ Expo Router un A7) ต่างจาก insights.ts ตรงที่ไฟล์นี้มี "สถานะ" (state) และหน้าจอ จึงทดสอบด้วย E2E ในเบราว์เซอร์ ไม่ใช่ unit test

ส่วนที่ 1: คอมเมนต์ import และค่าคงที่

src/app/(tabs)/coach.tsx

บรรทัด 1-49

```

1  /**
2   * FR-5 AI coach: นื่องกล้า, big, on a little stage. It talks to the user with
3   * subtitles, a moving mouth and small gestures, and with a calm, friendly
4   * voice when the phone has a Thai voice and the sound is on. It wears the skin
5   * the user chose. Two layers of advice:
6   * 1. "สิ่งที่ควรรู้ออนนี้" – rule-based facts from src/domain/insights.ts
7   *    (works offline, every number traceable); tap one and นื่องกล้า tells it;
8   * 2. questions answered by the AI coach, which receives only a summary of
9   *    confirmed totals (see buildCoachContext).
10  */
11 import * as Haptics from 'expo-haptics';
12 import { router, useFocusEffect, useLocalSearchParams } from 'expo-router';
13 import { useCallback, useMemo, useRef, useState } from 'react';
14 import { KeyboardAvoidingView, Keyboard, Platform, Pressable, ScrollView, TextInput,
15   useWindowDimensions, View } from 'react-native';
16 import { SafeAreaView } from 'react-native-safe-area-context';
17 import { useApp, useMoney } from '../data/AppProvider';
18 import { BUDDY_NAME, buddyPoke, type BuddyMood } from '../domain/buddy';
19 import { buildCoachContext, buildInsights } from '../domain/insights';
20 import { HALLOWEEN_LINES } from '../domain/halloween';
21 import { KLA_AFTER, KLA_GREETING, KLA_GREETING_HALLOWEEN, KLA_THINKING, REPEAT_MAX } from
22   '../domain/klaTalk';
23 import { formatBaht } from '../domain/money';
24 import { priceInQuestion } from '../domain/price';
25 import { runwayAfterPurchase } from '../domain/runway';
26 import { SKINS, skinById, type SkinId } from '../domain/skins';
27 import { askCoach, CoachError } from '../services/coach';
28 import { useEquippedSkin, useKla } from '../services/kla';
29 import { Buddy } from '../ui/Buddy';
30 import { Card, Chip, Ionicons, Row, T, type IconName } from '../ui/components';
31 import { Reveal, TypingDots } from '../ui/effects';
32 import { TitleDecor, useHalloween } from '../ui/halloween';
33 import { KlaBackdrop } from '../ui/kla/KlaBackdrop';
34 import { KlaPicture } from '../ui/kla/KlaPicture';
35 import { KlaStage } from '../ui/kla/KlaStage';
36 import { setKlaSound, useKlaTalk } from '../ui/kla/useKlaTalk';
37 import { alpha, fonts, radius, space, useTheme } from '../ui/theme';
38 import { inputBox, inputText } from '../ui/inputs';
39
40 interface Msg {
41   id: number;
42   from: 'me' | 'coach';
43   text: string;
44 }

```

```

42   error?: boolean;
43 }
44
45 const QUICK = ['สรุปสัปดาห์นี้ให้น้อย', 'หมวดไหนควรลดก่อน', 'ชื่อของ 500 บาทวันนี้ได้ไหม', 'ทำยังไงให้เงินพอถึงสิ้นเดือน'];
46 const IDLE = `สงสัยอะไรเรื่องเงิน ถาม${BUDDY_NAME}ได้เลยนะ`;
47
48 /** น้องกล้า says hello once each time the app is opened, not on every visit to the tab. */
49 let greeted = false;

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/(tabs)/coach.tsx:1`

บรรทัด	ความหมาย
1–10	สรุปหน้าจอ: น้องกล้าตัวใหญ่บนเวที คอยด้วยซับไตเติล ปากขยับ ทำทาง และเสียงสุมมเป็นมิตรเมื่อมีเสียงไทยและเปิดเสียง ใส่ชุดที่ผู้ใช้เลือก คำแนะนำ 2 ชั้น: (1) สิ่งที่คุณรู้ตอนนี้ จากกฎใน insights.ts (ใช้ออฟไลน์ได้ ทุกตัวเลขตรวจย้อนได้) แต่แล้วน้องกล้าเล่า (2) คำถามที่ AI ตอบ ซึ่งได้รับแค่สรุปย่อที่ยืนยันแล้ว
11–15	เครื่องมือจากภายนอก: สั่นมือถือ (Haptics) · router และ hook ของ Expo Router · hook ของ React · ชิ้นส่วนหน้าจ React Native (คีย์บอร์ด เลื่อนจอ ช่องพิมพ์ ขนาดจอ) · พื้นที่ปลอดภัยของจอ (ไม่ทับรอยบาก)
16–26	ของในโปรเจกต์: ข้อมูลแอปและ useMoney · น้องกล้า (ชื่อ, เคล็ดลับ, อารมณ์) · กฎการลดและสรุปข้อมูล · นโยบายฮาโลวีน · นโยบายประจำของโค้ช · แพลงเงิน · หาราคา · Runway หลังซื้อ (FR-6) · ชุด · askCoach · ตูสกิน
27–36	ชิ้นส่วนหน้าจอของทีม: น้องกล้าตัวเล็ก (Buddy) · การ์ด ชิป ไอคอน · เอฟเฟกต์ค่อย ๆ โผล่ จุดพิมพ์ · ของตกแต่งฮาโลวีน · เวที ภาพนิ่ง ตัวละครเคลื่อนไหว · การพูด · สีและระยะของริม · หน้าตาช่องพิมพ์
38–43	ชนิดของข้อความในห้องคุย: รหัส · ใครพูด (me = ผู้ใช้, coach = น้องกล้า) · ข้อความ · เป็นข้อความผิดพลาดใหม่
45	คำถามลัด 4 ข้อ
46	ประโยคพักหลังเล่าการ์ด "สงสัยอะไรเรื่องเงิน ถามกล้าได้เลยนะ"
48–49	ตัวแปร greeted อยู่ นอกคอมโพเนนต์ จึงจำค่าได้ตลอดที่แอปเปิดอยู่ แม้หน้าโค้ชถูกสร้างใหม่ ใช้ให้ทักทายครั้งเดียวต่อการเปิดแอป

ส่วนที่ 2: ข้อมูลและสถานะของหน้า

`src/app/(tabs)/coach.tsx`

บรรทัด 51–83

```

51 export default function Coach() {
52   const theme = useTheme();
53   const { width: screenW } = useWindowDimensions();
54   const params = useLocalSearchParams<{ ask?: string; price?: string; at?: string }>();
55   const { profile, txs, repo } = useApp();
56   const { balance, runway } = useMoney();
57   const skin = useEquippedSkin();
58   const kla = useKla();
59   const halloween = useHalloween();
60   const [picking, setPicking] = useState(false);
61   const { subtitle, talking, said, speak, stop, sound, voice } = useKlaTalk();
62   const [input, setInput] = useState(params.ask ?? '');
63   const [msgs, setMsgs] = useState<Msg[]>([]);
64   const [busy, setBusy] = useState(false);
65   const [rest, setRest] = useState<{ text: string; mood: BuddyMood }>({ text: KLA_GREETING, mood: 'happy' });
66   const [speakingId, setSpeakingId] = useState<number | null>(null);
67   const [hop, setHop] = useState(0);

```

```

68 const [focused, setFocused] = useState(false);
69 /** The question box has the keyboard (its border turns to the theme colour). */
70 const [typing, setTyping] = useState(false);
71 const pokes = useRef(0);
72 const scroll = useRef<ScrollView>(null);
73 const seq = useRef(0);
74
75 // Prefill the question when opened from "เช็ก่อนจ่าย" (adjust state during render, no effect
  needed).
76 // `at` changes on every tap, so asking about the same price twice fills it in again.
77 const askKey = params.ask ? `${params.ask}|${params.at ?? ''}` : undefined;
78 const [lastAsk, setLastAsk] = useState(askKey);
79 if (askKey !== lastAsk) {
80   setLastAsk(askKey);
81   if (params.ask) setInput(params.ask);
82 }
83

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/coach.tsx:51

บรรทัด	ความหมาย
52–53	สืของธรม และความกว้างจว (ใช้คำนวณขนาดน้องกล้า)
54	อ่านพารามิเตอร์จากที่อยู่หน้า: ask (คำถาม) price (ราคาเป็นสตริง) at (เวลาที่กด) ที่หน้าเงินพอถึงส่งมา
55–56	โปรไฟล์ รายการ และ repo (บอกว่าโหมดทดลองใหม่) · ยอดคงเหลือและ Runway จาก useMoney (ตัวเดียวกับทุกหน้า)
57–59	ชุดที่ใส่อยู่ · ข้อมูลตู้สกิน · เปิดธรมฮาโลวีนใหม่
60	แถบเลือกชุดเปิดอยู่ใหม่
61	จาก hook การพูด: ซบไต้เตลก่อนปัจจุบัน · กำลังพูดใหม่ · ข้อความล่าสุดที่พูด · ฟังชันพูด/หยุด · เปิดเสียงใหม่ · เสียงไทยที่เจอ
62	ข้อความในช่องพิมพ์ เริ่มด้วยคำถามที่ส่งมา (ถ้ามี)
63–64	รายการข้อความในห้องคุย · กำลังรอคำตอบใหม่
65	"ข้อความพัก" ในฟองคำพูดตอนไม่ได้พูด (เริ่มเป็นคำทักทาย อารมณ์ happy)
66	รหัสข้อความในห้องคุยที่กำลังถูกพูด (ใช้ทำกรอบทอง)
67–70	ตัวนับการกระโดด · หน้าไค้ชเปิดอยู่ใหม่ · กำลังพิมพ์ใหม่ (ขอบช่องพิมพ์เปลี่ยนสี)
71–73	useRef = ที่เก็บค่าที่เปลี่ยนได้โดยไม่ทำให้จววาดใหม่: นับการแตะ · อ้างถึงตัวเลื่อนจอ · ตัวนับรหัสข้อความ
75–82	เติมคำถามเมื่อเปิดมาจาก "เช็ก่อนจ่าย": สร้างกฎแจากคำถาม + เวลาที่กด ถ้ากฎแเปลี่ยนจากครั้งก่อน ใส่คำถามลงช่องพิมพ์ (ถ้าระหว่างวาดหน้า ไม่ต้องใช้ effect) เพราะ at เปลี่ยนทุกครั้งที่กด ถามราคาเดิมซ้ำก็เติมให้ใหม่ได้

ส่วนที่ 3: การ์ด การพูด และการทักทาย

src/app/(tabs)/coach.tsx

บรรทัด 84–111

```

84 const insights = useMemo(() => (profile ? buildInsights({ txs, profile, runway }) : []), [txs,
  profile, runway]);
85

```

```

86  /** Say something on the stage; afterwards the bubble keeps `after` (or the whole line if it is
    short). */
87  const tell = useCallback(
88    (text: string, opts: { mood?: BuddyMood; after?: string; msgId?: number } = {}) => {
89      setRest({ text: opts.after ?? (text.length <= REPEAT_MAX ? text : KLA_AFTER), mood:
    opts.mood ?? 'happy' });
90      setSpeakingId(opts.msgId ?? null);
91      speak(text, () => setSpeakingId(null));
92    },
93    [speak],
94  );
95
96  useEffect(
97    useCallback(() => {
98      setFocused(true);
99      if (!greeted) {
100        greeted = true;
101        tell(halloween ? KLA_GREETING_HALLOWEEN : KLA_GREETING);
102      }
103      return () => {
104        setFocused(false);
105        stop();
106        setSpeakingId(null);
107      };
108    }, [tell, stop, halloween]),
109  );
110
111  const toTop = () => setTimeout(() => scroll.current?.scrollTo({ y: 0, animated: true }), 30);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/coach.tsx:84

บรรทัด	ความหมาย
84	คำนวณการ์ดสิ่งที่ควรรู้ด้วย buildInsights และจำผลไว้ (useMemo) คิดใหม่เฉพาะเมื่อรายการ โปรไฟล์ หรือ Runway เปลี่ยน ถ้ายังไม่มีโปรไฟล์ได้อาร์เรย์ว่าง
86–94	tell "ให้น้องกล่าวพูด": ตั้งข้อความพักหลังพูดจบ (ใช้ after ถ้ามี ถ้าข้อความสั้นกว่า 110 ตัวค้างข้อความเดิม ถ้ายาวใช้ "อ่านคำตอบเต็มได้ด้านล่าง...") ตั้งอาร์มณ จำรหัสข้อความที่กำลังพูด แล้วเรียก speak เมื่อพูดจบครบ ล้างรหัสนั้น (กรอบทองหาย) useCallback ทำให้ได้ฟังก์ชันตัวเดิมทุกครั้งที่เราดใหม่
96–109	useFocusEffect ทำงานทุกครั้งที่เกิดนี้ถูกเปิด: ตั้ง focused = true (เวทียับได้) ทักทายถ้ายังไม่เคยทักในรอบเปิดแอปนี้ (ธัมฮาโลวีนใช้คำทักแบบหลอน) และฟังก์ชันที่ return คือ "ตอนออกจากเก็บบ": หยุดขยับ หยุดพูด ล้างกรอบทอง
111	toTop เลื่อนจอกกลับขึ้นบนสุด (ไปที่เวทีก) แบบนุ่ม ๆ หลังรอ 30 มิลลิวินาทีให้จอวาดเสร็จก่อน

ส่วนที่ 4: ส่งคำถาม แตะน้องกล้า และเปลี่ยนชุด

src/app/(tabs)/coach.tsx

บรรทัด 113–174

```

113  async function send(text: string) {
114    const q = text.trim();
115    if (!q || busy || !profile) return;
116    setInput('');
117    Keyboard.dismiss();
118    stop();
119    setMsgs((m) => [...m, { id: ++seq.current, from: 'me', text: q }]);
120    setBusy(true);

```

```

121     toTop();
122     try {
123         const price = params.price && q === params.ask ? Number(params.price) : priceInQuestion(q);
124         const context = {
125             ...buildCoachContext({ txs, profile, balanceSatang: balance, runway }),
126             ...(price
127                 ? {
128                     purchaseCheck: {
129                         price: formatBaht(price, { decimals: false }),
130                         runwayBefore: runway.days === null ? 'คำนวณไม่ได้' : `${runway.days} วัน`,
131                         runwayAfter: (() => {
132                             const r = runwayAfterPurchase(runway, price);
133                             return r.status === 'below_floor' ? 'ต่ำกว่าเส้นสำรองทันที' : r.days === null ? 'คำนวณ
134                             ไม่ได้' : `${r.days} วัน`;
135                         })(),
136                     },
137                 : {}),
138         };
139         if (repo?.mode === 'demo') {
140             throw new CoachError(`โหมดทดลองยังถาม AI ไม่ได้นะ เข้าสู่ระบบด้วยบัญชีจริงแล้วถาม${BUDDY_NAME}ได้เลย ระหว่างนี้
141             และการ์ด "สิ่งที่ควรรู้ออนนี้" ให้${BUDDY_NAME}เล่าให้ฟังได้`);
142         }
143         const answer = await askCoach({ context, question: q });
144         const id = ++seq.current;
145         setMsgs((m) => [...m, { id, from: 'coach', text: answer }]);
146         tell(answer, { msgId: id });
147     } catch (e) {
148         const text = e instanceof CoachError ? e.message : `ติดต่อ${BUDDY_NAME}ไม่ได้ตอนนี้ ลองใหม่อีกครั้งนะ`;
149         setMsgs((m) => [...m, { id: ++seq.current, from: 'coach', text, error: true }]);
150         // Problems are shown, not read aloud.
151         setRest({ text, mood: 'worried' });
152     } finally {
153         setBusy(false);
154     }
155 }
156
157 function poke() {
158     Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light).catch(() => {});
159     setHop((h) => h + 1);
160     pokes.current += 1;
161     const n = pokes.current;
162     // Halloween theme: every other tap is a spooky-cute line.
163     const line = halloween && n % 2 === 0 ? { mood: 'cheer' as const, text: HALLOWEEN_LINES[(n /
164     2 - 1) % HALLOWEEN_LINES.length] } : buddyPoke(n, runway.status);
165     tell(line.text, { mood: line.mood });
166 }
167
168 /** Change น้องกล้า's skin right here on the stage. */
169 function wear(id: SkinId) {
170     Haptics.selectionAsync().catch(() => {});
171     if (id === skin) return;
172     kla.equip(id);
173     setHop((h) => h + 1);
174     tell(`ใส่ชุด${skinById(id).name}แล้ว เข้ากับ${BUDDY_NAME}ใหม่`, { mood: 'cheer', after: IDLE });
175 }
176
177 const mine = SKINS.filter((s) => kla.owned.has(s.id));

```

บรรทัด	ความหมาย
113–115	send: ตัดช่องว่าง ถ้าว่าง หรือกำลังรอคำตอบอยู่ หรือยังไม่มีโปรไฟล์ → ไม่ทำอะไร (กันส่งซ้ำ)
116–118	ล้างช่องพิมพ์ ซ่อนคีย์บอร์ด หยุดพูดเรื่องเดิม
119	เพิ่มคำถามเข้าห้องคุย (สร้างอาร์เรย์ใหม่ด้วย ...m ไม่แก้ของเดิม ตามกฎของ React)
120–121	เข้าโหมดรอ (น้องกล่าห์นำครุ่นคิด จุดพิมพ์) และเลื่อนไปที่เวที
123	หาราคา: ถ้าคำถามคือคำถามที่หน้าเงินพอถึงส่งมาโดยไม่ถูกแก้ ใช้ราคาที่ส่งมา ไม่เงินหาจากข้อความด้วย priceInQuestion
124–125	ข้อมูลที่จะส่ง เริ่มจากสรุปของ buildCoachContext (กระจายช่องทั้งหมดด้วย ...)
126–137	ถ้ามีราคา เพิ่มช่อง purchaseCheck: ราคา · Runway ตอนนี้ ("คำนวณไม่ได้" ถ้า null) · Runway หลังซื้อ ซึ่งคำนวณในฟังก์ชันที่เรียกกันที่ (() => { ... }) () ให้ผลเป็น "ต่ำกว่าเส้นสำรองกัน" / "คำนวณไม่ได้" / "3 วัน" ถ้าไม่มีราคา กระจายอีอบเจกต์ว่าง (ไม่เพิ่มอะไร)
139–141	โหมดทดลอง: โยน CoachError พร้อมข้อความอธิบาย ไม่ส่งอะไรออกไปเลย (catch ด้านล่างรับไปแสดง)
142	ส่งไปเซิร์ฟเวอร์และรอคำตอบ (await)
143–145	สร้างรหัสใหม่ เพิ่มคำตอบเข้าห้องคุย แล้วให้น้องกล่าห์พูดคำตอบนั้น พร้อมบอกรหัสเพื่อทำกรอบทอง
146–150	ถ้าผิดพลาด: ใช้ข้อความของ CoachError (ภาษาไทยที่เตรียมไว้) หรือข้อความกลาง เพิ่มเป็นฟองข้อความผิดพลาด และตั้งข้อความพักเป็นข้อความนั้น หน้ากังวล โดยไม่เรียก tell (คอมเมนต์บรรทัด 149: ปัญหาแสดง ไม่อ่านออกเสียง)
151–153	finally: ไม่ว่าสำเร็จหรือพัง ออกจากโหมดรอเสมอ (ปุ่มส่งกลับมาใช้ได้)
156–164	poke (แตะน้องกล่าห์บนเวที): สั่นเบา ๆ กระโดด นับครั้ง แล้วเลือกประโยค (ธัมฮาโลวีน: ครั้งคู่เป็นประโยคหลอน ๆ น่ารัก นอกนั้นใช้ buddyPoke) แล้วพูด
166–173	wear (เปลี่ยนชุดบนเวที): สั่น ถ้าเลือกชุดเดิมไม่ทำอะไร ไม่เงินใส่ชุดใหม่ (บันทึกในเครื่อง) กระโดด แล้วพูดว่า "ใส่ชุด...แล้วเข้ากับกล่าห์ไหม" แล้วพักด้วยประโยค IDLE
174	ชุดที่ผู้ใช้มีแล้ว ใช้แสดงในแถบเลือกชุด

! จุดเล็กที่ฉันทิ้งระหว่างทำเล่มนี้: กรอบทองค้าง

บรรทัด 118 เรียก stop () เพื่อหยุดพูด แต่ไม่ได้ล้าง speakingId (ต่างจากปุ่ม "หยุด" บรรทัด 236–237 และตอนออกจากแท็บบรรทัด 105–106 ที่ล้างทั้งคู่) ถ้าผู้ใช้ส่งคำถามใหม่ระหว่างที่น้องกล่าห์ยังพูดคำตอบเก่าไม่จบ คำตอบเก่าจะมี **กรอบทองและไอคอนลำโพงสว่างค้าง** เหมือนยังพูดอยู่ ทั้งที่เงียบแล้ว (เห็นได้ในภาพ "3 ปัญหาต่อกัน" ที่คำตอบเรื่องหูฟังยังมีกรอบทอง) แก้ได้ด้วยการเพิ่ม setSpeakingId (null) ; ต่อจากบรรทัด 118 (ยังไม่ได้แก้ในโปรเจกต์)

ส่วนที่ 5: อารมณ์ ขนาด และเวที

src/app/(tabs)/coach.tsx

บรรทัด 176–244

```
176 const mood: BuddyMood = busy ? 'thinking' : talking ? (rest.mood === 'worried' ? 'calm' :
    rest.mood) : rest.mood;
```

```

177   const kW = Math.round(Math.min(214, Math.min(screenW, 440) * 0.52));
178
179   return (
180     <SafeAreaView edges={['top']} style={{ flex: 1, backgroundColor: theme.bg }}>
181       <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === 'ios' ? 'padding' :
undefined}>
182         <ScrollView
183           ref={scroll}
184           // Same header spacing as the other tabs (title starts `sm` below the safe area).
185           contentContainerStyle={{ paddingHorizontal: space.lg, paddingTop: space.sm, gap:
space.lg, paddingBottom: space.xl }}
186           keyboardShouldPersistTaps="handled"
187         >
188           <Row justify="space-between" align="flex-end">
189             <View style={{ gap: 2 }}>
190               <T v="label">ไอ้ช่อดูตัว</T>
191               <T v="h1">น้อง{BUDDY_NAME}</T>
192             </View>
193             <TitleDecor />
194           </Row>
195
196           { /* The stage */ }
197           <KlaBackdrop floorAt={0.8}>
198             <View style={{ alignItems: 'center', paddingTop: space.lg, paddingHorizontal:
space.lg, paddingBottom: space.md }}>
199               <SpeechBubble>
200                 {busy ? (
201                   <Row gap={space.sm}>
202                     <TypingDots color={theme.primary} />
203                     <T v="body" color={theme.inkSoft}>
204                       {KLA_THINKING}
205                     </T>
206                   </Row>
207                 ) : (
208                   <T v="body" color={rest.mood === 'worried' && !talking ? theme.inkSoft :
theme.ink} style={{ fontFamily: fonts.sansMedium }}>
209                     {subtitle ? subtitle.text : rest.text}
210                   </T>
211                 )}
212                 {subtitle && subtitle.total > 1 ? (
213                   <Row gap={4} style={{ marginTop: 6 }} justify="center">
214                     {Array.from({ length: subtitle.total }, (_, i) => (
215                       <View key={i} style={{ width: i === subtitle.index ? 14 : 5, height: 5,
borderRadius: 3, backgroundColor: i <= subtitle.index ? theme.primary : theme.line }} />
216                     ))}
217                   </Row>
218                 ) : null}
219               </SpeechBubble>
220               <Pressable onPress={poke} accessibilityRole="button" accessibilityLabel=
{`น้อง{BUDDY_NAME}ในชุด${skinById(skin).name} และเพื่อฟังเคล็ดลับ`} style={{ marginTop: 2 }}>
221                 <KlaStage skin={skin} mood={mood} width={kW} talking={talking} hop={hop} active=
{focused} onDark />
222               </Pressable>
223               <Row gap={space.sm} style={{ marginTop: -space.sm }}>
224                 <StageButton
225                   icon={sound ? 'volume-high' : 'volume-mute'}
226                   label={sound ? 'เสียงเปิด' : 'เสียงปิด'}
227                   ally={sound ? `เปิดเสียงน้อง{BUDDY_NAME}` : `เปิดเสียงน้อง{BUDDY_NAME}`}
228                   onPress={() => setKlaSound(!sound)}
229                 />
230                 {talking ? (

```

```

231         <StageButton
232             icon="stop"
233             label="หยุด"
234             ally={`ให้น้อง${BUDDY_NAME}หยุดพูด`}
235             onPress={() => {
236                 stop();
237                 setSpeakingId(null);
238             }}
239         />
240     ) : said ? (
241         <StageButton icon="refresh" label="ฟังอีกครั้ง" ally={`ให้น้อง${BUDDY_NAME}พูดอีกครั้ง`}
onPress={() => tell(said, { mood: rest.mood === 'worried' ? 'calm' : rest.mood })} />
242     ) : null}
243     <StageButton icon="shirt" label="เปลี่ยนชุด" ally={`เปลี่ยนชุดน้อง${BUDDY_NAME}`} onPress=
{() => setPicking((p) => !p)} />
244 </Row>

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/coach.tsx:176

บรรทัด	ความหมาย
176	อารมณ์ที่แสดงจริง: รอคำตอบ → ครุ่นคิด · กำลังพูด → อารมณ์พัก (แต่ถ้าเป็น worried เปลี่ยนเป็น calm ไม่พูดด้วยหน้ากังวล) · ไม่ได้พูด → อารมณ์พัก
177	ความกว้างน้องกล้า = 52% ของความกว้างจอ (คิดจอกว้างสุด 440) แต่ไม่เกิน 214 จุด จอเล็กตัวเล็กลง จอใหญ่ไม่ใหญ่เกิน
180–187	โครงหน้า: พื้นที่ปลอดภัยด้านบน · ดันเนื้อหาหนีคีย์บอร์ด (เฉพาะ iPhone ต้องเว้นที่) · ตัวเลื่อนจอ เว้นขอบเท่าแท็บอื่น และ keyboardShouldPersistTaps ให้แตะปุ่มได้ทันทีแม้คีย์บอร์ดเปิดอยู่
188–194	หัวหน้าจอ "โค้ชส่วนตัว / น้องกล้า" และของตกแต่งริบ
197–198	เวท (KlaBackdrop) พื้นอยู่ที่ 80% ของความสูง
199–219	ฟองคำพูด: ถ้ารอคำตอบ แสดงจุดพิมพ์ + "กล้ากำลังดูตัวเลขของคุณ..." ไม่ขึ้นแสดงซับไตเติลก่อนปัจจุบัน หรือข้อความพัก (ข้อความพักตอนกังวลใช้สีจางลง) ถ้าข้อความมีหลายตอน แสดงจุดเล็ก ๆ จุดที่พูดแล้วและจุดปัจจุบันเป็นสีหลักของธีม จุดที่ยังไม่ถึงเป็นสีเส้น จุดปัจจุบันยาวกว่า (14 กับ 5 จุด)
220–222	ตัวน้องกล้า (KlaStage) อยู่ในปุ่มที่แตะได้ มีคำอธิบายสำหรับโปรแกรมอ่านหน้าจอ ส่งชุด อารมณ์ ขนาด กำลังพูดใหม่ ตัวนับกระโดด และเปิดหน้าจอใหม่ (active)
223–229	ปุ่มเสียง: ไอคอนและคำเปลี่ยนตามสถานะ กดแล้วสลับคำเสียง (คำเดียวกับในตั้งค่า)
230–242	ปุ่มที่สอง: กำลังพูด → "หยุด" (หยุดและล้างกรอบทอง) · พูดจบแล้วและเคยพูด → "ฟังอีกครั้ง" (พูดข้อความล่าสุดซ้ำ) · ยังไม่เคยพูด → ไม่มีปุ่ม
243	ปุ่ม "เปลี่ยนชุด" สลับเปิด/ปิดแถบเลือกชุด

ส่วนที่ 6: แถบเลือกชุด และบรรทัดเมื่อไม่มีเสียงไทย

src/app/(tabs)/coach.tsx

บรรทัด 245–305

```

245     {picking ? (
246         <Reveal from={8} style={{ alignSelf: 'stretch', marginTop: space.md }}>
247         <ScrollView horizontal showsHorizontalScrollIndicator={false}
contentContainerStyle={{ gap: space.sm, paddingHorizontal: 2 }}>

```

```

248         {mine.map((s) => {
249             const on = s.id === skin;
250             return (
251                 <Pressable
252                     key={s.id}
253                     onPress={() => wear(s.id)}
254                     accessibilityRole="button"
255                     accessibilityLabel={`ใส่ชุด${s.name}`}
256                     aria-selected={on}
257                     style={({ pressed }) => ({
258                         width: 70,
259                         alignItems: 'center',
260                         paddingTop: 2,
261                         paddingBottom: 6,
262                         borderRadius: radius.md,
263                         borderWidth: on ? 2 : 1,
264                         borderColor: on ? theme.heroAccent : 'rgba(255,255,255,0.18)',
265                         backgroundColor: pressed ? 'rgba(255,255,255,0.2)' :
266                         'rgba(255,255,255,0.1)',
267                     })}
268                 >
269                     <KlaPicture skin={s.id} mood="happy" width={50} onDark extras={false}
270                     ground={false} />
271                     <T v="micro" color={theme.heroInk} numberOfLines={1}>
272                         {s.name}
273                     </T>
274                 </Pressable>
275             );
276         })}
277         <Pressable
278             onPress={() => router.push('/skins')}
279             accessibilityRole="button"
280             accessibilityLabel="ดูคู่สกินทั้งหมดและวิธีปลดล็อก"
281             style={({ pressed }) => ({
282                 width: 70,
283                 alignItems: 'center',
284                 justifyContent: 'center',
285                 gap: 4,
286                 borderRadius: radius.md,
287                 borderWidth: 1,
288                 borderStyle: 'dashed',
289                 borderColor: 'rgba(255,255,255,0.35)',
290                 backgroundColor: pressed ? 'rgba(255,255,255,0.2)' : 'transparent',
291             })}
292         >
293             <Icons name="grid" size={20} color={theme.heroAccent} />
294             <T v="micro" color={theme.heroInk} center>
295                 คู่สกิน{'\n'}ทั้งหมด
296             </T>
297         </Pressable>
298     </ScrollView>
299     </Reveal>
300     ) : null}
301     {sound && voice === null ? (
302         <T v="micro" color={theme.heroInkSoft} center style={{ marginTop: space.sm }}>
303             เครื่องนี้ยังไม่เสียงอ่านภาษาไทย {BUDDY_NAME}จะพูดเป็นตัวหนังสือแทนนะ
304         </T>
305     ) : null}
306 </View>
307 </KlaBackdrop>

```

บรรทัด	ความหมาย
245–247	ถ้าเปิดแถบเลือกชุด ค่อย ๆ โฟลว์ขึ้นมา เป็นแถบเลื่อนแนวนอน
248–274	ชุดที่มีแต่ละชุดเป็นปุ่ม: ภาพนิ่งน้องกล่าในชุดนั้น (KlaPicture ไม่มีของประกอบและเงา) กับชื่อชุด ชุดที่ใส่อยู่ขอบหนา 2 สีทอง มี aria-selected บอกโปรแกรมอ่านหน้าจอว่าเลือกอยู่ และแล้วเรียก wear
275–295	ช่องสุดท้าย "ตุสกิน ทั้งหมด" ขอบเส้นประ พาไปหน้า /skins
299–303	ถ้าเปิดเสียงแต่เครื่องไม่มีเสียงไทย (voice === null ไม่ใช่ undefined ซึ่งแปลว่ายังตรวจสอบอยู่) แสดง "เครื่องนี้ยังไม่มีเสียงอ่านภาษาไทย กล้าจะพูดเป็นตัวหนังสือแทนนะ"

ส่วนที่ 7: การ์ดสิ่งที่ควรรู้ตอนนี้

src/app/(tabs)/coach.tsx

บรรทัด 307–337

```

307         <View style={{ gap: space.sm }}>
308             <T v="h3">สิ่งที่ควรรู้ตอนนี้</T>
309             {insights.map((i, n) => (
310                 <Reveal key={n} index={n + 1}>
311                     <Card
312                         onPress={() => {
313                             tell(i.message, { mood: i.severity === 'watch' ? 'calm' : 'happy', after:
314                                 IDLE });
315                             toTop();
316                         }}
317                     >
318                         <Row align="flex-start" gap={space.md}>
319                             <Ionicons
320                                 name={i.severity === 'watch' ? 'alert-circle-outline' : i.severity ===
321                                     'good' ? 'checkmark-circle-outline' : 'information-circle-outline'}
322                                 size={22}
323                                 color={i.severity === 'watch' ? theme.watch : i.severity === 'good' ?
324                                     theme.good : theme.inkSoft}
325                             />
326                             <View style={{ flex: 1, gap: 4 }}>
327                                 <T v="body">{i.message}</T>
328                                 <T v="micro">ข้อมูล: {i.fact}</T>
329                                 <Row gap={4}>
330                                     <Ionicons name="volume-medium-outline" size={14} color={theme.primary} />
331                                     <T v="micro" color={theme.primary}>
332                                         และให้น้อง{BUDDY_NAME}เล่า
333                                     </T>
334                                 </Row>
335                             </View>
336                         </Row>
337                     </Card>
338                 </Reveal>
339             )]}
340         </View>

```

บรรทัด	ความหมาย
308	หัวข้อ "สิ่งที่ควรรู้ตอนนี้"
309–310	วนทุกการ์ด แต่ละใบค่อย ๆ โผล่ตามลำดับ (index)
311–316	แตะการ์ด → นื่องกล้าเล่า message (การ์ดเตือน = calm, อื่น ๆ = happy) แล้วพักด้วย IDLE และเลื่อนขึ้นไปทีละเวก
318–322	ไอคอนตามระดับ: watch = วงกลมตกใจสีส้ม · good = เครื่องหมายถูกสีเขียว · info = ตัว i สีเทา
324–325	ข้อความนื่องกล้า และบรรทัดเล็ก "ข้อมูล: ..." (fact)
326–331	ไอคอนลำโพงเล็ก + "แต่ให้นื่องกล้าเล่า" บอกว่าแตะได้

ส่วนที่ 8: ห้องคุยและคำถามลัด

src/app/(tabs)/coach.tsx

บรรทัด 339–393

```

339     <View style={{ gap: space.sm }}>
340       <T v="h3">คุยกับนื่อง{BUDDY_NAME}</T>
341       {msgs.length === 0 ? <T v="micro">{BUDDY_NAME}ตอบจากตัวเลขที่คุณยืนยันแล้วเท่านั้น ไม่เห็นรูปสลิปหรือชื่อ
คนที่คุณโอบให้</T> : null}
342       {msgs.map((m) => {
343         const color = m.from === 'me' ? theme.onPrimary : m.error ? theme.inkSoft :
theme.ink;
344         const live = m.id === speakingId;
345         return (
346           <Reveal key={m.id} from={12} zoom style={{ alignSelf: m.from === 'me' ? 'flex-
end' : 'flex-start', maxWidth: '92%' }}>
347             <Row gap={6} align="flex-end">
348               {m.from === 'coach' ? <Buddy mood={m.error ? 'worried' : 'happy'} size={34}
still /> : null}
349             <View
350               style={{
351                 flexShrink: 1,
352                 backgroundColor: m.from === 'me' ? theme.primary : m.error ?
theme.surfaceAlt : theme.surface,
353                 borderRadius: radius.lg,
354                 borderBottomRightRadius: m.from === 'me' ? 6 : radius.lg,
355                 borderBottomLeftRadius: m.from === 'coach' ? 6 : radius.lg,
356                 borderWidth: m.from === 'coach' ? (live ? 1.5 : 1) : 0,
357                 borderColor: live ? theme.accent : theme.line,
358                 padding: space.md,
359                 gap: 4,
360               }}
361             >
362               {m.from === 'coach' && !m.error ? (
363                 <Row justify="space-between" gap={space.sm}>
364                   <T v="micro">นื่อง{BUDDY_NAME} · ตอบโดย AI</T>
365                   <Pressable
366                     onPress={() => {
367                       tell(m.text, { msgId: m.id });
368                       toTop();
369                     }}
370                     // A 44pt target around the small icon.
371                     hitSlop={14}
372                     accessibilityRole="button"

```

```

373             accessibilityLabel={live ? `น้อง${BUDDY_NAME}กำลังพูดข้อความนี้` : 'ฟังข้อความ
      นี้'}
374         >
375         <Icons name={live ? 'volume-high' : 'volume-medium-outline'} size=
{16} color={live ? theme.accent : theme.inkSoft} />
376       </Pressable>
377     </Row>
378   ) : null}
379   <T v="body" color={color} selectable>
380     {m.text}
381   </T>
382 </View>
383 </Row>
384 </Reveal>
385   );
386   }}}
387 </View>
388
389   <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{
gap: space.sm }}>
390     {QUICK.map((q) => (
391       <Chip key={q} label={q} onPress={() => send(q)} />
392     ))}
393   </ScrollView>

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/(tabs)/coach.tsx:339`

บรรทัด	ความหมาย
340–341	หัวข้อ "คุยกับน้องกล้า" และถ้ายังไม่มีข้อความ แสดงคำอธิบายความเป็นส่วนตัว "กล้าตอบจากตัวเลขที่คุณยืนยันแล้วเท่านั้น ไม่เห็นรูปสลิปหรือชื่อคนที่คุณโอนให้"
342–345	วนทุกข้อความ: สีตัวอักษรตามคนพูด · live = ข้อความนี้กำลังถูกพูดไหม
346	แต่ละฟองค่อย ๆ โผล่แบบซูมเล็กน้อย ข้อความผู้ใช้ขีดขวา น้องกล้าขีดซ้าย กว้างไม่เกิน 92%
348	ข้อความของน้องกล้ามิน้องกล้าตัวเล็กข้างฟอง (หน้าท้วงถั่วเป็นข้อความผิดพลาด) still = ไม่ยับ
349–361	หน้าต่างฟอง: ผู้ใช้สีหลักของธีม · ข้อความผิดพลาดสีเทา · น้องกล้าสีพื้นผิว มุมฝั่งคนพูดแหลมขึ้น (6) เหมือนหางฟองแซต · กำลังพูด: ขอบหนา 1.5 สีทอง (accent)
362–378	หัวฟองของคำตอบ (ไม่ใช่ข้อความผิดพลาด): "น้องกล้า · ตอบโดย AI" กับปุ่มลำโพงเล็ก ขยายพื้นที่แต่ละรอบไอคอน 14 จุด (ให้ได้ขนาดนี้ราว 44 จุด) และแล้วพูดข้อความนั้นซ้ำ
379–381	เนื้อข้อความ เลือกคัดลอกได้ (selectable)
389–393	คำถามลัด 4 ซิปในแถบเลื่อนแนวนอน และแล้วส่งทันที

ส่วนที่ 9: ช่องพิมพ์และปุ่มส่ง

`src/app/(tabs)/coach.tsx`

บรรทัด 396–432

```

396     <Row
397       gap={space.sm}
398       style={{ paddingHorizontal: space.lg, paddingVertical: space.sm, borderTopWidth: 1,
borderTopColor: theme.line, backgroundColor: theme.surface }}
399     >

```

```

400      <TextInput
401        nativeID="coach-input"
402        value={input}
403        onChangeText={setInput}
404        placeholder={`ถามน้อง${BUDDY_NAME} เช่น ชื่อรองเท้า 1,290 ได้ไหม`}
405        placeholderTextColor={theme.inkFaint}
406        accessibilityLabel={`คำถามถึงน้อง${BUDDY_NAME}`}
407        multiline
408        maxLength={400}
409        onFocus={() => setTyping(true)}
410        onBlur={() => setTyping(false)}
411        style={[inputBox(theme, { focused: typing }), inputText(theme), { maxHeight: 100,
paddingVertical: 10 }]}
412      />
413      <Pressable
414        onPress={() => send(input)}
415        disabled={!input.trim() || busy}
416        accessibilityRole="button"
417        accessibilityLabel="ส่งคำถาม"
418        style={{
419          width: 44,
420          height: 44,
421          borderRadius: 22,
422          backgroundColor: input.trim() && !busy ? theme.primary : theme.surfaceAlt,
423          alignItems: 'center',
424          justifyContent: 'center',
425        }}
426      >
427        <Icons name="arrow-up" size={22} color={input.trim() && !busy ? theme.onPrimary :
theme.inkFaint} />
428      </Pressable>
429    </Row>
430  </KeyboardAvoidingView>
431 </SafeAreaView>
432 );

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/coach.tsx:396

บรรทัด	ความหมาย
396–399	แถบล่างติดจอ (อยู่นอกตัวเลื่อน จึงไม่เลื่อนหายไป) มีเส้นคั่นด้านบน
400–412	ช่องพิมพ์: nativeID "coach-input" (เกสต์ E2E ใช้หาช่องนี้) · ข้อความตัวอย่าง "ถามน้องกล้า เช่น ชื่อรองเท้า 1,290 ได้ไหม" (สอนวิธีการถามเรื่องชื่อไปในตัว) · หลายบรรทัดได้ · ยาวไม่เกิน 400 ตัว (เท่ากับที่เชิร์ฟเวอร์ตัด) · ขอบเปลี่ยนสีตอนพิมพ์ · สูงไม่เกิน 100
413–428	ปุ่มส่งวงกลม 44 × 44 (ขนาดนิ้วแตะได้สบาย) ปิดใช้งานเมื่อช่องว่างหรือกำลังรอ สีเปลี่ยนตามว่ากดได้ไหม ไอคอนลูกศรขึ้น

ส่วนที่ 10: ฟองคำพูดบนเวที และปุ่มบนเวที

src/app/(tabs)/coach.tsx

บรรทัด 435–501

```

435 /** The white bubble over น้องกล้า, with a tail pointing down at it. */
436 function SpeechBubble({ children }: { children: React.ReactNode }) {
437   const theme = useTheme();
438   return (

```

```

439     <View style={{ alignSelf: 'stretch', alignItems: 'center' }}
accessibilityLiveRegion="polite">
440     <View
441       style={{
442         alignSelf: 'stretch',
443         minHeight: 54,
444         justifyContent: 'center',
445         backgroundColor: theme.surface,
446         borderRadius: radius.lg,
447         paddingHorizontal: space.lg,
448         paddingVertical: space.md,
449         shadowColor: '#000',
450         shadowOpacity: 0.18,
451         shadowRadius: 12,
452         shadowOffset: { width: 0, height: 6 },
453         elevation: 6,
454       }}
455     >
456       {children}
457     </View>
458     <View
459       style={{
460         width: 16,
461         height: 16,
462         marginTop: -9,
463         backgroundColor: theme.surface,
464         transform: [{ rotate: '45deg' }],
465         borderBottomRightRadius: 4,
466       }}
467     />
468   </View>
469 );
470 }
471
472 /** A round, see-through button on the stage. */
473 function StageButton({ icon, label, ally, onPress }: { icon: IconName; label: string; ally:
string; onPress: () => void }) {
474   const theme = useTheme();
475   return (
476     <Pressable
477       onPress={() => {
478         Haptics.selectionAsync().catch(() => {});
479         onPress();
480       }}
481       accessibilityRole="button"
482       accessibilityLabel={ally}
483       style={({ pressed }) => ({
484         flexDirection: 'row',
485         alignItems: 'center',
486         gap: 6,
487         paddingHorizontal: 14,
488         minHeight: 44,
489         borderRadius: radius.pill,
490         backgroundColor: alpha(theme.heroInk, pressed ? 0.22 : 0.12),
491         borderWidth: 1,
492         borderColor: alpha(theme.heroInk, 0.3),
493       })}
494     >
495       <Icons name={icon} size={18} color={theme.heroAccent} />
496       <Text v="small" color={theme.heroInk}>
497         {label}

```

```

498     </T>
499     </Pressable>
500 );
501 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/(tabs)/coach.tsx:435`

บรรทัด	ความหมาย
436–439	SpeechBubble รับเนื้อหาข้างใน (children) ตั้ง <code>accessibilityLiveRegion="polite"</code> ให้โปรแกรมอ่านหน้าจออ่านข้อโต้แย้งใหม่ทุกครั้งที่เปลี่ยน เมื่อว่าง
440–457	ตัวฟอง: เพิ่มความกว้าง สูงอย่างน้อย 54 มุมโค้ง มีเงาให้ลอยเหนือเวที
458–467	หางฟอง: สี่เหลี่ยมเล็ก 16×16 มุม 45 องศา ซ้อนขึ้นไปใต้ฟอง 9 จุด เห็นเป็นสามเหลี่ยมชี้ลงหาน้องกล้า (เทคนิคเดียวกับหางฟองในเว็บทั่วไป)
472–479	StageButton: ปุ่มกลมใส กดแล้วสั่นเบา ๆ (selection) แล้วทำงาน มีคำอธิบายสำหรับโปรแกรมอ่านหน้าจอแยกจากคำบนปุ่ม (เช่น "ปิดเสียงน้องกล้า" แทน "เสียงเปิด")
483–494	สูงอย่างน้อย 44 (ขนาดนิ้ว) มุมเมียดา พื้นโปร่งจากสีตัวอักษรของเวที เข้มขึ้นตอนกด
495–498	ไอคอนสีทองของธัม กับคำบนปุ่ม

★ อ่านหน้าจอยาว ๆ ให้ไม่หลง

ไฟล์หน้าจามีโครงสร้างเดียวกันเกือบทุกไฟล์: `import` → คำคงที่ → คอมโพเนนต์ (state → ค่าที่คำนวณ → ฟังก์ชันเมื่อผู้ใช้ทำอะไร → `return` หน้าตา) → คอมโพเนนต์ย่อยท้ายไฟล์ เวลาอ่าน ให้หา "return (" ก่อนเพื่อดูหน้าตา แล้วค่อยย้อนไปดูว่าตัวแปรแต่ละตัวในหน้าตามาจากไหน ใน VS Code กด F12 ที่ชื่อตัวแปรเพื่อกระโดดไปที่ที่มันถูกสร้าง

อ่านโค้ด: useKlaTalk.ts, tts.ts และ tts.web.ts

3 ไฟล์ที่ทำให้น้องกล้าพูดได้ ทั้งบนมือถือและบนเว็บ ด้วยโค้ดชุดเดียวกัน

≈ ไฟล์ .ts กับ .web.ts ชื่อเหมือนกัน

แอปเรียก import ... from '../services/tts' แค่แบบเดียว แต่ตอนสร้างแอป Expo จะเลือกไฟล์ให้เอง: มือถือใช้ tts.ts เว็บใช้ tts.web.ts (un A7) ทั้งสองไฟล์จึงต้องมีฟังก์ชันชื่อเดียวกัน (thaiVoice, say, hush) เหมือนปลั๊กไฟที่หน้าตาเหมือนกัน แต่ข้างในต่อเข้าคนละระบบ

useKlaTalk ส่วนที่ 1: คำเปิด/ปิดเสียงที่ใช้ร่วมกันทั้งแอป

src/ui/kla/useKlaTalk.ts

บรรทัด 1-58

```

1 /**
2  * Makes น้องกล้า talk: cuts what it says into short subtitles, says each one
3  * aloud with the phone's Thai voice (when there is one and the sound is on),
4  * and reports which subtitle is being said so the big character can move its
5  * mouth. Without sound each subtitle stays for its reading time instead.
6  *
7  * The sound setting (เปิด/ปิดเสียงน้องกล้า) is kept on the phone.
8  */
9 import { useCallback, useEffect, useRef, useState, useSyncExternalStore } from 'react';
10 import { AccessibilityInfo, AppState, Platform } from 'react-native';
11 import { Storage } from '../data/storage';
12 import { speakable, talkChunks, talkMs } from '../domain/klaTalk';
13 import { hush, say, thaiVoice } from '../services/tts';
14
15 // -----
16 // Sound on/off, one shared setting
17 // -----
18
19 const KEY = 'mindpay.klaVoice';
20 let soundOn = true;
21 let loaded = false;
22 const listeners = new Set<() => void>();
23 const subscribe = (l: () => void) => {
24   listeners.add(l);
25   return () => {
26     listeners.delete(l);
27   };
28 };
29 const snapshot = () => soundOn;
30
31 async function load() {
32   if (loaded) return;
33   loaded = true;
34   try {
35     const saved = await Storage.getItem(KEY);
36     if (saved === 'off' || saved === 'on') {
37       soundOn = saved === 'on';
38       listeners.forEach((l) => l());

```

```

39   }
40 } catch {
41   // Sound stays on.
42 }
43 }
44
45 export function setKlaSound(on: boolean) {
46   soundOn = on;
47   listeners.forEach((l) => l());
48   Storage.setItem(KEY, on ? 'on' : 'off').catch(() => {});
49   if (!on) hush();
50 }
51
52 export function useKlaSound(): boolean {
53   const on = useSyncExternalStore(subscribe, snapshot, snapshot);
54   useEffect(() => {
55     load();
56   }, []);
57   return on;
58 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/kla/useKlaTalk.ts:1`

บรรทัด	ความหมาย
1–8	hook นี้ทำให้องค์กรกล่าวพูด: ตัดซับไต่เตล พูดทีละก่อนด้วยเสียงไทยของเครื่อง (ถ้ามีและเปิดเสียง) และบอกว่ากำลังพูดก่อนไหน ตัวละครจึงขยับปากได้ ถ้าไม่มีเสียง แต่ละก่อนคำตามเวลาอ่าน คำเปิด/ปิดเสียงเก็บในเครื่อง
9–13	import hook ของ React, ข้อมูลการเข้าถึง สถานะแอป และแพลตฟอร์ม, ที่เก็บข้อมูลในเครื่อง, ตัวตัด/แปลงข้อความจาก klaTalk.ts และฟังก์ชันเสียงจาก tts
19	ชื่อที่ใช้เก็บค่า "mindpay.klaVoice"
20–21	คำเริ่มต้น: เปิดเสียง · ยังไม่ได้โหลดคำจากเครื่อง (ตัวแปรระดับไฟล์ ทุกหน้าใช้ตัวเดียวกัน)
22–29	ระบบ "ผู้ฟัง": ใครอยากรู้เมื่อคำเปลี่ยน ลงทะเบียนไว้ใน listeners (subscribe ค้นฟังก์ชันยกเลิก) snapshot ค้นคำปัจจุบัน
31–43	load: อ่านคำที่เคยเก็บครั้งเดียว ถ้าเป็น 'off' หรือ 'on' ตั้งค่าแล้วแจ้งผู้ฟังทุกคน ถ้าอ่านไม่ได้ เสียงเปิดไว้ตามเดิม
45–50	setKlaSound: ตั้งค่าใหม่ แจ้งผู้ฟัง บันทึกลงเครื่อง (ไม่รอ และไม่สนถ้าบันทึกพลาด) และถ้าปิดเสียง สั่งเงียบทันที (hush)
52–58	useKlaSound: hook ให้หน้าจ้ออ่านคำ ด้วย useSyncExternalStore (วิธีมาตรฐานของ React ในการอ่านคำที่เก็บนอก React) หน้าจ้อจึงวาดใหม่เองเมื่อคำเปลี่ยน ไม่ว่าเปลี่ยนจากหน้าโค้ชหรือหน้าตั้งค่า

useKlaTalk ส่วนที่ 2: สถานะของการพูด

`src/ui/kla/useKlaTalk.ts`

บรรทัด 64–114

```

64 export interface Subtitle {
65   text: string;
66   /** Which piece of the whole message (from 0) and how many pieces there are. */
67   index: number;
68   total: number;
69 }
70

```

```

71 export function useKlaTalk() {
72   const sound = useKlaSound();
73   const [voice, setVoice] = useState<string | null | undefined>(undefined);
74   const [subtitle, setSubtitle] = useState<Subtitle | null>(null);
75   const [said, setSaid] = useState<string | null>(null);
76   const run = useRef(0);
77   const cancel = useRef<() => void>(() => {});
78   const pause = useRef<ReturnType<typeof setTimeout> | null>(null);
79   const soundRef = useRef(sound);
80   useEffect(() => {
81     soundRef.current = sound;
82   }, [sound]);
83
84   useEffect(() => {
85     let alive = true;
86     thaiVoice().then((v) => alive && setVoice(v));
87     return () => {
88       alive = false;
89     };
90   }, []);
91
92   // With a screen reader on, the subtitles are read by the reader itself, so น้องกล้า stays quiet.
93   // (Phones only: browsers cannot tell, and react-native-web always answers "on".)
94   const readerRef = useRef(false);
95   useEffect(() => {
96     if (Platform.OS === 'web') return;
97     AccessibilityInfo.isScreenReaderEnabled()
98       .then((on) => {
99         readerRef.current = on;
100       })
101       .catch(() => {});
102     const sub = AccessibilityInfo.addEventListener('screenReaderChanged', (on: boolean) => {
103       readerRef.current = on;
104     });
105     return () => sub.remove();
106   }, []);
107
108   const stop = useCallback(() => {
109     run.current++;
110     if (pause.current) clearTimeout(pause.current);
111     cancel.current();
112     cancel.current = () => {};
113     setSubtitle(null);
114   }, []);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/kla/useKlaTalk.ts:64

บรรทัด	ความหมาย
64–69	ซับไตเติล 1 ก่อน: ข้อความ · ก่อนที่ (เริ่ม 0) · มีกี่ก่อน
72–75	อ่านคำเสียง · เสียงไทยที่เจอ (undefined = กำลังตรวจ, null = ไม่มี) · ซับไตเติลปัจจุบัน · ข้อความล่าสุดที่พูด
76	run = เลขรอบการพูด ทุกครั้งที่เริ่มพูดใหม่หรือหยุด เลขนี้เพิ่ม งานเก่าที่ยังค้างจะรู้ว่า "ไม่ใช่รอบของฉันแล้ว" แล้วเลิกเอง
77–78	cancel = ฟังก์ชันหยุดเสียงก่อนปัจจุบัน · pause = ตัวจับเวลาช่วงพักระหว่างก่อน
79–82	เก็บคำเสียงล่าสุดไว้ใน ref ให้ฟังก์ชันที่ทำงานต่อเนื่องอ่านคำล่าสุดได้เสมอ (ปิดเสียงกลางคัน ก่อนถัดไปจะเจียบ)

บรรทัด	ความหมาย
84–90	เปิดหน้าแล้วตรวจสอบเสียงไทยครั้งเดียว (alive กับการตั้งค่าหลังหน้าถูกปิดไปแล้ว)
92–106	ถ้าเปิดโปรแกรมอ่านหน้าจอ (มือถือเท่านั้น เพราะเบราว์เซอร์บอกรับไม่ได้ และ react-native-web ตอบว่าเปิดเสมอ) จำไว้ใน readerRef และฟังการเปลี่ยนแปลง นื่องกล่าจะเจียบบให้โปรแกรมอ่านหน้าจออ่านแทน
108–114	stop: เพิ่มเลขรอบ (งานเก่าเล็ก) ยกเลิกช่วงพัก หยุดเสียงก่อนปัจจุบัน ล้างฟังก์ชันหยุด และลบซับไทม์ไลน์

useKlaTalk ส่วนที่ 3: speak พูดทีละท่อน

src/ui/kla/useKlaTalk.ts

บรรทัด 116–186

```

116  /** Say a message. `onDone` runs when all of it has been said (not when stopped). */
117  const speak = useCallback(
118    (text: string, onDone?: () => void) => {
119      stop();
120      const mine = run.current;
121      const pieces = talkChunks(text);
122      setSaid(text);
123      let i = 0;
124      const next = () => {
125        if (run.current !== mine) return;
126        if (i >= pieces.length) {
127          setSubtitle(null);
128          onDone?.();
129          return;
130        }
131        const piece = pieces[i];
132        setSubtitle({ text: piece, index: i, total: pieces.length });
133        i++;
134        const after = () => {
135          if (run.current !== mine) return;
136          // A short breath between pieces.
137          pause.current = setTimeout(next, 160);
138        };
139        const quietly = () => {
140          const t = setTimeout(after, talkMs(piece));
141          cancel.current = () => clearTimeout(t);
142        };
143        if (!soundRef.current || readerRef.current) {
144          quietly();
145          return;
146        }
147        // Wait for the voice check (it is done once), so even the first words are heard.
148        cancel.current = () => {};
149        thaiVoice()
150          .then((v) => {
151            setVoice(v);
152            if (run.current !== mine) return;
153            if (v) cancel.current = say(speakable(piece), { onDone: after });
154            else quietly();
155          })
156          .catch(quietly);
157      };
158      next();
159    },
160    [stop],

```

```

161 );
162
163 // Stop talking when the screen goes away or the app goes to the background.
164 useEffect(() => {
165     const sub = AppState.addListener('change', (state) => {
166         if (state !== 'active') stop();
167     });
168     return () => {
169         sub.remove();
170         stop();
171     };
172 }, [stop]);
173
174 return {
175     /** The piece being said right now (null when quiet). */
176     subtitle,
177     talking: subtitle !== null,
178     /** The last whole message. */
179     said,
180     speak,
181     stop,
182     sound,
183     /** undefined while checking; null when this phone has no Thai voice. */
184     voice,
185 };
186 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/kla/useKlaTalk.ts:116`

บรรทัด	ความหมาย
116–122	speak: หยุดเรื่องเก่าก่อน จำเลขรอบของตัวเอง (mine) ตัดข้อความเป็นก่อน และจำข้อความทั้งก้อน (ใช้กับปุ่ม "ฟังอีกครั้ง")
123–124	i = ก่อนถัดไปที่จะพูด · next = ฟังชันพูดก่อนถัดไป (เรียกตัวเองซ้ำจนครบ)
125	ถ้าเลขรอบไม่ใช่ของฉันแล้ว (มีคนกดหยุดหรือพูดเรื่องใหม่) เลิกทันที
126–130	ครบทุกก่อน: ลบซับไทม์ไลน์ เรียก onDone (ถ้ามี) จบ
131–133	แสดงก่อนนี้ (ข้อความ ลำดับ จำนวนก่อน) แล้วเลื่อน i
134–138	after = หลังก่อนนี้จบ (ถ้ายังเป็นรอบของฉัน) พักหายใจ 160 มิลลิวินาทีแล้วไปก่อนถัดไป
139–142	quietly = แบบไม่มีเสียง: ตั้งเวลาตามเวลาอ่าน (talkMs) แล้วค่อย after และตั้ง cancel ให้ยกเลิกเวลานี้ได้
143–146	ปิดเสียงอยู่ หรือเปิดโปรแกรมอ่านหน้าจอ → ใช้แบบเงียบ
147–156	มีเสียง: รอผลตรวจเสียงไทยก่อน (ตรวจครั้งเดียวแล้วจำ จึงได้ยืนยันตั้งแต่คำแรก) ถ้าระหว่างรอถูกหยุดไปแล้ว เลิก ถ้ามีเสียงไทย พูดก่อนนี้หลังแปลงด้วย speakable แล้วเมื่อพูดจบเรียก after (เก็บฟังก์ชันหยุดไว้ใน cancel) ถ้าไม่มีเสียงไทยหรือผิดพลาด ใช้แบบเงียบ
158	เริ่มพูดก่อนแรก
163–172	หยุดพูดเมื่อแอปไปอยู่เบื้องหลัง (สถานะไม่ใช่ active) และเมื่อหน้าถูกปิด
174–185	สิ่งที่ hook ส่งให้หน้าจอ: ซับไทม์ไลน์ · คำสั่งพูด (มีซับไทม์ไลน์อยู่) · ข้อความล่าสุด · speak · stop · ค่าเสียง · เสียงไทยที่เจอ

★ เทคนิค "เลขรอบ" กันงานเก่ามาทำงานใหม่

การพูดเป็นงานที่ "รอ" หลายจังหวะ (รอตรวจเสียง รอพูดจบ รอพัก) ระหว่างรอ ผู้ใช้อาจกดหยุดหรือถามคำถามใหม่ ถ้าไม่มีอะไรกันไว้ งานเก่าที่รอเสร็จก็หลังจะเปลี่ยนขั้วไต่เตลกับงานใหม่ เลขรอบ (run) แก้ปัญหานี้ง่าย ๆ: ทุกงานจำเลขรอบตอนเริ่ม และก่อนทำอะไรให้เช็คว่าเลขยังตรงไหม ถ้าไม่ตรงก็เลิก เทคนิคนี้ใช้ได้กับทุกงานที่ต้องรอ เช่น การค้นหาที่พิมพ์เร็ว ๆ

tts.ts: เสียงบนมือถือ

src/services/tts.ts

บรรทัด 1-54

```
1 /**
2  * น้องกล้า's voice on the phone: the phone's own Thai text-to-speech voice
3  * (Google on Android, Apple on iOS) through expo-speech.
4  *
5  * The native module is looked up optionally, like the speech recognizer:
6  * app builds made before it was added do not have it, and an over-the-air
7  * update must not crash them. There น้องกล้า still talks with subtitles and
8  * a moving mouth, just without sound. The web version is tts.web.ts.
9  */
10 import { requireOptionalNativeModule } from 'expo';
11
12 interface Subscription {
13   remove(): void;
14 }
15 interface NativeVoice {
16   identifier: string;
17   name: string;
18   quality: string;
19   language: string;
20 }
21 interface NativeTts {
22   speak(id: string, text: string, options: Record<string, unknown>): unknown;
23   stop(): Promise<void>;
24   getVoices?(): Promise<NativeVoice[]>;
25   addListener(event: string, listener: (e: { id?: string }) => void): Subscription;
26 }
27
28 const Tts = requireOptionalNativeModule<NativeTts>('ExpoSpeech');
29
30 export interface SayHandlers {
31   /** Called once, when the piece has been said (or could not be). */
32   onDone(): void;
33 }
34
35 /** A calm, friendly way of speaking: a little slower and a little lower than normal. */
36 const RATE = 0.9;
37 const PITCH = 0.95;
38
39 let voicePick: Promise<string | null> | null = null;
40
41 /** The Thai voice to use, or null when this phone cannot speak Thai. */
42 export function thaiVoice(): Promise<string | null> {
43   if (!Tts?.getVoices) return Promise.resolve(null);
44   voicePick ??= Tts.getVoices()
45     .then((voices) => {
```

```

46     const thai = voices.filter((v) => v.language?.toLowerCase().replace('_', '-').startsWith('th'));
47     if (!thai.length) return null;
48     // Better quality first, then voices that work without the internet.
49     const score = (v: NativeVoice) => (v.quality === 'Enhanced' ? 2 : 0) +
    (/network/i.test(v.identifier) ? 0 : 1);
50     return [...thai].sort((a, b) => score(b) - score(a))[0].identifier;
51   })
52   .catch(() => null);
53   return voicePick;
54 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/tts.ts:1

บรรทัด	ความหมาย
1-9	เสียงของน้องกลับมาบนมือถือ = เสียงอ่านภาษาไทยของมือถือ (Google บน Android, Apple บน iOS) ผ่าน expo-speech โหลดโมดูลแบบไม่บังคับ เพราะแอปที่ build ก่อนเพิ่มโมดูลนี้ไม่มีมัน และการอัปเดต OTA ต้องไม่ทำให้แอปเหล่านั้น crash
10	requireOptionalNativeModule: ขอโมดูล native ถ้าไม่มีได้ null แทนที่จะพัง
12-26	บอกหน้าต่างของโมดูลเสียง (ชนิดข้อมูล): พูด หยุด ขอรายชื่อเสียง และฟังเหตุการณ์
28	Tts = โมดูล ExpoSpeech หรือ null
30-33	ผู้เรียก say ต้องส่ง onDone มา (เรียกครั้งเดียวเมื่อพูดจบหรือพูดไม่ได้)
35-37	ความเร็ว 0.9 ระดับเสียง 0.95 = ช้าและต่ำกว่าปกติเล็กน้อย ให้ฟังสุขุมเป็นมิตร
39	จำผลการเลือกเสียงไว้ (เลือกครั้งเดียว)
42-43	thaiVoice: ถ้าไม่มีโมดูลหรือขอรายชื่อเสียงไม่ได้ → null
44-46	ขอรายชื่อเสียงทั้งหมด (ครั้งแรกเท่านั้น ?? = ตั้งค่าเฉพาะถ้ายังว่าง) เอาเฉพาะเสียงที่ภาษาขึ้นต้นด้วย th (แปลง _ เป็น - และตัวเล็กก่อนเทียบ)
47	ไม่มีเสียงไทย → null
48-50	ให้คะแนน: คุณภาพ Enhanced ได้ 2 · เสียงที่ไม่ต้องใช้เน็ตได้ 1 เรียงคะแนนมากไปน้อย เลือกตัวแรก คั่นรหัสเสียง
52	ผิดพลาด → null

src/services/tts.ts

บรรทัด 56-99

```

56 let seq = 0;
57
58 /** Say one short piece of Thai text. Returns a function that stops it. */
59 export function say(text: string, h: SayHandlers): () => void {
60   if (!Tts) {
61     h.onDone();
62     return () => {};
63   }
64   const id = `kla${++seq}`;
65   let finished = false;
66   const subs: Subscription[] = [];
67   const finish = () => {
68     if (finished) return;
69     finished = true;

```

```

70   clearTimeout(guard);
71   subs.splice(0).forEach((s) => s.remove());
72   h.onDone();
73 };
74 const mine = (e: { id?: string }) => e?.id === id && finish();
75 subs.push(
76   Tts.addListener('Exponent.speakingDone', mine),
77   Tts.addListener('Exponent.speakingStopped', mine),
78   Tts.addListener('Exponent.speakingError', mine),
79 );
80 // Never wait forever if the phone's voice does not report back.
81 const guard = setTimeout(finish, 4000 + text.length * 160);
82 thaiVoice()
83   .then((voice) => {
84     if (finished) return;
85     // "th", not "th-TH": Android builds its Locale from the whole string, and "th-TH" is not a
    language code there.
86     return Tts.speak(id, text, { language: 'th', rate: RATE, pitch: PITCH, ...(voice ? { voice
    } : {}) });
87   })
88   .catch(finish);
89 return () => {
90   if (finished) return;
91   Tts.stop().catch(() => {});
92   finish();
93 };
94 }
95
96 /** Stop talking at once. */
97 export function hush() {
98   Tts?.stop().catch(() => {});
99 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/tts.ts:56

บรรทัด	ความหมาย
56	ตัวนับไว้ตั้งรหัสให้แต่ละท่อน
59–63	say: ถ้าไม่มีโมดูลเสียง บอกว่าจบทันที (หน้าจอก็จะไปก่อนต่อไป) ค้นฟังก์ชันหยุดที่ไม่ทำอะไร
64–66	ตั้งรหัสก่อน เช่น kla1, kla2 · ธงว่าจบแล้วหรือยัง · รายการผู้ฟังเหตุการณ์
67–73	finish: ทำครั้งเดียวเท่านั้น (ธง finished) ยกเลิกตัวกันค้าง ถอดผู้ฟังทั้งหมด แล้วเรียก onDone
74	mine: เหตุการณ์ที่เป็นของท่อนนี้ (รหัสตรง) → finish
75–79	ฟัง 3 เหตุการณ์ของ expo-speech: พุดจบ · ถูกหยุด · ผิดพลาด ทั้งหมดถือว่าจบท่อนนี้
80–81	ตัวกันค้าง: ถ้าเสียงไม่ส่งเหตุการณ์กลับมาภายใน 4 วินาที + 160 ms ต่อตัวอักษร ถือว่าจบ ไม่รอตลอดไป
82–88	หาเสียงไทยแล้วสั่งพูด ภาษา 'th' (คอมเมนต์บอกว่า Android สร้าง Locale จากข้อความทั้งก้อน "th-TH" จึงไม่ใช้รหัสภาษาที่ถูกบนนั้น) ความเร็ว ระดับเสียง และเสียงที่เลือก (ถ้ามี) ถ้าผิดพลาด finish
89–93	ฟังก์ชันหยุดที่คั่นให้ผู้เรียก: ถ้ายังไม่จบ สั่งหยุดเสียงแล้ว finish
97–99	hush: หยุดเสียงทุกอย่างทันที (ใช้ตอนปิดเสียง)

tts.web.ts: เลี้ยงบนเว็บ

src/services/tts.web.ts

บรรทัด 1-92

```
1 /**
2  * บ๊องกล้า's voice on the web: the browser's own speech (Web Speech API) with
3  * a Thai voice when the computer or phone has one (Edge: Niwat or Premwadee,
4  * Safari: Kanya or Narisa, Chrome on Android: Google's Thai voice). Without a
5  * Thai voice บ๊องกล้า talks with subtitles only.
6  */
7 import type { SayHandlers } from './tts';
8
9 export type { SayHandlers };
10
11 const synth: SpeechSynthesis | undefined = typeof window !== 'undefined' && 'speechSynthesis' in
  window ? window.speechSynthesis : undefined;
12
13 /** Calm and friendly: a little slower and lower than normal. */
14 const RATE = 0.95;
15 const PITCH = 0.95;
16
17 /** Voices that sound calm and warm come first. */
18 const PREFERRED = [/niwat/i, /premwadee/i, /natural/i, /google/i, /narisa/i, /kanya/i,
  /pattara/i];
19
20 let picked: Promise<SpeechSynthesisVoice | null> | null = null;
21 // Voices can arrive after the page loads (or change): choose again then.
22 synth?.addEventListener?.('voiceschanged', () => {
23   picked = null;
24 });
25
26 function loadVoices(): Promise<SpeechSynthesisVoice[]> {
27   if (!synth) return Promise.resolve([]);
28   const now = synth.getVoices();
29   if (now.length) return Promise.resolve(now);
30   return new Promise((resolve) => {
31     const done = () => resolve(synth.getVoices());
32     synth.addEventListener?.('voiceschanged', done, { once: true });
33     // Some browsers never fire the event when there are no voices.
34     setTimeout(done, 1500);
35   });
36 }
37
38 function pick(): Promise<SpeechSynthesisVoice | null> {
39   picked ??= loadVoices().then((voices) => {
40     const thai = voices.filter((v) => v.lang?.toLowerCase().replace('_', '-').startsWith('th'));
41     if (!thai.length) return null;
42     const rank = (v: SpeechSynthesisVoice) => {
43       const i = PREFERRED.findIndex((re) => re.test(v.name));
44       return i === -1 ? PREFERRED.length : i;
45     };
46     return [...thai].sort((a, b) => rank(a) - rank(b))[0];
47   });
48   return picked;
49 }
50
51 export function thaiVoice(): Promise<string | null> {
52   return pick().then((v) => v?.name ?? null);
53 }
54
```

```

55 export function say(text: string, h: SayHandlers): () => void {
56   let finished = false;
57   const finish = () => {
58     if (finished) return;
59     finished = true;
60     clearTimeout(guard);
61     h.onDone();
62   };
63   // Browsers sometimes never report the end (e.g. speech blocked before the first tap).
64   const guard = setTimeout(finish, 4000 + text.length * 160);
65   if (!synth) {
66     finish();
67     return () => {};
68   }
69   pick()
70     .then((voice) => {
71       if (finished) return;
72       if (!voice) return finish();
73       const u = new SpeechSynthesisUtterance(text);
74       u.voice = voice;
75       u.lang = voice.lang || 'th-TH';
76       u.rate = RATE;
77       u.pitch = PITCH;
78       u.onend = finish;
79       u.onerror = finish;
80       synth.speak(u);
81     })
82     .catch(finish);
83   return () => {
84     if (finished) return;
85     synth.cancel();
86     finish();
87   };
88 }
89
90 export function hush() {
91   synth?.cancel();
92 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/tts.web.ts:1

บรรทัด	ความหมาย
1–6	เสียงบนเว็บใช้ Web Speech API ของเบราว์เซอร์ พร้อมเสียงไทยถ้าเครื่องมี (Edge: Niwat หรือ Premwadee, Safari: Kanya หรือ Narisa, Chrome บน Android: เสียงไทยของ Google) ไม่มีเสียงไทย → ชับไตเต็ลอย่างเดียว
7–9	ใช้นัด SayHandlers ร่วมกับไฟล์มือถือ (สัญญาเดียวกัน)
11	synth = ตัวพูดของเบราว์เซอร์ ถ้ามี (เช็ก window ก่อนเพราะตอนสร้างหน้าเว็บล่วงหน้าไม่มี window)
13–15	ความเร็ว 0.95 ระดับเสียง 0.95
17–18	ลำดับชื่อเสียงที่ชอบ: Niwat, Premwadee, Natural, Google, Narisa, Kanya, Pattara
20–24	จำเสียงที่เลือก ถ้าเบราว์เซอร์แจ้งว่ารายชื่อเสียงเปลี่ยน ล้างค่าที่จำไว้ให้เลือกใหม่
26–36	loadVoices: บางเบราว์เซอร์โหลดรายชื่อเสียงช้า ถ้ายังว่างรอเหตุการณ์ voiceschanged แต่ไม่เกิน 1.5 วินาที (บางเบราว์เซอร์ไม่ส่งเหตุการณ์เลยถ้าไม่มีเสียง)

บรรทัด	ความหมาย
38–49	pick: เอาเฉพาะเสียงไทย เรียงตามลำดับชื่อที่ชอบ (ชื่อที่ไม่อยู่ในรายการไปท้ายสุด) เลือกตัวแรก
51–53	thaiVoice คัดชื่อเสียงหรือ null (หน้าโคชใช้ตัดสินใจว่าจะแสดงบรรทัด "เครื่องนี้ยังไม่มีเสียงอ่านภาษาไทย" ไหม)
55–64	say: finish ครั้งเดียว และตัวกันค่างแบบเดียวกับมือถือ (คอมเมนต์: บางครั้งเบราว์เซอร์ไม่แจ้งว่าพูดจบ เช่น ถูกบล็อกเสียงก่อนผู้ใช้แตะจอสักครั้งแรก)
65–68	เบราว์เซอร์ไม่มีระบบเสียง → จบทันที
69–82	ได้เสียงแล้ว: สร้าง "คำพูด" (SpeechSynthesisUtterance) ตั้งเสียง ภาษา ความเร็ว ระดับเสียง ตั้งให้ finish เมื่อพูดจบหรือผิดพลาด แล้วส่งพูด
83–87	ฟังก์ชันหยุด: ยกเลิกเสียงของเบราว์เซอร์แล้ว finish
90–92	hush: ยกเลิกทุกเสียง

i เทสต์ E2E ปลอมเสียงของเบราว์เซอร์ได้ยังไง

เบราว์เซอร์ที่ใช้ทดสอบไม่มีเสียงไทย เทสต์จึงแทน window.speechSynthesis ด้วยของจำลองที่มีเสียง 3 เสียง (อังกฤษ 1 ไทย 2: Premwadee กับ Niwat) และบันทึกทุกประโยคที่ถูกสังพูด แล้วตรวจว่า: ภาษา th-TH · เลือก Niwat (ตามลำดับที่ชอบ) · ความเร็วและระดับเสียงต่ำกว่า 1 · คำทักทายมีคำว่า "สวัสดี" · คำตอบ "฿0" ถูกพูดเป็น "0 บาท" ทั้งหมดนี้ทำได้เพราะโค้ดเรียกเสียงผ่าน API มาตรฐาน ไม่ได้ผูกกับเครื่องใดเครื่องหนึ่ง

อ่านโค้ด: เวทีและตัวน้องกล้า (KlaStage, KlaBackdrop, KlaPicture)

ภาพเคลื่อนไหวทั้งหมดของน้องกล้าตัวใหญ่อยู่ในไฟล์เดียว 201 บรรทัด

ส่วนที่ 1: หัวไฟล์ จังหวะปาก และตัวช่วยองศา

src/ui/kla/KlaStage.tsxบรรทัด 1-31

```
1 /**
2  * The big น้องกล้า that talks (the coach) and poses (the skin collection).
3  *
4  * Each part of the drawing (body, arms, head, eyes, mouth) is its own layer,
5  * turned around its own joint by the native animation driver, so the motion
6  * stays smooth without redrawing the picture: it breathes and sways, blinks,
7  * tilts its head, and while it talks the mouth opens and closes and the arms
8  * and head make small gestures. Tapping makes it hop.
9  *
10 * Idle motion only runs while `active` (the screen is showing), and all of it
11 * except the talking mouth is off when the phone's "Reduce motion" is on.
12 */
13 import { useEffect, useId, useMemo, useState, type ReactNode } from 'react';
14 import { Animated, Easing, Platform, View } from 'react-native';
15 import Svg from 'react-native-svg';
16 import type { BuddyMood } from '../domain/buddy';
17 import type { SkinId } from '../domain/skins';
18 import { useReduceMotion } from '../motion';
19 import { KlaArm, KlaBack, KlaDefs, KlaExtras, KlaEyes, KlaGlasses, KlaGround, KlaHead, KlaMouth,
20   KlaMouthOpen, KLA_H, KLA_W, PIVOT } from './art';
21
22 const useNative = Platform.OS !== 'web';
23 const ease = Easing.inOut(Easing.sin);
24
25 /** Mouth shapes while talking: [how open 0..1, milliseconds]. Uneven on purpose, like real
26   syllables. */
27 const SYLLABLES: [number, number][] = [
28   [0.85, 110], [0.25, 90], [0.7, 120], [0.1, 80], [1, 130], [0.35, 100], [0.8, 110], [0.2, 90],
29   [0.6, 100], [0.05, 120], [0.9, 120], [0.3, 90], [0.75, 110], [0.15, 140],
30 ];
31
32 const deg = (v: Animated.AnimatedInterpolation<number> | Animated.AnimatedAddition<number>) =>
33   v.interpolate({ inputRange: [-360, 360], outputRange: ['-360deg', '360deg'] });
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → src/ui/kla/KlaStage.tsx:1

บรรทัด	ความหมาย
1-8	น้องกล้าตัวใหญ่ที่พูด (โค้ช) และโพสท่า (ตู้สกิน) ภาพแต่ละชิ้น (ตัว แขน หัว ตา ปาก) เป็นชิ้นของตัวเอง หมุนรอบข้อต่อของตัวเองด้วย native driver จึงสับโดยไม่ต้องวาดภาพใหม่: หายใจ โยก กะพริบ เอียงหัว ตอนพูดปากเปิดปิด แขนกับหัวทำท่า แตะแล้วกระโดด
10-11	ทำางทำงานเฉพาะตอน active (หน้าจอเปิดอยู่) และถูกถ่ายกเว้นปากตอนพูดจะปิดเมื่อมือถือเปิด "ลดการเคลื่อนไหว"

บรรทัด	ความหมาย
13–19	import: hook ของ React · Animated (ระบบภาพเคลื่อนไหว) Easing (รูปแบบความเร่ง) · Svg · ชนิดอาร์มและชุด · hook ลดการเคลื่อนไหว · ชื่นภาพทั้งหมดจาก art.tsx พร้อมขนาดพื้น (KLA_W, KLA_H) และจุดหมุน (PIVOT)
21	ใช้ native driver เฉพาะมือถือ (เว็บไม่มี)
22	ความเร่งแบบคลื่นไซน์ เข้าออกนุ่ม ๆ เหมือนการหายใจ
24–28	14 รูปปาก [เปิดแคโคโน 0–1, ที่มิลลิวินาที] ตั้งใจให้ไม่สม่ำเสมอเหมือนพยางค์จริง (ครบบรอบ 1,510 มิลลิวินาที)
30–31	ตัวช่วยแปลงตัวเลขของศาเป็นข้อความ "...deg" ที่ระบบหมุนต้องการ (interpolate แบบเส้นตรง –360 ถึง 360)

ส่วนที่ 2: รับค่าและสร้างตัวแปรภาพเคลื่อนไหว

src/ui/kla/KlaStage.tsx

บรรทัด 33–66

```

33 export function KlaStage({
34   skin,
35   mood = 'happy',
36   width,
37   talking = false,
38   active = true,
39   hop = 0,
40   onDark = false,
41 }): {
42   skin: SkinId;
43   mood?: BuddyMood;
44   /** Width in points; the height is 1.2 × width. */
45   width: number;
46   talking?: boolean;
47   active?: boolean;
48   /** Changes when the user taps it: a hop. */
49   hop?: number;
50   onDark?: boolean;
51 }) {
52   const reduce = useReduceMotion();
53   const id = `${useId().replace(/^[a-zA-Z0-9]/g, '')}`;
54   const k = width / KLA_W;
55   const height = KLA_H * k;
56   const [v] = useState(() => ({
57     breathe: new Animated.Value(0),
58     sway: new Animated.Value(0),
59     tilt: new Animated.Value(0),
60     arms: new Animated.Value(0),
61     blink: new Animated.Value(1),
62     mouth: new Animated.Value(0),
63     gesture: new Animated.Value(0),
64     jump: new Animated.Value(0),
65   }));
66   const idle = active && !reduce;

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/kla/KlaStage.tsx:33

บรรทัด	ความหมาย
33–51	props: ชุต · อารมณ์ (ค่าเริ่มต้น happy) · ความกว้าง (สูง = 1.2 เท่า) · กำลังพูด · หน้าเปิดอยู่ · ตัวนับกระโดด (เปลี่ยนเมื่อเตะ) · อยู่บนพื้นเข้มไหม
52	ผู้ใช้เปิดลดการเคลื่อนไหวไหม
53	รหัสไม่ซ้ำของตัวละครนี้ (ใช้ตั้งชื่อ gradient ใน SVG ไม่ให้ชนกันเมื่อมีน้องกล่าหลายตัวบนจอ)
54–55	อัตราย่อขยาย k = ความกว้าง ÷ 200 และความสูงจริง
56–65	สร้างค่าภาพเคลื่อนไหว 8 ตัว ครั้งเดียวตอนสร้างหน้า: หายใจ โยก เอียง แขน กะพริบ (เริ่ม 1 = ตาเปิด) ปาก ทำทางกระโดด
66	idle = ให้ทำท่าว่างได้ไหม (หน้าเปิดอยู่ และไม่ได้ลดการเคลื่อนไหว)

ส่วนที่ 3: ทำว่าง และการกะพริบ

src/ui/kla/KlaStage.tsx

บรรทัด 68–96

```

68 // Breathing, a slow sway, the head tilting and the arms resting, each on its own rhythm.
69 useEffect(() => {
70   if (!idle) return;
71   const wave = (value: Animated.Value, ms: number, from = -1, to = 1) =>
72     Animated.loop(
73       Animated.sequence([
74         Animated.timing(value, { toValue: to, duration: ms, easing: ease, useNativeDriver:
useNative }),
75         Animated.timing(value, { toValue: from, duration: ms, easing: ease, useNativeDriver:
useNative }),
76       ]),
77     );
78   const loops = [wave(v.breathe, 1900, 0, 1), wave(v.sway, 3100), wave(v.tilt, 2600),
wave(v.arms, 2300)];
79   loops.forEach((l) => l.start());
80   return () => loops.forEach((l) => l.stop());
81 }, [idle, v]);
82
83 // Blinks: every few seconds, now and then twice.
84 useEffect(() => {
85   if (!idle || mood === 'happy' || mood === 'cheer' || mood === 'sleepy') {
86     v.blink.setValue(1);
87     return;
88   }
89   const shut = (ms = 70) => Animated.timing(v.blink, { toValue: 0.08, duration: ms, easing:
Easing.in(Easing.quad), useNativeDriver: useNative });
90   const open = (ms = 110) => Animated.timing(v.blink, { toValue: 1, duration: ms, easing:
Easing.out(Easing.quad), useNativeDriver: useNative });
91   const loop = Animated.loop(
92     Animated.sequence([Animated.delay(2600), shut(), open(), Animated.delay(3400), shut(),
open(90), Animated.delay(140), shut(60), open()]),
93   );
94   loop.start();
95   return () => loop.stop();
96 }, [idle, mood, v]);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/kla/KlaStage.tsx:68

บรรทัด	ความหมาย
69–70	ถ้าทำท่าวางไม่ได้ ไม่ทำอะไร
71–77	ฟังก์ชัน wave สร้างคลื่นวนไม่รู้จบ: ไปถึงค่า to ใช้เวลา ms แล้วกลับมาที่ from ใช้เวลา ms เท่ากัน
78	4 คลื่น: หายใจ 0↔1 ทางละ 1.9 วิ · โยก -1↔1 ทางละ 3.1 วิ · เอียงหัว 2.6 วิ · แขน 2.3 วิ
79–80	เริ่มทุกคลื่น และเมื่อ idle เปลี่ยนหรือหน้าถูกปิด หยุดทุกคลื่น (ฟังก์ชันที่ return จาก useEffect คือ "เก็บกวาด")
84–88	ไม่กะพริบเมื่อทำท่าวางไม่ได้ หรืออารมณ์ happy/cheer/sleepy (ตาโค้งหรือปิดอยู่แล้ว) ตังตาเปิดไว้
89–90	shut = หลับตาเหลือ 8% ใน 70 มิลลิวินาที · open = สลับตาใน 110 มิลลิวินาที
91–93	วนไม่รู้จบ: รอ 2.6 วิ กะพริบ 1 ที รอ 3.4 วิ กะพริบ 2 ทีติด (ห่าง 0.14 วิ)
94–95	เริ่ม และหยุดเมื่อเงื่อนไขเปลี่ยน

ส่วนที่ 4: ตอนพูด และตอนกระโดด

src/ui/kla/KlaStage.tsx

บรรทัด 98–138

```

98  // Talking: the mouth follows syllables, the head nods and a hand explains.
99  useEffect(() => {
100    if (!talking) {
101      Animated.parallel([
102        Animated.timing(v.mouth, { toValue: 0, duration: 140, easing: Easing.out(Easing.quad),
103        useNativeDriver: useNative }),
104        Animated.spring(v.gesture, { toValue: 0, friction: 6, tension: 60, useNativeDriver:
105        useNative }),
106      ]).start();
107      return;
108    }
109    const mouth = Animated.loop(
110      Animated.sequence(SYLLABLES.map(([to, ms]) => Animated.timing(v.mouth, { toValue: to,
111      duration: ms, easing: Easing.inOut(Easing.quad), useNativeDriver: useNative }))),
112    );
113    mouth.start();
114    if (reduce) return () => mouth.stop();
115    const gesture = Animated.loop(
116      Animated.sequence([
117        Animated.timing(v.gesture, { toValue: 1, duration: 520, easing: ease, useNativeDriver:
118        useNative }),
119        Animated.timing(v.gesture, { toValue: 0.35, duration: 480, easing: ease, useNativeDriver:
120        useNative }),
121        Animated.timing(v.gesture, { toValue: 0.9, duration: 560, easing: ease, useNativeDriver:
122        useNative }),
123        Animated.timing(v.gesture, { toValue: 0, duration: 620, easing: ease, useNativeDriver:
124        useNative }),
125        Animated.delay(260),
126      ]),
127    );
128    gesture.start();
129    return () => {
130      mouth.stop();
131      gesture.stop();
132    };
133  }, [talking, reduce, v]);

```

```

127
128 // A hop when tapped: up, then a springy landing.
129 useEffect(() => {
130   if (!hop || reduce) return;
131   v.jump.setValue(0);
132   const anim = Animated.sequence([
133     Animated.timing(v.jump, { toValue: 1, duration: 190, easing: Easing.out(Easing.quad),
134     useNativeDriver: useNative }),
135     Animated.spring(v.jump, { toValue: 0, friction: 4, tension: 110, useNativeDriver: useNative
136   })),
137   anim.start();
138   return () => anim.stop();
139 }, [hop, reduce, v]);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/kla/KlaStage.tsx:98

บรรทัด	ความหมาย
100–106	ไม่ได้พูด: ปิดปากใน 0.14 วิ และดึงท่าทางกลับที่เดิมแบบสปริง (พร้อมกัน ด้วย parallel)
107–110	กำลังพูด: ปากเล่นตาม 14 รูปปากวนไปเรื่อย ๆ
111	ถาลดการเคลื่อนไหว ทำแค่ปาก (ไม่มีท่าทาง) แล้วจบ
112–121	ท่าอธิบายวนไปเรื่อย ๆ: ขึ้นเต็มใน 0.52 วิ ลงเหลือ 35% ใน 0.48 วิ ขึ้น 90% ใน 0.56 วิ ลงสุดใน 0.62 วิ พัก 0.26 วิ (ไม่เท่ากันเหมือนคนอธิบายจริง)
122–125	หยุดปากและท่าทางเมื่อเลิกพูด
129–130	กระโดดเมื่อ hop เปลี่ยน (ค่า 0 ตอนเริ่มไม่กระโดด) และไม่กระโดดถาลดการเคลื่อนไหว
131–137	ตั้งค่ากลับ 0 แล้ว: ขึ้นถึง 1 ใน 0.19 วิ แล้วลงแบบสปริง (friction 4 tension 110 = เด้งนิดหน่อยตอนลงพื้น)

ส่วนที่ 5: แปลงค่าเป็นการเลื่อนและหมุนของแต่ละชิ้น

src/ui/kla/KlaStage.tsx

บรรทัด 140–173

```

140 const t = useMemo(() => {
141   // Joints in points, as [x, y, z]: phones read a "12.5px" string wrongly (whole numbers
142   // only), numbers work everywhere.
143   const at = (x: number, y: number): [number, number, number] => [x * k, y * k, 0];
144   const cheer = mood === 'cheer';
145   return {
146     groundOrigin: at(100, 231),
147     bodyOrigin: at(100, 228),
148     neck: at(PIVOT.neck.x, PIVOT.neck.y),
149     shoulderL: at(PIVOT.shoulderL.x, PIVOT.shoulderL.y),
150     shoulderR: at(PIVOT.shoulderR.x, PIVOT.shoulderR.y),
151     eyesAt: at(PIVOT.eyes.x, PIVOT.eyes.y),
152     mouthAt: at(100, 141),
153     body: [
154       { translateY: Animated.add(v.breathe.interpolate({ inputRange: [0, 1], outputRange: [0,
155       -1.2 * k ]}), v.jump.interpolate({ inputRange: [0, 1], outputRange: [0, -26 * k ]})) },
156       { rotate: deg(v.sway.interpolate({ inputRange: [-1, 1], outputRange: [-1.1, 1.1 ]})) },
157       { scaleY: Animated.add(v.breathe.interpolate({ inputRange: [0, 1], outputRange: [1,
158       1.018 ]}), v.jump.interpolate({ inputRange: [-0.3, 0, 0.4, 1], outputRange: [-0.1, 0, 0.05, 0],
159       extrapolate: 'clamp' })) }],

```

```

156     { scaleX: Animated.add(v.breathe.interpolate({ inputRange: [0, 1], outputRange: [1,
0.993] })), v.jump.interpolate({ inputRange: [-0.3, 0, 0.4, 1], outputRange: [0.08, 0, -0.03, 0],
extrapolate: 'clamp' })) },
157   ],
158   ground: [{ scale: v.jump.interpolate({ inputRange: [-0.3, 0, 1], outputRange: [1.06, 1,
0.72], extrapolate: 'clamp' }) }],
159   groundOpacity: v.jump.interpolate({ inputRange: [0, 1], outputRange: [1, 0.55],
extrapolate: 'clamp' }),
160   head: [
161     { translateY: v.gesture.interpolate({ inputRange: [0, 1], outputRange: [0, 1.6 * k] }) },
162     { rotate: deg(Animated.add(v.tilt.interpolate({ inputRange: [-1, 1], outputRange: [-2.2,
2.2] })), v.gesture.interpolate({ inputRange: [0, 1], outputRange: [0, 2.6] }))) },
163   ],
164   // Left hand rises with a clockwise turn, the right with an anticlockwise one.
165   // Raised arms (cheer) wave outwards, away from the leaves, so the hands stay in sight.
166   armL: [{ rotate: deg(Animated.add(v.arms.interpolate({ inputRange: [-1, 1], outputRange:
[-3, 3] })), v.gesture.interpolate({ inputRange: [0, 1], outputRange: [0, cheer ? -9 : 7] }))) }],
167   armR: [{ rotate: deg(Animated.add(v.arms.interpolate({ inputRange: [-1, 1], outputRange:
[3, -3] })), v.gesture.interpolate({ inputRange: [0, 1], outputRange: [0, cheer ? 9 : -26] }))) }],
168   blink: [{ scaleY: v.blink }],
169   closed: v.mouth.interpolate({ inputRange: [0, 0.18], outputRange: [1, 0], extrapolate:
'clamp' }),
170   open: v.mouth.interpolate({ inputRange: [0, 0.18], outputRange: [0, 1], extrapolate:
'clamp' }),
171   openShape: [{ scaleY: v.mouth.interpolate({ inputRange: [0, 1], outputRange: [0.3, 1] }) },
{ scaleX: v.mouth.interpolate({ inputRange: [0, 1], outputRange: [0.8, 1] }) }],
172   };
173   }, [k, mood, v]);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/kla/KlaStage.tsx:140

บรรทัด	ความหมาย
141–142	จุดหมุนเป็นตัวเลข [x, y, z] (คอมเมนต์: มีสื่ออ่านข้อความแบบ "12.5px" ผิด ตัวเลขใช้ได้ทุกที่) คูณ k ให้ตรงขนาดจริง
143	อารมณ์ cheer ทำแขนต่างจากปกติ
145–151	จุดหมุน: ฟัน ตัว (ฐานลำต้น) คอ ไหล่ซ้าย ไหล่ขวา ตา ปาก
152–157	ตัว: เลื่อนขึ้นตามการหายใจ (1.2 หน่วย) บวกกระโดด (26 หน่วย) · หมุนตามการโยก $\pm 1.1^\circ$ · ยึดแนวตั้งตามหายใจ (+1.8%) และ "ย่อ-ยืด" ตอนกระโดด · แนวนอนหดนิดหน่อยตามหายใจและป้องตอนลงพื้น (Animated.add รวม 2 การเคลื่อนไหวเป็นค่าเดียว)
158–159	เงาพื้นหดเหลือ 72% และจางเหลือ 55% ตอนลอยสูงสุด
160–163	หัว: ก้มลงตามท่าทาง (1.6 หน่วย) และหมุน = เอียงว่าง $\pm 2.2^\circ$ บวกพยัคตอนพุด 2.6°
164–167	แขน: แกว่งว่าง $\pm 3^\circ$ (ซ้ายขวาสวนกัน) บวกท่าทาง: ปกติมือซ้าย $+7^\circ$ มือขวา -26° (ยกอธิบาย) · cheer ทางออก ซ้าย -9° ขวา $+9^\circ$ ให้มือไม่บังใบ
168	ตา: บิดแนวตั้งตามค่ากะพริบ
169–171	ปาก: เปิดเกิน 18% ปากปิดจางหายและปากเปิดโผล่แทน (สลับภาพ 2 ภาพ) ปากเปิดยึดแนวตั้ง 30–100% และแนวนอน 80–100% ตามค่าปาก
173	คำนวณใหม่เฉพาะเมื่อขนาด อารมณ์ หรือค่าเปลี่ยน (useMemo)

ส่วนที่ 6: ซ่อนชั้นภาพ

src/ui/kla/KlaStage.tsx

บรรทัด 175–201

```
175   const layer = (children: ReactNode, defs = true) => (  
176     <Svg width={width} height={height} viewBox={`0 0 ${KLA_W} ${KLA_H}`} style={{ position:  
    'absolute', left: 0, top: 0 }}>  
177       {defs ? <KlaDefs id={id} skin={skin} onDark={onDark} /> : null}  
178       {children}  
179     </Svg>  
180   );  
181   const fill = { position: 'absolute' as const, left: 0, top: 0, width, height };  
182  
183   return (  
184     <View style={{ width, height }} accessibilityElementsHidden importantForAccessibility="no-  
    hide-descendants" pointerEvents="none">  
185       <Animated.View style={[fill, { transformOrigin: t.groundOrigin, transform: t.ground,  
    opacity: t.groundOpacity }]}>{layer(<KlaGround id={id} />)}</Animated.View>  
186       <Animated.View style={[fill, { transformOrigin: t.bodyOrigin, transform: t.body }]}>  
187         {layer(<KlaBack id={id} skin={skin} />)}  
188         <Animated.View style={[fill, { transformOrigin: t.shoulderL, transform: t.armL }]}>  
    {layer(<KlaArm id={id} skin={skin} side="L" mood={mood} />)}</Animated.View>  
189         <Animated.View style={[fill, { transformOrigin: t.shoulderR, transform: t.armR }]}>  
    {layer(<KlaArm id={id} skin={skin} side="R" mood={mood} />)}</Animated.View>  
190         <Animated.View style={[fill, { transformOrigin: t.neck, transform: t.head }]}>  
    {layer(<KlaHead id={id} skin={skin} mood={mood} />)}  
191         <Animated.View style={[fill, { transformOrigin: t.eyesAt, transform: t.blink }]}>  
    {layer(<KlaEyes id={id} mood={mood} />)}</Animated.View>  
192         {layer(<KlaGlasses skin={skin} />, false)}  
193         <Animated.View style={[fill, { opacity: t.closed }]}>{layer(<KlaMouth mood={mood} skin=  
    {skin} />, false)}</Animated.View>  
194         <Animated.View style={[fill, { opacity: t.open, transformOrigin: t.mouthAt, transform:  
    t.openShape }]}>{layer(<KlaMouthOpen skin={skin} />, false)}</Animated.View>  
195         {layer(<KlaExtras mood={mood} />, false)}  
196       </Animated.View>  
197     </View>  
198   );  
199 }  
200 }  
201 }
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/kla/KlaStage.tsx:175

บรรทัด	ความหมาย
175–180	layer = พื้น SVG ขนาดเท่าตัวละคร วางทับกันที่มุมซ้ายบน ใส่ defs (สีไล่ระดับของชุด) เมื่อจำเป็น
181	สไตล์ "เติมกรอบ" ใช้ซ้ำทุกชั้น
184	กรอบนอก: ซ่อนจากโปรแกรมอ่านหน้าจอ (ภาพตกแต่ง) และไม่รับการแตะ (ปุ่มที่ห่ออยู่รับแทน)
185	ชั้นพื้น: หดและจางตอนกระโดด
186–198	ชั้นตัว (หมุนรอบฐานลำต้น) ข้างในซ้อนต่อ: หลัง/ลำต้น · แขนซ้าย (หมุนรอบไหล่ซ้าย) · แขนขวา · หัว (หมุนรอบคอ) ซึ่งข้างในมี ตา (กะพริบ) · แขน · ปากปิด · ปากเปิด (ยิ้ม) · ของประกอบอารมณ์ การซ่อนแบบนี้ทำให้ หัวหมุนตามตัว และ ตา หมุนตามหัว เหมือนข้อต่อจริง

≈ เหมือนหุ่นเชิดกระดาด

มาถึงหุ่นกระดาดที่ตัดเป็นชิ้นแล้วเจาะหลอดที่ข้อต่อ ขยับแขนก็หมุนรอบหลอดโหล่ ขยับตัวทั้งตัว แขนและหัวก็ไปด้วย
KlaStage ทำแบบเดียวกันในโค้ด: "หลอด" คือ transformOrigin และ "ชิ้นที่ติดกัน" คือ View ที่ซ้อนอยู่ข้างใน

KlaBackdrop: เวท

src/ui/kla/KlaBackdrop.tsx

บรรทัด 14-49

```
14 export function KlaBackdrop({
15   children,
16   style,
17   floorAt = 0.8,
18 }): {
19   children: ReactNode;
20   style?: StyleProp<ViewStyle>;
21   /** Where the floor is, as a share of the height (the character's feet). */
22   floorAt?: number;
23 } {
24   const theme = useTheme();
25   const id = `b${useId().replace(/^[a-zA-Z0-9]/g, '')}`;
26   return (
27     <View style={[{ borderRadius: radius.xl, overflow: 'hidden' }, style]>
28       <LinearGradient colors={[theme.heroTop, theme.hero, theme.heroDeep]} start={{ x: 0.1, y: 0
29       }} end={{ x: 0.9, y: 1 }} style={{ position: 'absolute', left: 0, right: 0, top: 0, bottom: 0 }}
30     />
31     <Aurora cycles={2} strength={0.3} seed={11} />
32     <Svg width="100%" height="100%" viewBox="0 0 100 100" preserveAspectRatio="none" style={{
33     position: 'absolute', left: 0, top: 0 }}>
34       <Defs>
35         <RadialGradient id={`${id}spot`} cx="50%" cy="50%" r="50%">
36           <Stop offset="0" stopColor={theme.heroAccent} stopOpacity={0.3} />
37           <Stop offset="0.6" stopColor={theme.heroAccent} stopOpacity={0.08} />
38           <Stop offset="1" stopColor={theme.heroAccent} stopOpacity={0} />
39         </RadialGradient>
40         <RadialGradient id={`${id}floor`} cx="50%" cy="50%" r="50%">
41           <Stop offset="0" stopColor="#FFFFFF" stopOpacity={0.22} />
42           <Stop offset="1" stopColor="#FFFFFF" stopOpacity={0} />
43         </RadialGradient>
44       </Defs>
45       <Ellipse cx={50} cy={floorAt * 100 - 30} rx={42} ry={40} fill={`url(#${id}spot)`} />
46       <Ellipse cx={50} cy={floorAt * 100} rx={40} ry={7} fill={`url(#${id}floor)`} />
47     </Svg>
48     {theme.colorTheme === 'halloween' ? <SpookyBackdropDecor /> : null}
49     {children}
50   </View>
51 );
52 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/kla/KlaBackdrop.tsx:14

บรรทัด ความหมาย

14-23 รับสิ่งที่จะวางบนเวที สโตน และตำแหน่งพื้น (สัดส่วนความสูง ค่าเริ่ม 0.8)

27-28 กล้องมุมโค้งที่ตัดส่วนเกิน พื้นหลังไล่สีตามธีม (บน → กลาง → ล่าง)

บรรทัด	ความหมาย
29	แสงออโรราเบา ๆ (เล่น 2 รอบแล้วหยุด)
30–44	SVG กับเติมกรอบ: วงแสงสปอตไลท์สีทองของริมด้านหลังตัวละคร และวงแสงขาวจาง ๆ บนพื้นใต้เท้า ทำให้ดูเหมือน "ยืนอยู่ในห้อง" ไม่ใช่แปะบนการ์ด
45	ริมหาดโลว์มีของตกแต่งหลอน ๆ เพิ่ม
46	วางเนื้อหา (น้องกล้ำ ฟองคำพูด ปุ่ม) กับด้านบน

KlaPicture: น้องกล้ำภาพนิ่ง

src/ui/kla/KlaPicture.tsx

บรรทัด 16–49

```

16 export function KlaPicture({
17   skin,
18   mood = 'calm',
19   width,
20   blink,
21   onDark = false,
22   extras = true,
23   ground = true,
24 }): {
25   skin: SkinId;
26   mood?: BuddyMood;
27   width: number;
28   blink?: boolean;
29   onDark?: boolean;
30   /** Z's, sparkles or thought bubbles for the mood. */
31   extras?: boolean;
32   ground?: boolean;
33 } {
34   const id = `k${useId().replace(/^[a-zA-Z0-9]/g, '')}`;
35   return (
36     <Svg width={width} height={(width * KLA_H) / KLA_W} viewBox={`0 0 ${KLA_W} ${KLA_H}`}>
37       <KlaDefs id={id} skin={skin} onDark={onDark} />
38       {ground ? <KlaGround id={id} /> : null}
39       <KlaBack id={id} skin={skin} />
40       <KlaArm id={id} skin={skin} side="L" mood={mood} />
41       <KlaArm id={id} skin={skin} side="R" mood={mood} />
42       <KlaHead id={id} skin={skin} mood={mood} />
43       <KlaEyes id={id} mood={mood} closed={blink} />
44       <KlaGlasses skin={skin} />
45       <KlaMouth mood={mood} skin={skin} />
46       {extras ? <KlaExtras mood={mood} /> : null}
47     </Svg>
48   );
49 }

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/ui/kla/KlaPicture.tsx:16

บรรทัด	ความหมาย
16–33	รับชุด อาร์มรณ ความกว้าง หลับตาใหม่ พื้นเข้มใหม่ แสดงของประกอบอาร์มรณใหม่ แสดงเงาพื้นใหม่
34	รหัสไม่ซ้ำสำหรับ gradient

บรรทัด	ความหมาย
36–47	SVG พื้นเดียว วาดทุกชั้นตามลำดับจากหลังไปหน้า: defs → พื้น → หลัง → แขนซ้าย → แขนขวา → หัว → ตา → แวน → ปาก → ของประกอบ (ชั้นเดียวกับ KlaStage แต่ไม่แยกชั้น ไม่ขยับ จึงเบามาก เหมาะกับตัวเล็ก ๆ หลายตัวบนจอ)

อ่านโค้ด: ตู้สกินและภารกิจ (skins.ts, missions.ts, kla.ts)

กฎของชุดอยู่ใน domain (มีเกสต์) ส่วนการเก็บในเครื่องอยู่ใน services

skins.ts ส่วนที่ 1: ชนิดข้อมูลของชุด

src/domain/skins.ts

บรรทัด 1-56

```

1 /**
2  * Skins (ชุด) for บ็องกล่า. Three kinds:
3  *   - the starter look everyone has;
4  *   - mission skins, unlocked by a money mission (src/domain/missions.ts);
5  *   - limited skins, given to everyone who opens the app during a set window
6  *     (the Halloween ones need candies collected in that window,
7  *     src/domain/halloween.ts). Once the window has passed they cannot be
8  *     collected any more and are shown as "สกินลิบิเต็ด".
9  *
10 * A skin, once owned, stays owned. Drawings: src/ui/kla/art.tsx.
11 * Unit tests: __tests__/skins.test.ts.
12 */
13 import { formatThaiDay } from './dates';
14 import { HALLOWEEN } from './halloween';
15
16 export type SkinId =
17   | 'classic'
18   | 'thai'
19   | 'graduate'
20   | 'detective'
21   | 'dj'
22   | 'saver'
23   | 'chill'
24   | 'hero'
25   | 'sakura'
26   | 'pioneer'
27   | 'songkran2569'
28   | 'loykrathong2568'
29   | 'newyear2570'
30   | 'pumpkin'
31   | 'sheetghost'
32   | 'witch'
33   | 'mummy'
34   | 'vampire'
35   | 'frankenstein';
36
37 export type SkinKind = 'starter' | 'mission' | 'limited';
38
39 export interface Skin {
40   id: SkinId;
41   name: string;
42   /** One line about the look. */
43   blurb: string;
44   kind: SkinKind;
45   /** Limited skins: the Bangkok days (inclusive) when opening the app gives it. */

```

```

46 window?: { from: string; to: string };
47 /** Halloween skins: candies needed (collected during the window) instead of just opening the
    app. */
48 candies?: number;
49 /** Shown together in the collection (e.g. the Halloween set). */
50 event?: 'halloween2569';
51 /** Colour behind the skin in the collection. */
52 tint: string;
53 }
54
55 export const DEFAULT_SKIN: SkinId = 'classic';
56

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/skins.ts:1

บรรทัด	ความหมาย
1–11	ชุดของน้องกล่ามี 3 แบบ: ชุดเริ่มต้นที่ทุกคนมี · ชุดภารกิจ ปลดล็อกด้วยการกิจการเงิน · ชุดลิมิเต็ด แจกทุกคนที่เปิดแอป ในช่วงที่จำกัด (ชุดฮาโลวีนต้องเก็บลูกอมในช่วงนั้น) พันช่วงแล้วเก็บไม่ได้อีก · ได้แล้วอยู่ตลอด · ภาพอยู่ที่ art.tsx เทสต์ ใน skins.test.ts
12–13	ใช้ตัวจัดรูปวันที่ไทย และช่วงเวลาฮาโลวีนจาก halloween.ts
15–34	SkinId = รหัสชุด 19 ค่า (ชนิด union: ค่าได้แค่ในรายการนี้ พิมพ์ผิด TypeScript ฟ้องทันที)
36	ชนิดของชุด 3 แบบ
38–52	ข้อมูลของชุด 1 ชุด: รหัส ชื่อ คำบรรยาย แบบ · ช่วงเวลา (เฉพาะลิมิเต็ด) · ลูกอมที่ต้องใช้ (เฉพาะฮาโลวีน) · กลุ่มงาน (event) · สีพื้นหลังในตู้
54	ชุดเริ่มต้นคือ classic (ต้นกล้า)
56	เริ่มรายการชุดทั้งหมด (อ่านรายละเอียดทุกชุดในตาราง "สกินลิมิเต็ด" และ "ภารกิจ" บก "ตู้สกินและการกิจการเงิน")

skins.ts ส่วนที่ 2: สถานะของชุดและข้อความ

src/domain/skins.ts

บรรทัด 108–177

```

108 const BY_ID = new Map(SKINS.map((s) => [s.id, s]));
109
110 export function skinById(id: string | null | undefined): Skin {
111   return BY_ID.get(id as SkinId) ?? BY_ID.get(DEFAULT_SKIN)!;
112 }
113
114 export function isSkinId(id: unknown): id is SkinId {
115   return typeof id === 'string' && BY_ID.has(id as SkinId);
116 }
117
118 /** Where a skin stands for this user today. */
119 export type SkinState =
120   | { kind: 'owned' }
121   /** A mission skin not unlocked yet. */
122   | { kind: 'locked' }
123   /** A limited skin that can be collected right now (opening the app gives it). */
124   | { kind: 'limited_open'; until: string }
125   /** A Halloween skin: collect this many candies before the event ends. */
126   | { kind: 'candy'; need: number; have: number; until: string }

```

```

127  /** A limited skin whose window has not started. */
128  | { kind: 'limited_soon'; from: string }
129  /** A limited skin whose window has passed: it can no longer be collected. */
130  | { kind: 'limited_ended' };
131
132  export function skinState(skin: Skin, owned: ReadonlySet<string>, today: string, candies = 0):
  SkinState {
133    if (skin.kind === 'starter' || owned.has(skin.id)) return { kind: 'owned' };
134    if (skin.kind === 'mission') return { kind: 'locked' };
135    const w = skin.window!;
136    if (today < w.from) return { kind: 'limited_soon', from: w.from };
137    if (today > w.to) return { kind: 'limited_ended' };
138    if (skin.candies) return { kind: 'candy', need: skin.candies, have: Math.min(candies,
  skin.candies), until: w.to };
139    return { kind: 'limited_open', until: w.to };
140  }
141
142  /** Limited skins that opening the app today gives (the free ones, and the candy ones with enough
  candies). */
143  export function limitedOpenToday(today: string, candies = 0): SkinId[] {
144    return SKINS.filter((s) => s.kind === 'limited' && s.window && today >= s.window.from && today
  <= s.window.to && (s.candies ?? 0) <= candies).map(
145      (s) => s.id,
146    );
147  }
148
149  /** "10-16 พ.ย. 2569" or "25 ธ.ค. 2569 - 5 ม.ค. 2570". */
150  export function formatWindow(w: { from: string; to: string }): string {
151    if (w.from.slice(0, 7) === w.to.slice(0, 7)) return `${Number(w.from.slice(8, 10))}-
  ${formatThaiDay(w.to)}`;
152    const sameYear = w.from.slice(0, 4) === w.to.slice(0, 4);
153    return `${formatThaiDay(w.from, { year: !sameYear })} - ${formatThaiDay(w.to)}`;
154  }
155
156  /** Short status line for a skin card (the mission's own progress is shown separately). */
157  export function skinStatusLine(skin: Skin, state: SkinState): string {
158    switch (state.kind) {
159      case 'owned':
160        return skin.kind === 'limited' ? 'สกินลิมิเต็ด · มีแล้ว' : 'มีแล้ว';
161      case 'locked':
162        return 'ทำภารกิจเพื่อปลดล็อก';
163      case 'limited_open':
164        return `สกินลิมิเต็ด · รับฟรีเมื่อเปิดแอปถึง ${formatThaiDay(state.until)}`;
165      case 'candy':
166        return `สกินลิมิเต็ดฮาโลวีน · สะสมลูกอม ${state.need} เม็ดภายใน ${formatThaiDay(state.until)} (มี
  ${state.have})`;
167      case 'limited_soon':
168        return `สกินลิมิเต็ด · เร็ว ๆ นี้ เปิดแอปช่วง ${formatWindow(skin.window!)} เพื่อรับ`;
169      case 'limited_ended':
170        return `สกินลิมิเต็ด · หมดเวลาแล้ว หาไม่ได้อีก (แจกช่วง ${formatWindow(skin.window!)})`;
171    }
172  }
173
174  /** The skin to draw: the chosen one if the user owns it, otherwise the starter look. */
175  export function wearable(equipped: string | null | undefined, owned: ReadonlySet<string>): SkinId
  {
176    return isSkinId(equipped) && (equipped === DEFAULT_SKIN || owned.has(equipped)) ? equipped :
  DEFAULT_SKIN;
177  }

```

บรรทัด	ความหมาย
108	ตารางค้นชุดจากรหัสอย่างรวดเร็ว (Map)
110–112	skinById: หาชุด ถ้าไม่เจอ (รหัสแปลก) ค้นชุดเริ่มต้น ไม่พ้อง
114–116	isSkinId: ตรวจสอบว่าค่าที่ได้มา (เช่น จากที่เก็บในเครื่อง) เป็นรหัสชุดจริงไหม ชนิดค้นค่าแบบ id is SkinId ทำให้ TypeScript รู้ว่าหลังผ่าน if นี้ id เป็น SkinIdแน่นอน
118–130	สถานะของชุดสำหรับผู้ใช้นี้ 6 แบบ: มีแล้ว · ล็อก (ภารกิจ) · ลิมิตเปิดรับ · ลิมิตใช้ลูกอม · ยังไม่ถึงช่วง · หมดช่วงแล้ว (แต่ละแบบเก็บข้อมูลที่ต้องใช้ต่างกัน เช่น until, need, have)
132–140	skinState ตัดสินตามลำดับ: ชุดเริ่มต้นหรือมีแล้ว → owned · ชุดภารกิจที่ยังไม่มี → locked · ลิมิตเปิด: ก่อนช่วง → soon · หลังช่วง → ended · ต้องใช้ลูกอม → candy (บอกต้องมีเท่าไร มีเท่าไร) · ไม่จิ้น → เปิดรับ
142–147	limitedOpenToday: ชุดลิมิตเปิดที่ "เปิดแอปวันนี้แล้วได้เลย" = อยู่ในช่วง และลูกอมที่ต้องใช้ (ไม่ระบุ = 0) ไม่เกินที่มี
149–154	formatWindow: เขียนช่วงวันแบบไทย เดือนเดียวกัน "10–16 เม.ย. 2569" ข้ามเดือน/ปี "25 ธ.ค. 2569 – 5 ม.ค. 2570"
156–172	skinStatusLine: ข้อความใต้การ์ดชุดในตู้ตามสถานะ เช่น "ทำภารกิจเพื่อปลดล็อก" · "สกินลิมิตเปิด · รับฟรีเมื่อเปิดแอปถึง ..." · "สกินลิมิตฮาโลวีน · สะสมลูกอม 12 เบ็ดภายใน ... (มี 3)" · "สกินลิมิตเปิด · หมดเวลาแล้ว หาไม่ได้อีก (แจกช่วง ...)"
174–177	wearable: ชุดที่จะวาดจริง = ชุดที่เลือก ถ้าเป็นรหัสจริงและ (เป็นชุดเริ่มต้นหรือมีแล้ว) ไม่จิ้นใส่ต้นกล้า ก็นการ "ใส่ชุดที่ไม่มี"

missions.ts ส่วนที่ 1: ชนิดข้อมูลและวันเปิดแอปติดกัน

src/domain/missions.ts

บรรทัด 1–75

```

1 /**
2  * ภารกิจการเงิน: money missions on the achievements screen. Each one unlocks a
3  *  skin for น่องกล้า (src/domain/skins.ts). Progress is worked out from the
4  *  user's own records, plus the days the app was opened (for "open the app 30
5  *  days in a row"), so nothing can go out of sync.
6  *  Unit tests: __tests__/missions.test.ts.
7  */
8  import { VOICE_NOTE } from './achievements';
9  import { addDays, bkkDayKey, daysBetween, formatThaiMonth, previousMonth } from './dates';
10 import { formatBaht } from './money';
11 import type { SkinId } from './skins';
12 import type { Transaction } from './types';
13
14 export type MissionId = 'open_30' | 'streak_14' | 'slips_20' | 'voice_10' | 'goal_done' |
15   'thrifty_week' | 'budget_month' | 'badges_8';
16
17 export interface Mission {
18   id: MissionId;
19   title: string;
20   /** What to do. */
21   how: string;
22   /** Icons name. */
23   icon: string;
24   /** The skin it unlocks. */
25   skin: SkinId;

```

```

25   done: boolean;
26   progress: { value: number; target: number };
27   /** A line with the user's own numbers ("7 วันนี้ ฿1,200 · 7 วันก่อน ฿1,500"). */
28   note?: string;
29 }
30
31 export interface MissionInput {
32   txs: Transaction[];
33   /** Bangkok days the app was opened (any order). */
34   openDays: readonly string[];
35   /** Longest run of days in a row with a record (src/domain/achievements.ts). */
36   recordBest: number;
37   /** Achievement badges earned so far. */
38   badgesEarned: number;
39   goals: { savedSatang: number; targetSatang: number }[];
40   monthlyBudgetSatang: number | null;
41   today: string;
42 }
43
44 export const OPEN_TARGET = 30;
45 export const RECORD_TARGET = 14;
46 export const SLIPS_TARGET = 20;
47 export const VOICE_TARGET = 10;
48 export const BADGES_TARGET = 8;
49 /** A month counts for "คุณจบได้ทั้งเดือน" only if it was really recorded. */
50 export const BUDGET_MONTH_MIN_RECORDS = 10;
51
52 /** Days in a row the app was opened, ending today (0 if not opened today). */
53 export function openStreak(openDays: readonly string[], today: string): number {
54   const days = new Set(openDays);
55   let run = 0;
56   for (let d = today; days.has(d); d = addDays(d, -1)) run++;
57   return run;
58 }
59
60 /** Add today to the list of open days (sorted, without repeats, only the most recent `keep`). */
61 export function recordOpen(openDays: readonly string[], today: string, keep = 120): string[] {
62   const all = [...new Set([...openDays, today])].sort();
63   return all.slice(Math.max(0, all.length - keep));
64 }
65
66 const baht = (s: number) => formatBaht(s, { decimals: false });
67
68 function spendBetween(txs: Transaction[], today: string, fromAge: number, toAge: number): number {
69   let sum = 0;
70   for (const t of txs) {
71     if (t.status !== 'confirmed' || t.kind !== 'expense') continue;
72     const age = daysBetween(bkkDayKey(t.occurredAt), today);
73     if (age >= fromAge && age < toAge) sum += t.amountSatang;
74   }
75   return sum;

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/missions.ts:1

บรรทัด ความหมาย

- 1-7** การจัดการเงินในหน้าความสำเร็จ แต่ละข้อปลดล็อกชุด ความคืบหน้าคำนวณจากรายการของผู้ใช้เอง บวกวันที่เปิดแอป (สำหรับภารกิจ 30 วัน) "จึงไม่มีอะไรเพี้ยน" (ไม่มีตัวนับแยกที่อาจไม่ตรงกับข้อมูลจริง)

บรรทัด	ความหมาย
14	รหัสภารกิจ 8 คำ
16–29	ข้อมูลภารกิจ: ชื่อ วิธีทำ ไอคอน ชุดที่ได้ สำเร็จหรือยัง ความคืบหน้า (ค่า/เป้า) และบรรทัดที่ใช้ตัวเลขของผู้ใช้เอง
31–41	ข้อมูลที่ต้องใช้คำนวณ: รายการ · วันที่เปิดแอป · สถิติต่อเนื่องสูงสุด · จำนวนเหรียญ · กระปุกออม · งบรายเดือน · วัน นี้
43–49	เป้าของแต่ละภารกิจเป็นค่าคงที่ (30, 14, 20, 10, 8) และเดือนที่นับว่าคุณงบได้ต้องจดอย่างน้อย 10 รายการ
51–57	openStreak: นับย้อนจากวันนี้ทีละวัน トラブใดที่วันนั้นเปิดแอป (อยู่ในเซต) นับเพิ่ม เจอวันที่ไม่ได้เปิด หยุด (วันนี้ไม่ได้เปิด = 0)
59–63	recordOpen: เพิ่มวันนี้ ไม่ซ้ำ เรียงลำดับ เกือบแค่ 120 วันล่าสุด (พอสำหรับภารกิจ 30 วัน และไม่ให้อัปเดตไม่สิ้นสุด)
67–75	spendBetween: รวมรายจ่ายที่ยืนยันแล้วว่า "อายุ" (จำนวนวันจากวันนั้นถึงวันนี้) อยู่ในช่วง [fromAge, toAge) เช่น 0–7 = 7 วันล่าสุด 7–14 = 7 วันก่อนหน้า

missions.ts ส่วนที่ 2: computeMissions

src/domain/missions.ts

บรรทัด 91–161

```

91 export function computeMissions(p: MissionInput): Mission[] {
92   const out: Mission[] = [];
93   const add = (m: Omit<Mission, 'done'>, done: boolean) => out.push({ ...m, done });
94
95   const open = openStreak(p.openDays, p.today);
96   add(
97     {
98       id: 'open_30',
99       title: 'มาหากลับทุกวัน',
100      how: `เปิดแอปติดต่อกัน ${OPEN_TARGET} วัน`,
101      icon: 'calendar',
102      skin: 'thai',
103      progress: { value: Math.min(open, OPEN_TARGET), target: OPEN_TARGET },
104      note: open > 0 ? `ตอนนี้ต่อเนื่อง ${open} วัน ถ้าเว้นวันไหนจะเริ่มนับใหม่` : undefined,
105    },
106    open >= OPEN_TARGET,
107  );
108
109   add(
110     {
111       id: 'streak_14',
112       title: 'จดครบสองสัปดาห์',
113       how: `จดรายการต่อเนื่อง ${RECORD_TARGET} วัน`,
114       icon: 'flame',
115       skin: 'graduate',
116       progress: { value: Math.min(p.recordBest, RECORD_TARGET), target: RECORD_TARGET },
117     },
118     p.recordBest >= RECORD_TARGET,
119   );
120
121   const slips = p.txs.filter((t) => t.source === 'slip' && t.status === 'confirmed').length;
122   add(
123     {
124       id: 'slips_20',

```

```

125     title: 'บักสืบสลিপ',
126     how: `บับทึกสลิปเข้ายอดเงิน ${SLIPS_TARGET} ใบ`,
127     icon: 'receipt',
128     skin: 'detective',
129     progress: { value: Math.min(slips, SLIPS_TARGET), target: SLIPS_TARGET },
130   },
131   slips >= SLIPS_TARGET,
132 );
133
134 const voice = p.txs.filter((t) => t.note === VOICE_NOTE).length;
135 add(
136   {
137     id: 'voice_10',
138     title: 'พูดแล้วจอด',
139     how: `จอดด้วยเสียง ${VOICE_TARGET} รายการ`,
140     icon: 'mic',
141     skin: 'dj',
142     progress: { value: Math.min(voice, VOICE_TARGET), target: VOICE_TARGET },
143   },
144   voice >= VOICE_TARGET,
145 );
146
147 // Closest jar to its target, in percent.
148 const best = p.goals.reduce((m, g) => (g.targetSatang > 0 ? Math.max(m, Math.min(100,
Math.floor((g.savedSatang / g.targetSatang) * 100))) : m), 0);
149 const reached = p.goals.some((g) => g.targetSatang > 0 && g.savedSatang >= g.targetSatang);
150 add(
151   {
152     id: 'goal_done',
153     title: 'ออมครบเป้า',
154     how: `หยอดกระปุกออมจนครบเป้า 1 ใบ`,
155     icon: 'wallet',
156     skin: 'saver',
157     progress: { value: reached ? 100 : best, target: 100 },
158     note: p.goals.length === 0 ? 'ยังไม่มีกระปุก ตั้งได้ทีหน้าหลัก' : reached ? undefined : `กระปุกที่ใกล้สุดได้
${best}% แล้ว`,
159   },
160   reached,
161 );

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/missions.ts:91

บรรทัด	ความหมาย
91–93	อาร์เรย์ผลลัพธ์ และฟังก์ชัน add (รับภารกิจ + สำเร็จไหม)
95–107	มาหากลับทุกวัน: ความคืบหน้า = วันเปิดติดกัน (ไม่เกิน 30) บรรทัดบอก "ตอนนี้ต่อเนื่อง ... วัน ถ้าวันไหนจะเริ่มนับใหม่" สำเร็จเมื่อ ≥ 30 → ชุดไทย
109–119	จดครบสองสัปดาห์: ใช้สถิติต่อเนื่องสูงสุด ≥ 14 → บันทึกรายการการเงิน
121–132	บักสืบสลิป: นับรายการที่มาจากสลิปและยืนยันแล้ว ≥ 20 → บักสืบสลิป
134–145	พูดแล้วจอด: นับรายการที่ note เป็นเครื่องหมายของการพูดจอด (VOICE_NOTE) ≥ 10 → ดีเจเสียงใส
147–161	ออมครบเป้า: หากกระปุกที่ใกล้เป้าที่สุดเป็น % (ปัดลง ไม่เกิน 100) และมีกระปุกที่ครบเป้าไหม บรรทัดบอก "ยังไม่มีกระปุกตั้งได้ทีหน้าหลัก" หรือ "กระปุกที่ใกล้สุดได้ ...% แล้ว" → เศรษฐีน้อย

```

163  const last = spendBetween(p.txs, p.today, 0, 7);
164  const before = spendBetween(p.txs, p.today, 7, 14);
165  const thrifty = last > 0 && before > 0 && last < before;
166  add(
167    {
168      id: 'thrifty_week',
169      title: 'สัปดาห์ประหยัด',
170      how: 'ใช้จ่าย 7 วันล่าสุดให้น้อยกว่า 7 วันก่อนหน้า',
171      icon: 'trending-down',
172      skin: 'chill',
173      progress: { value: thrifty ? 1 : 0, target: 1 },
174      note: before > 0 ? `7 วันนี้ ${baht(last)} · 7 วันก่อน ${baht(before)}` : 'ต้องมีรายจ่ายอย่างน้อย 2
สัปดาห์ก่อนนะ',
175    },
176    thrifty,
177  );
178
179  const thisMonth = p.today.slice(0, 7);
180  const lastMonth = previousMonth(thisMonth);
181  const prev = monthSpend(p.txs, lastMonth);
182  const budget = p.monthlyBudgetSatang;
183  const kept = !!budget && prev.records >= BUDGET_MONTH_MIN_RECORDS && prev.spent <= budget;
184  const now = monthSpend(p.txs, thisMonth);
185  add(
186    {
187      id: 'budget_month',
188      title: 'คุมงบได้ทั้งเดือน',
189      how: 'ตั้งงบรายเดือน แล้วใช้ไม่เกินงบจนจบเดือน',
190      icon: 'shield-checkmark',
191      skin: 'hero',
192      progress: { value: kept ? 1 : 0, target: 1 },
193      note: !budget
194        ? 'ตั้งงบรายเดือนในหน้าตั้งค่าก่อนนะ'
195        : kept
196          ? `${formatThaiMonth(lastMonth)} ใช้ ${baht(prev.spent)} จากงบ ${baht(budget)}`
197          : `เดือนนี้ใช้ไป ${baht(now.spent)} จากงบ ${baht(budget)} จนเดือนแบบไม่เกินงบเพื่อปลดล็อก`,
198    },
199    kept,
200  );
201
202  add(
203    {
204      id: 'badges_8',
205      title: 'นักสะสมเหรียญ',
206      how: `สะสมเหรียญความสำเร็จ ${BADGES_TARGET} เหรียญ`,
207      icon: 'medal',
208      skin: 'sakura',
209      progress: { value: Math.min(p.badgesEarned, BADGES_TARGET), target: BADGES_TARGET },
210    },
211    p.badgesEarned >= BADGES_TARGET,
212  );
213
214  return out;
215 }

```

บรรทัด	ความหมาย
163–165	สัปดาห์ประหยั: รายจ่าย 7 วันล่าสุด กับ 7 วันก่อนหน้า สำเร็จเมื่อทั้งคู่มากกว่า 0 และล่าสุดน้อยกว่า (ต้องมีรายจ่ายทั้งสองช่วง กันคนที่ไม่จดเลยได้ไปฟรี ๆ)
166–177	บรรทัดบอกตัวเลขจริง "7 วันนี้ ฿... · 7 วันก่อน ฿..." หรือ "ต้องมีรายจ่ายอย่างน้อย 2 สัปดาห์ก่อนนะ" → ชิลล์ริมทะเล
179–183	คุมงบได้ทั้งเดือน: ดู "เดือนที่แล้ว" (เดือนที่จบแล้ว) สำเร็จเมื่อมีงบ และเดือนนั้นจดยรายจ่าย ≥ 10 รายการ และใช้ไม่เกินงบ
184–200	บรรทัดบอก: ยังไม่ตั้งงบ → ชวนตั้ง · สำเร็จ → "ก.ย. 2569 ใช้ ฿... จากงบ ฿..." · ยังไม่สำเร็จ → บอกเดือนนี้ใช้ไปเท่าไร "จบเดือนแบบไม่เกินงบเพื่อปลดล็อก" → ฮีโร่คุมงบ
202–212	นักสะสมเหรียญ: เหรียญความสำเร็จ ≥ 8 → ชากุระ
214	คืนภารกิจทั้ง 8

kla.ts: เก็บตุ้สกินในเครื่อง และแจกชุด

src/services/kla.ts

บรรทัด 20–104

```

20 interface Wardrobe {
21   owned: SkinId[];
22   /** Skins the user has already been told about. */
23   seen: SkinId[];
24   equipped: SkinId;
25   /** Bangkok days the app was opened. */
26   opens: string[];
27   /** Halloween ghosts caught per Bangkok day. */
28   ghosts: GhostCatches;
29 }
30
31 const EMPTY: Wardrobe = { owned: [DEFAULT_SKIN], seen: [DEFAULT_SKIN], equipped: DEFAULT_SKIN,
  opens: [], ghosts: {} };
32 const key = (who: string) => `mindpay.kla.${who}`;
33
34 type Snap = { who: string; data: Wardrobe } | null;
35 let current: Snap = null;
36 const listeners = new Set<() => void>();
37 const emit = () => listeners.forEach((l) => l());
38 const subscribe = (l: () => void) => {
39   listeners.add(l);
40   return () => {
41     listeners.delete(l);
42   };
43 };
44 const snapshot = () => current;
45 const loading = new Set<string>();
46
47 function clean(raw: unknown): Wardrobe {
48   if (!raw || typeof raw !== 'object') return EMPTY;
49   const r = raw as Partial<Record<keyof Wardrobe, unknown>>;
50   const ids = (v: unknown) => (Array.isArray(v) ? v.filter(isSkinId) : []);
51   const owned = [...new Set<SkinId>([DEFAULT_SKIN, ...ids(r.owned)])];
52   return {
53     owned,
54     seen: [...new Set<SkinId>([DEFAULT_SKIN, ...ids(r.seen)])],
55     equipped: isSkinId(r.equipped) ? r.equipped : DEFAULT_SKIN,

```

```

56     opens: Array.isArray(r.opens) ? r.opens.filter((d): d is string => typeof d === 'string' &&
    /\d{4}-\d{2}-\d{2}$/.test(d)) : [],
57     ghosts:
58       r.ghosts && typeof r.ghosts === 'object'
59       ? Object.fromEntries(
60         Object.entries(r.ghosts as Record<string, unknown>).filter(
61           (e): e is [string, number] => /\d{4}-\d{2}-\d{2}$/.test(e[0]) && typeof e[1] ===
    'number' && e[1] > 0,
62         ),
63       )
64       : {},
65   };
66 }
67
68 async function load(who: string) {
69   if (current?.who === who || loading.has(who)) return;
70   loading.add(who);
71   let data = EMPTY;
72   try {
73     const raw = await Storage.getItem(key(who));
74     if (raw) data = clean(JSON.parse(raw));
75   } catch {
76     // Start fresh.
77   }
78   loading.delete(who);
79   current = { who, data };
80   emit();
81 }
82
83 function save(who: string, change: (w: Wardrobe) => Wardrobe) {
84   if (current?.who !== who) return;
85   const next = change(current.data);
86   if (next === current.data) return;
87   current = { who, data: next };
88   emit();
89   Storage.setItem(key(who), JSON.stringify(next)).catch(() => {});
90 }
91
92 const whoOf = (repo: { mode: string } | null | undefined, userId: string | null | undefined) =>
    (repo?.mode === 'demo' ? 'demo' : (userId ?? null));
93
94 /** The skin บ๊องกล้า wears right now (the starter look before sign-in). */
95 export function useEquippedSkin(): SkinId {
96   const app = useAppMaybe();
97   const who = app ? whoOf(app.repo, app.userId) : null;
98   const snap = useSyncExternalStore(subscribe, snapshot, snapshot);
99   useEffect(() => {
100     if (who) load(who);
101   }, [who]);
102   if (!snap || !who || snap.who !== who) return DEFAULT_SKIN;
103   return wearable(snap.data.equipped, new Set(snap.data.owned));
104 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/kla.ts:20

บรรทัด ความหมาย

20–29 ข้อมูลตุ้สกิน 5 อย่าง: ชุดที่มี · ชุดที่แจ้จแล้ว · ชุดที่ใส่ · วันที่เปิดแอป · ฝัที่จับได้รายวัน

31 ค่าเริ่มต้นของคนใหม่: มีและใส่ตันกล้า

บรรทัด	ความหมาย
32	ชื่อที่เก็บ: mindpay.kla.<ใคร>
34–45	ที่เก็บกลางในหน่วยความจำ (current) + ระบบผู้ฟัง เหมือนคำเสียงใน useKlaTalk ทุกหน้าที่มีน้องกล้าเห็นชุดเดียวกันกับที่เมื่อเปลี่ยน
47–66	clean: ทำความสะอาดข้อมูลที่อ่านจากเครื่อง (อาจเก่าหรือเสีย) เก็บเฉพาะรหัสชุดที่มีจริง ใส่ต้นกล้าไว้เสมอ ไม่ซ้ำ วันที่ต้องเป็นรูปแบบ YYYY-MM-DD จำนวนปีต้องเป็นตัวเลขบวก ใช้หลัก "อย่าเชื่อข้อมูลที่อ่านมาจากภายนอก"
68–81	load: อ่านครั้งเดียวต่อคน (กันโหลดซ้อน) ถ้าอ่านฟังเริ่มใหม่จากค่าเริ่มต้น แล้วแจ้งทุกหน้า
83–90	save: เปลี่ยนข้อมูลด้วยฟังก์ชัน change ถ้าไม่มีอะไรเปลี่ยนจริง (คืนอ็อบเจกต์เดิม) ไม่ทำอะไร ไม่ขึ้นแจ้งทุกหน้าแล้วบันทึกลงเครื่อง
92	whoOf: โหมดทดลองใช้ชื่อ 'demo' ไม่ขึ้นใช้รหัสผู้ใช้
94–104	useEquippedSkin: hook ที่ทุกที่ใช้หาชุดที่ใส่ ก่อนเข้าสู่ระบบหรือก่อนโหลดเสร็จ ใช้ต้นกล้า

src/services/kla.ts

บรรทัด 106–195

```

106 /** Everything the achievements screen and the skin collection need. */
107 export function useKla() {
108   const { txs, goals, today, userId, repo, profile } = useApp();
109   const { streak, badges } = useAchievements();
110   const who = whoOf(repo, userId);
111   const snap = useSyncExternalStore(subscribe, snapshot, snapshot);
112   useEffect(() => {
113     if (who) load(who);
114   }, [who]);
115   const data = snap && who && snap.who === who ? snap.data : null;
116   const opens = data?.opens;
117   const earnedBadges = badges.filter((b) => b.earned).length;
118   const missions = useMemo(
119     () =>
120       computeMissions({
121         txs,
122         openDays: opens ?? [today],
123         recordBest: streak.best,
124         badgesEarned: earnedBadges,
125         goals,
126         monthlyBudgetSatang: profile?.monthlyBudgetSatang ?? null,
127         today,
128       }),
129     [txs, opens, today, streak.best, earnedBadges, goals, profile?.monthlyBudgetSatang],
130   );
131   const owned = useMemo(() => new Set<SkinId>(data?.owned ?? [DEFAULT_SKIN]), [data?.owned]);
132   const fresh = useMemo(() => (data ? data.owned.filter((id) => !data.seen.includes(id)) : []),
133     [data]);
134   const equipped = wearable(data?.equipped, owned);
135   const equip = useCallback(
136     (id: SkinId) => {
137       if (!who) return;
138       save(who, (w) => (w.owned.includes(id) || id === DEFAULT_SKIN ? { ...w, equipped: id, seen:
139         [...new Set([...w.seen, id])] } : w));
140     },
141     [who],

```

```

141 );
142 const markSeen = useCallback(
143   (ids: SkinId[]) => {
144     if (!who || !ids.length) return;
145     save(who, (w) => (ids.every((id) => w.seen.includes(id)) ? w : { ...w, seen: [...new
Set([...w.seen, ...ids])] })));
146   },
147   [who],
148 );
149
150 // Halloween: candies from caught ghosts and from days with a record.
151 const ghosts = data?.ghosts;
152 const candies = useMemo(() => countCandies({ txs, caught: ghosts ?? {} })), [txs, ghosts]);
153 const catchGhost = useCallback(() => {
154   if (!who) return;
155   save(who, (w) => (ghostsLeft(w.ghosts, today) > 0 ? { ...w, ghosts: addGhost(w.ghosts, today)
} : w));
156 }, [who, today]);
157
158 return {
159   ready: !!data,
160   missions,
161   owned,
162   equipped,
163   fresh,
164   equip,
165   markSeen,
166   candies,
167   ghostsLeft: data ? ghostsLeft(data.ghosts, today) : 0,
168   catchGhost,
169 };
170 }
171
172 /**
173  * Mounted once inside the app: records today's visit and gives the skins the
174  * user has earned (finished missions, limited events open today, Halloween
175  * skins with enough candies).
176  */
177 export function useKlaSync() {
178   const { today, userId, repo } = useApp();
179   const who = whoOf(repo, userId);
180   const { missions, ready, candies } = useKla();
181   const doneSkins = missions
182     .filter((m) => m.done)
183     .map((m) => m.skin)
184     .join(',');
185   useEffect(() => {
186     if (!who || !ready) return;
187     const give = new Set<SkinId>([...limitedOpenToday(today, candies.total), ...(doneSkins ?
(doneSkins.split(',') as SkinId[]) : [])]);
188     save(who, (w) => {
189       const opens = w.opens.includes(today) ? w.opens : recordOpen(w.opens, today);
190       const owned = [...new Set([...w.owned, ...give])];
191       if (opens === w.opens && owned.length === w.owned.length) return w;
192       return { ...w, opens, owned };
193     });
194   }, [who, ready, today, doneSkins, candies.total]);
195 }

```

บรรทัด	ความหมาย
106–116	useKla: hook ของหน้าความสำเร็จและตู้สกิน อ่านข้อมูลแอป สถิติเหรียญ และตู้สกินของคนปัจจุบัน
117–130	คำนวณภารกิจทั้ง 8 ด้วย computeMissions (คิดใหม่เฉพาะเมื่อข้อมูลที่ใช้เปลี่ยน)
131–133	ชุดที่มี · ชุดใหม่ที่ยังไม่ได้แจ้ง (ใช้ทำการ์ด "น้องกล้าได้ชุดใหม่" บนหน้าหลัก) · ชุดที่ใส่ได้จริง
135–141	equip: ใส่ชุดได้เฉพาะชุดที่มี (หรือต้นกล้า) และถือว่าแจ้งแล้ว
142–148	markSeen: บันทึกว่าแจ้งชุดเหล่านี้แล้ว (ถ้าแจ้งครบอยู่แล้วไม่บันทึกซ้ำ)
150–156	ฮาโลวีน: นับลูกอมจากผีและวันที่จืด · catchGhost เพิ่มผีวันนี้ถ้ายังจับได้
158–169	คืนทุกอย่างให้หน้าจอใช้
172–176	useKlaSync: ติดตั้งครั้งเดียวในแอป (ในกรอบเก็บ บรรทัด 177 ของ _layout.tsx) บันทึกการเปิดแอปวันนี้ และแจกชุดที่ได้
181–184	รหัสชุดของภารกิจที่สำเร็จ ต่อเป็นข้อความเดียว (ใช้เป็นค่าที่ effect ติดตาม เปลี่ยนเมื่อมีภารกิจสำเร็จเพิ่ม)
185–194	เมื่อพร้อม: รวมชุดที่ควรได้ = ลิ้มเต็ดที่เปิดวันนี้ (ตามลูกอม) + ชุดภารกิจที่สำเร็จ แล้วบันทึกวันเปิดแอป (ถ้ายังไม่มีวันนี้) และเพิ่มชุดใหม่ ถ้าไม่มีอะไรเปลี่ยนคืนข้อมูลเดิม (ไม่บันทึกซ้ำ)

★ แยก "กฎ" ออกจาก "การเก็บ" อีกแล้ว

เหมือน FR-6 ที่แยกสูตร (runway.ts) ออกจากหน้าจอ ตู้สกินแยกกฎ (skins.ts, missions.ts ใน domain มีเกสต์ TC-73 ถึง TC-75) ออกจากการเก็บข้อมูล (kla.ts ใน services) วันหนึ่งถ้าทีมอยากย้ายตู้สกินไปเก็บบนเซิร์ฟเวอร์ แค่ kla.ts กฎทั้งหมดและเกสต์ใช้ต่อได้เลย

การทดสอบ FR-5

กฎทดสอบด้วย unit test · หน้าจอ เสียง และ error ทดสอบด้วย E2E · คำตอบจริงของ AI ทดสอบด้วยมือ (MT-08)

157/157

unit test ทั้งโปรเจกต์ผ่าน (รันซ้ำ 9 ต.ค. 2569)

20 TC

ที่เกี่ยวกับ FR-5 โดยตรง (การ์ด น้ำเสียง เสียงพูด ราคา ตูสกิน AI)

E2E

โคซ์ เสียงจำลอง error 3 แบบ เปลี่ยนชุด โหมดทดลอง

รายการเทสต์ของ FR-5

TC	ตรวจอะไร	ไฟล์
TC-31	หาหมวดที่สูงขึ้น: ปกติ ฿300/สัปดาห์ สัปดาห์นี้ ฿600 → อาหาร 100%	domain.test.ts
TC-32	ไม่มีข้อมูล → no_data ที่มีคำว่า "กล้า" · โทน friend กับ senior ได้ผลเหมือนกันทุกตัวอักษร · ทุกการ์ดมี fact · ไม่มีคำของบุคลิกเก่า	domain.test.ts
TC-34	ใช้เงินจบ: ข้อความมี "เงินจบ" และ "฿600" ไม่มี "นิดนึ่ง/นิดหน่อย"	domain.test.ts
TC-49	ประโยชน์บนหน้าหลักเลือกเรื่องด่วนก่อน และอารมณ์ถูกต้องทุกสถานะ	buddy.test.ts
TC-50	ไม่มีคำดู (เยอะเกินไป ฟุ่มเฟือย ผิด แยะ) ในทุกสถานะ	buddy.test.ts
TC-51	นับเฉพาะรายจ่ายยื่นยันแล้ววันนี้ · ป้าย "ใหม่" ของสลิปที่บันทึกวันนี้	buddy.test.ts
TC-53	วันใหม่เริ่มเที่ยงคืนเวลาไทย และน้องกล้าทักวันใหม่ (เรื่องด่วนยังมาก่อน)	buddy.test.ts
TC-58	แต่น้องกล้า: ไม่ซ้ำติดกัน 20 ครั้ง · เงินตั้งให้กำลังใจก่อน · ครั้งที่ 6 จึกจี้	buddy.test.ts
TC-66	แต่นั่นไม้: อธิบายใบทองอย่างใจดีทุกสถานะ	buddy.test.ts
TC-73	ชุดมีรหัสไม่ซ้ำและชื่อไทย · สลิปเด้งรับได้เฉพาะในช่วง · ชุดภารกิจลือจอนได้ · ใส่ได้เฉพาะชุดที่มี	skins.test.ts
TC-74, 75	นับวันเปิดแอปติดกัน · การจิกนับจากรายการจริงและบอกชุดที่ได้ · สัปดาห์ประหยัด และเดือนคุ่มงบต้องเป็นเดือนที่จงดจริง	missions.test.ts
TC-76	ตัดซับไตเติลไม่ตัดกลางคำ ไม่มีคำหาย ≤ 64 ตัว · ฿/%/ลบ/อีโมจิ สำหรับเสียง · เวลาอ่าน 1.4–9 วินาที	klaTalk.test.ts
TC-77	ฮาโลวีน: ช่วงงาน ฆีวันละ 3 ลูกอมจากผีและวันทักชุดฮาโลวีนหมดหลังงาน	halloween.test.ts
TC-89	หาราคาในคำถามได้ทุกรูปแบบ และไม่เอา "30 วัน" "20%" "120 วัน" มาเป็นราคา	audit.test.ts
TC-38–40, 54, 55	ฝั่งเซิร์ฟเวอร์: เลือก Claude/Gemini · แยกกฎแฉผิดจากติดลิมิต · อ่านคำตอบ (ข้ามส่วนความคิด ล้าง markdown) · อ่านกฎแฉที่พิมพ์ผิดบ่อย · รุ่นที่ไม่ว่างส่งต่อรุ่นถัดไป	helpers.test.ts

อ่านเทสต์ของการ์ดสิ่งที่ควรรู้

ในไฟล์นี้ "วันนี้" ถูกล็อกเป็น 27 ก.ย. 2569 เวลา 14:00 โทษ (ตัวแปร NOW บสทัด 36) ฟังก์ชัน tx() สร้างรายจ่ายหมวดอาหารที่ยืนยันแล้ว และไปรไฟล์ทดสอบมิงบ ฿9,000

src/domain/__tests__/domain.test.ts

บรรทัด 358-398

```
358 describe('FR-5 coach insights', () => {
359   it('TC-31 finds a category that rose this week', () => {
360     const txs = [
361       // usual: ฿300/week for 3 weeks
362       tx({ day: '2026-09-08', amount: 300 }),
363       tx({ day: '2026-09-14', amount: 300 }),
364       tx({ day: '2026-09-18', amount: 300 }),
365       // this week: ฿600
366       tx({ day: '2026-09-25', amount: 600 }),
367     ];
368     const rise = findCategoryIncrease(txs, NOW);
369     expect(rise).toMatchObject({ key: 'food', week: 60_000, usualWeek: 30_000, changePct: 100 });
370   });
371
372   it('TC-32 speaks as น้องกล้า in one calm voice, with the facts behind it, and never without data',
373     () => {
374     const runway = computeRunway({ balanceSatang: 100_000, floorSatang: 50_000, averageSatang:
375       10_000, now: NOW });
376     const empty = buildInsights({ txs: [], profile, runway, now: NOW });
377     expect(empty[0].kind).toBe('no_data');
378     expect(empty[0].message).toContain('กล้า');
379     const txs = [tx({ day: '2026-09-27', amount: 100 })];
380     // The old tone setting no longer changes anything: there is one coach now.
381     const a = buildInsights({ txs, profile: { ...profile, coachTone: 'friend' }, runway, now: NOW
382     });
383     const b = buildInsights({ txs, profile: { ...profile, coachTone: 'senior' }, runway, now: NOW
384     });
385     expect(a).toEqual(b);
386     for (const i of a) {
387       expect(i.fact.length).toBeGreaterThan(0);
388       expect(i.message).not.toMatch(/เกินไป!|พี่|น้องตั้ง/);
389     }
390   });
391
392   describe('FR-5 budget wording', () => {
393     it('TC-34 says "over budget" plainly when spending passed the budget, without shaming', () => {
394       const txs = [tx({ day: '2026-09-10', amount: 9500 }), tx({ day: '2026-09-27', amount: 100
395       })];
396       const runway = computeRunway({ balanceSatang: 900_000, floorSatang: 50_000, averageSatang:
397         10_000, now: NOW });
398       const budget = buildInsights({ txs, profile, runway, now: NOW }).find((i) => i.kind ===
399         'budget_pace')!;
400       expect(budget.message).toContain('เกินงบ');
401       expect(budget.message).toContain('฿600');
402       expect(budget.message).not.toMatch(/มิดมิง|มิดน้อย/);
403     });
404   });
405 });
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:358

บรรทัด	ความหมาย
360–367	TC-31: 3 สัปดาห์ก่อนใช้ ฿300 สามครั้ง (8, 14, 18 ก.ย. อยู่ในช่วงฐาน 31 ส.ค.–20 ก.ย.) สัปดาห์นี้ ฿600 (25 ก.ย.)
368–369	ต้องได้ หมวด food สัปดาห์นี้ 60,000 สตางค์ ปกติ 30,000 เพิ่ม 100% (toMatchObject = ตรวจสอบเฉพาะช่องที่ระบุ)
373	TC-32: Runway ใกล้เส้นสำรอง (เหลือ ฿500 เหลือเงิน ใช้วันละ ฿100 = 5 วัน)
374–376	ไม่มีรายการเลย → การ์ดแรกต้องเป็น no_data และพูดถึง "กล้า"
377–381	มีรายการ 1 รายการ ลองโปรไฟล์โทน friend กับ senior ผลต้องเท่ากันทั้งหมด (toEqual เทียบทุกช่องทุกตัวอักษร) พิสูจน์ว่าการตั้งค่าโทนเท่าไม่ผลแล้ว
382–385	ทุกการ์ดต้องมี fact และ message ต้องไม่มี "เกินไป!" และคำของบุคลิกเก่า ("พี่" "น้องตัง")
391	TC-34: ใช้ ฿9,500 + ฿100 = ฿9,600 จน ฿9,000
393–396	หากการ์ดจบ ข้อความต้องมี "เกินงบ" และ "฿600" และต้องไม่มี "นิดนึ่ง" "บิ๊ดหน้อย" (! หลัง find บอกว่ามั่นใจว่าเจอ)

อ่านทดสอบการพูด (TC-76)

src/domain/__tests__/klaTalk.test.ts

บรรทัด 1–36

```

1  /**
2   * TC-76: how น้องกล้า talks – subtitles cut at spaces, text made easy for the
3   * Thai voice, and reading time when there is no sound.
4   */
5  import { describe, expect, it } from 'vitest';
6  import { KLA_GREETING, speakable, talkChunks, talkMs } from '../klaTalk';
7
8  describe('Talking', () => {
9    it('TC-76 cuts long answers into short subtitles at spaces, never inside a word, keeping every
10     word', () => {
11       const answer =
12         'สัปดาห์นี้ใช้ไป ฿1,290 ส่วนใหญ่เป็นค่าอาหาร สูงกว่าปกติราว 20% ถ้าลดค่ากาแฟวันละแก้ว จะเหลือเงินเพิ่มอีกหลายร้อยบาท ลองดู
13         ว่าแบบไหนเหมาะกับเรา';
14       const parts = talkChunks(answer);
15       expect(parts.length).toBeGreaterThan(1);
16       expect(parts.join(' ')).toBe(answer);
17       for (const p of parts) expect(p.length).toBeLessThanOrEqual(64);
18       expect(talkChunks('สั้น ๆ')).toEqual(['สั้น ๆ']);
19       expect(talkChunks('บรรทัดแรก\nบรรทัดสอง')).toEqual(['บรรทัดแรก', 'บรรทัดสอง']);
20       expect(talkChunks(KLA_GREETING)[0]).toContain('ตามเรื่องเงินได้ทุกเรื่อง');
21       // A word that starts the next thought ("เช่น") opens the next subtitle instead of ending this
22       one.
23       for (const p of talkChunks(KLA_GREETING)) expect(p).not.toMatch(/ (เช่น|หรือ|แต่)$/);
24       expect(talkChunks(KLA_GREETING).join(' ')).toBe(KLA_GREETING);
25     });
26
27     it('TC-76 reads baht, percent and minus the way a Thai voice should, without emoji', () => {
28       expect(speakable('ซื้อได้ ฿1,290 🍷')).toBe('ซื้อได้ 1290 บาท');
29       expect(speakable('เพิ่มขึ้น 25%')).toBe('เพิ่มขึ้น 25 เปอร์เซ็นต์');
30       expect(speakable('ยอด -฿500 เดือนนี้')).toBe('ยอด ลบ 500 บาท , เดือนนี้');
31       expect(speakable('“ลองปรับ”')).toBe('ลองปรับ');
32     });
33
34     it('TC-76 a silent subtitle stays long enough to read, but not forever', () => {
35       expect(talkMs('สั้น')).toBe(1400);
36     });
37   });

```

```

33     expect(talkMs('ก'.repeat(60))).toBe(420 + 60 * 72);
34     expect(talkMs('ก'.repeat(500))).toBe(9000);
35   });
36 });

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/klaTalk.test.ts:1

บรรทัด	ความหมาย
10–12	คำตอบยาวตัวอย่าง (มี ฿1,290 และ 20%) ตัดเป็นก่อน
13–15	ต้องได้มากกว่า 1 ก่อน · ต่อทุกก่อนกลับต้องได้ข้อความเต็มพอดี (ไม่มีคำหยาบ ไม่ตัดกลางคำ) · ทุกก่อนยาวไม่เกิน 64
16–17	ข้อความสั้นได้ก่อนเดียว · ขึ้นบรรทัดใหม่ได้ 2 ก่อน
18–21	คำทักทาย: ก่อนแรกมี "ถามเรื่องเงินได้ทุกเรื่อง" · ไม่มีก่อนไหนจบด้วย เช่น/หรือ/แต่ (คำพวกนี้ต้องไปขึ้นต้นก่อนถัดไป) · ต่อกลับได้ข้อความเต็ม
25–28	speakeable: "ซื้อได้ ฿1,290 🎉" → "ซื้อได้ 1290 บาท" · 25% → "25 เปอร์เซ็นต์" · "ยอด -฿500 · เดือนนี้" → "ยอดลบ 500 บาท , เดือนนี้" · เครื่องหมายคำพูดหยาบ
32–34	เวลาอ่าน: ข้อความสั้นได้ขั้นต่ำ 1,400 · 60 ตัวได้ 420 + 60 × 72 · 500 ตัวได้ขั้นสูง 9,000

อ่านทดสอบการหาราคา (TC-89)

src/domain/__tests__/audit.test.ts

บรรทัด 268–284

```

268 describe('TC-89 prices in coach questions', () => {
269   it('finds the price, wherever the buying word is', () => {
270     expect(priceInQuestion('ซื้อของ 500 บาทวันนี้ได้ไหม')).toBe(50_000);
271     expect(priceInQuestion('฿1,290 แพงไหม')).toBe(129_000);
272     expect(priceInQuestion('ซื้อรองเท้า 1290 ได้ไหม')).toBe(129_000);
273     expect(priceInQuestion('กระเป๋า 2500 ซื้อได้ไหม')).toBe(250_000);
274     expect(priceInQuestion('หูฟัง 1,990 คู่ไหม')).toBe(199_000);
275     expect(priceInQuestion('อยากได้มือถือ 15000 ควรซื้อไหม')).toBe(1_500_000);
276   });
277
278   it('does not take other numbers for a price', () => {
279     expect(priceInQuestion('ทำยังไงให้เงินพอถึงสิ้นเดือน อีก 30 วัน')).toBeNull();
280     expect(priceInQuestion('ลดกาแฟ 20% ได้ไหม')).toBeNull();
281     expect(priceInQuestion('สรุปสัปดาห์นี้ให้หน่อย')).toBeNull();
282     expect(priceInQuestion('อีก 120 วันจะเก็บได้เท่าไร')).toBeNull();
283   });
284 });

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/audit.test.ts:268

บรรทัด	ความหมาย
270–275	หาเจอทุกตำแหน่งของคำว่าซื้อ: "ซื้อของ 500 บาท..." · "฿1,290 แพงไหม" · "ซื้อรองเท้า 1290..." · "กระเป๋า 2500 ซื้อได้ไหม" (ราคาอยู่ก่อนคำว่าซื้อ จึงใช้ขั้นที่ 4) · "หูฟัง 1,990 คู่ไหม" · "อยากได้มือถือ 15000 ควรซื้อไหม"
279–282	ไม่เอาตัวเลขอื่น: "อีก 30 วัน" · "ลดกาแฟ 20%" · ไม่มีตัวเลข · "อีก 120 วัน" (3 หลักแต่ตามด้วย "วัน")

E2E: โค้ชในเบราว์เซอร์จริง (ข้อ 13)

e2e/web-auth.mjs

บรรทัด 495–571

```
495 // 13. Coach: an answer from the AI, and a clear message when the key is missing
496 {
497   state.confirmEmail = false;
498   const { ctx, page } = await freshPage('coach');
499   // A stand-in for the browser's speech (the test browser has no Thai voice): records what is
      said.
500   await ctx.addInitScript(() => {
501     window.__spoken = [];
502     const voices = [
503       { name: 'Google US English', lang: 'en-US', default: true, localService: false, voiceURI:
        'en' },
504       { name: 'Microsoft Premwadee Online (Natural) - Thai (Thailand)', lang: 'th-TH', default:
        false, localService: false, voiceURI: 'th1' },
505       { name: 'Microsoft Niwat Online (Natural) - Thai (Thailand)', lang: 'th-TH', default:
        false, localService: false, voiceURI: 'th2' },
506     ];
507     class FakeUtterance {
508       constructor(text) {
509         this.text = text;
510         this.lang = '';
511         this.rate = 1;
512         this.pitch = 1;
513         this.voice = null;
514         this.onend = null;
515         this.onerror = null;
516       }
517     }
518     const synth = {
519       speaking: false,
520       getVoices: () => voices,
521       addEventListener() {},
522       speak(u) {
523         window.__spoken.push({ text: u.text, lang: u.lang, rate: u.rate, pitch: u.pitch, voice:
        u.voice?.name ?? '' });
524         setTimeout(() => u.onend?.(), 250);
525       },
526       cancel() {},
527     };
528     Object.defineProperty(window, 'speechSynthesis', { value: synth, configurable: true });
529     Object.defineProperty(window, 'SpeechSynthesisUtterance', { value: FakeUtterance,
        configurable: true, writable: true });
530   });
531   await page.goto(APP);
532   await visible(page, 'ลองใช้ด้วยข้อมูลตัวอย่าง', 8000);
533   await introGone(page);
534   await page.getByText('สมัครสมาชิก', { exact: true }).click();
535   await page.fill('#name', 'โค้ช');
536   await page.fill('#email', 'coach@example.com');
537   await page.fill('#password', 'secret123');
538   await button(page, 'สร้างบัญชี').click();
539   await visible(page, 'สมัครบัญชีสำเร็จ');
540   await button(page, 'ไปตั้งค่าเงิน').click();
541   await page.fill('#ob-balance', '2000');
542   await button(page, 'เริ่มใช้ MindPay').click();
543   await visible(page, 'ยอดคงเหลือ', 8000);
544   await page.goto(`${APP}coach`);
```

```

545     await introGone(page);
546     check('Coach: the companion greets before the first question', await visible(page, 'ถามเรื่องเงิน
ได้ทุกเรื่อง'));
547     const greeted = await page.waitForFunction(() => window.__spoken.length > 0, null, { timeout:
6000 }).then(() => true, () => false);
548     const firstSaid = greeted ? (await page.evaluate(() => window.__spoken))[0] : null;
549     check(
550         'Coach voice: น้องกล้า speaks Thai with a calm Thai voice, a little slower and lower than
normal',
551         !!firstSaid && firstSaid.lang === 'th-TH' && firstSaid.voice.includes('Niwat') &&
firstSaid.rate < 1 && firstSaid.pitch < 1 && firstSaid.text.includes('สวัสดี'),
552         JSON.stringify(firstSaid),
553     );
554     check('Coach voice: no "no Thai voice" note on a device that has one', (await
page.getByText('เครื่องนี้ยังไม่มีเสียงอ่านภาษาไทย').count()) === 0);
555     await page.getByText('สรุปสปีดาค์นี้ให้หน่อย').first().click();
556     check('Coach: the AI answer appears', await visible(page, 'สปีดาค์นี้ใช้ไป ฿0'));
557     const readAloud = await page.waitForFunction(() => window.__spoken.some((u) =>
u.text.includes('ใช้ไป 0 บาท')), null, { timeout: 8000 }).then(() => true, () => false);
558     check('Coach voice: the answer is read aloud, with "฿0" said as "0 บาท"', readAloud);
559     state.coach = { status: 503, body: { error: { code: 'not_configured', message: 'x' } } };
560     await page.fill('#coach-input', 'หมวดไหนควรลดก่อน');
561     await button(page, 'ส่งคำถาม').click();
562     check('Coach: a missing key says exactly what to set', await visible(page,
'GEMINI_API_KEY'));
563     state.coach = { status: 503, body: { error: { code: 'busy', message: 'x' } } };
564     await page.fill('#coach-input', 'อีกข้อ');
565     await button(page, 'ส่งคำถาม').click();
566     check('Coach: a busy AI asks to try again shortly', await visible(page, 'รอสักครู่'));
567     await shot(page, '18-coach');
568     await ctx.close();
569     state.confirmEmail = true;
570 }
571

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:495

บรรทัด	ความหมาย
497–498	ปิดการยืนยันอีเมลในเซิร์ฟเวอร์จำลอง (สมัครแล้วเข้าได้เลย) เปิดเบราว์เซอร์ใหม่
499–530	เสียงจำลอง: ก่อนหน้าเว็บโหลด แถบ speechSynthesis ของเบราว์เซอร์ด้วยของปลอมที่มี 3 เสียง (อังกฤษ, Premwadee, Niwat) ทุกครั้งที่ถูกส่งพูด บันทึกข้อความ ภาษา ความเร็ว ระดับเสียง ชื่อเสียงไว้ใน window.__spoken แล้วแจ้งว่าพูดจบหลัง 0.25 วินาที
531–543	สมัครบัญชีใหม่ชื่อ "โค้ช" ตั้งยอดเงิน ฿2,000 เข้าหน้าหลัก
544–546	เปิดหน้าโค้ช ต้องเห็นคำทักทาย "ถามเรื่องเงินได้ทุกเรื่อง"
547–553	รอนมีการพูด แล้วตรวจสอบประโยคแรก: ภาษา th-TH · เสียง Niwat · ความเร็ว < 1 · ระดับเสียง < 1 · มีคำว่า "สวัสดี"
554	เครื่องที่มีเสียงไทย ต้องไม่มีบรรทัด "เครื่องนี้ยังไม่มีเสียงอ่านภาษาไทย"
555–558	กดคำถามลัด "สรุปสปีดาค์นี้ให้หน่อย" เซิร์ฟเวอร์จำลองตอบ "สปีดาค์นี้ใช้ไป ฿0 ยังไม่มีรายจ่ายเลยนะ" ต้องเห็นคำตอบ และต้องถูกพูดเป็น "ใช้ไป 0 บาท"
559–562	ตั้งเซิร์ฟเวอร์ให้ตอบ 503 not_configured แล้วถาม ต้องเห็นคำว่า GEMINI_API_KEY (บอกผู้ดูแลชัด ๆ ว่าต้องตั้งอะไร)

บรรทัด	ความหมาย
563–566	ตั้งให้ตอบ 503 busy ต้องเห็น "รอสักครู"
567–569	ถ่ายภาพหน้าจอ ปิดเบราว์เซอร์ ค้นค่าการยืนยันอีเมล

นอกจากข้อ 13 ยังมีข้อในส่วนตุสกิน (บรรทัด 964–987 ของไฟล์เดียวกัน) ที่ตรวจ: น้องกล้าตัวใหญ่ใส่ชุดที่เลือก · ไม่มีโค้ช 3 โทนแล้ว · แตะน้องกล้าแล้วพูดพร้อมปุ้มหยุด · หยุดแล้วมี "ฟังอีกครั้ง" · ปิดเสียงได้ · เปลี่ยนชุดบนเวทีได้เฉพาะชุดที่มี · โหมดทดลองอธิบายว่า AI ต้องใช้บัญชีจริง และข้อในหน้าตั้งค่า (บรรทัด 280): ไม่มีตัวเลือก "โทนของโค้ช" และมีสวิตช์ "เสียงน้องกล้า"

ทดสอบด้วยมือ: MT-08 และเช็กलिस्टบนมือถือ

คำตอบจริงของ AI ทดสอบอัตโนมัติไม่ได้ (เปลี่ยนทุกครั้ง และเรียก AI จริงมีค่าใช้จ่าย) แผนทดสอบใน TRACEABILITY จึงมี MT-08: ถามโค้ช "ชื่อของ 500 บาทวันนี้ได้ไหม" → ต้องตอบภาษาไทย อ้างตัวเลข Runway ก่อน/หลัง ไม่ตำหนิ (สถานะในเอกสารยังเป็น ☐ คือทีมยังไม่ได้กรอกผล ควรทำก่อนนำเสนอ)

#	ทำ	ต้องเห็น
1	บัญชีจริง ถาม "ชื่อของ 500 บาทวันนี้ได้ไหม"	Runway ก่อน/หลังตรงกับที่คิดเองด้วยสูตร FR-6 ไม่มีคำดู ไม่มีครับ/ค่ะ ไม่มีรายการบูลเล็ต
2	ถาม "ลงทุนคริปโตได้ไหม"	ปฏิเสธอย่างสุภาพว่าแนะนำเรื่องนี้ไม่ได้
3	ถามเรื่องข้อมูลที่ไม่มี เช่น "เดือนที่แล้วกินกาแฟกี่แก้ว"	บอกว่าข้อมูลไม่พอ ไม่แต่งตัวเลข
4	มือถือ Android ที่ลง APK ใหม่ เปิดเสียง	ได้ยินเสียงไทย ช้าและต่ำเล็กน้อย ชับไตเติลเปลี่ยนตามเสียง
5	เปิดโปรแกรมอ่านหน้าจอ (TalkBack)	น้องกล้าไม่พูดซ้อน โปรแกรมอ่านชับไตเติลให้
6	เปิด "ลดการเคลื่อนไหว"	น้องกล้านิ่ง แต่ปากยังขยับตอนพูด
7	ถาม 61 ครั้งใน 24 ชั่วโมง	ครั้งที่ 61 ได้ข้อความครบโควตา

FR-5 เชื่อมกับ FR อื่นยังไ้

FR-5 คือ "ปาก" ของระบบ เล่าสิ่งที่ FR อื่นคำนวณไว้

FR	ความเกี่ยวข้องกับโค้ช
FR-1 จัดการรายการ	ทุกรายการที่ยืนยันแล้วเป็นวัตถุดิบของการัดและข้อมูลที่ส่งให้ AI หมวดของรายการ (categoryKey) คือสิ่งที่ findCategoryIncrease เทียบ · parseBahtToSatang ของ FR-1 ใช้แปลงราคาในคำถาม
FR-2 ภาพรวม	ยอดคงเหลือ (computeBalance) และรายจ่ายเดือนนี้ (monthExpense) อยู่ในข้อมูลที่ส่งให้ AI และในการดงบ
FR-3 พุดจด	ภารกิจ "พุดแล้วจด" นับรายการจากเสียง และเคล็ดลับของน้องกล้าชวนให้ลองพุดจด
FR-4 สแกนสลิป	สลิปที่ยังไม่ยืนยันไม่ถูกลบ · น้องกล้าบนหน้าหลักบอก "มีสลิป ... ใบที่กล้าอ่านไม่ชัด" · ภารกิจ "นักสืบสลิป" · AI ของโค้ชและของสลิปใช้ตัวช่วยกลางชุดเดียวกัน (common.ts)
FR-6 Money Runway	Runway คือการัดแรกเสมอ · ส่ง lowLine, averageDailySpend7d, runway ให้ AI · ปุ่ม "ถามโค้ชเรื่องนี้" ส่งคำถามพร้อมราคามา · runwayAfterPurchase คัด purchaseCheck · ต้นไม้เงินกับใบทองเป็นภาษาร่วมกัน
กระปุกออม	เงินในกระปุกอยู่ใน lowLine (ผ่าน FR-6) · ภารกิจ "ออมครบเป้า"

“สรุปเป็นประโยคเดียว

FR-1, FR-3, FR-4 เป็นทางเข้าข้อมูล FR-2 กับ FR-6 เป็นสมองที่คำนวณ และ FR-5 คือน้องกล้า ผู้ที่เล่าผลการคำนวณนั้นให้ผู้ฟังเป็นภาษาคน ด้วยน้ำเสียงที่ใจดี และไม่เคยแต่งตัวเลขเอง

ทำไมฉันออกแบบแบบนี้

ทุกการตัดสินใจมีทางเลือกอื่น นี่คือเหตุผลที่เลือก

แยก 2 ชั้น: กฎ + AI	เรื่องสำคัญ (Runway หมวดที่สูงขึ้น งบ) ต้องถูกต้อง ตรวจสอบได้ และใช้ได้เสมอ จึงใช้กฎ ส่วนคำถามอิสระที่กฎตอบไม่ได้จึงใช้ AI ถ้าใช้ AI อย่างเดียว ไม่มีเน็ตหรือ AI ล่มแล้ว โค้ชใช้ไม่ได้เลย
แอปคิดเลข AI แค่อธิบาย	AI ภาษาคิดเลขพลาดได้ และอาจแต่งตัวเลขที่ฟังดูจริง แอปจึงส่งตัวเลขที่คำนวณแล้ว (รวม purchaseCheck) และ prompt ห้ามแต่ง
ส่งเฉพาะยอดรวมตามหมวด	พอสำหรับคำแนะนำ และไม่มีชื้อคน ร้าน เลขบัญชี รูปสลิป หลุดไปถึงผู้ให้บริการ AI (หลักใช้ข้อมูลน้อยที่สุดเท่าที่จำเป็น)
ส่งเป็นข้อความเงินที่จัดรูปแล้ว	AI ยกตัวเลขไปพูดต่อได้เลย ไม่ต้องแปลงสตริงเป็นบาทเอง
น้องกล้าตัวเดียว แทน 3 โท	ผู้ใช้ขอ (29 ก.ย.) และตัวละครเดียวง่าย ผู้กพันได้ ทดสอบง่ายกว่า (1 น้ำเสียงแทน 3) ตัวละครเดียวกันทั้งแอป
พูดออกเสียงด้วยเสียงของเครื่อง	ฟรี ไม่ต้องส่งข้อความไปบริการเสียงภายนอก ทำงานออฟไลน์ได้ถ้าเครื่องมีเสียงไทยในตัว ข้อเสียคือคุณภาพเสียงต่างกันตามเครื่อง
ซับไตเติลทีละก่อน	อ่านตามกัน ขยับปากตรงจังหวะ หยุดกลางทางได้ ค้างแค่ก่อนเดียวถ้าเสียงมีปัญหา
ภาพวาดแยกชั้น ไม่ใช่โมเดล 3D	อัปเดต OTA ได้ ไม่ต้องลง APK ใหม่ สั้นบนมือถือราคาประหยัด ประหยัดแบตเตอรี่
ข้อความผิดพลาดไม่อ่านออกเสียง	เรื่องเทคนิคไม่ควรเป็นเสียงที่ทำให้ตกใจ อ่านบนจอพอ
โควตา 60 ครั้ง/วัน และจำกัดความยาว	กันค่าใช้จ่ายบานปลายจากบั๊กหรือบัญชีถูกขโมย 60 ครั้งเพียงพอสำหรับการใช้ปกติ
Claude ถ้ามีคุญแจ ไม่จัน Gemini	ทีมยืนยันตัวตนซื้อเครดิต Anthropic ไม่ผ่าน จึงเพิ่ม Gemini (ฟรี) เป็นทางเลือก ใส่ คุญแจตัวไหนก็ใช้ตัวนั้น (AI_USAGE_LOG แถวที่ 8)
เกณฑ์ 25% และ ฿100	เตือนเฉพาะเรื่องที่ถึง "เปลี่ยนมาก" และ "เป็นเงินมากพอ" ไม่จู้จี้ เพราะการเตือนบ่อยเกินไปทำให้คนไม่ฟัง
ภารกิจผูกกับนิสัยการเงินจริง	ให้รางวัลพฤติกรรมที่ทำให้ข้อมูลครบ (จดทุกวัน สลิป เสียง) และนิสัยที่ดี (ออม คุมงบ ใช้น้อยลง) และกันการโกง (ต้องมีรายจ่ายจริง)

ข้อจำกัด และสิ่งที่จะทำต่อ

ระบบที่ดีต้องรู้จักอ่อนของตัวเอง

ข้อจำกัดตอนนี้

ข้อจำกัด	ผลกระทบ	ทางแก้ที่เป็นไปได้
ไม่จำบทสนทนา: ทุกคำถามส่งแค่ข้อมูลสรุป + คำถามเดียว	ถามต่อว่า "แล้วถ้าลดครึ่งหนึ่งล่ะ" AI ไม่รู้ว่า "ครึ่งหนึ่งของอะไร"	ส่ง 2-3 คำถาม/คำตอบล่าสุดไปด้วย (แลกกับค่าใช้จ่ายที่เพิ่ม)
คำตอบของ AI เปลี่ยนทุกครั้ง และ "ไม่เกิน 90 คำ" เป็นคำสั่ง ไม่ได้บังคับในโค้ด	บางคำตอบยาวกว่าที่ตั้งใจ	ตัดความยาวฝั่งเซิร์ฟเวอร์ หรือวัดและบันทึกความยาวคำตอบ
โหมดทดลองถาม AI ไม่ได้	คนที่ลองแอปไม่เห็นความสามารถของ AI	คำตอบตัวอย่างที่เขียนไว้ล่วงหน้าสำหรับคำถามลัด 4 ข้อ
การ์ดหมวดที่สูงขึ้นไม่เห็นหมวดใหม่ (3 สัปดาห์ก่อนเป็น 0)	เริ่มใช้เงินหมวดใหม่ก้อนใหญ่ จะไม่ถูกเตือน	กฎเสริมสำหรับหมวดใหม่ที่เกินจำนวนหนึ่ง
เกณฑ์ 25%, ฿100, 10% เท่ากันทุกคน	คนใช้เงินน้อยมากหรือมากมากอาจได้การเตือนไม่พอดี	ปรับตามขนาดรายจ่ายของแต่ละคน
คุณภาพเสียงขึ้นกับเครื่อง และ Android ต้องลง APK ใหม่ถึงจะมีเสียง	บางเครื่องไม่มีเสียงไทย ได้แค่ซับไตเติล	แนะนำวิธีติดตั้งเสียงไทยในตั้งค่ามือถือ
ตัดซับไตเติลที่เว้นวรรคเท่านั้น	ภาษาไทยที่ไม่เว้นวรรคยาว ๆ อาจได้ก่อนยาวถึงราว 102 ตัว หรือวงเล็บถูกแยก เช่น "(฿248 จากปกติราว" "฿128)"	ใช้ตัวตัดคำภาษาไทย (Intl.Segmenter)
ตุ้สกินเก็บในเครื่อง	เปลี่ยนมือถือแล้ววันเปิดแอปและซัดลิมีเต็มหาย	ตารางบนเซิร์ฟเวอร์ (แก้แค่ kla.ts)
ผู้ใช้อาจพิมพ์ข้อมูลส่วนตัวลงในคำถามเอง	ข้อความนั้นไปถึง AI	ตรวจและปิดบังเลขบัญชี/เบอร์โทรในคำถามก่อนส่ง

บั๊กเล็ก 2 จุดที่ฉันทเเจระหว่างทำเล่มนี้ (ยังไม่ได้แก้ในโปรเจกต์)

บั๊ก	เกิดเมื่อ	แก้ยังไง
การ์ดงบ "ใช้เงินจบไปแล้ว" แต่ขึ้นไอคอนลูกสีเขียว	วันท้าย ๆ ของเดือน ใช้เงินจบไม่มาก (เช่น งบ ฿6,000 ใช้ ฿6,300 วันที่ 31 ต.ค.) เพราะระดับดูแค่ ahead	insights.ts ussgัด 127: severity: over ahead ? 'watch' : 'good' + เทสต์วันสิ้นเดือน
กรอบทอง "กำลังพูด" ค้างที่คำตอบเก่า	ส่งคำถามใหม่ระหว่างที่น้องกลายังพูดคำตอบเก่า	coach.tsx หลังussgัด 118 เพิ่ม setSpeakingId(null);

★ ถ้าอาจารย์ถามว่า "ระบบมีบั๊กไหม"

ตอบตรง ๆ พร้อมหลักฐานจะได้คะแนนดีกว่าตอบว่าไม่มี: "เจอ 2 จุดเล็ก ๆ ที่ไม่กระทบตัวเลข คือสไลด์คอนของการดงบ ในวันสิ้นเดือน กับกรอบไฮไลต์ที่ค้าง แก้ได้จุดละบรรทัดเดียว และควรเพิ่มเทสต์ที่ครอบคลุมนั้น" แสดงว่าเข้าใจโค้ดจริง และรู้วิธีปรับปรุงด้วยกระบวนการ

ไอดีพัฒนาต่อ

- ให้น้องกล้า **เตือนล่วงหน้า** เมื่อสถานะ Runway เปลี่ยน (แจ้งเตือนบนมือถือ)
- **สรุปรายสัปดาห์อัตโนมัติ** ทุกวันอาทิตย์ เป็นเสียงน้องกล้าสั้น ๆ
- ให้ผู้ใช้ **ให้คะแนนคำตอบ** (มีประโยชน์ไหม) เพื่อปรับ prompt จากข้อมูลจริง
- วัดผล UX ตามเป้าใน TRACEABILITY: ผู้ใช้ตอบได้ใหม่ว่า "เงินพออีกกี่วัน" หลังฟังน้องกล้า

คำถามที่อาจารย์อาจถาม พร้อมคำตอบ

ฝึกตอบให้คล่อง แต่ละข้อพูดได้ใน 20–30 วินาที

Q1 FR-5 ทำอะไร?

โค้ชการเงินชื่อน้องกล้า อธิบายข้อมูลที่ยืนยันแล้วให้เข้าใจง่าย มี 2 ชั้น: การ์ดสิ่งที่ควรรู้จากกฎในแอป และคำถามอิสระที่ AI ตอบ น้องกล้าพูดออกเสียงพร้อมซับไตเติล

Q2 ทำไมไม่ให้ AI ทำทุกอย่าง?

เพราะเรื่องสำคัญต้องถูกต้อง ตรวจสอบได้ และใช้ได้แม้ไม่มีเน็ต กฎในแอปทำได้ทั้งสามอย่างและมีเกสต์ ส่วน AI ใช้เฉพาะคำถามที่กฎตอบไม่ได้

Q3 กันไม่ให้ AI มั่วตัวเลขยังงัย?

แอปคำนวณตัวเลขทั้งหมดเอง ส่งไปในส่วน DATA และ prompt สั่งให้ใช้เฉพาะตัวเลขใน DATA ถ้าข้อมูลไม่พอให้บอกว่าขาดอะไร เรื่องชื่อของ แอปคิด Runway ก่อน/หลังให้ใน purchaseCheck

Q4 ข้อมูลอะไรถูกส่งไปที่ AI?

แค่ยอดรวม: ยอดคงเหลือ เส้นสำรอง ค่าเฉลี่ย Runway งบ รายจ่ายเดือนนี้ รายจ่ายตามหมวด 7 และ 30 วัน รายรับ 30 วัน และคำถาม ไม่มีรูปสลิป ชื่อคน ชื่อร้าน หรือเลขบัญชี

Q5 ทำไมแอปไม่เรียก AI เอง?

ถ้าใส่กุญแจ AI ในแอป ใครก็แกะเอาไปใช้ได้ กุญแจจึงอยู่บนเซิร์ฟเวอร์ (Supabase Edge Function) ซึ่งตรวจตัวตนและนับโควตาก่อนเรียก AI (บท A10)

Q6 หมวดที่ "สูงกว่าปกติ" คิดยังงัย?

เทียบ 7 วันล่าสุดกับค่าเฉลี่ยต่อสัปดาห์ของ 3 สัปดาห์ก่อนหน้า ต้องเพิ่มอย่างน้อย 25% และอย่างน้อย ฿100 เลือกหมวดที่เพิ่มมากที่สุดหมวดเดียว (TC-31)

Q7 ทำไมเปลี่ยนจาก 3 โทนเป็นน้องกล้า?

ผู้ใช้ขอเมื่อ 29 ก.ย. ทีมบันทึกการเปลี่ยนแปลงใน TRACEABILITY แก่ prompt โค้ด หน้าตั้งค่า และเกสต์ TC-32 ให้ตรวจว่าการตั้งค่าโทนเก่าไม่มีผลแล้ว และต้องแก้ข้อความ FR-5 ใน SRS

Q8 น้องกล้าพูดได้ยังงัย?

ตัดคำตอบเป็นก่อน ≤ 64 ตัวที่เว้นวรรค แปลง ฿ % ลบ ให้อ่านถูก แล้วใช้เสียงภาษาไทยของมือถือหรือเบราว์เซอร์อ่านทีละก่อน ช้าและต่ำลงเล็กน้อยให้ฟังสุ่ม ไม่มีเสียงไทยก็ใช้ซับไตเติลตามเวลาอ่าน (TC-76)

Q9 ทำไมไม่ใช้โมเดล 3D?

ต้องใช้ไลบรารี native ใหม่ ทุกคนต้องลงแอปใหม่ และหนักสำหรับมือถือราคาประหยัด ทีมวาดด้วยโค้ดแยกชิ้นแล้วหมุนด้วย native driver ได้ความรู้สึกใกล้เคียงกัน สั้นและอัปเดต OTA ได้

Q10 ถ้า AI ล่ม หรือไม่มีกุญแจ ผู้ใช้เห็นอะไร?

ข้อความภาษาไทยที่บอกตรง ๆ ว่าเกิดอะไรและทำอะไรได้ เช่น "รอสักครู่แล้วถามใหม่" ไม่อ่านออกเสียง และการ์ดสิ่งที่ควรรู้ยังใช้ได้ (E2E ตรวจ)

Q11 กันค่าใช้จ่าย AI ยังงัย?

โควตา 60 ครั้งต่อ 24 ชั่วโมงต่อบัญชี (นับในฐานข้อมูลแบบล็อกกันนับพร้อมกัน) คำถามไม่เกิน 400 ตัว ข้อมูลไม่เกิน 6,000 ตัว คำตอบไม่เกิน 500 token และใช้รุ่นเล็กก่อน

Q12 แอปรู้ได้ยังงิว่าคำถามมีราคา?

priceInQuestion ลอง 4 ชั้น: ฿ตัวเลข · ตัวเลขบาท · หลังคำว่าชื่อ/ราคา/จ่าย · ตัวเลข 3 หลักขึ้นไปในคำถามเรื่องชื่อ และไม่เอาตัวเลขที่ตามด้วย วัน เดือน ขึ้น % ฯลฯ (TC-89)

Q13 น้องกล้าไม่ดู ทดสอบยังงิ?

เกสตร์ตรวจว่าข้อความทุกสถานะไม่มีคำต้องห้าม (TC-32, TC-50, TC-58, TC-66) และ prompt ห้ามตำหนิ ส่วนคำตอบ AI ตรวจด้วยมือ (MT-08)

Q14 คนตามอดใช้ได้ไหม?

ฟองคำพูดเป็น live region โปรแกรมอ่านหน้าจ้ออ่านซั้บไ้เตลให้ และน้องกล้าเจียบเองเมื่อเปิดโปรแกรมอ่านหน้าจ้อ ปุ่มทุกปุ่มมีคำอธิบาย ปุ่มสูง 44 จุด

Q15 การกิจกับสกินเกี่ยวกับโค้ชยังงิ?

โค้ชช่วยได้เมื่อข้อมูลครบ การกิจให้รางวัลกับนิสัยที่ทำให้ข้อมูลครบและการเงินดีขึ้น เช่น จดต่อเนื่อง 14 วัน ออมครบเป้า คุมงบได้ทั้งเดือน และกันการโกงด้วยการต้องมีรายการจริง

Q16 มีอะไรที่ยังทำไม่ได้?

ยังไม่จำบทสนทนาต่อเนื่อง โหมดทดลองถาม AI ไม่ได้ เภทที่เตือนยังเท่ากันทุกคน และเจอบักเล็ก 2 จุดที่แก้ได้บรรทัดเดียว (บกข้อจำกัด)

แบบฝึกหัดเชิงความเข้าใจ

ลองคิดเองก่อน แล้วค่อยดูเฉลย ทุกข้อ "วันนี้" = 15 ต.ค. 2569

1. หมวดข้อปึง 3 สัปดาห์ก่อน (24-4 ต.ค.) ใช้ ฿300 สามครั้ง สัปดาห์นี้ (9-15 ต.ค.) ใช้ ฿200 กับ ฿220 ได้การ์ดใหม่ ถ้าได้ขึ้นกี่ %
2. เหมือนข้อ 1 แต่สัปดาห์นี้ใช้ ฿180 กับ ฿200 ได้การ์ดใหม่ เพราะอะไร
3. งบเดือนละ ฿6,200 เดือนนี้ใช้ไป ฿3,400 การ์ดงบบพูดว่าอะไร ข้อมูลใต้การ์ดเขียนว่าอะไร
4. คำถามต่อไปนี้ priceInQuestion ได้ราคาเท่าไร: (ก) "ซื้อหนังสือ 3 เล่ม 450 บาทได้ไหม" (ข) "อีก 15 วันได้เงินเดือน" (ค) "ตัวคอนเสิร์ต 2,500 คู่ไหม" (ง) "ค่าโทรศัพท์เดือนละ 299 แพงไหม"
5. speakable("เหลือ ฿2,566.00 · ลดลง 5%") ได้ข้อความอะไร
6. ชับไตเติล "สวัสดิ์ กล้าเองนะ" (16 ตัวอักษร) คำนวณเท่าไรถ้าไม่มีเสียง
7. สถานะ watch ใช้ได้วันละ ฿120 ไม่มีสลิปปรอตรวจ น้องกลับมาบนหน้าหลักพูดอะไร อารมณ์อะไร
8. แตะน้องกล้าครั้งที่ 12 ตอนสถานะ healthy และครั้งที่ 4 ตอนสถานะ critical ได้ประโยชน์อะไร
9. ยอด ฿3,000 สำรอง ฿500 ใช้วันละ ฿100 ถามว่าซื้อของ ฿700 ได้ไหม purchaseCheck เป็นอะไร ถ้าเปลี่ยนเป็น ฿2,600 ละ
10. (คิดลึก) คำตอบ "ถ้าซื้อวันนี้ เงินจะพอใช้อีก 18 วัน แต่ถ้ารอถึงสิ้นเดือน จะเหลือเงินสำรองเพื่อฉุกเฉินมากกว่า ลองคิดดูว่าจำเป็นแค่ไหนนะ" ถูกตัดเป็นกี่ก่อน คำว่า "แต่" ถูกย้ายไปขึ้นต้นก่อนใหม่ไหม เพราะอะไร

เฉลย (รับกับฟังก์ชันจริงทุกข้อแล้ว)

ข้อ	วิธีคิด	คำตอบ
1	ปกติ = $900 \div 3 = \text{฿}300$ · สัปดาห์นี้ $\text{฿}420$ · เพิ่ม $\text{฿}120 (\geq \text{฿}100 \checkmark)$ · $120 \div 300 = 40\% (\geq 25 \checkmark)$	ได้การ์ด ช้อปปี้งสูงกว่าปกติ 40% ($\text{฿}420$ จากปกติราว $\text{฿}300$)
2	สัปดาห์นี้ $\text{฿}380$ · เพิ่ม $\text{฿}80$ แม้จะเป็น 27%	ไม่ได้ เพราะเพิ่มไม่ถึง $\text{฿}100$
3	ควรใช้ไปแล้ว = $6,200 \times 15 \div 31 = \text{฿}3,000$ · $\times 1.1 = \text{฿}3,300$ · ใช้ $\text{฿}3,400$ มากกว่า → เร็วกว่าเวลา · $3,400 \div 6,200 = 54.8\% \rightarrow 55\% \cdot 15 \div 31 = 48\%$	"งบเดือนนี้ไปเร็วกว่าเวลาอยู่บ้าง ใช้ไปแล้ว 55% ช่วงที่เหลือค่อย ๆ ผ่อนลงก็ยังทันนะ" · ข้อมูล: "เดือนนี้ใช้ไป $\text{฿}3,400$ จากงบ $\text{฿}6,200$ (55%) ผ่านเดือนมาแล้ว 48%" (ระดับ watch)
4	(ก) ขึ้น 2 "450 บาท" (ไม่ใช่ 3 เพราะ 3 ตามด้วย "เล่ม" ไม่ใช่ "บาท") · (ข) ไม่ใช่คำถามเรื่องซื้อ และ 15 ตามด้วย "วัน" · (ค) ขึ้น 4 (คุ้ม) · (ง) ขึ้น 4 (แพง, 299 มี 3 หลัก)	(ก) $\text{฿}450$ (ข) ไม่มี (ค) $\text{฿}2,500$ (ง) $\text{฿}299$
5	$\text{฿}2,566.00 \rightarrow 2566.00$ บาท · $\rightarrow , \cdot 5\% \rightarrow 5$ เปอร์เซ็นต์	"เหลือ 2566.00 บาท , ลดลง 5 เปอร์เซ็นต์"
6	$420 + 72 \times 16 = 1,572$ ซึ่งมากกว่า 1,400	1.572 วินาที
7	ไม่มีเรื่องด่วนกว่า → ตามสถานะ watch	"วันนี้ใช้ได้ราว $\text{฿}120$ เงินจะพอถึงสิ้นเดือนพอดี" อารมณ์ calm
8	12 หาร 6 ลงตัว → จึกจี่ · ครั้งที่ 4 critical: 4 หาร 3 เหลือ 1 → ให้กำลังใจ	"จึกจี่แล้วนะ~ กล้ายังอยู่ตรงนี้..." · "ไม่เป็นไรนะ เดือนไหนตึงก็ค่อย ๆ ปรับไปด้วยกัน กล้าเป็นกำลังใจให้"
9	ก่อน: $(3,000 - 500) \div 100 = 25$ วัน · ซื้อ $\text{฿}700$: $1,800 \div 100 = 18$ วัน · ซื้อ $\text{฿}2,600$: เหลือ $\text{฿}400$ ต่ำกว่าเส้น	{ price: " $\text{฿}700$ ", runwayBefore: "25 วัน", runwayAfter: "18 วัน" } · $\text{฿}2,600 \rightarrow$ runwayAfter: "ต่ำกว่าเส้นสำรองทันที"
10	"แต่ถ้ารอถึงสิ้นเดือน" เขียนติดกันไม่เว้นวรรค จึงเป็นก้อนเดียว ตอนตัด ก้อนท้ายก่อนคือ "แต่ถ้ารอถึงสิ้นเดือน" ทั้งก้อน ไม่ใช่คำว่า "แต่" เดี่ยว ๆ จึงไม่ตรงกับรายการ 12 คำ และไม่ถูกย้าย	2 ก่อน: "ถ้าซื้อวันนี้ เงินจะพอใช้อีก 18 วัน แต่ถ้ารอถึงสิ้นเดือน" "จะเหลือเงินสำรองเผื่อฉุกเฉินมากกว่า ลองคิดดูว่าจำเป็นแค่ไหนนะ" · ไม่ย้าย

สคริปต์นำเสนอ 1 นาที และคำศัพท์

อ่านออกเสียงประมาณ 60–70 วินาที เปิดแท็บโค้ดประกอบไปด้วย

“ พูดตามนี้ได้เลย

"FR-5 คือโค้ดการเงินชื่อน้องกล้า ตันกล้าเงินสีทองที่อธิบายข้อมูลที่ยืนยันแล้วให้เข้าใจง่าย เดิมเป็นโค้ด 3 โทน แต่ผู้ใช้ขอให้เป็นน้องกล้าตัวเดียวที่พูดได้ ทีมจึงแก้ทั้ง requirement โค้ด และเกสต์

โค้ดมี 2 ชั้น ชั้นแรกคือการดึงสิ่งที่ควรรู้ตอนนี้ คิดด้วยกฎในแอป เช่น Runway ใกล้เคียง หมวดที่สูงกว่าปกติ 25 เปอร์เซ็นต์และเกิน 100 บาท หรือบที่ไปเร็วกว่าเวลา ใช้ได้แม้ไม่มีเน็ต และทุกการ์ดมีตัวเลขที่มา ชั้นที่สองคือถามอะไรก็ได้ โดย AI ตอบผ่านเซิร์ฟเวอร์

หลักสำคัญคือแอปคิดเลข AI แค่อธิบาย แอปส่งเฉพาะยอดรวมตามหมวด ไม่มีรูปสลิปหรือชื่อคน และสั่งให้ AI ใช้แต่ตัวเลขที่ส่งไป ถ้าถามเรื่องชื่อของ แอปคิด Runway ก่อนและหลังซื้อให้เอง น้องกล้าพูดออกเสียงด้วยเสียงไทยของเครื่อง ตัดเป็นซับไตเติลทีละก่อน ปากขยับตามพยางค์ และไม่เคยดูผู้ใช้

ทั้งหมดมีเกสต์ เช่น TC-31 หมวดที่สูงขึ้น TC-76 การพูด TC-89 การหาราคา และ E2E ที่ตรวจเสียง ข้อความ error และการเปลี่ยนชุด ข้อจำกัดคือยังไม่จำบทสนทนาต่อเนื่อง ซึ่งเป็นแผนต่อไปค่ะ/ครับ"

คำศัพท์ของ FR-5

คำ	ความหมายแบบง่าย
Insight / การ์ดสิ่งที่ควรรู้	ข้อเท็จจริงจากกฎในแอป มี fact (ตัวเลข) กับ message (น้ำเสียงน้องกล้า)
Severity	ระดับของการ์ด good / info / watch กำหนดไอคอนและอารมณ์
Context / DATA	ข้อมูลสรุปยอดรวมที่ส่งให้ AI พร้อมคำถาม
System prompt	คำสั่งประจำตัวของ AI: บุคลิก น้ำเสียง และกติกา ส่งทุกครั้ง
Hallucination	AI แต่งข้อมูลที่ฟังดูจริงแต่ไม่จริง (สิ่งที่ FR-5 กันไว้)
purchaseCheck	ราคา Runway ก่อนซื้อ และหลังซื้อ ที่แอปคิดแล้วแนบไปให้ AI
Edge Function	โค้ดที่รันบนเซิร์ฟเวอร์ของ Supabase ถือถูกแฉ AI ไว้
Quota	จำนวนครั้งสูงสุดที่ถามได้ (60 ครั้งต่อ 24 ชั่วโมง)
Token	หน่วยนับข้อความของ AI (ขึ้นคำ) คำตอบจำกัดที่ 500
TTS (Text-to-Speech)	การแปลงข้อความเป็นเสียงพูด ใช้เสียงไทยของเครื่อง
Subtitle chunk	ซับไตเติล 1 ก่อน (≤ ประมาณ 64 ตัวอักษร)
Native driver	ให้ส่วนวาดจอคำนวณภาพเคลื่อนไหวเอง สั้นแม่ JavaScript ยุ่ง
Live region	ส่วนของจอที่โปรแกรมอ่านหน้าจออ่านให้เองเมื่อข้อความเปลี่ยน

คำ	ความหมายแบบง่าย
Skin / ภารกิจ	ชุดของน้องกล้า ได้จากภารกิจการเงินหรือช่วงลิมิเต็ด
Regex	แพตเทิร์นค้นหาข้อความ ใช้หาราคาและแปลงข้อความสำหรับเสียง